



哈爾濱工業大學
HARBIN INSTITUTE OF TECHNOLOGY



分布式存储与计算 (II)

从 MapReduce 到新型计算框架

王宏志

哈尔滨工业大学

计算学部 海量数据计算研究中心

wangzh@hit.edu.cn

<http://homepage.hit.edu.cn/wang>

目录

Contents

- 1 MapReduce与计算框架
- 2 数据的分片与路由
- 3 分布式文件系统
- 4 分布式计算框架例析：Hadoop
- 5 分布式计算框架例析：Spark
- 6 分布式计算框架例析：Storm

目录

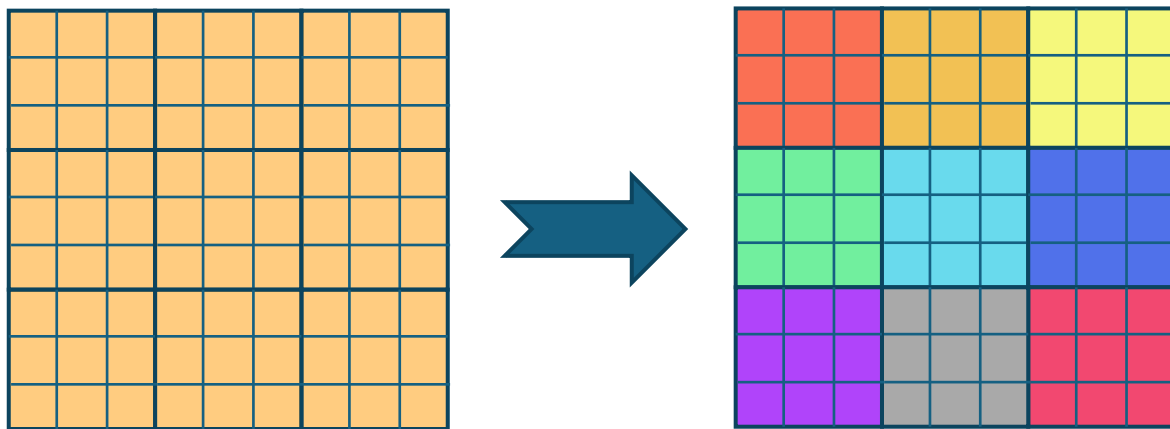
Contents

- 1 **MapReduce与计算框架**
- 2 数据的分片与路由
- 3 分布式文件系统
- 4 分布式计算框架例析：Hadoop
- 5 分布式计算框架例析：Spark
- 6 分布式计算框架例析：Storm

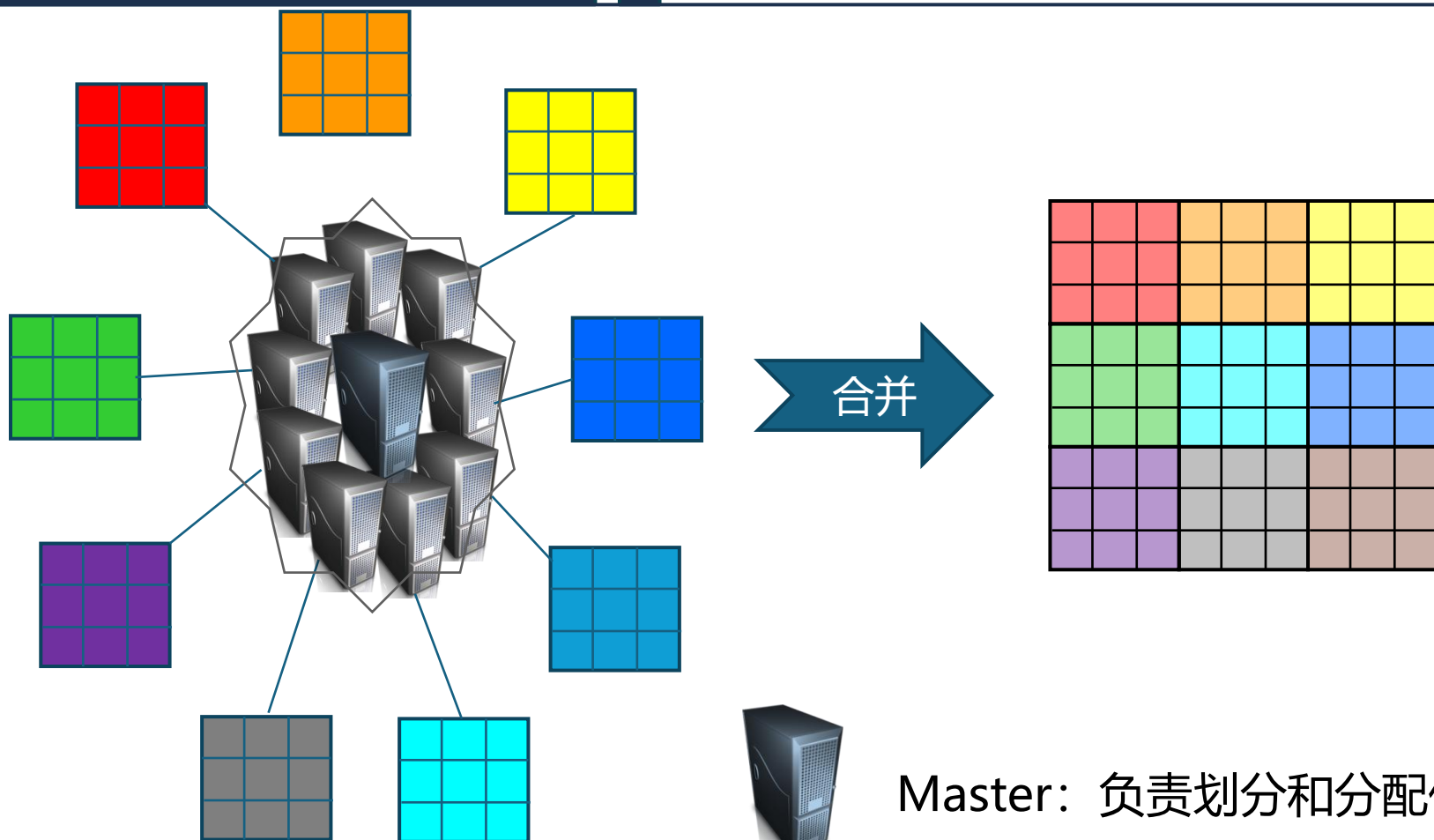
分布式并行计算

计算任务无法由单台计算机完成时，需要由多台机器共同承担计算任务

例如：假设有一个巨大的2维数据需要处理(比如求每个元素的开立方)，其中对每个元素的处理是相同的,并且数据元素间不存在数据依赖关系,可以考虑不同的划分方法将其划分为子数组,由一组处理器并行处理



并行计算

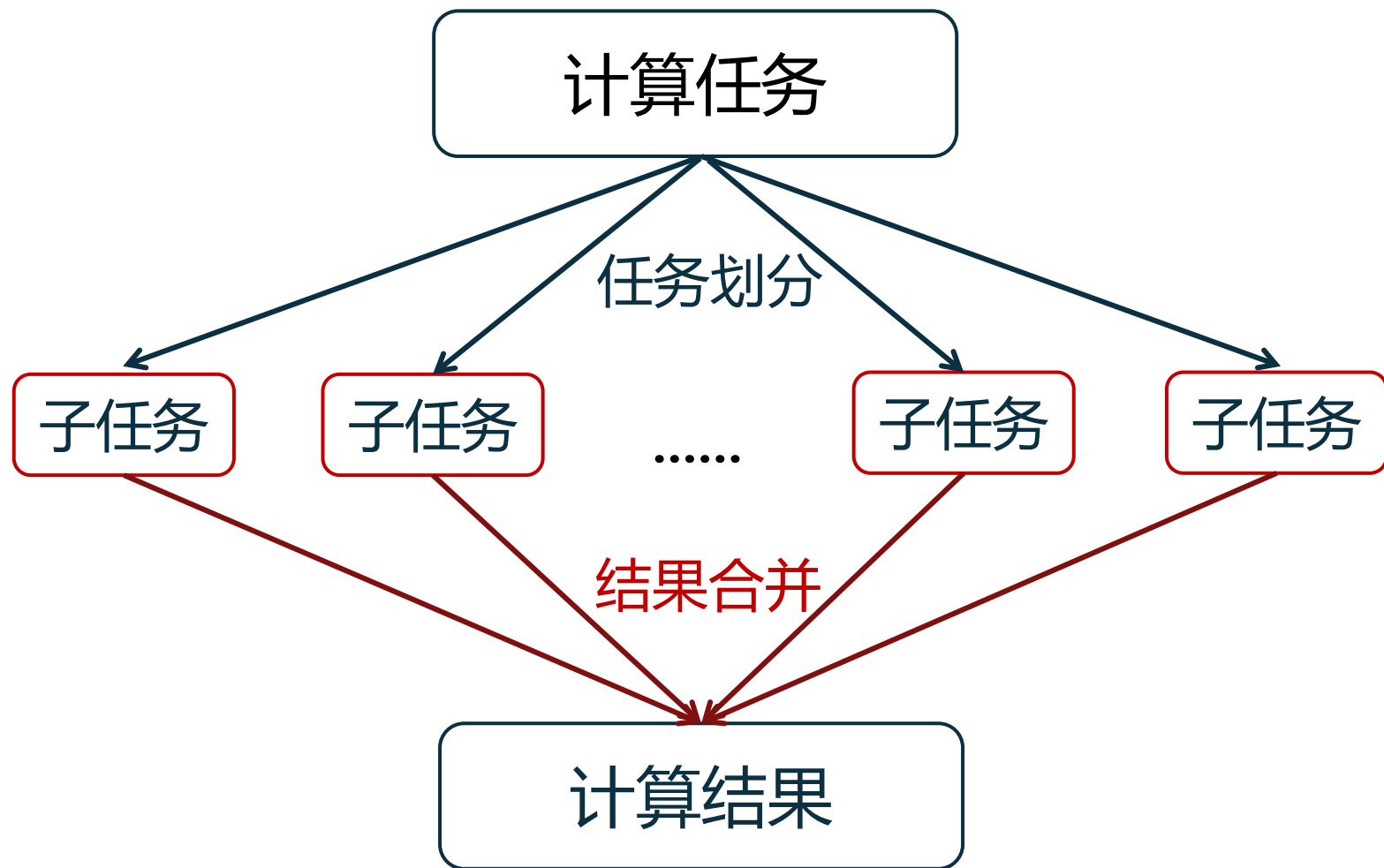


Master: 负责划分和分配任务



Worker: 负责数据块计算

计算任务划分和并行计算模型



计算框架的缘起

- 在分布式环境中进行并行处理，除了访问存储系统，还涉及计算任务的分工，计算负荷的分配，计算机间的数据迁移等工作，并且要考虑计算机或网络发生故障时的数据安全，情况复杂
- 对开发人员的挑战
 - ✓ 编程难度大、调试难度大
- 对系统设计人员的挑战
 - ✓ 系统设计需要考虑的因素多
- 对系统运维人员的挑战
 - ✓ 运维工具缺少，容易积累错误



从一个例子说起

从很多很多销售记录中统计各种商品销售额，一台机器无法处理，需要许多台计算机一起来统计。

需要解决的问题：

问题一：如何为每台机器分配任务，机向各台机器分发一部分销售记录？

问题二：上述两种方式都涉及数据的

问题三：如何定义每台机器处理的数据？是主动发送，接收方处理不过来怎

问题四：会不会任务分配不均，有的理完后才能开始执行？

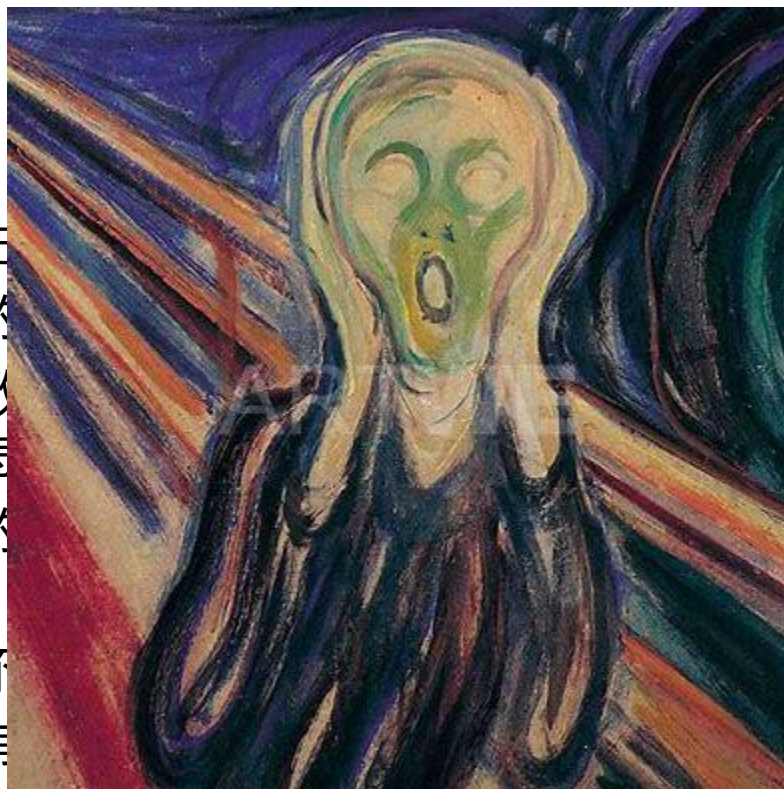
问题五：如果增加一台机器，它能不能

问题六：如果一台机器挂了，它没有

问题七：统计过程中，机器之间如何协调，是否需要专门的一台机器指挥调度其他机器？如果这台机器挂了呢？

问题八：如果销售记录在源源不断地增加，统计还没执行完新记录又来了，如何保证统计结果的准确性？能不能保证结果是实时更新的？再次统计时能不能避免大量重复计算？

问题九：能不能让用户执行一个语句就可以得到结果？



同机器处理不同商品种类的销售记录，还是按商品种类合并？

该在哪台机器上执行排序过程？

据是主动发送，还是接收方申请时才发送？

方应该保存数据多久？

忙着？甚至，闲着的机器需要等忙着的机器

务执行时间？

计或重复统计？

为什么有计算框架？

- 除了问题一外，其余的都与具体任务无关，在其他分布式计算的场合也会遇到，而且解决起来都相当困难
- 问题一中的分组、统计，在很多数据处理场合也会涉及，只是具体方式不同
- 如果能把这些问题的解决方案封装到一个计算框架中，则可大大简化这类应用程序的开发

问题一：如何为每台机器分配任务，是先按商品种类对销售记录分组，不同机器处理不同商品种类的销售记录，还是随机向各台机器分发一部分销售记录进行统计，最后把各台机器的统计结果按商品种类合并？

问题二：上述两种方式都涉及数据的排序问题，应选择哪种排序算法？应该在哪个机器上执行排序过程？

问题三：如何定义每台机器处理的数据从哪里来，处理结果到哪里去？数据是主动发送，还是接收方申请时才发送？如果是主动发送，接收方处理不过来怎么办？如果是申请时才发送，那发送方应该保存数据多久？

问题四：会不会任务分配不均，有的机器很快就处理完了，有的机器一直忙着？甚至，闲着的机器需要等忙着的机器处理完后才能开始执行？

问题五：如果增加一台机器，它能不能减轻其他机器的负荷，从而缩短任务执行时间？

问题六：如果一台机器挂了，它没有完成的任务该交给谁？会不会遗漏统计或重复统计？

问题七：统计过程中，机器之间如何协调，是否需要专门的一台机器指挥调度其他机器？如果这台机器挂了呢？

问题八：如果销售记录在源源不断地增加，统计还没执行完新记录又来了，如何保证统计结果的准确性？能不能保证结果是实时更新的？再次统计时能不能避免大量重复计算？

问题九：能不能让用户执行一个语句就可以得到结果？

什么是计算框架？

- 计算框架：一种抽象，在其中提供相应的通用功能供用户编写代码以实现具体功能，从而形成面向应用的软件。
- 简史
 - 2004年前后，Google提出分布式文件系统GFS、并行计算模型MapReduce、非关系数据存储系统BigTable，第一次提出了针对大数据分布式处理的可重用方案。
 - 在Google论文的启发下，Yahoo的工程师DougCutting和MikeCafarella开发了Hadoop
 - 在借鉴和改进Hadoop的基础上，又先后诞生了数十种应用于分布式环境的大数据计算框架
- 优势
 - 无需造轮子
 - 提高开发效率
 - 降低开发成本
 - 降低运维难度
 - 提高代码执行的可靠性
- 今天的分布式计算系统的开发过程中几乎离不开大数据计算框架！

```
Class Mapper
method map(String input_key, String input_value):
    // input_key: text document name
    // input_value: document contents
    for each word w in input_value:
        EmitIntermediate(w, "1");
```

```
Class Reducer
method reduce(String output_key,
               Iterator intermediate_values):
    // output_key: a word
    // output_values: a list of counts
    int result = 0;
    for each v in intermediate_values:
        result += ParseInt(v);
    Emit(AsString(result));
```


计算框架做了什么？

- 计算任务的划分和调度
- 数据的分布存储和划分
- 处理数据与计算任务的同步
- 结果数据的收集整理(sorting, combining, partitioning,...)
- 系统通信、负载均衡、计算性能优化处理
- 处理系统节点出错检测和失效恢复



目录

Contents

- 1 MapReduce与计算框架
- 2 数据的分片与路由
- 3 分布式文件系统
- 4 分布式计算框架例析：Hadoop
- 5 分布式计算框架例析：Spark
- 6 分布式计算框架例析：Storm

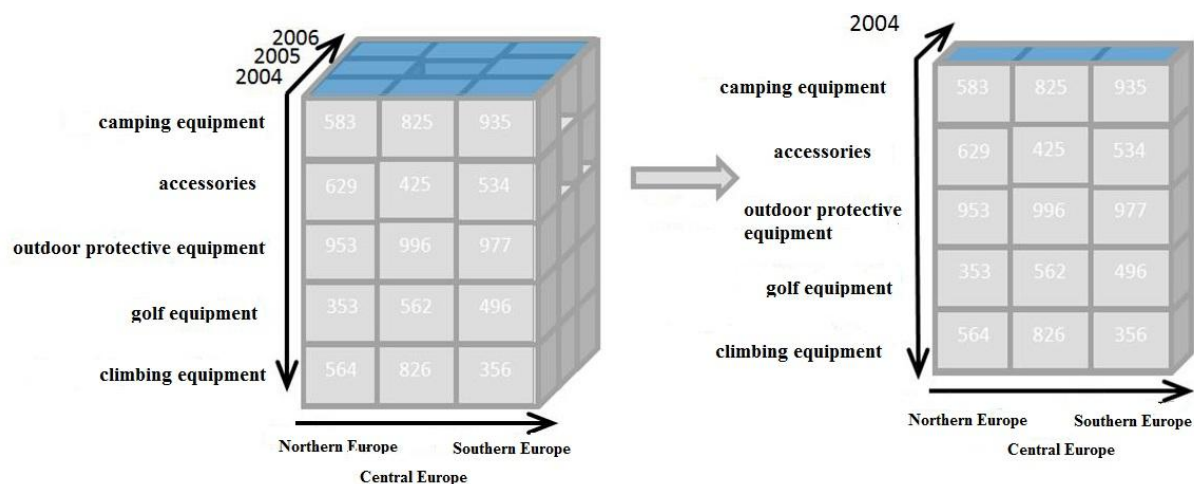
数据分片与路由

数据分片

将待存储处理的数据进行切分并分配到各个机器中去。

数据路由

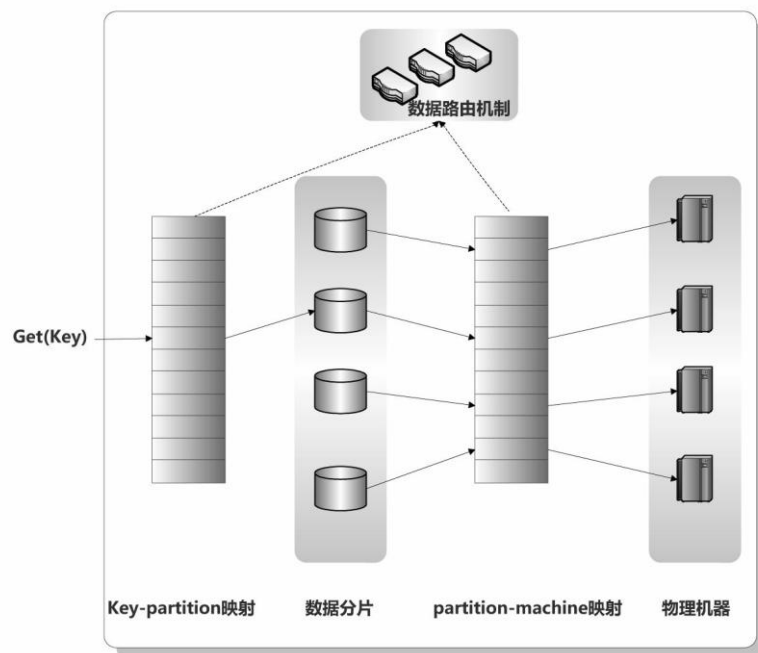
数据分片后，找到某条记录的存储位置。



*图片: <https://pediaa.com/what-is-the-difference-between-slice-and-dice-in-data-warehouse/>

数据分片

- 可以将其看作是一个二级映射关系
- 第一级映射是key-partition映射，其将数据记录映射到数据分片空间
- 第二级映射是partition-machine映射，其将数据分片映射到物理机器中

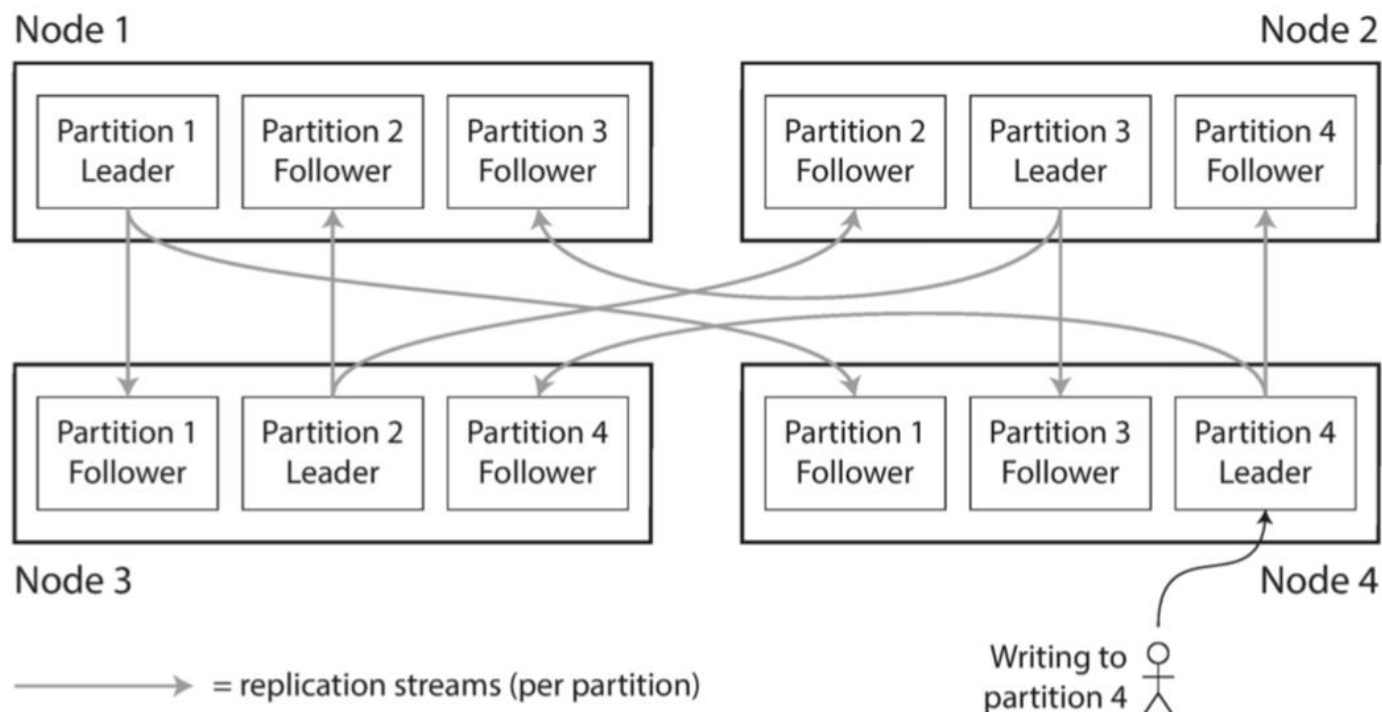


* 图片: https://fred_yufeng.gitbooks.io/massive_data_collect/content/cun-chu/shu-ju-fen-pian/ha-xi-fen-pian.html

分片与复制

分片通常与复制结合使用：每个分片的副本存储在多个节点上。

- 这意味着，即使每条记录属于一个分片，它仍然可以存储在多个不同的节点上以获得容错能力。
- 一个节点可能存储多个分片。使用主从复制模型，分片复制的组合如图所示。每个节点可能是某些分片的领导者，同时是其他分片的追随者。



键值数据的分片

- 假设有大量数据并且想要分片,如何决定在哪些节点上存储哪些记录呢?



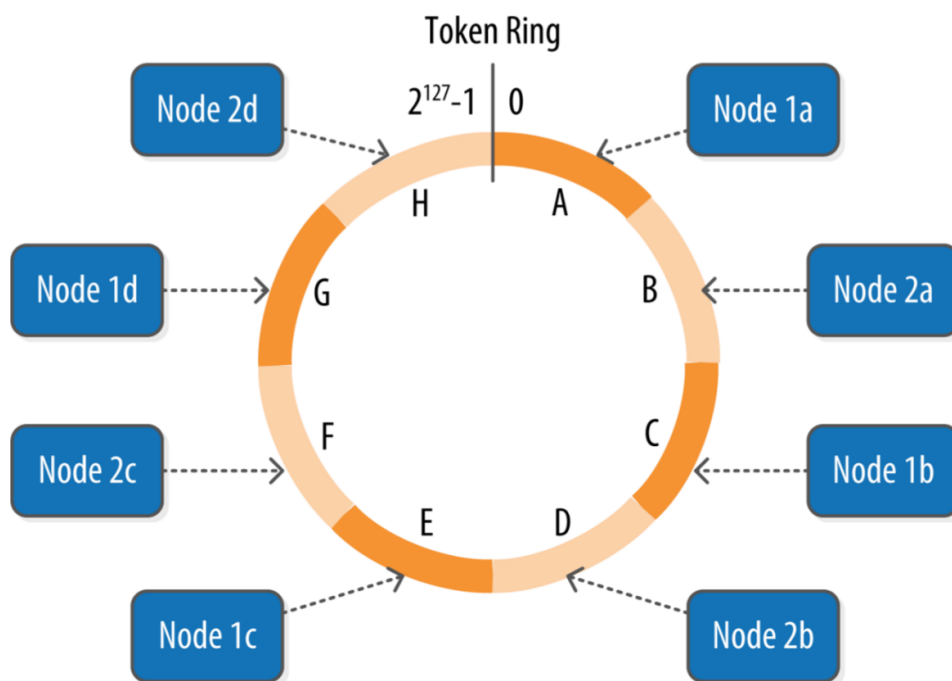
- ✓ 分片目标是将数据和查询负载均匀分布在各个节点上。
- ✓ 如果每个节点公平分享数据和负载，理论上10个节点应该能够处理10倍的数据量和10倍的单个节点的读写吞吐量（忽略复制）。

- 如果分片是不公平的，一些分片比其他分片有更多的数据或查询，称之为**偏斜 (skew)**。
- 数据偏斜的存在使分片效率下降很多。在极端的情况下，所有的负载可能压在一个分片上，瓶颈落在这一个繁忙的节点上。不均衡导致的高负载的分片被称为**热点 (hot spot)**。

键值数据的分片

避免热点最简单的方法：将记录随机分配给节点。

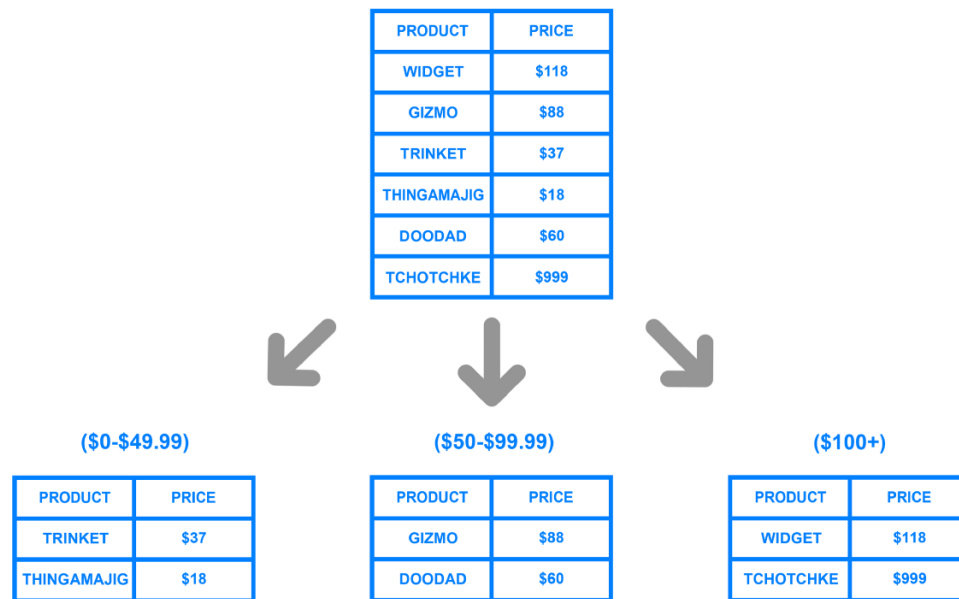
- 这将在所有节点上平均分配数据。
- 缺点：试图读取一个特定的值时，无法知道它在哪个节点上，所以必须并行地查询所有的节点。



范围分片

范围分片

- 首先将所有记录的主键进行排序。
- 然后在排好序的主键空间里将记录划分成数据分片，每个数据分片存储有序的主键空间片段内的所有记录

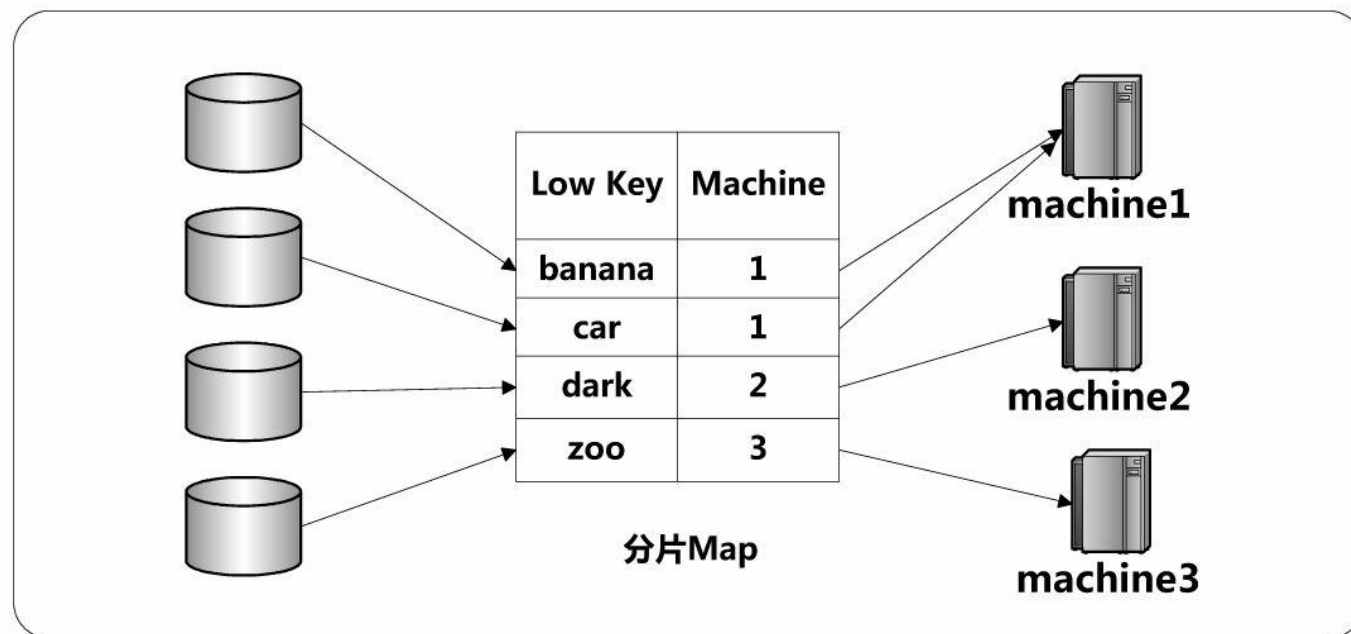


*图片: <https://www.digitalocean.com/community/tutorials/understanding-database-sharding>

范围分片

在实现具体存储系统时，往往保持一个数据分片的映射表，记录表每一项记载数据分片的最小主键及其对应的物理机地址

- ✓ 在对记录 / 增 / 删 / 改时，查找映射表就可以找到对应存储这个记录所在数据分片的物理机
- ✓ 数据分片在物理机的管理方式往往采用LSM树



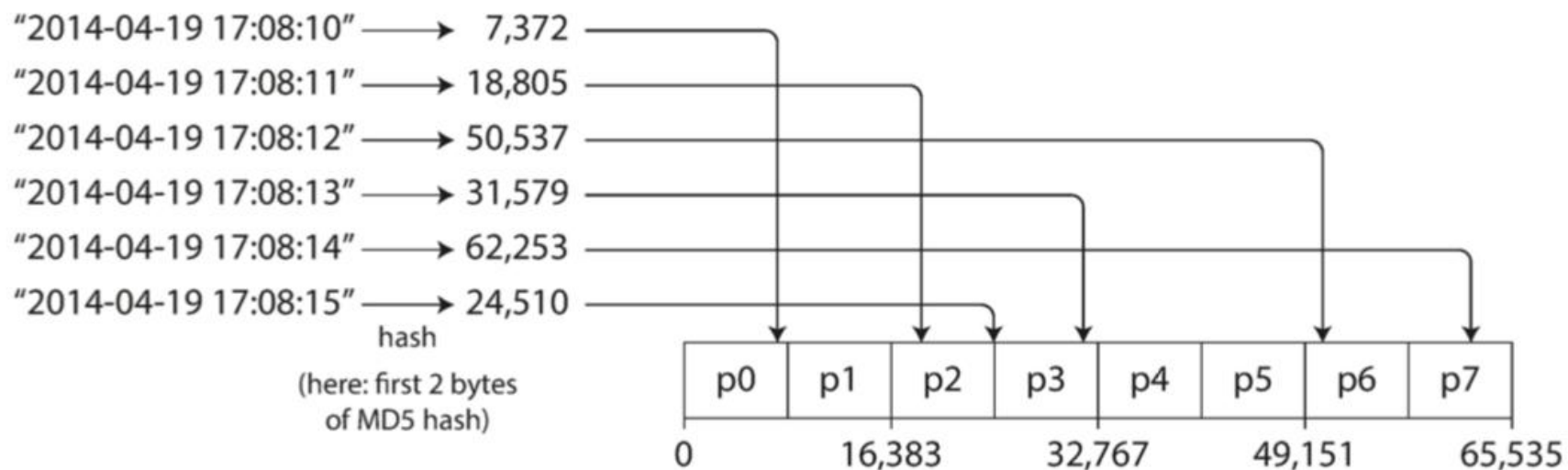
键的保存

- 在每个分片中，可以按照一定的顺序保存键。
- **好处是**进行范围扫描非常简单，您可以将键作为联合索引来处理，以便在一次查询中获取多个相关记录
- **缺点是**某些特定的访问模式会导致热点。例如，如果主键是时间戳，由于在测量发生时将数据从传感器器写入数据库，因此所有写入操作都会转到同一个分片（即今天的分片），这样分片可能会因写入而过载，而其他分片则处于空闲状态。
- 为了避免这个问题，需要使用除了时间戳以外的其他东西作为主键的第一个部分。

根据键的哈希分片

由于偏斜和热点的风险：许多分布式数据存储使用散列函数来确定给定键的分片。

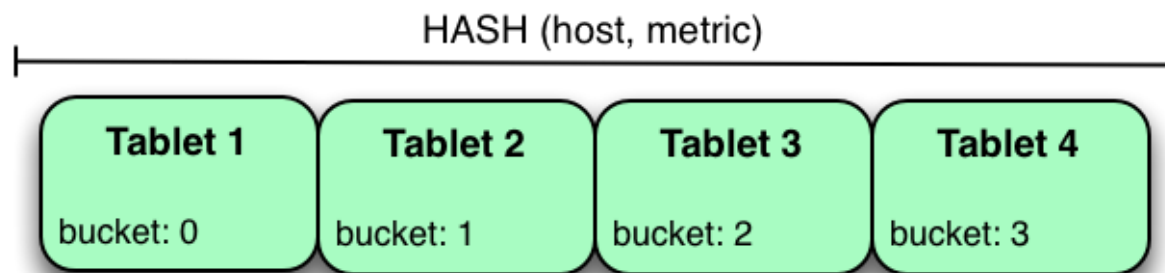
- 一个好的散列函数可以将偏斜的数据均匀分布。出于分片的目的，散列函数不需要多么强壮的算法。
- 一旦有一个合适的键散列函数，可以为每个分片分配一个散列范围，每个哈希散列落在分片范围内的键将被存储在该分片中。



哈希分片

哈希分片

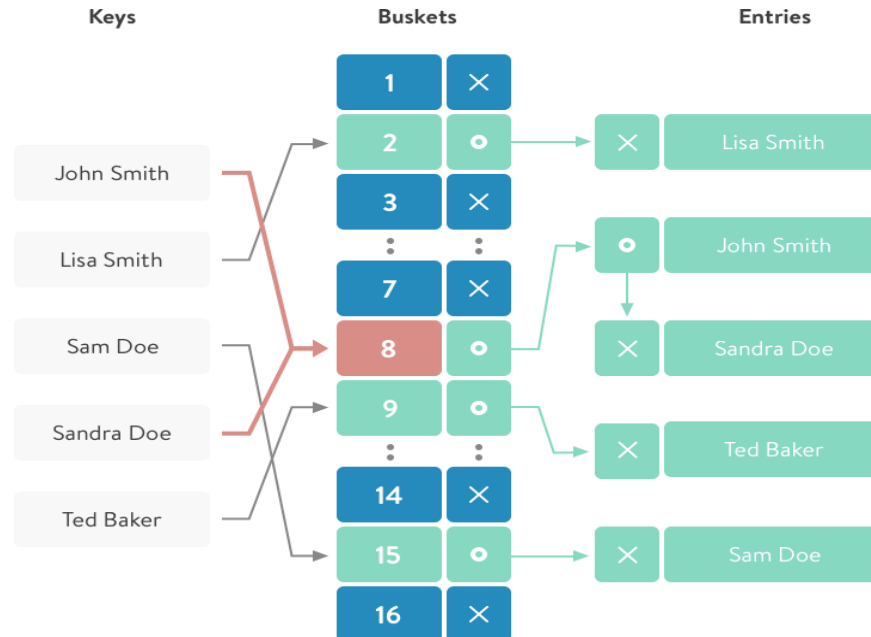
- 通过哈希函数来进行数据分片。
- 主要包括：Round Robin、虚拟桶及一致性哈希方法。



Round Robin

Round Robin

- 即，哈希取模法。
- 假设有K台物理机，通过以下哈希函数即可实现数据分片：
$$H(\text{key}) = \text{hash}(\text{key}) \bmod K$$
- 对于以key为主键的记录， $H(\text{key})$ 即是存储该数据的物理机编号

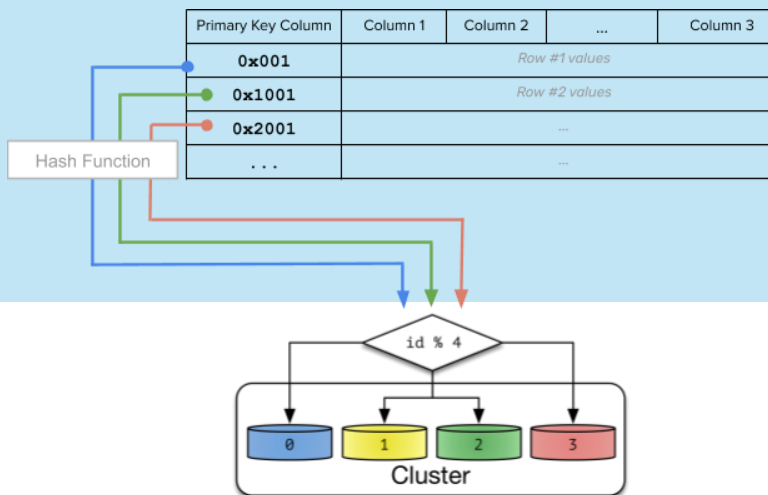


*图片: <https://anadea.info/blog/how-hash-table-works-ruby-examples>

Round Robin

Round Robin

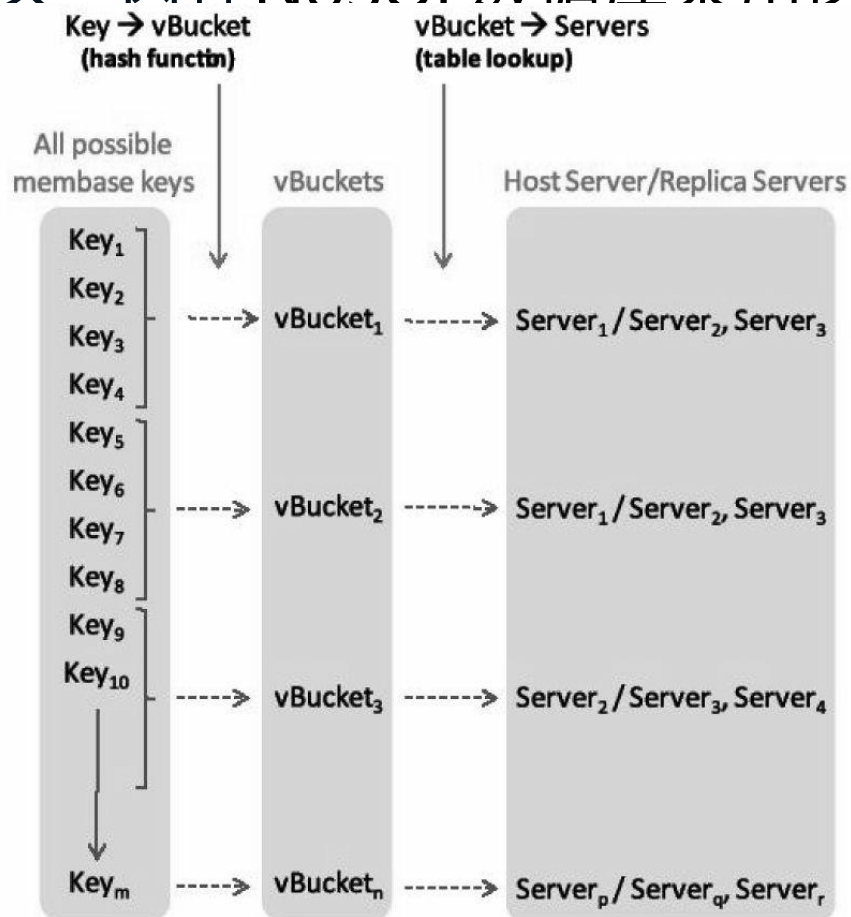
- ✓ 优点：实现起来非常简单
- × 缺点：缺乏灵活性



- 比如，新增一台物理机到分布式存储系统，那么哈希函数就变成了：
$$H(key) = hash(key) \bmod (K + 1)$$
- 之前已经分配的所有数据与存储该数据的物理机之间的映射关系被完全打乱，只能将所有数据再次分配一遍

虚拟桶

为了解决上述问题，对于数据分片管理，提出了虚拟桶的实现方式，Couchbase这一内存NoSQL 数据库采用这个策略：



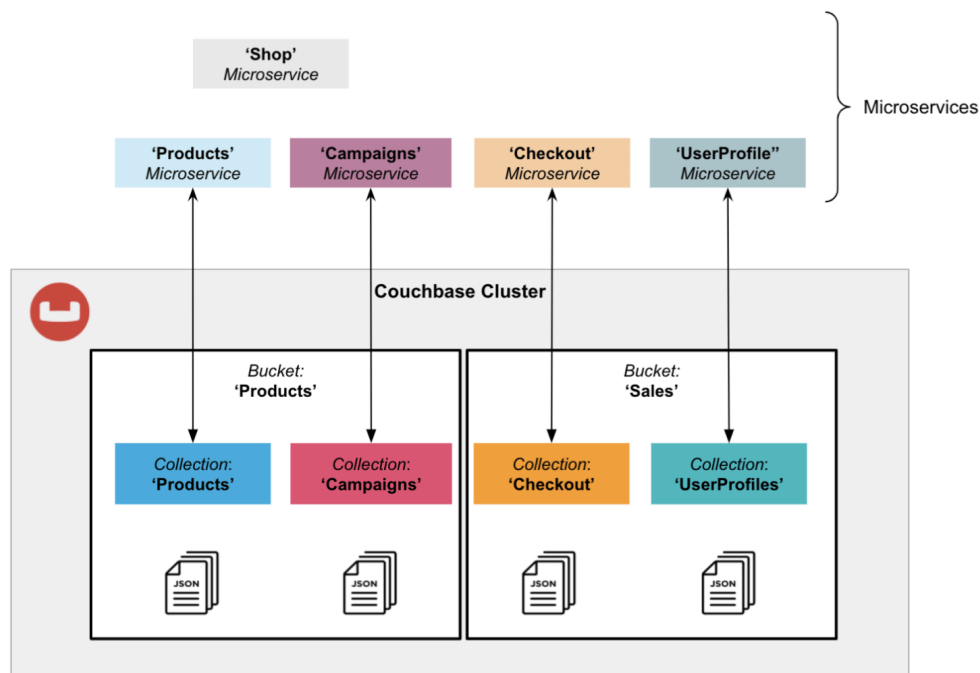
基本原理

- Membase在待存储记录和物理机之间引入了**虚拟桶层**，所有记录首先通过**哈希函数**映射到对应的虚拟桶，记录和虚拟桶是**多对一**的映射关系，即一个虚拟桶包含多条记录信息将快照复制到新的从库节点。
- 第二层映射是虚拟桶和物理机之间的映射关系，同样也是多对一映射，一个物理机可以容纳多个虚拟桶，其具体实现方式是通过查表来实现的

虚拟桶

优点

- 引入虚拟桶层，这样将原先由记录直接到物理机的单层映射解耦成**两级映射**，大大加强了系统的扩展灵活性。
- 当新加入机器时，将某些虚拟桶从原先分配的机器重新分配给新机器，只需要修改partition-machine映射表中受影响的个别条目就能实现扩展

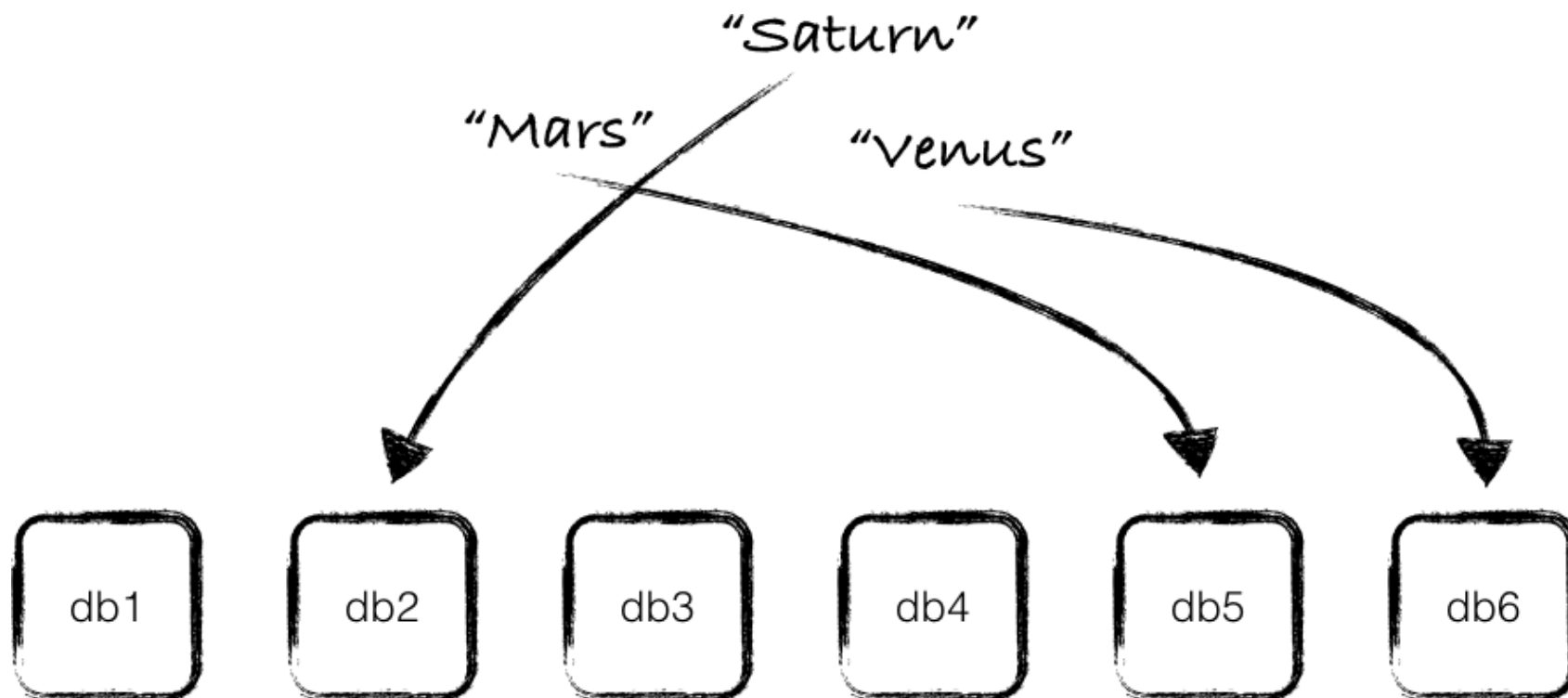


*图片: <https://www.couchbase.com/blog/microservices-architecture-in-couchbase/>

一致性哈希

分布式哈希表 (DHT) 是P2P (Peer-to-Peer) 网络 (对等联网) 和分布式存储中常见的一项技术。

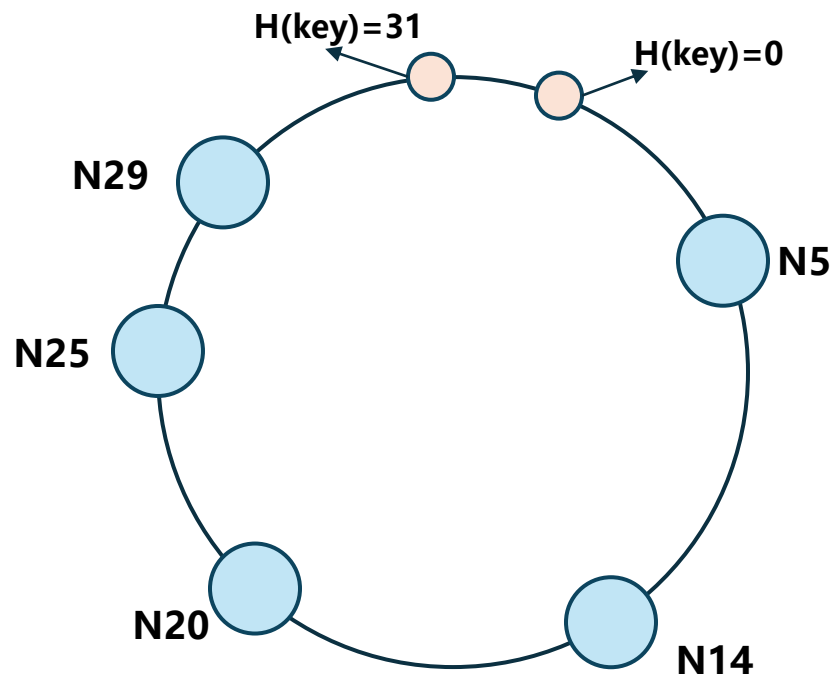
- DHT的实现方式有很多，一致性哈希是其中一种实现方式。



一致性哈希

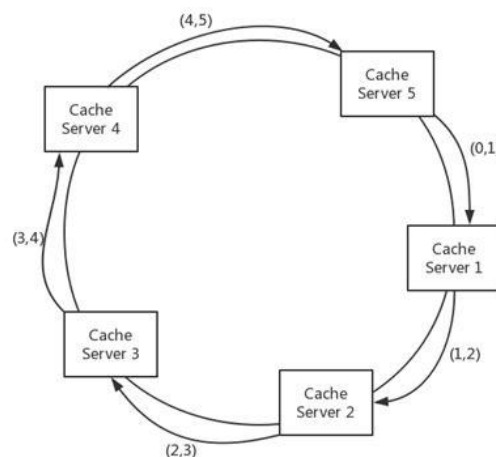
考虑哈希空间表示为长度为5的二进制数值情况：其哈希空间可以表达的数值范围为0 ~ 31。

- 哈希数值空间按照大小组成一个首尾相接的环状序列。
- 根据机器的IP和端口号经过哈希函数映射到哈希数值空间，那么这台机器则负责存储落在一段有序哈希空间内的数据。
- 每个机器节点记录环中的前趋节点和后继节点地址位置



路由问题

- 在P2P环境下意味着无中心管理节点，那么如何根据数据记录的主键以及哈希函数H来定位到记录内容呢？



一种直观的解决办法：沿着有向环顺序查找，接收到查询请求的节点根据哈希函数获得待查找主键的哈希值j。

- 首先判断是否在自身管理范围内
- 如果不在，则将其转交给后继节点继续查找，如此循环
- 直到找到某个机器节点 N_x ， x 是大于等于 j 的最小编号节点。

路由问题

为了加快查找速度，可以在每个机器节点配置路由表。

- 路由表存储m条路由信息
- 其中第i项 ($0 \leq i \leq m-1$) 路由信息代表距离当前节点为 2^i 的哈希空间数值所在的机器节点编号。

距离	1 (2^0)	2 (2^1)	4 (2^2)	8 (2^3)	16 (2^4)
机器节点	N20	N20	N20	N25	N5

- 表中第3项代表与N14节点距离为4的哈希值（即 $14+4=18$ ）落在N20节点上。
- 同理第5项代表与N14节点距离为16的哈希值落在N5节点上。

路由问题

- 有了路由表，假设机器节点 N_i 接收到了主键为 key 的查询请求，如果 $H(key) = j$ 不在 N_i 的管理范围，应该如何进行查找操作呢？

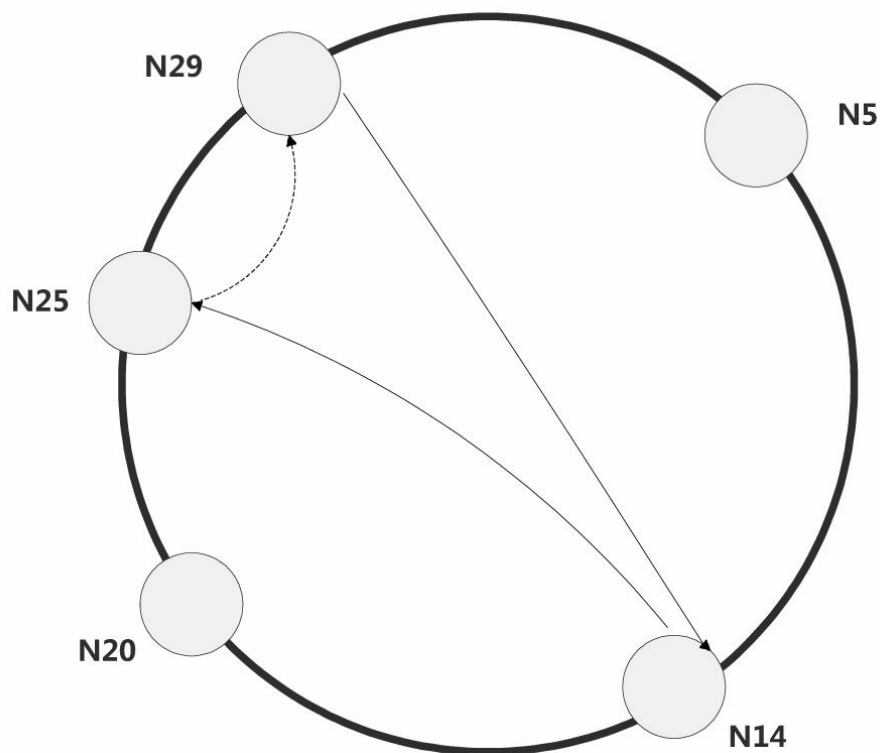
一致性哈希路由算法

- 假设当前执行操作的节点为 N_c ，其初始值是 N_i ， N_c 的后继节点为 N_s ，重复执行下列步骤。
 - ✓ 判断是否 $c < j \leq s$ ，如果为真，则结束查找，说明 key 如果存在，则在 N_c 的后继节点 N_s 上，所以 N_c 发送消息给 N_s 查找 key 的值 $value$ ， N_s 将查询结果返回给 N_i
 - ✓ 否则， N_c 查找其对应的路由表，找到小于 j 的最大编号节点 N_h ， N_c 向 N_h 发送消息，请求它代表 N_i 查找 key 的值 $value$ ， N_h 此时成为当前节点 N_c ，继续按照步骤1和步骤2递归进行查找操作。

路由问题的例子

N14节点接到查询key的键值请求，其中 $H(\text{key})=27$

- N14发现27不在后继节点N20中，进入步骤2
- 查找路由表，找到小于27的最大编号节点N25，重复步骤1



加入新节点

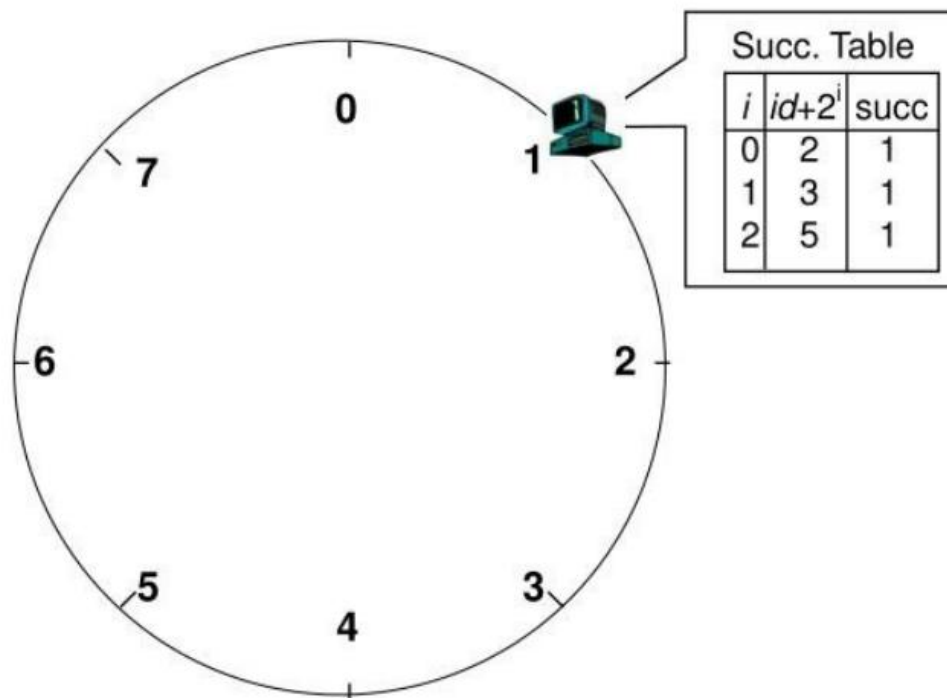
如果P2P网络中新加入一个机器节点 N_{new} ，首先 N_{new} 必须能够和目前P2P网络中任意一个节点 N_x 建立联系。

- 根据 $H(N_{new}) = new$ 找到 N_{new} 的后继节点 N_s 。
- 假设 N_s 的前趋节点为 N_p ，那么为了能够将 N_{new} 加入P2P网络，需要做以下两件事情：

- ✓ 改变 N_p 、 N_{new} 和 N_s 对应已经发生变化的前趋节点和后继节点记录，以体现新的网络架构。
- ✓ 数据的重新分片与分布，具体而言就是将 N_s 节点中存储的应该由 N_{new} 承载的数据（即 N_s 节点上哈希值小于等于 new 的记录）迁移到 N_{new} 节点上。

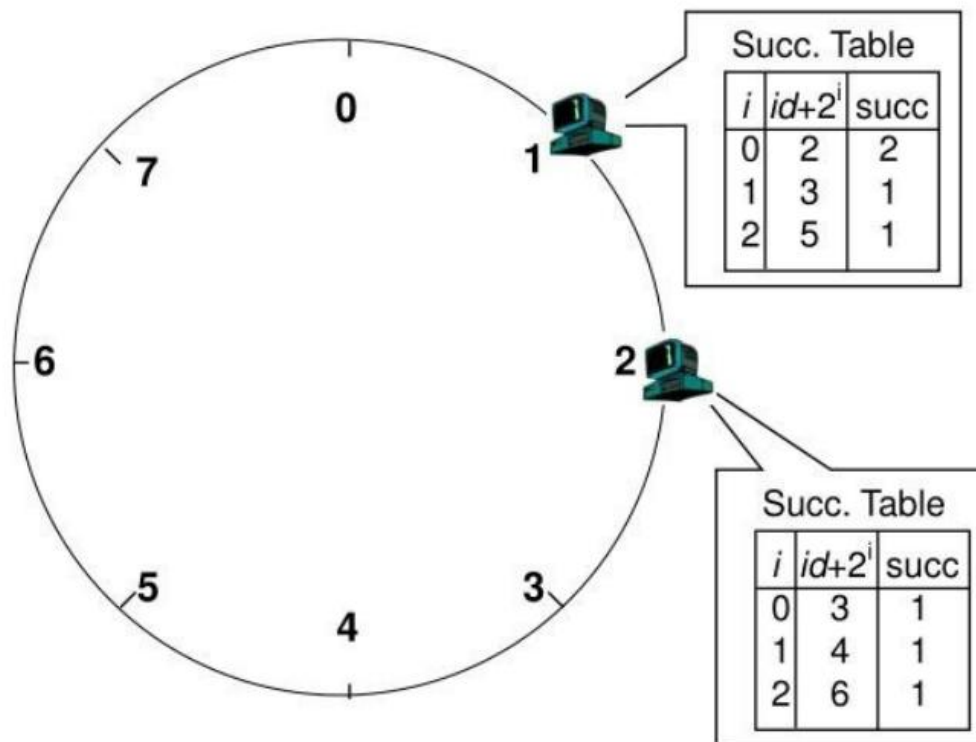
加入新节点

- 假设哈希空间为 $[0, 2^i]$ 添加Node1



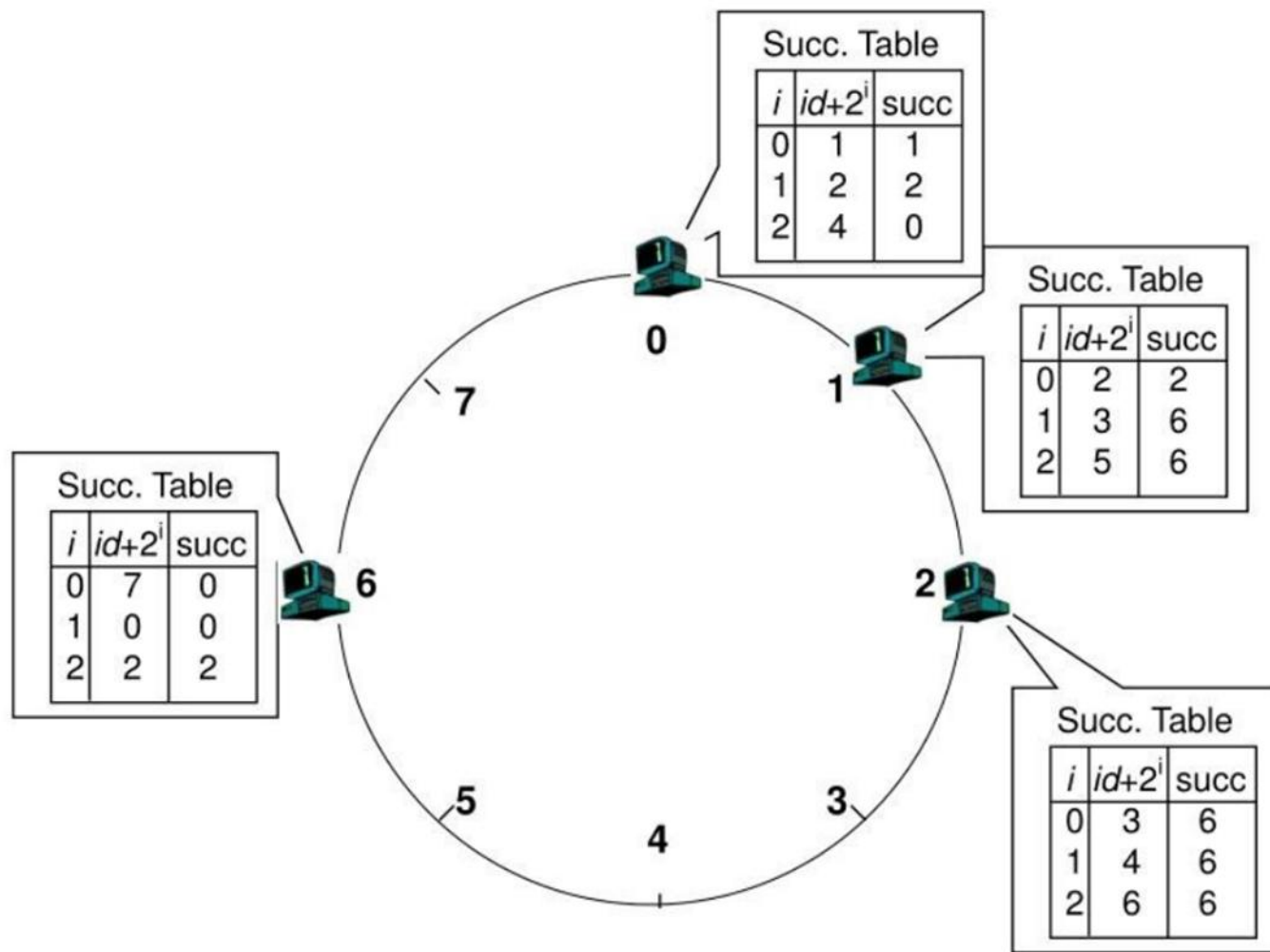
加入新节点

- 假设哈希空间为 $[0..8]$ 。添加Node2?



加入新节点

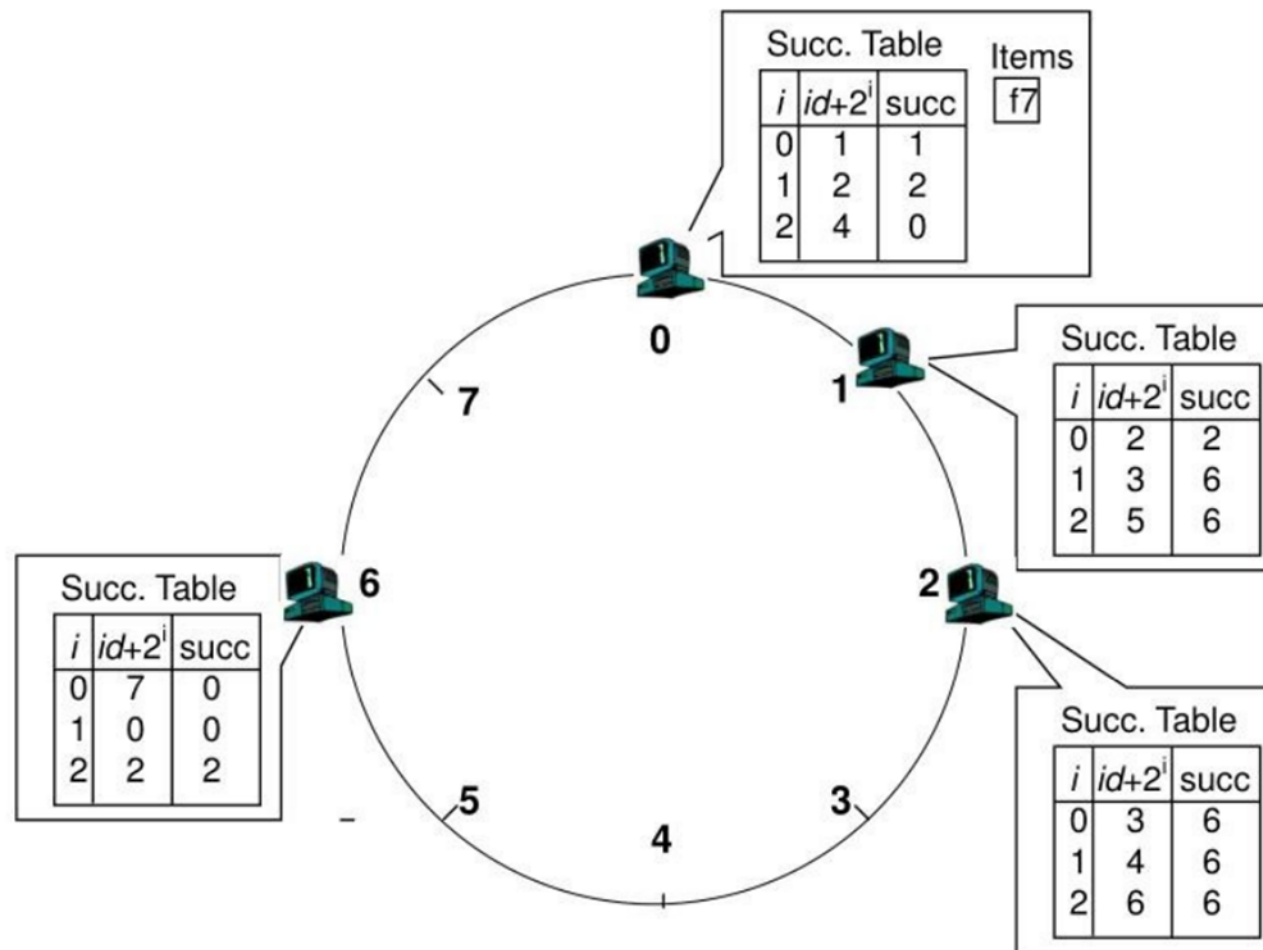
- 假设



*图片: <https://www.slideserve.com/cyndi/distributed-hash-tables>

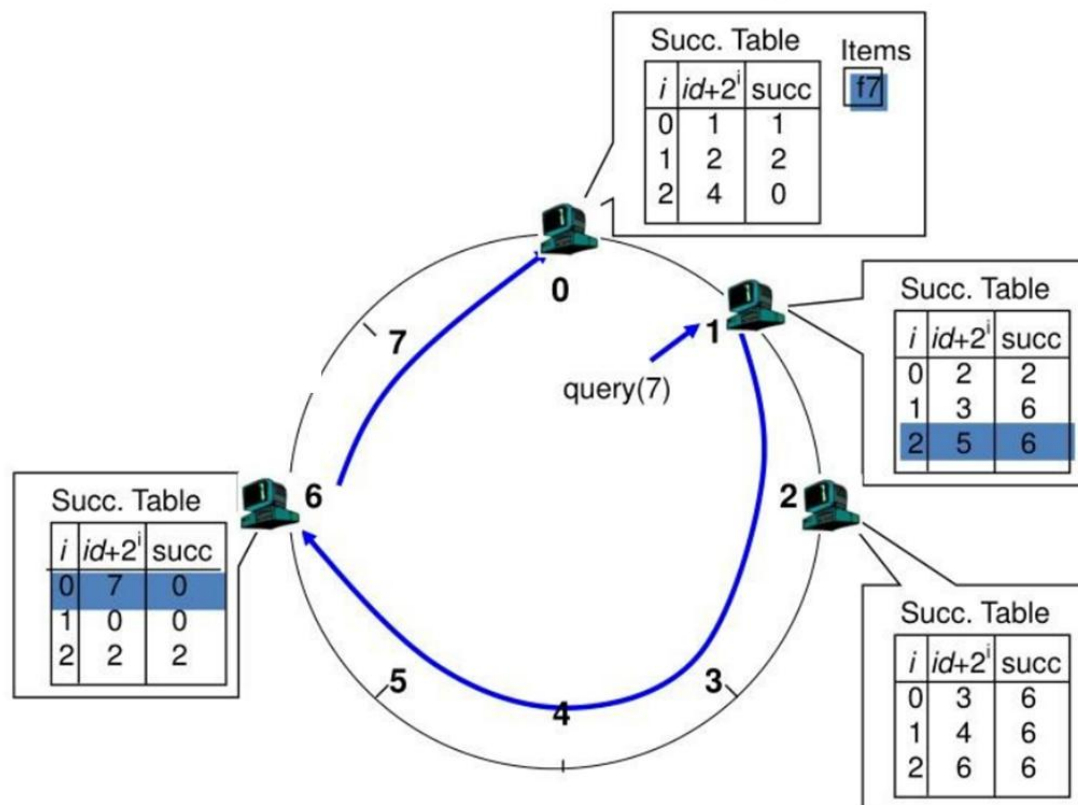
加入新节点

- 假设哈希空间为 $[0..8]$ ，添加数据 $f7$



加入新节点

- 假设哈希空间为 $[0..8]$ ，查询数据 $f7$
- ✓ 检查是否在本地存储
- ✓ 如果不是，则将该查询转发给其最大的继承者，该继承者id不超过 id

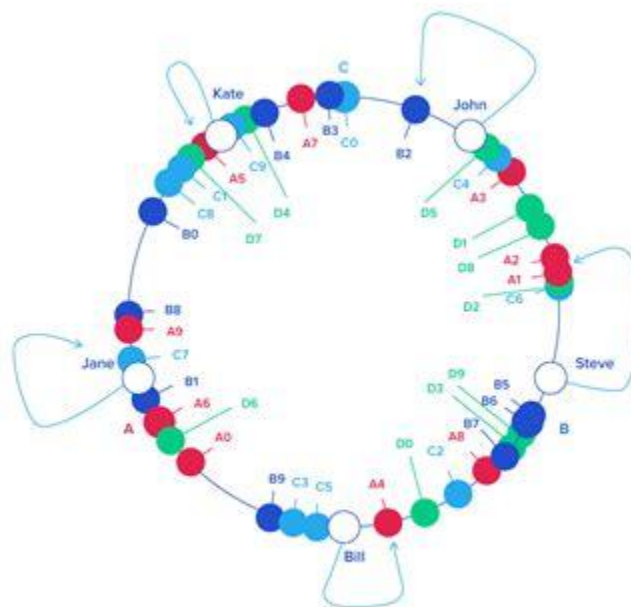


*图片: <https://www.slideserve.com/cyndi/distributed-hash-tables>

加入新节点

在并发环境下，可能在 N_p 和 N_s 之间同时有多个新节点要加入，此时可以按照以下两步操作。

- 将 N_{new} 的后继节点指向 N_s ，前继节点置为空值null。
- P2P网络中每个节点定期执行稳定性检测，完成前趋和后继节点的更新及数据迁移。



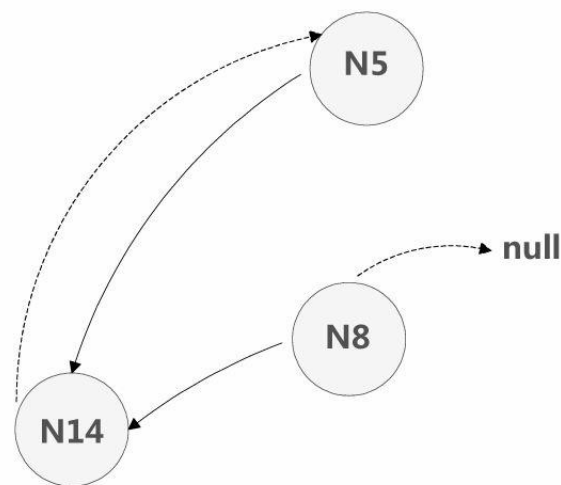
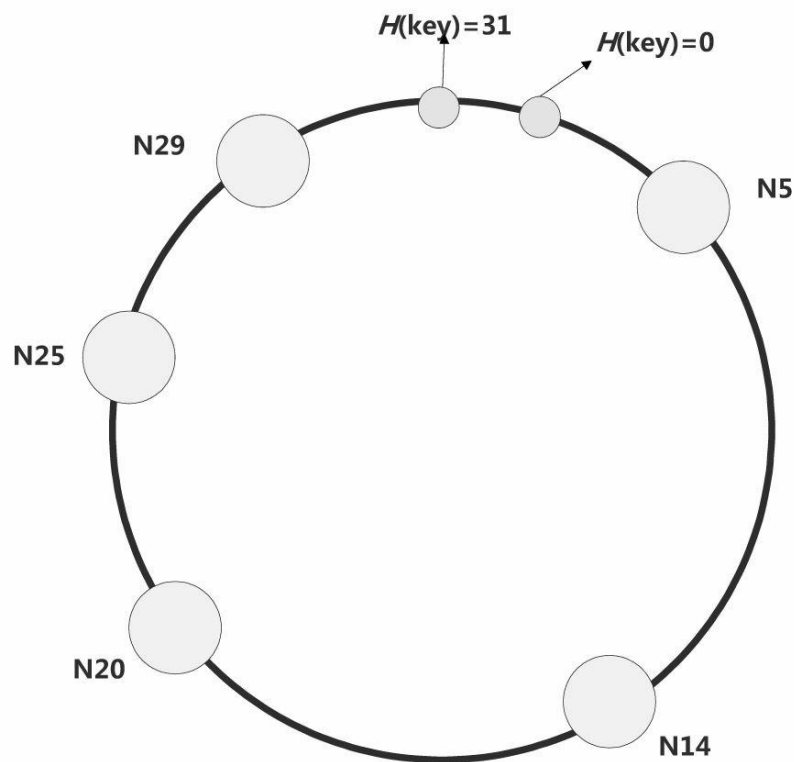
稳定性检测

- 假设 N_s 为 N_c 的后继节点, N_c 向 N_s 询问其前趋节点 N_p , N_s 向 N_c 答复, 一般情况下, 如果这样则转第4步。
- 如果 N_p 介于 N_c 和 N_s 之间, N_c 记录下 N_p 为其后继节点。
- 令 N_x 是 N_c 的当前后继节点, 其可能是 N_s 也可能是 N_p , 这取决于步骤2的判断结果。如果 N_x 的前趋节点为空或者 N_c 位于 N_x 和它的前趋节点之间, 那么 N_c 给 N_x 发消息告诉 N_x 说 N_c 就是 N_x 的前趋节点, N_x 将其前趋节点设置为 N_c 。
- N_x 将其部分数据迁移到 N_c , 即将 N_x 上哈希值小于等于 c 的记录迁移到 N_c 上。

加入新节点的例子

假设在左图的P2P网络例子中新加入节点N8

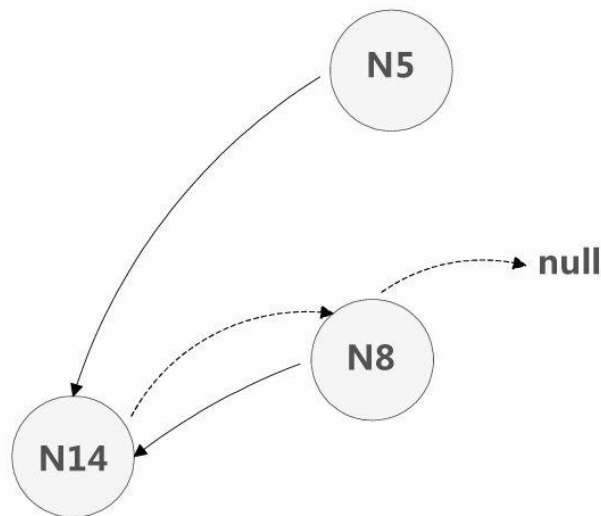
- 根据步骤1，首先将N8的后继节点置为N14，其前趋节点置为空，节点状态如右图所示



加入新节点的例子

假设在左图的P2P网络例子中新加入节点N8

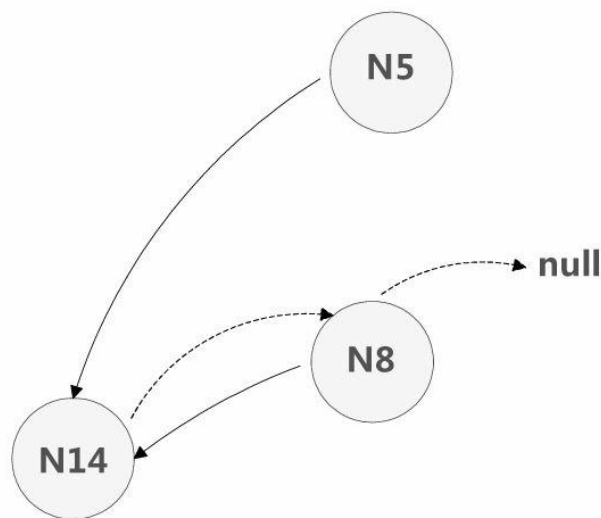
- 之后N8节点开始执行稳定性检测
 - ✓ 在检测过程的步骤1中，N8发现N14的前趋节点是N5而不是自己
 - ✓ 进入步骤2，由于N5没有介于N8和N14之间，所以步骤2无动作
 - ✓ 进入步骤3，其中 $N_x = N_s = N14$ ，因为N8位于N14和它的前趋节点N5之间，于是N8给N14发消息告知其前趋节点应该为N8，N14将其前趋节点指向N8,节点状态如图所示



加入新节点的例子

假设在左图的P2P网络例子中新加入节点N8。

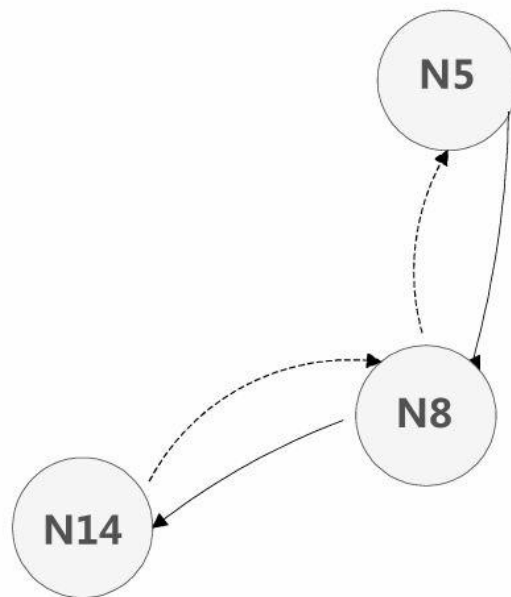
- 之后N8节点开始执行稳定性检测
- ✓ 在接下来的步骤4中，N14中主键哈希值落在6到8之间的记录被迁移到N8节点上。节点N8完成了一次稳定性检测。
- ✓ 此时，数据得到了迁移，N14的前趋节点指向N8，N5还需要更改其后继节点指向，N8也需要落实其前趋节点指向。



加入新节点的例子

假设在左图的P2P网络例子中新加入节点N8。

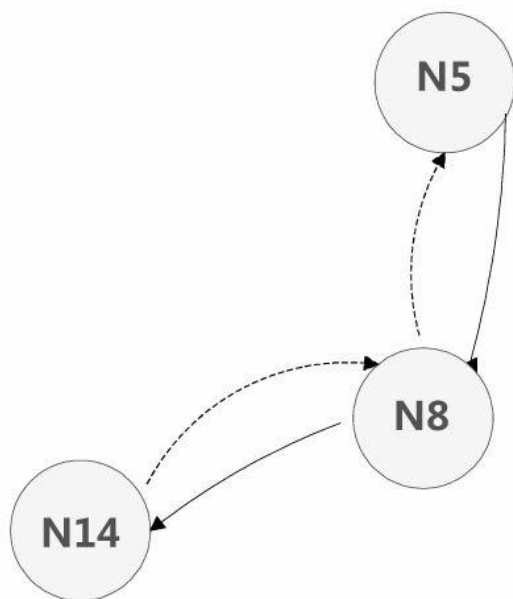
- 一段时间后，N5节点开始执行稳定性检测
- ✓ 经过步骤1，N5被N14告知其前趋节点是N8而非N5本身。
- ✓ 进入步骤2，此时步骤2条件满足，即N8介于N5和N14之间，所以N5将后继节点改为N8。
- ✓ 在步骤3， $N_x = N8$ ，因为N8的前趋节点为空，所以N5告知N8将其作为前趋节点，N8将前趋节点改为N5



加入新节点的例子

假设在左图的P2P网络例子中新加入节点N8。

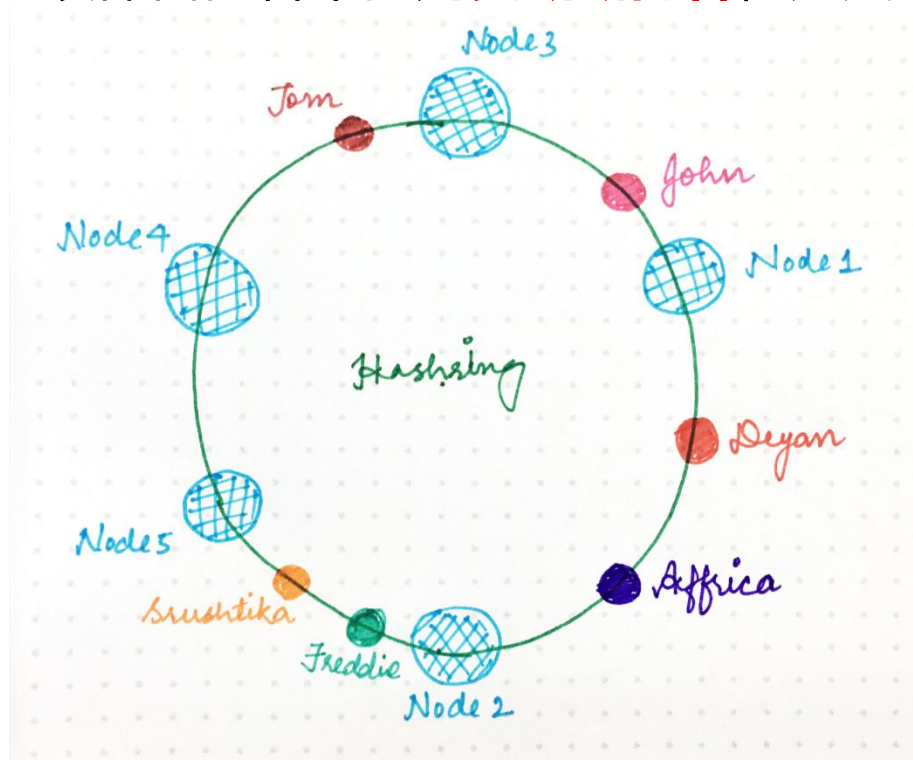
- 一段时间后，N5节点开始执行稳定性检测
- ✓ 步骤4没有数据需要分配，因为其后继节点N8存储数据主键值都大于5。
- ✓ 这样N5节点完成了稳定性检测，经过这步后，系统状态转变为如下图所示。



加入新节点

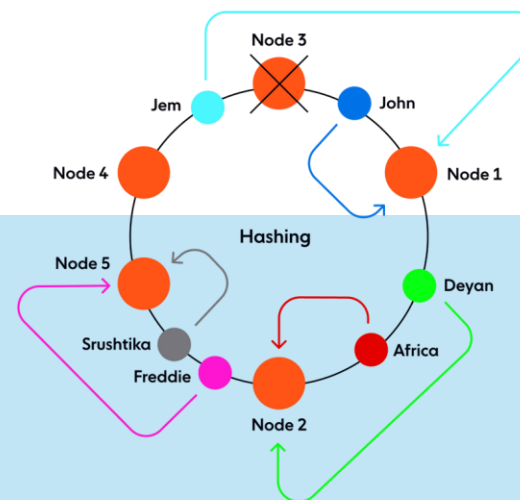
除了上述工作，因为新加入节点后，原先节点的路由表可能不再正确，把本应该指向新加入节点的路由项指向了旧的节点。

- 每个节点需要**周期性检查路由表**，对表内每一项 $k=2^i$ ($0 \leq i \leq m-1$)加上本机节点编号执行路由算法，得到每个路由项经过查询获得的机器节点
- 如果与目前路由项保留内容不同则**更新新内容**，完成路由表的更新



节点离开P2P网络有两种方式：正常离开与异常离开。

FAILURE
OF NODE 3



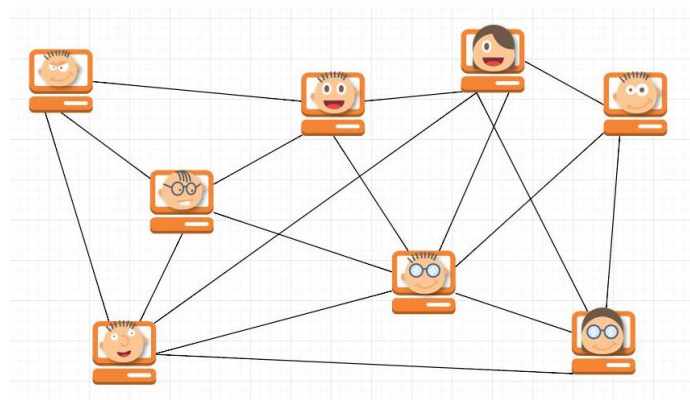
正常离开的节点

- 在离开前可以做些准备工作。
- 包括通知相应节点更新其前趋和后继节点以及将本身持有数据迁移到后继节点上，由于其离开造成的其他机器路由表失效可以通过“加入新节点时的情形”介绍的方式获得更新。

节点离开P2P网络有两种方式：正常离开与异常离开。

异常离开的节点

- 往往是机器故障导致，此时故障机器保持的数据可能丢失。
- 为了避免这个问题，可以采用将同一份数据在多台机器上保留副本的方式



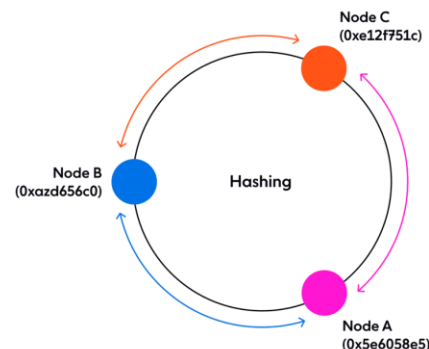
一致性哈希算法有两个潜在问题：



- 机器映射到环状结构的位置随机，导致机器负载不均衡
- 一致性哈希并未考虑到机器异质性

- ✓ Dynamo对一致性哈希进行了改造，引入“**虚拟节点**”的概念
- ✓ 即将一个物理节点虚拟成若干**虚拟节点**，分别映射到一致性哈希的环状结构不同位置。
- ✓ 一方面可以导致更佳的**负载均衡**，也可以兼顾到**机器异质性问题**。

NODE C
ADDED



和其他哈希方法对比

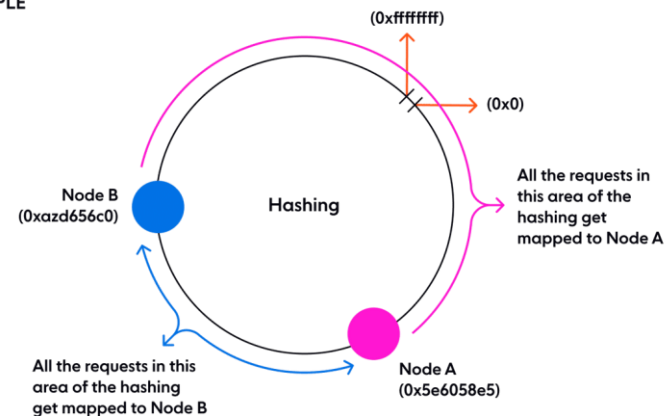
相比于Round Robin数据分片方法:

- 将集群机器数目这一变量从哈希函数中移出，转而将机器及记录主键都映射到哈希数值空间
- 不论是新加入机器还是机器故障导致某些机器失效，都只影响当前机器的后继节点上的数据分布



- 由于其管理P2P网络的复杂性，也导致一致性哈希算法维护成本很高

WORKING EXAMPLE



基于键哈希分片的缺点



- 失去键范围分片的一个属性：高效执行范围查询的能力。
- 曾经相邻的键现在分散在所有分片中。

- ✓ 在MongoDB中，如果使用基于散列的分片模式，则任何范围查询都必须发送到所有分片。
- ✓ Cassandra采取了折衷的策略。Cassandra中的表可以使用由多个列组成的复合主键来声明。
 - 键中只有第一列会作为散列的依据，而其他列则被用作Cassandra的SSTables中排序数据的连接索引。
 - 尽管查询无法在复合主键的第一列中按范围扫表，但如果第一列已经指定了固定值，则可以对键的其他列执行有效的范围扫描。

负载倾斜与消除热点

哈希分片不能完全避免热点：在极端情况下，所有的读写操作都是针对同一个键的，所有的请求都会被路由到同一个分片。

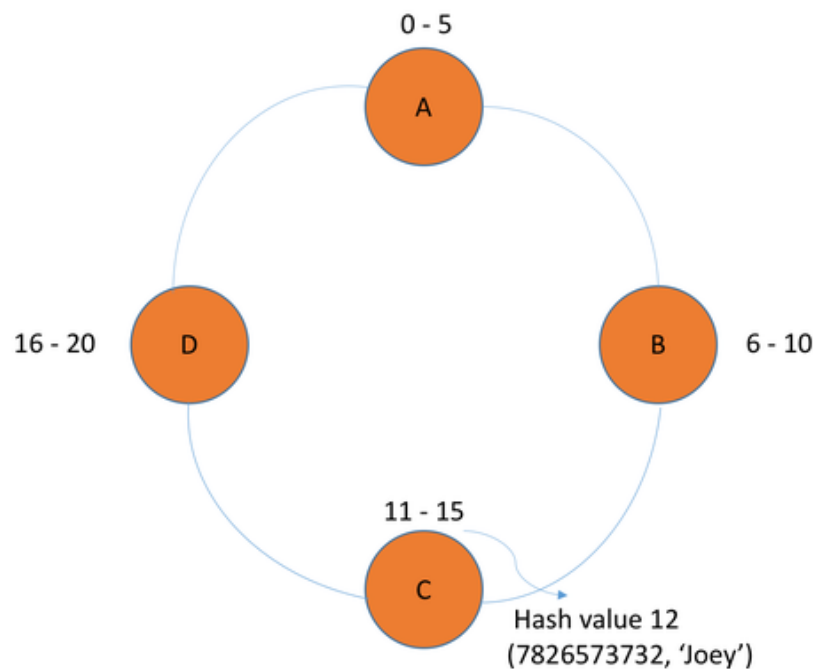
- 例如，在社交媒体网站上，一个拥有数百万追随者的名人用户在做某事时可能会引发一场风暴。
- 这个事件可能导致大量写入同一个键（键可能是名人的用户ID，或者人们正在评论的运动的ID）

- ✓ 如今，大多数数据系统无法自动补偿这种高度偏斜的负载，因此应用程序有责任减少偏斜。
- ✓ 如果一个主键被认为是非常火爆的，一个简单的方法是在主键的开始或结尾添加一个随机数。

负载倾斜与消除热点

将主键进行分割之后，任何读取都必须要做额外的工作：因为他们必须从所有主键分布中读取数据并将其合并

- 此技术还需要额外的记录：只需要对少量热点附加随机数；对于写入吞吐量低的绝大多数主键来是不必要的开销
- 还需要一些方法来跟踪哪些键需要被分割

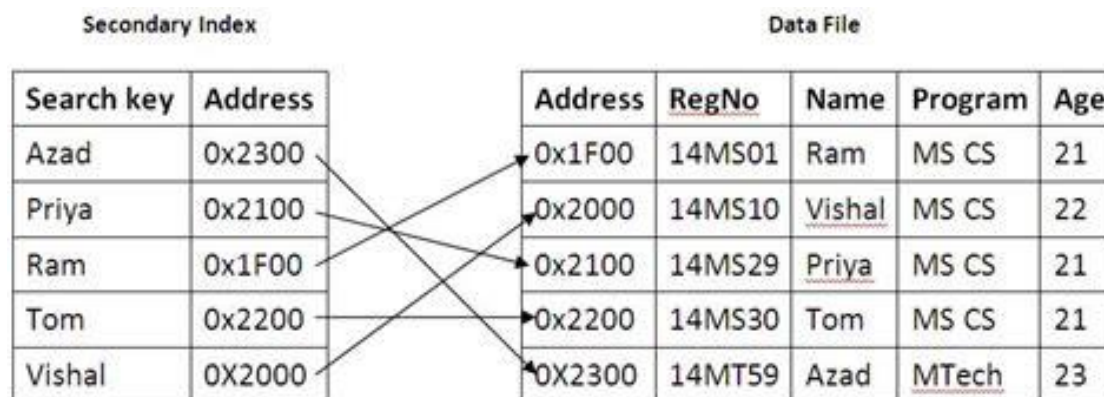


分片与次级索引

到目前为止讨论的分片方案依赖于键值数据模型。

- 如果只通过主键访问记录，可以从该键确定分片，并使用它来将读写请求路由到负责该键的分片。
- 如果涉及次级索引，情况会变得更加复杂。次级索引是关系型数据库的基础，并且在文档数据库中也很普遍。

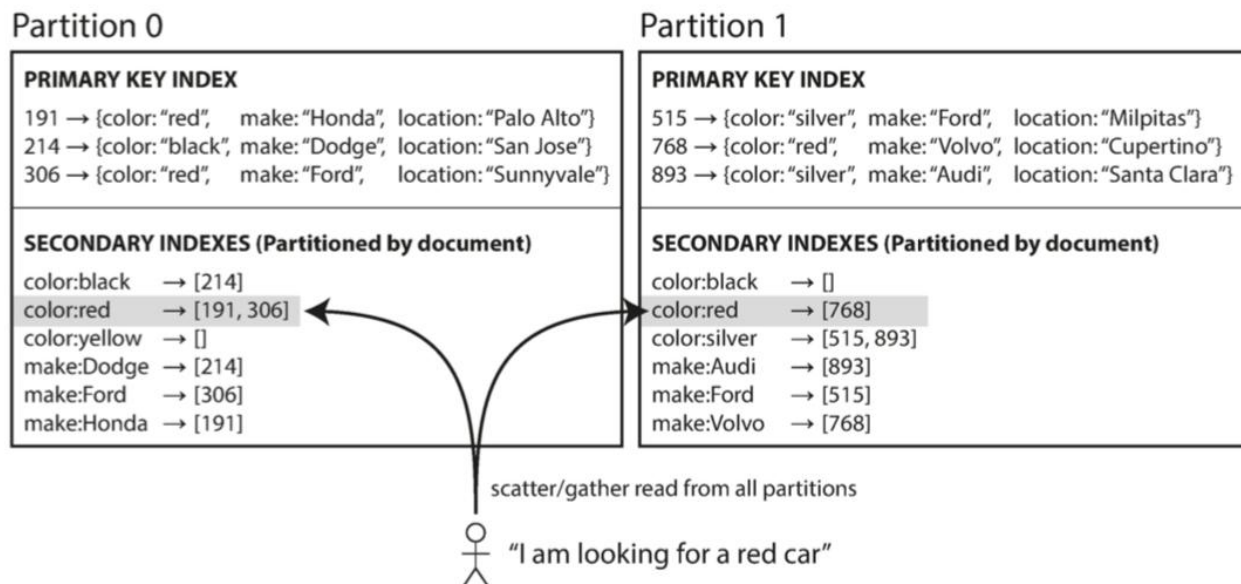
- ✓ 次级索引的问题是它们不能整齐地映射到分片。
- ✓ 有两种用二级索引对数据库进行分片的方法：基于文档的分片和基于关键词的分片。



按文档的二级索引

考虑一个销售二手车的网站，每个列表都有一个唯一的ID(文档ID)并且用文档ID对数据库进行分片。

- 允许用户通过颜色和厂商过滤，需要一个在颜色和厂商上的次级索引
- 如果声明了索引，则数据库可以自动执行索引 2。例如，无论何时将红色汽车添加到数据库，数据库分片都会自动将其添加到索引条目 color: red 的文档ID列表中。



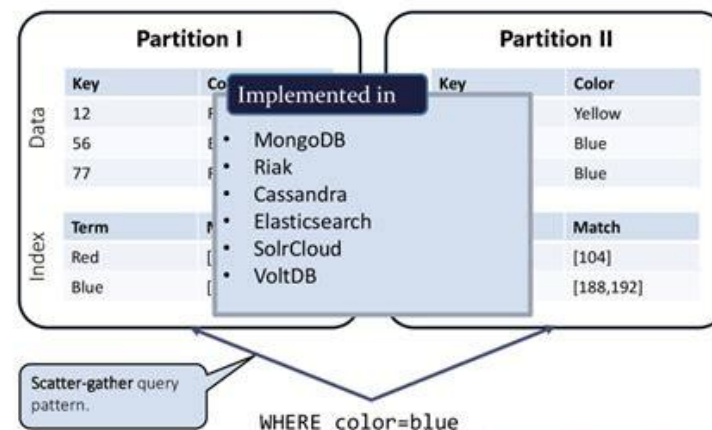
按文档的二级索引

按文档的二级索引

- 每个分片是**完全独立的**：每个分片维护自己的二级索引，仅覆盖该分片中的文档。
- 从文档分片索引中读取**需要注意**：除非您对文档ID做了特别的处理，否则没有理由将所有具有特定颜色或特定品牌的汽车放在同一个分片中。

这种查询分片数据库的方法有时被称为**分散/聚集**，并且可能会使二级索引上的读取查询相当昂贵。即使并行查询分片，分散/聚集也容易导致**尾部延迟放大**

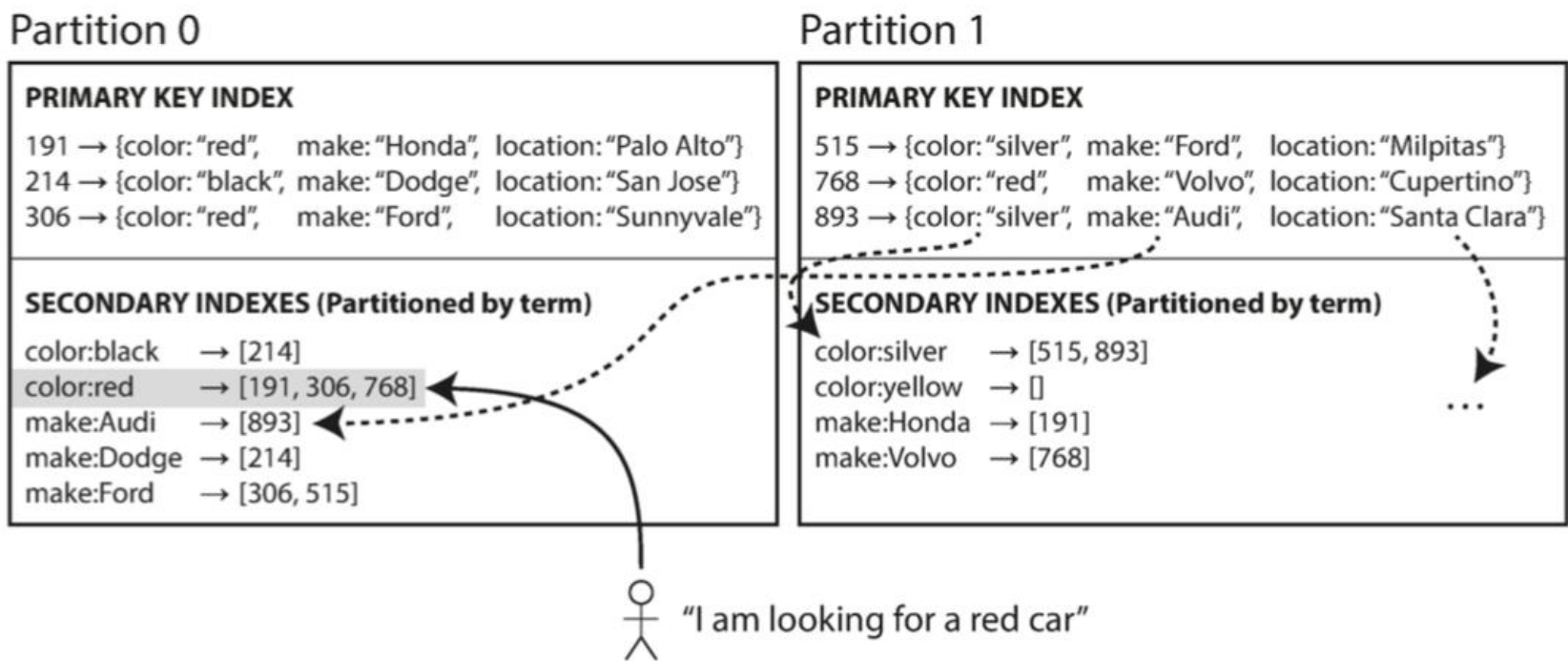
Local Secondary Indexing
Partitioning By Document



根据关键词(Term)的二级索引

可以构建一个覆盖所有分片数据的全局索引，而不是给每个分片创建自己的次级索引。

- 如图，来自所有分片的红色汽车在红色索引中，并且索引是分片的，首字母从a 到 r 的颜色在分片0中， s 到 z 的在分片1。



根据关键词(Term)的二级索引

这种索引称为关键词分片，寻找的关键词决定了索引的分片方式。

- 可以通过关键词本身或者它的散列列进行索引分片。

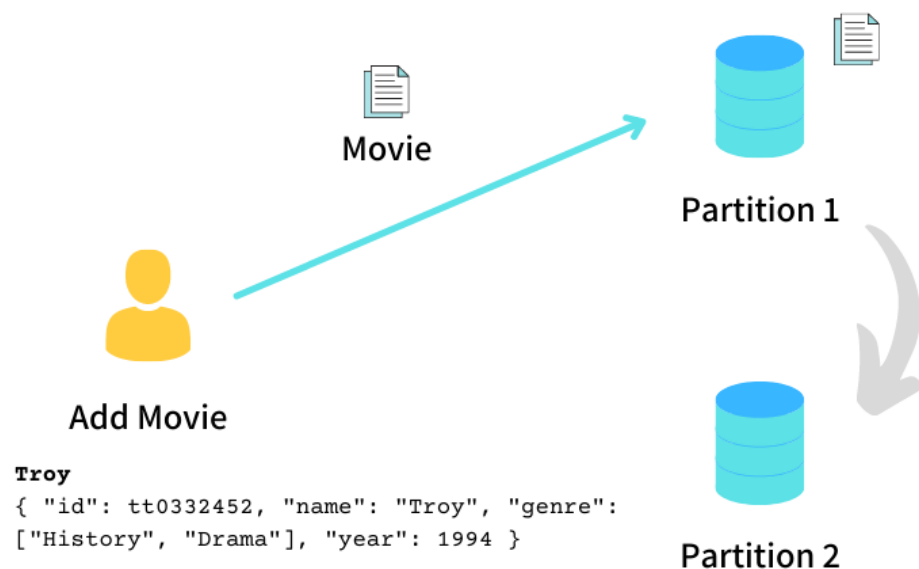
根据关键词(Term)的二级索引

- 优点：使读取更有效率，不需要分散/收集所有分片，客户端只需要向包含关键词的分片发出请求。
- 缺点：写入速度较慢且较为复杂，因为写入单个文档现在可能会影响索引的多个分片

根据关键词(Term)的二级索引

理想情况下，索引总是最新的，写入数据库的每个文档都会立即反映在索引中。

- 但是，在关键词分片索引中，这需要跨分片的分布式事务，并不是所有数据库都支持。
- 在实践中，对全局二级索引的更新通常是异步的（也就是说，如果在写入之后不久读取索引，刚才所做的更改可能尚未反映在索引中）

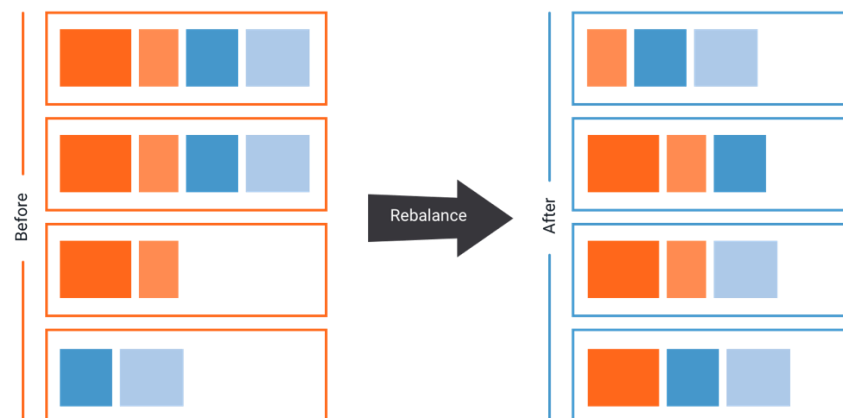


*图片: <https://arpitbhayani.me/blogs/how-indexes-work-on-partitioned-and-sharded-data/>

分片再平衡

随着时间的推移，数据库会有各种变化：

- 查询吞吐量增加，添加更多的CPU来处理负载。
- 数据集大小增加，添加更多的磁盘和RAM来存储它。
- 机器出现故障，其他机器需要接管故障机器的责任。



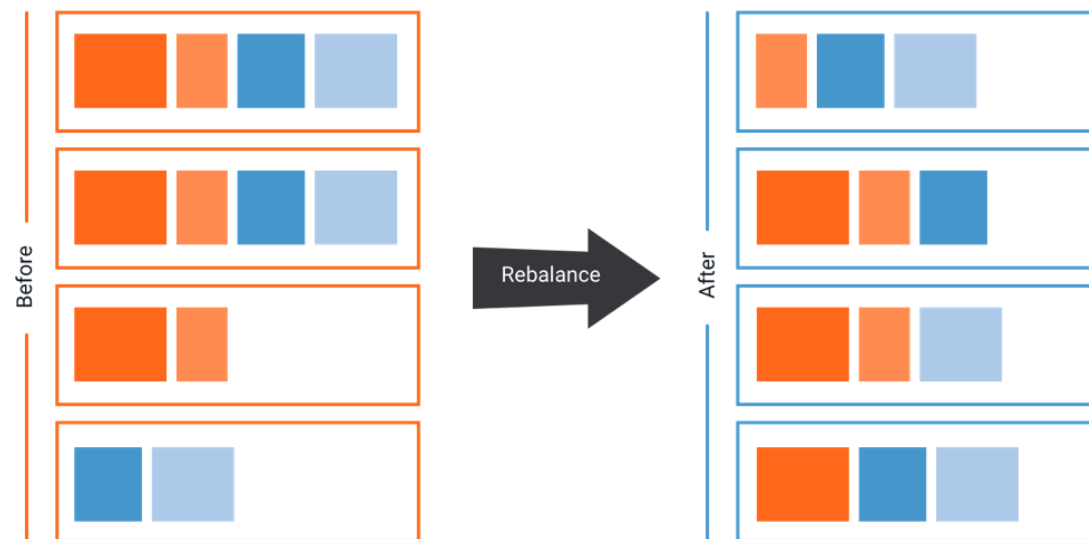
- ✓ 所有这些更改都需要数据和请求从一个节点移动到另一个节点。
- ✓ 将负载从集群中的一个节点向另一个节点移动的过程称为**再平衡**。

*图片：<https://www.confluent.io/blog/introducing-apache-kafka-for-the-enterprise/>

分片再平衡

再平衡通常要满足一些最低要求：

- 再平衡之后，负载（数据存储，读取和写入请求）应该在集群中的节点之间公平地共享。
- 再平衡发生时，数据库应该继续接受读取和写入。
- 节点之间只移动必须的数据，以便快速再平衡，减少网络磁盘I/O负载。

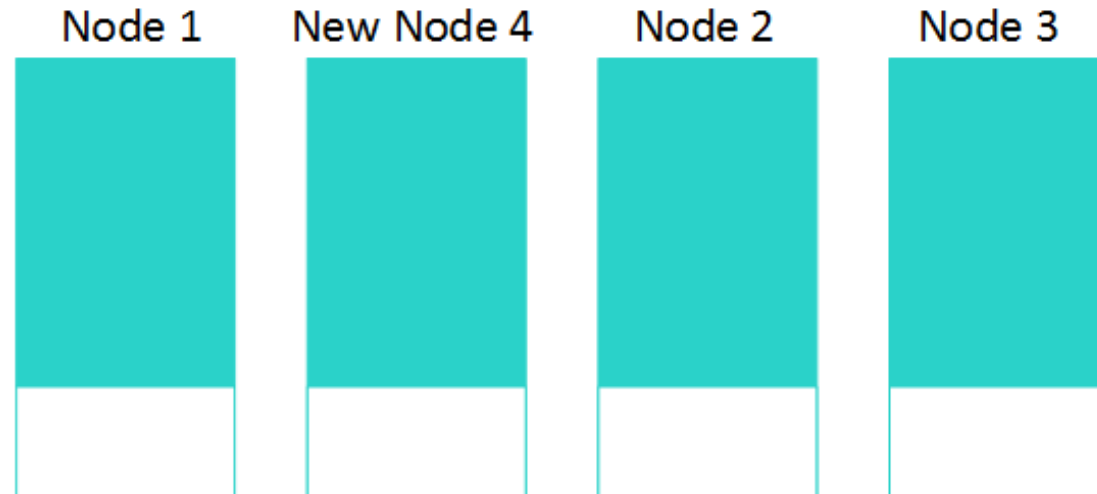


- 有几种不同的分片分配方法, 依次简要讨论一下。

反面教材：hash mod N

考虑为什么不用mod，如果有10个节点，编号为0到9，这似乎是将每个键分配给一个节点的简单方法。

- 如果节点数量N变化，大多数键将需要从一个节点移动到另一个节点
- 需要一种只移动必需数据的方法



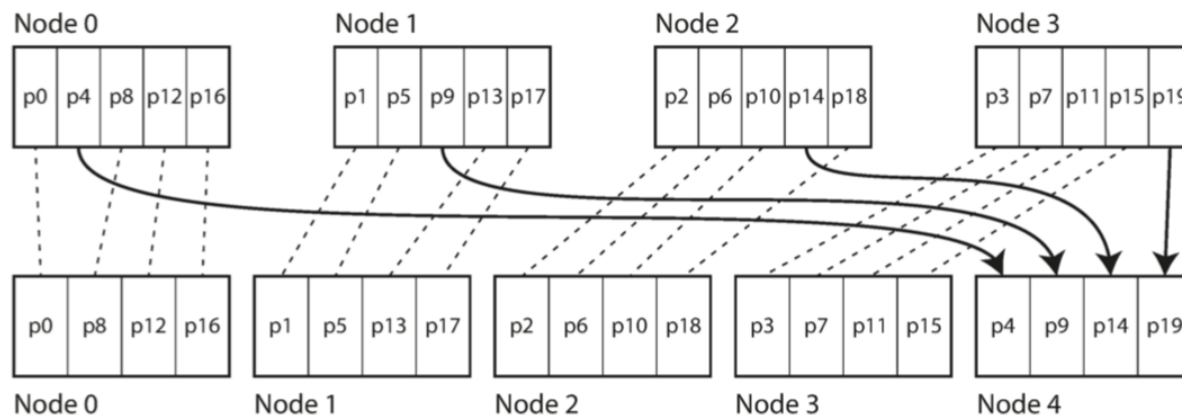
*图片: <https://www.vertica.com/kb/Understanding-Rebalancing-Part-1-What-Happens-During-Rebalancing/Content/BestPractices/Understanding-Rebalancing-Part-1-What-Happens-During-Rebalancing.htm>

固定数量的分片

一个相当简单的解决方案：创建比节点更多的分片，并为每个节点分配多个分片。

- 如果一个节点被添加到集群中，新节点可以从当前每个节点中窃取一些分片，直到分片再次公平分配。
- 如果从集群中删除一个节点，则会发生相反的情况。

Before rebalancing (4 nodes in cluster)



After rebalancing (5 nodes in cluster)

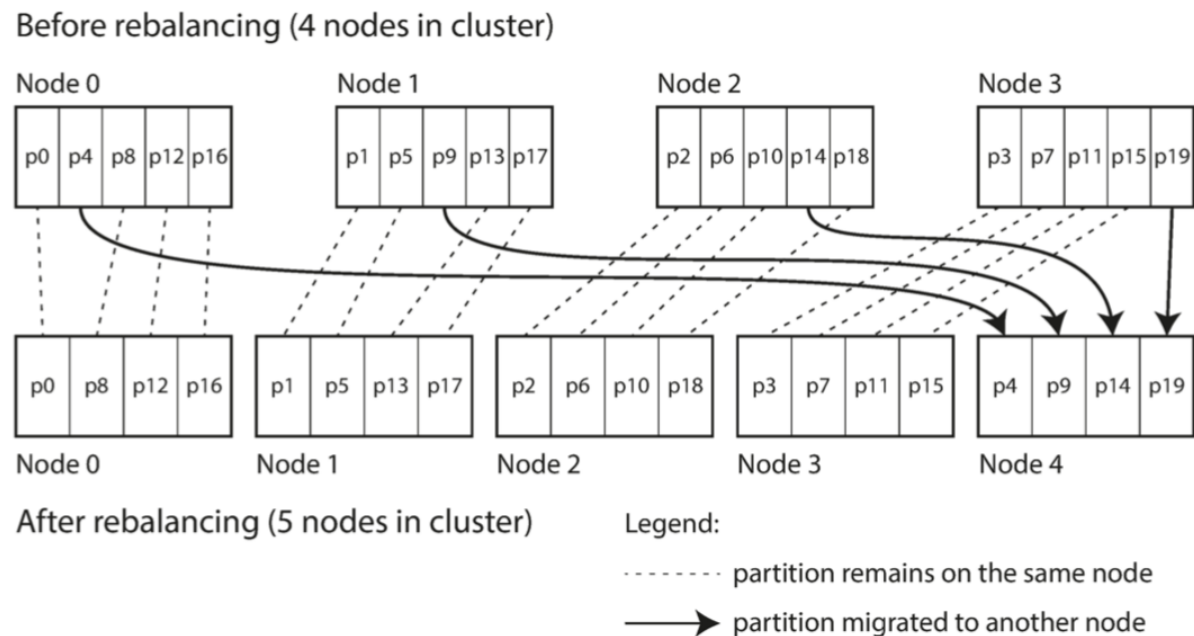
Legend:

- partition remains on the same node
- > partition migrated to another node

固定数量的分片

只有分片在节点之间的移动： 分片的数量不会改变，键所指定的分片也不会改变。

- 唯一改变的是分片所在的节点。
- 这种变更并不是即时的 — 在网络上传输大量的数据需要一些时间 — 所以在传输过程中，原有分片仍然会接受读写操作。

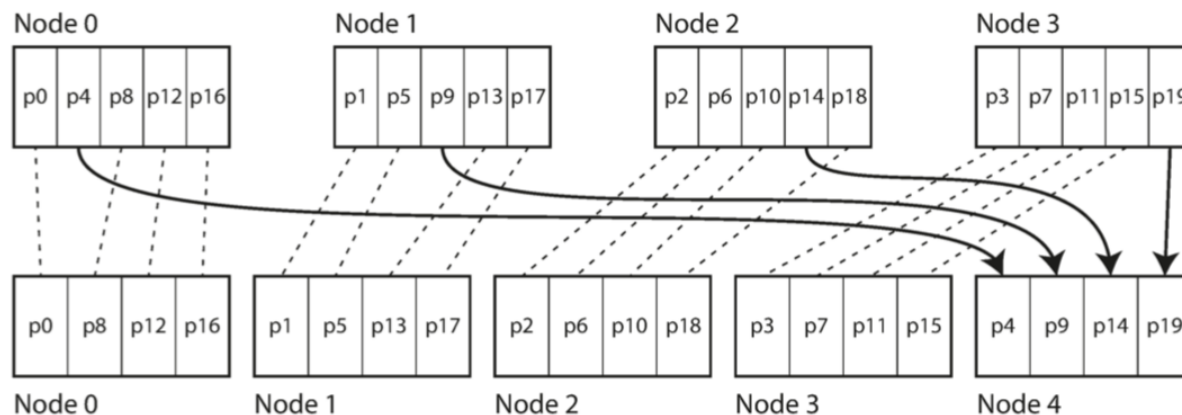


固定数量的分片

解决集群中的硬件不匹配问题：通过为更强大的节点分配更多的分片，可以强制这些节点承载更多的负载多。

- 这种配置中，分片数量通常在数据库第一次建立时确定，之后不会改变。
- 一开始配置的分片数就是可以拥有的最大节点数量，所以需要选择足够多的分片以适应未来的增长。

Before rebalancing (4 nodes in cluster)



After rebalancing (5 nodes in cluster)

Legend:

- partition remains on the same node
- > partition migrated to another node

动态分片

对于使用键范围分片的数据库：具有固定边界的固定数量的分片将非常不便。

- 如果出现边界错误，则可能会导致一个分片中的所有数据或者其他分片中的所有数据为空。
- 手动重新配置分片边界将非常繁琐。

- ✓ 出于这个原因，按键的范围进行分片的数据库会**动态创建分片**。
- ✓ 当分片增长到超过配置大小时，会分成两个分片，每个分片约占一半数据。
- ✓ 与之相反，如果大量数据被删除并且分片缩小到某个阈值以下，则可以将其与相邻分片合并。

动态分片

每个分片分配给一个节点：每个节点可以处理多个分片，就像固定数量的分片一样。

- 大型分片拆分后，可以将其中的一半转移到另一个节点，以平衡负载。
- 优点：分片数量适应总数据量。如果只有少量的数据，少量的分片就足够了，所以开销很小；如果有大量的数据，每个分片的大小被限制在一个可配置的最大值。

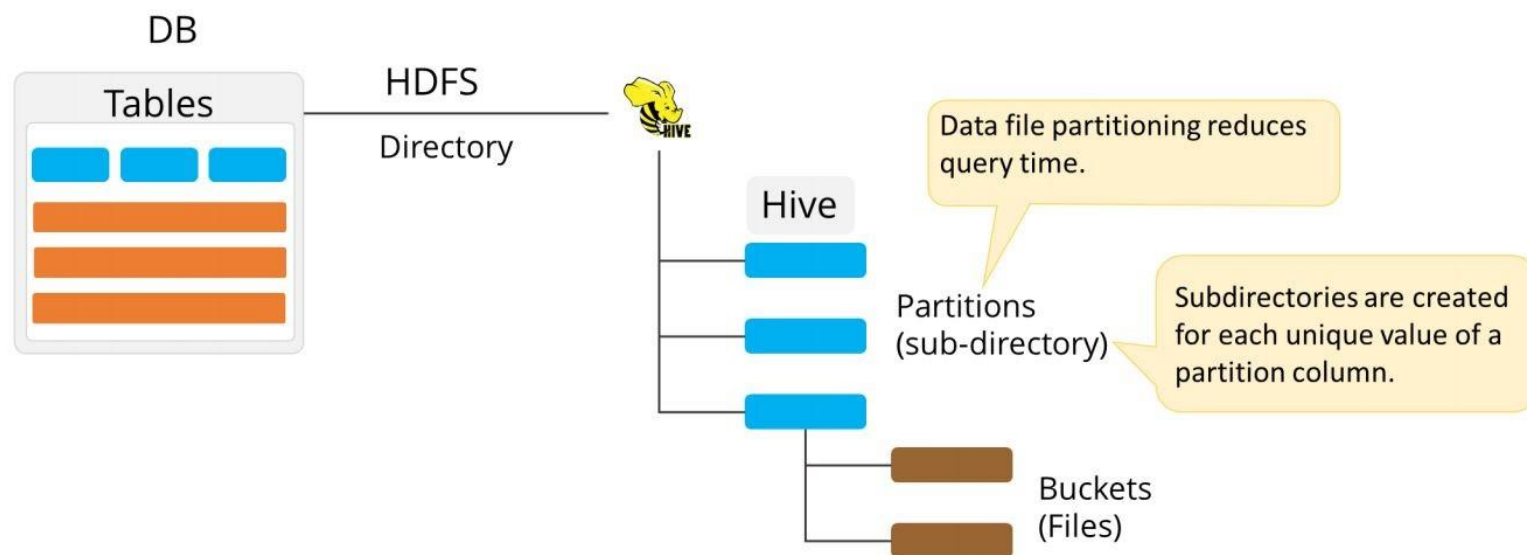
需要注意的是，一个空的数据库从一个分片开始，因为没有关于在哪里绘制分片边界的先验信息。

- 数据集开始时很小，直到达到第一个分片的分割点，所有写入操作都必须由单个节点处理，而其他节点则处于空闲状态

按节点比例分片

第三种方法是使分片数与节点数成正比：每个节点具有固定数量的分片。

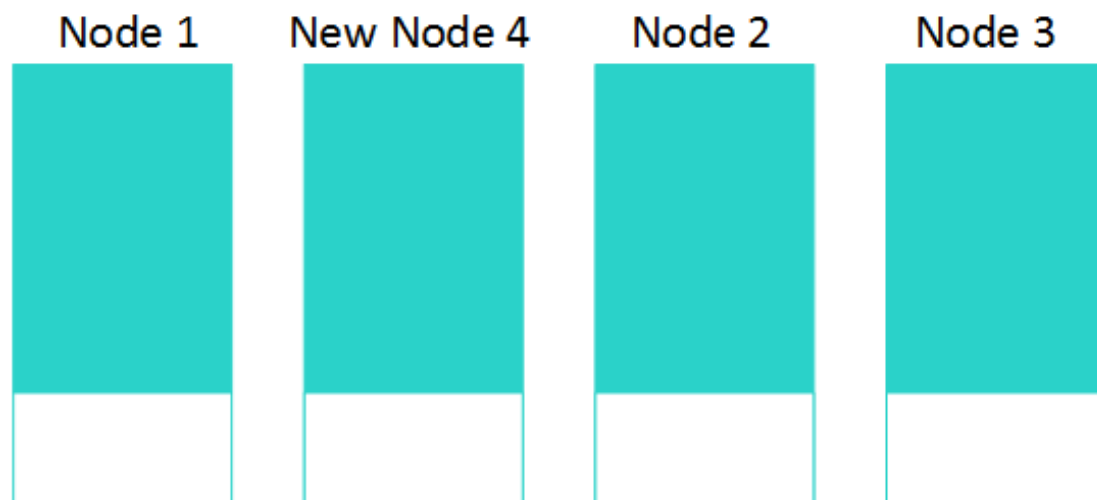
- 在这种情况下，每个分片的大小与数据集大小成比例地增长，而节点数量保持不变，但是当增加节点数时，分片将再次变小。
- 由于较大的数据量通常需要较大数量的节点进行存储，因此这种方法也使每个分片的大小较为稳定。



按节点比例分片

当一个新节点加入集群时：它随机选择固定数量的现有分片进行拆分，然后占有这些拆分分片中每个分片的一半，同时将每个分片的另一半留在原地。

- 随机化可能会产生不公平的分割，但是平均在更大数量的分片上时，新节点最终从现有节点获得公平的负载份额
- 随机选择分片边界要求使用基于散列的分片



*图片: <https://www.vertica.com/kb/Understanding-Rebalancing-Part-1-What-Happens-During-Rebalancing/Content/BestPractices/Understanding-Rebalancing-Part-1-What-Happens-During-Rebalancing.htm>

运维：手动还是自动平衡

- 关于再平衡有一个重要问题：自动还是手动进行？

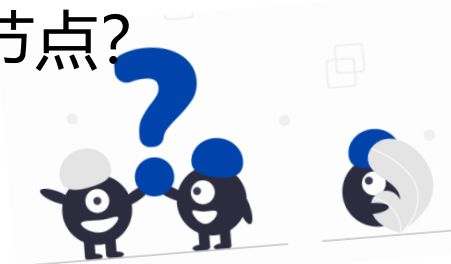


在全自动重新平衡和完全手动之间有一个权衡：

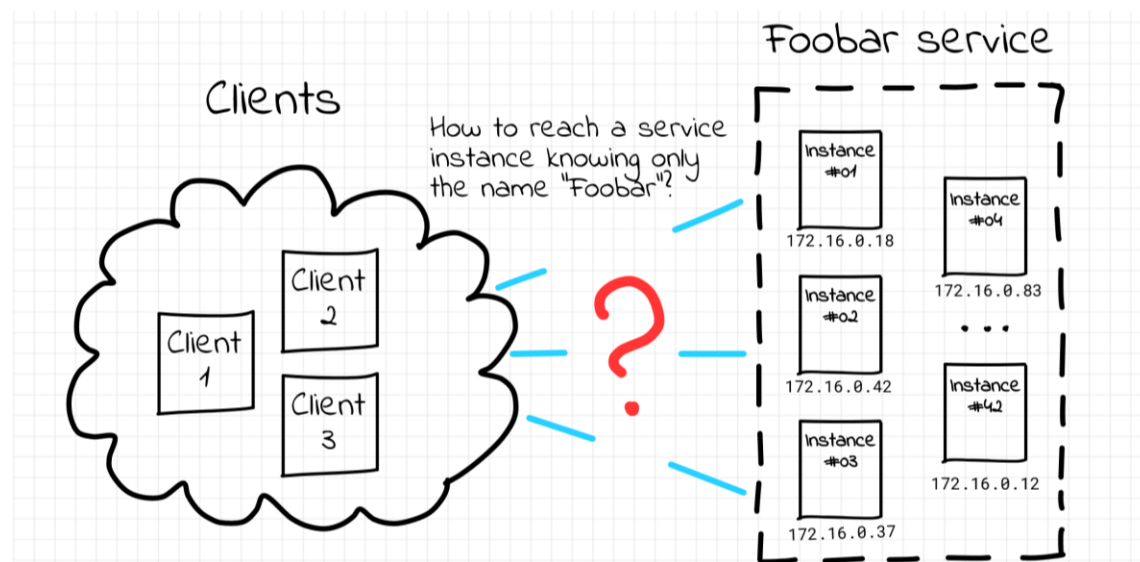
- 全自动重新平衡可以很方便，因为正常维护的操作工作较少。
- 但是，这可能是不可预测的。再平衡是一个昂贵的操作，因为它需要重新路由请求并将大量数据从一个节点移动到另一个节点。如果没有做好，这个过程可能会使网络或节点负载过重，降低其他请求的性能。
- 这种自动化与自动故障检测相结合可能十分危险。例如，假设一个节点过载，并且对请求的响应暂时很慢。其他节点得出结论：过载的节点已经死亡，并自动重新平衡集群，使负载离开它。这会对已经超负荷的节点，其他节点和网络造成额外的负载，从而使情况变得更糟，并可能导致级联失败。

服务发现

- 当客户想要发出请求时，如何知道要连接哪个节点？



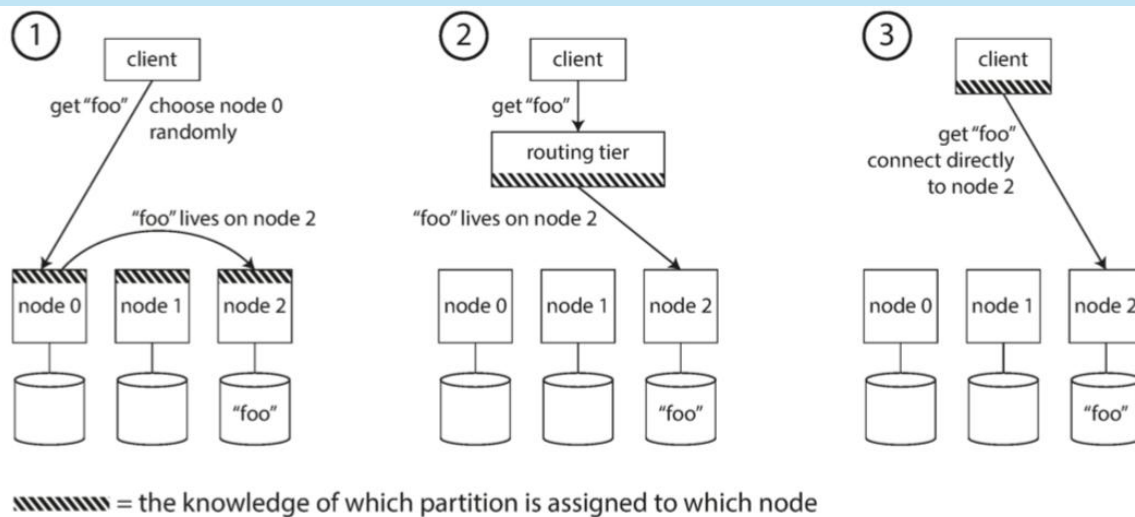
这个问题可以概括为服务发现：它不仅限于数据库。任何通过网络访问的软件都有这个问题，特别是如果它的目标是高可用。



*图片: <https://iximiuz.com/en/posts/service-discovery-in-kubernetes/>

服务发现(service discovery) 方案

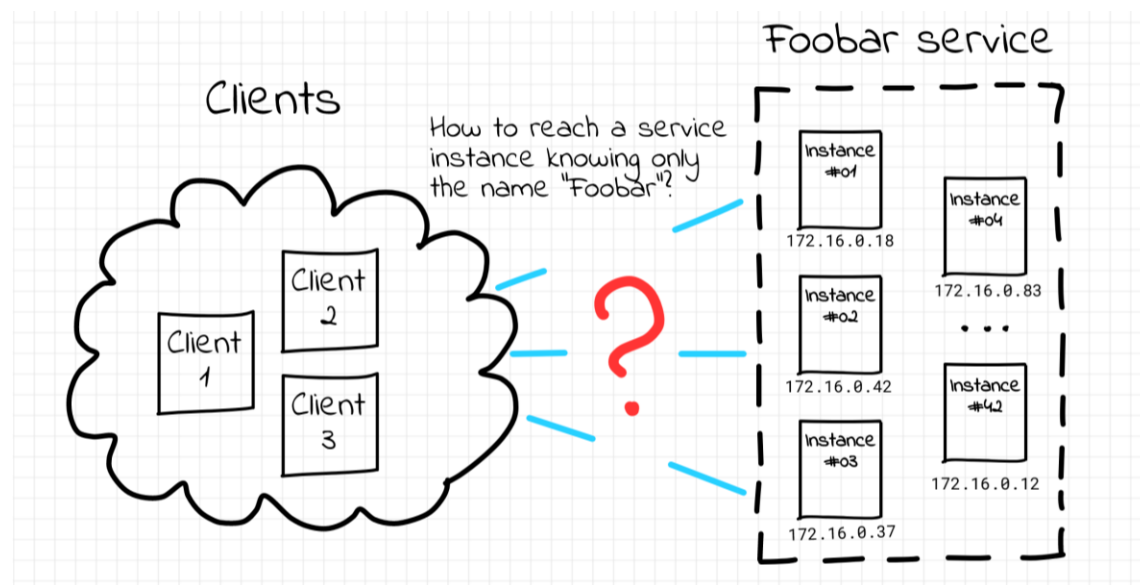
- 允许客户联系任何节点。如果该节点恰巧拥有请求的分片，则可以直接处理该请求;否则，将请求转发到适当节点，接收回复并传递给客户端。
- 首先将所有来自客户端的请求发送到路由层，决定应该处理请求的节点，并相应转发。此路由层本身不处理任何请求；它仅负责分片的负载均衡。
- 要求客户端知道分片和节点的分配。在这种情况下，客户端可以直接连接到适当的节点，而不需要任何中介。



服务发现关键问题

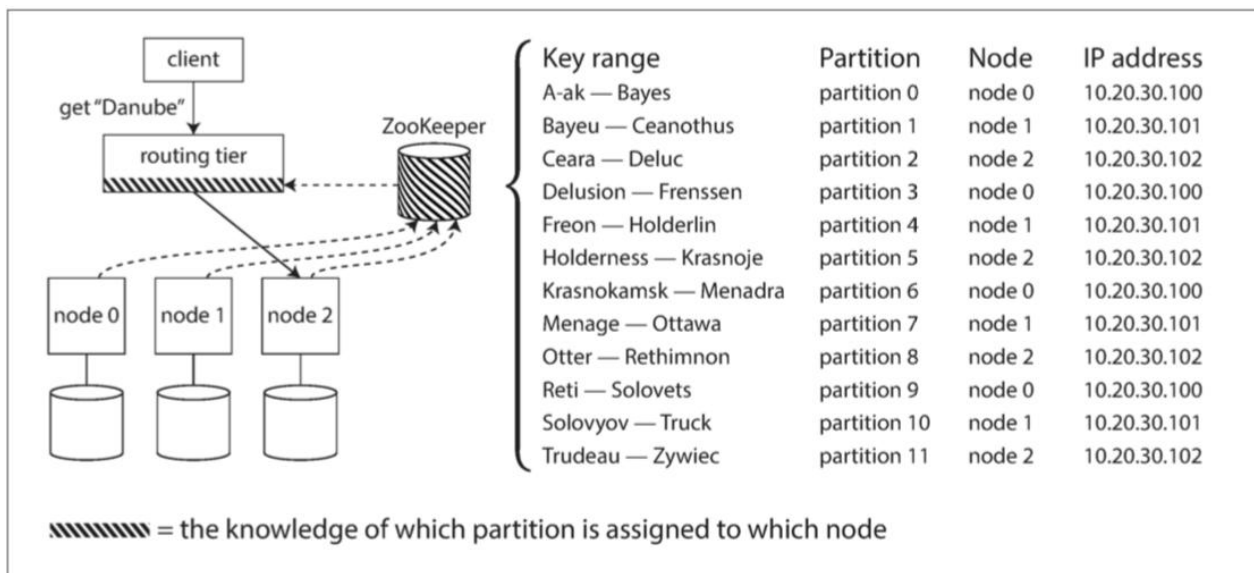
关键问题：作出路由决策的组件（可能是节点之一，还是路由层或客户端）如何了解分片-节点之间的分配关系变化？

- 这是一个具有挑战性的问题，因为重要的是所有参与者都同意 - 否则请求将被发送到错误的节点，而不是正确处理。
- 在分布式系统中有达成共识的协议，但很难正确地实现。



独立协调服务

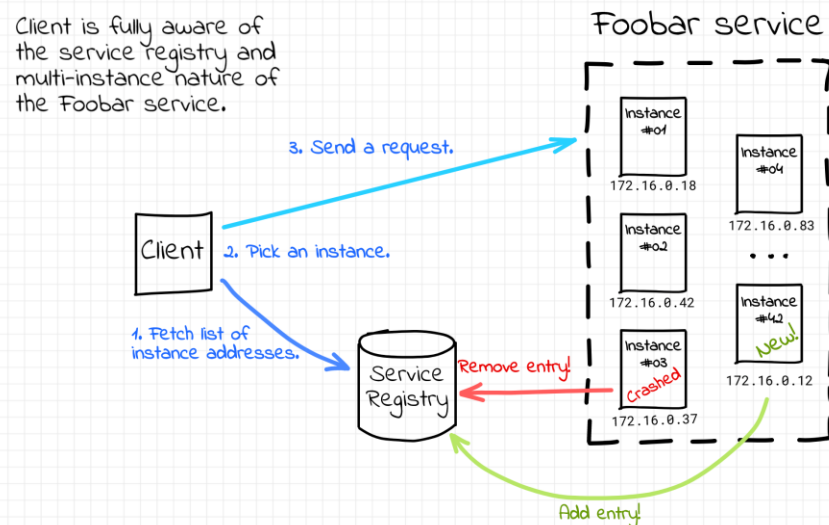
- ✓ 许多分布式数据系统都依赖于一个独立的协调服务。
- ✓ 比如ZooKeeper来跟踪集群元数据，每个节点在ZooKeeper中注册自己，ZooKeeper维护分片到节点的可靠映射。
- ✓ 其他参与者（如路由层或分片感知客户端）在ZooKeeper中订阅此信息。
- ✓ 只要分片分配发生的改变，或者集群中添加或删除了一个节点，ZooKeeper就会通知路由层使路由信息保持最新状态。



不同的策略

Cassandra和Riak采取不同的方法：他们在节点之间使用流言协议来传播群集状态的变化。

- 请求可以发送到任意节点，该节点转发到包含所请求的分片的适当节点。
- Couchbase不会自动重新平衡，通常情况下，它配置了一个名为moxi的路由层，它会从集群节点了解路由变化。
- 当使用路由层或向随机节点发送请求时，客户端仍然需要找到要连接的IP地址。这些地址不像分片的节点分布变化的快，所以使用DNS就够了。

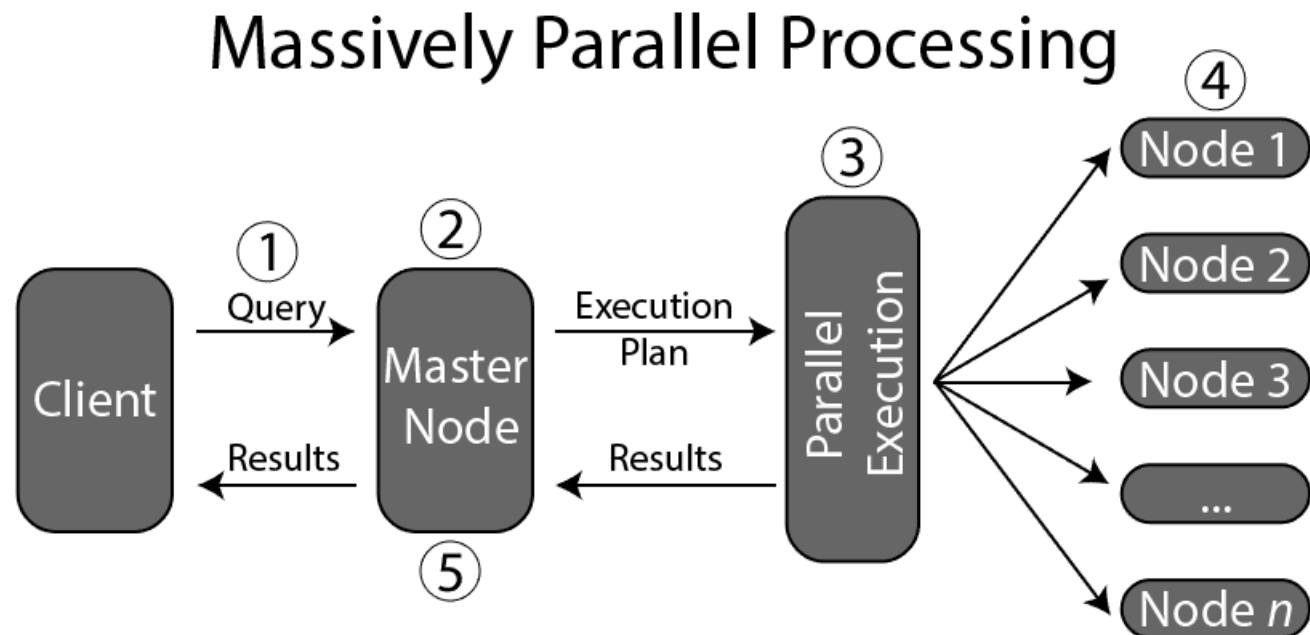


*图片: <https://iximiuz.com/en/posts/service-discovery-in-kubernetes/>

不同的策略

对于用于分析的大规模并行处理关系型数据库产品：其支持的查询类型方面要复杂得多。

- 一个典型的数据仓库查询包含多个连接，过滤，分组和聚合操作。
- MPP查询优化器将这个复杂的查询分解成许多执行阶段和分片，其中许多可以在数据库集群的不同节点上并行执行。



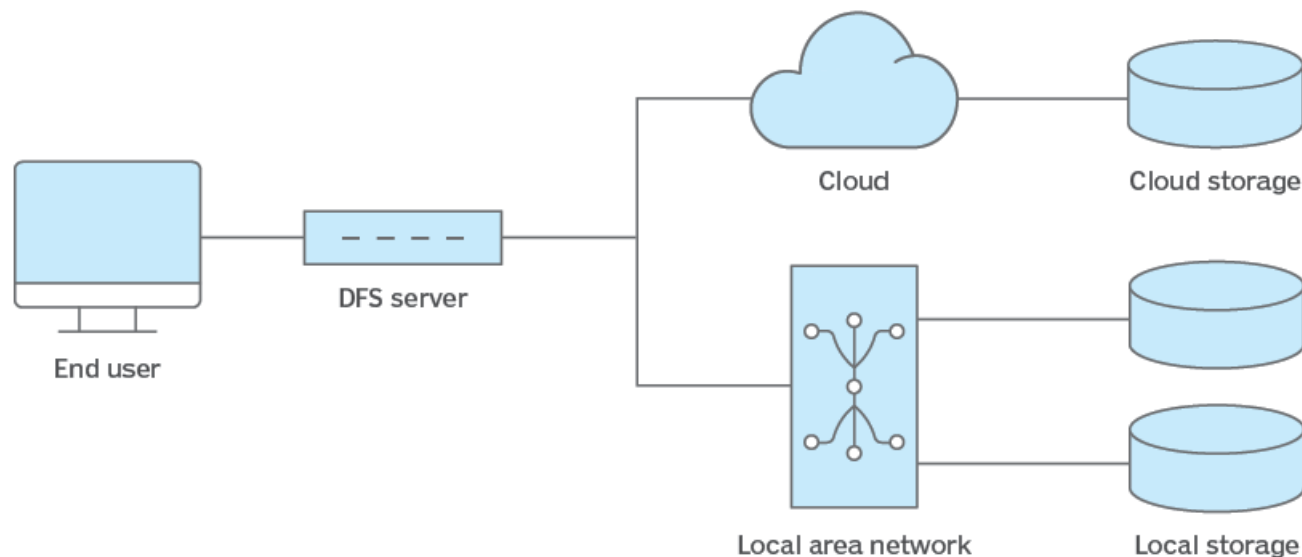
目录

Contents

- 1 MapReduce与计算框架
- 2 数据的分片与路由
- 3 分布式文件系统**
- 4 分布式计算框架例析：Hadoop
- 5 分布式计算框架例析：Spark
- 6 分布式计算框架例析：Storm

分布式文件系统（DFS） 是跨越多个文件服务器或多个位置的文件系统。

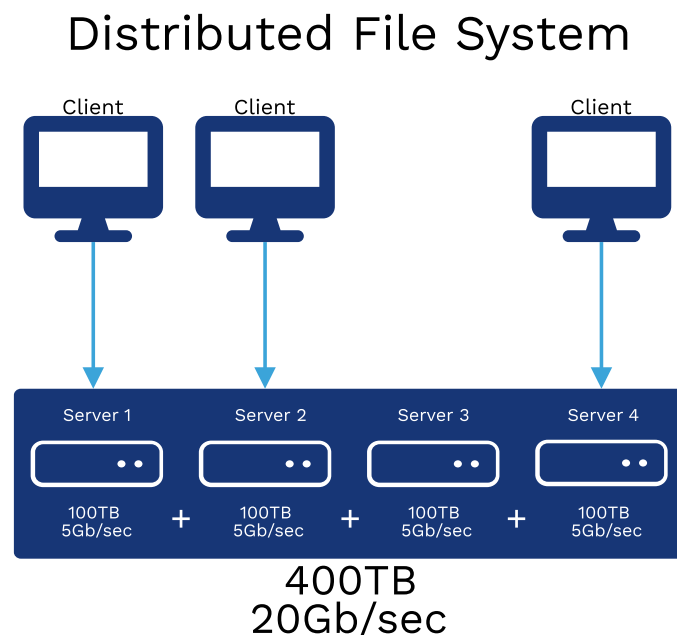
- 例如位于不同物理位置的文件服务器，可以从任何设备和网络上的任何位置访问文件，就像它们存储在本地一样。
- DFS 可以方便地以受控和授权的方式在网络上用户之间共享信息和文件。



分布式文件系统

选择DFS的主要原因是提供从多个位置访问相同数据能力。

- 例如，有一个分布在世界各地的团队，但他们必须能够访问相同的文件才能进行协作。
- 或者在当今日益混合的云世界中，每当需要访问从数据中心到边缘、再到云的相同数据时，都会希望使用 DFS。



*图片: <https://www.techtarget.com/searchstorage/definition/distributed-file-system-DFS>

Google文件系统（GFS） 是Google公司为了能够存储以百亿计的海量网页信息而专门开发的文件系统。

- GFS提供了海量非结构化信息的存储平台。
- 提供了数据的冗余备份、成千台服务器的自动负载均衡以及失效服务器检测等各种完备的分布式存储功能。

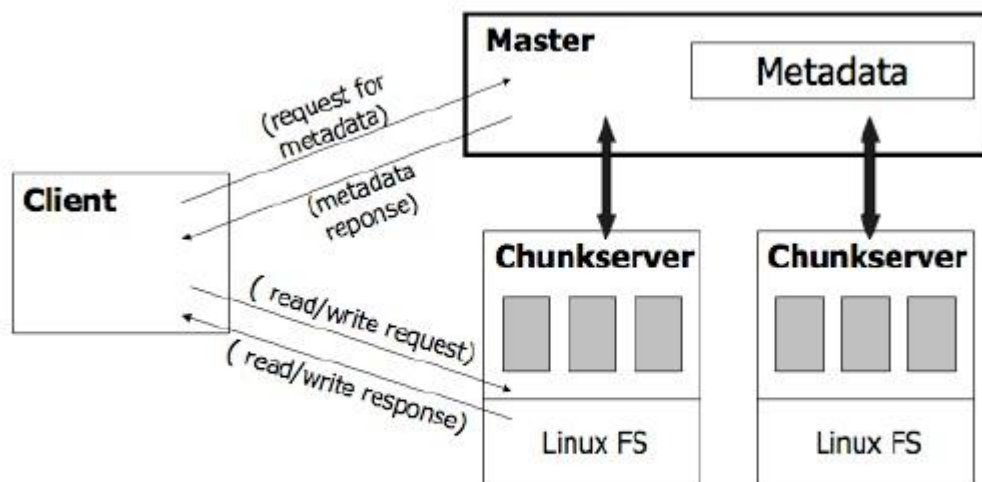


Figure 1

GFS设计原则

GFS设计原则

考虑到搜索引擎的应用环境，GFS在设计之初就定下几个基本的设计原则。

- 首先，GFS采用大量商业PC来构建存储集群。
- PC因为是面向普通用户设计的，所以其稳定性并没有很高的保障，尤其是大量PC构成的集群系统，每天都有机器死机或者发生硬盘故障。
- 数据冗余备份、自动检测机器是否还在有效提供服务、故障机器的自动恢复等都列在GFS的设计目标里。



GFS设计原则

考虑到搜索引擎的应用环境，GFS在设计之初就定下几个基本的设计原则。

- 其次，GFS文件系统所存储的文件绝大多数都是大文件。
- 系统的设计应该针对这种大文件的读 / 写操作做出优化。
- 尽管GFS也支持小文件读 / 写，但是不作为重点，也不会进行有针对性的操作优化。



GFS

优化



大文件

GFS设计原则

考虑到搜索引擎的应用环境，GFS在设计之初就定下几个基本的设计原则。

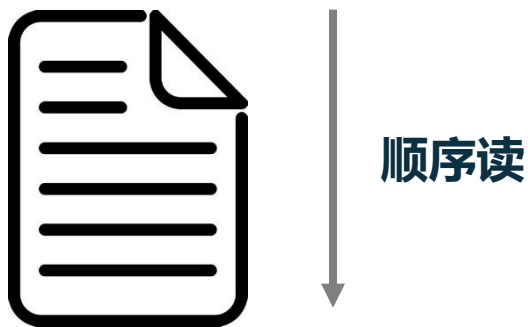
- 再次，系统中存在大量的“追加写”操作。
- 即将新增内容追加到已有文件的末尾，已经写入的内容一般不做更改。
- 很少有文件的“随机写”行为，即指定已有文件中间的某个位置，在这个位置之后写入数据。



GFS设计原则

考虑到搜索引擎的应用环境，GFS在设计之初就定下几个基本的设计原则。

- 最后，对于数据读取操作来说，绝大多数读文件操作都是“顺序读”，少量的操作是“随机读”。
- 即按照数据在文件中的顺序，一次顺序读入较大量的数据，而不是不断地定位到文件指定的位置，读取少量数据。

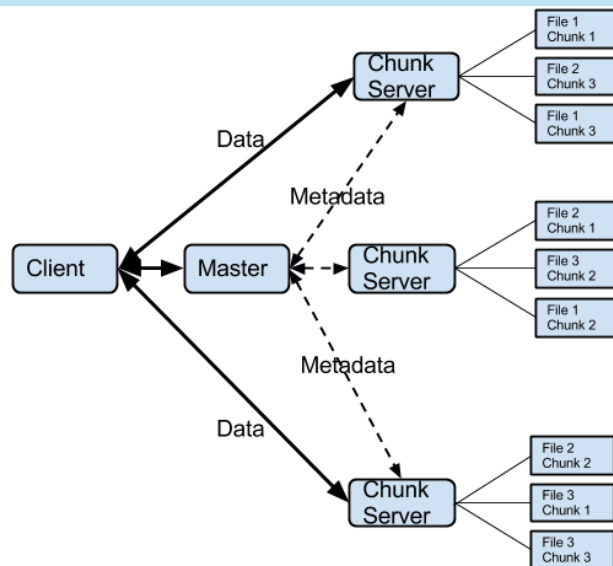


GFS整体架构

GFS文件系统

主要由3个组成部分构成：唯一的“主控服务器”（Master）、众多的“Chunk服务器”和“GFS客户端”。

- “主控服务器” 主要做管理工作。
- “Chunk服务器” 负责实际数据存储并响应“GFS客户端”读 / 写请求。
- 分布式存储系统的细节对应用开发者来说是不可见的。

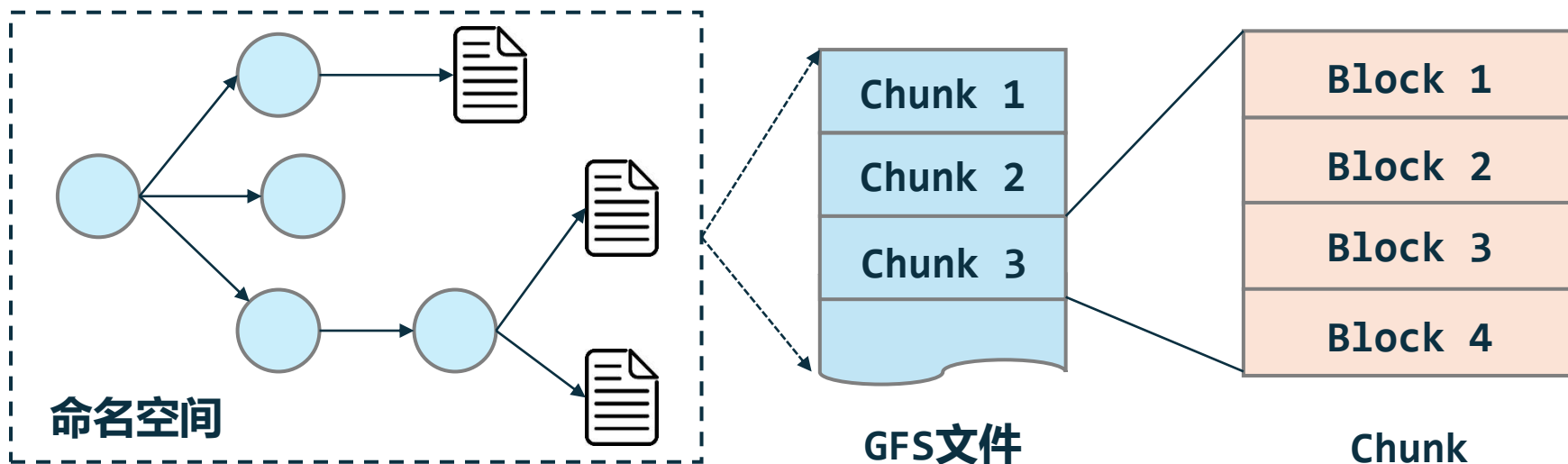


GFS整体架构

GFS文件系统及其文件

GFS文件系统类似于Linux文件系统或者是Windows操作系统提供的文件系统，即由目录和存放在某个目录下的文件构成的树形结构。

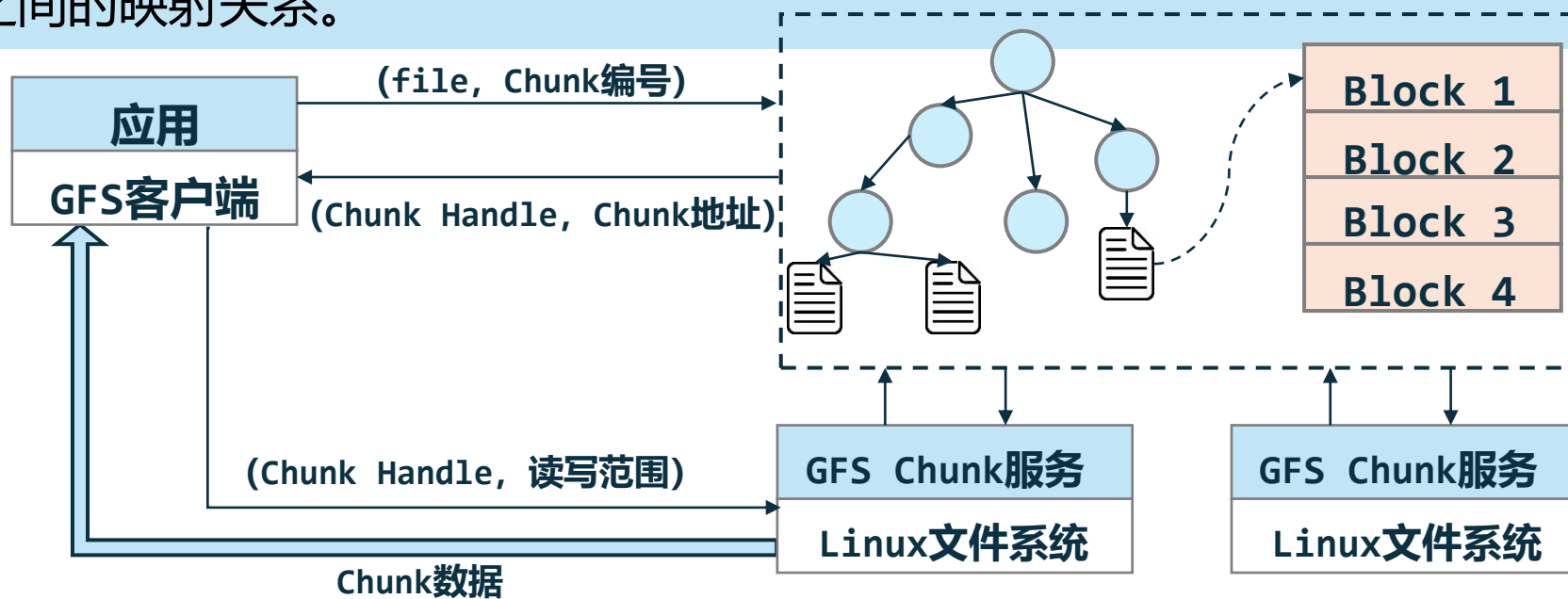
- GFS命名空间由目录和GFS文件构成，GFS文件由固定大的Chunk构成。
- 每个Chunk又由更小粒度的Block构成，Chunk是GFS中基本的存储单元，而Block是基本的读取单元。



GFS整体架构

GFS整体架构

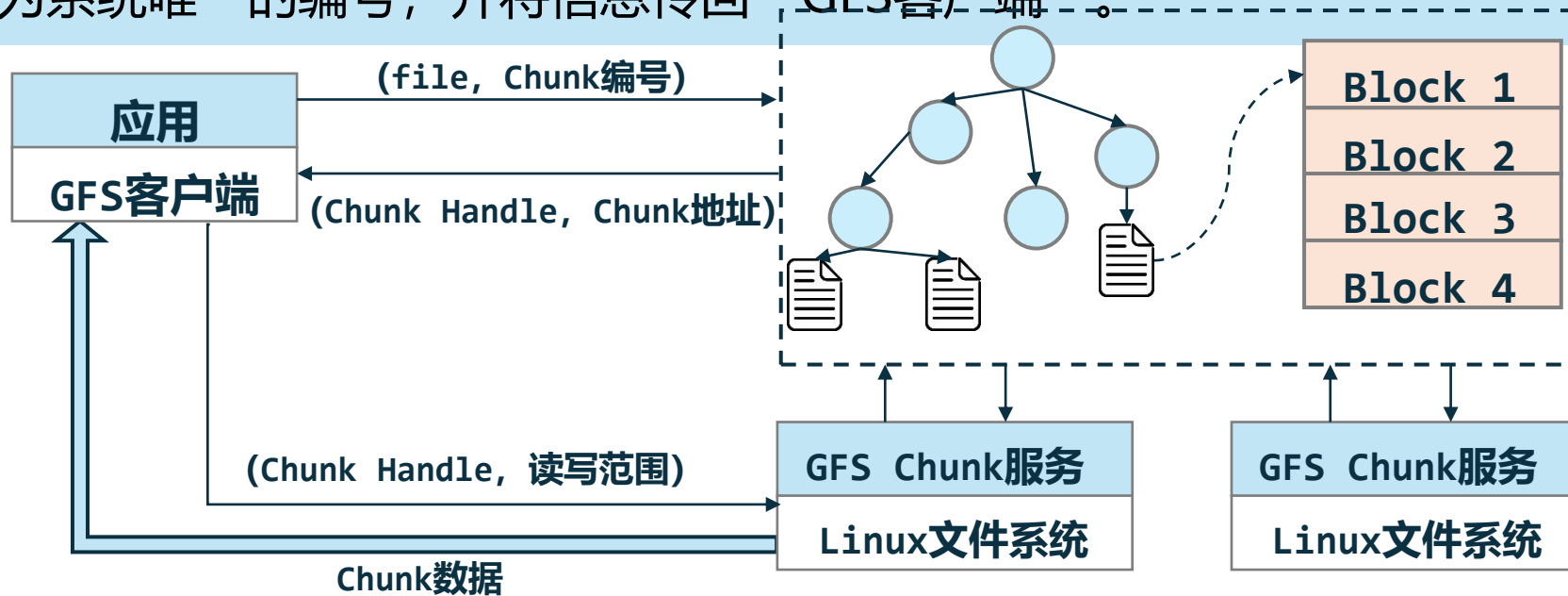
- “主控服务器”不仅维护GFS命名空间，还维护Chunk的命名空间。
- 为了能够识别不同的Chunk，每个Chunk都会被赋予一个独一无二的编号，所有Chunk的编号构成了Chunk命名空间。
- 因为GFS文件被切割成了Chunk，GFS系统内部就需要维护文件名称到其对应的多个Chunk之间的映射关系。



GFS整体架构

“GFS客户端” 读取数据

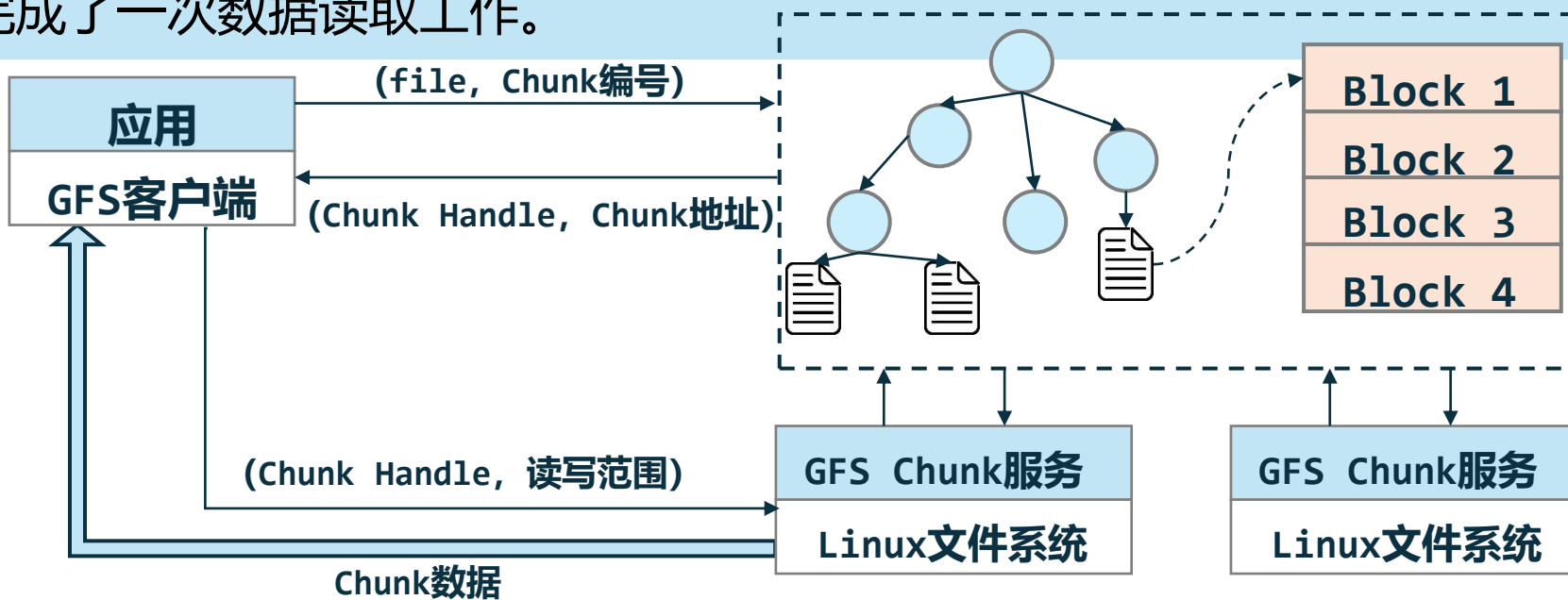
- 读数据请求是：读取文件file，从某个位置P开始读，读大小为L的数据。
- GFS系统在接收到这种请求后从位置P和大小L可以推算出要读的数据位于文件file中第几个Chunk中，转换为<文件名file,Chunk序号>的形式。
- GFS系统通过“主控服务器”得知要读的数据在哪台“Chunk服务器”，将Chunk序号转换为系统唯一的编号，并将信息传回“GES客户端”。



GFS整体架构

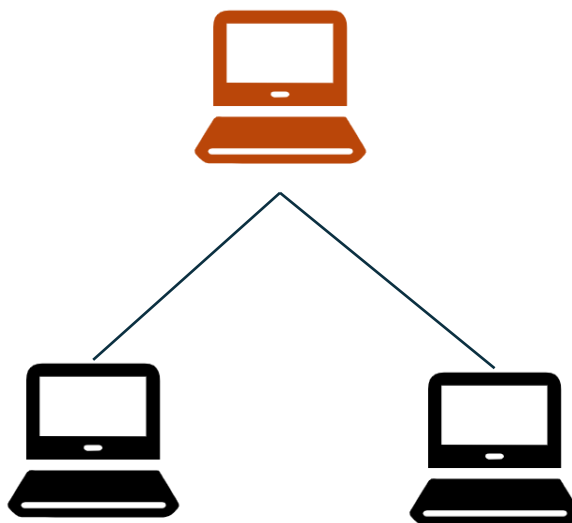
“GFS客户端” 读取数据

- “GFS客户端” 知道了应该去哪台 “Chunk服务器” 读取数据后，会和 “Chunk服务器” 建立联系。
- 发送要读取的Chunk编号以及读取范围，“Chunk服务器” 在接收到请求后，将请求数据发送给 “GFS客户端”。
- 如此就完成了一次数据读取工作。



Google的云存储平台一个显著特点是大量采用“主从结构”。

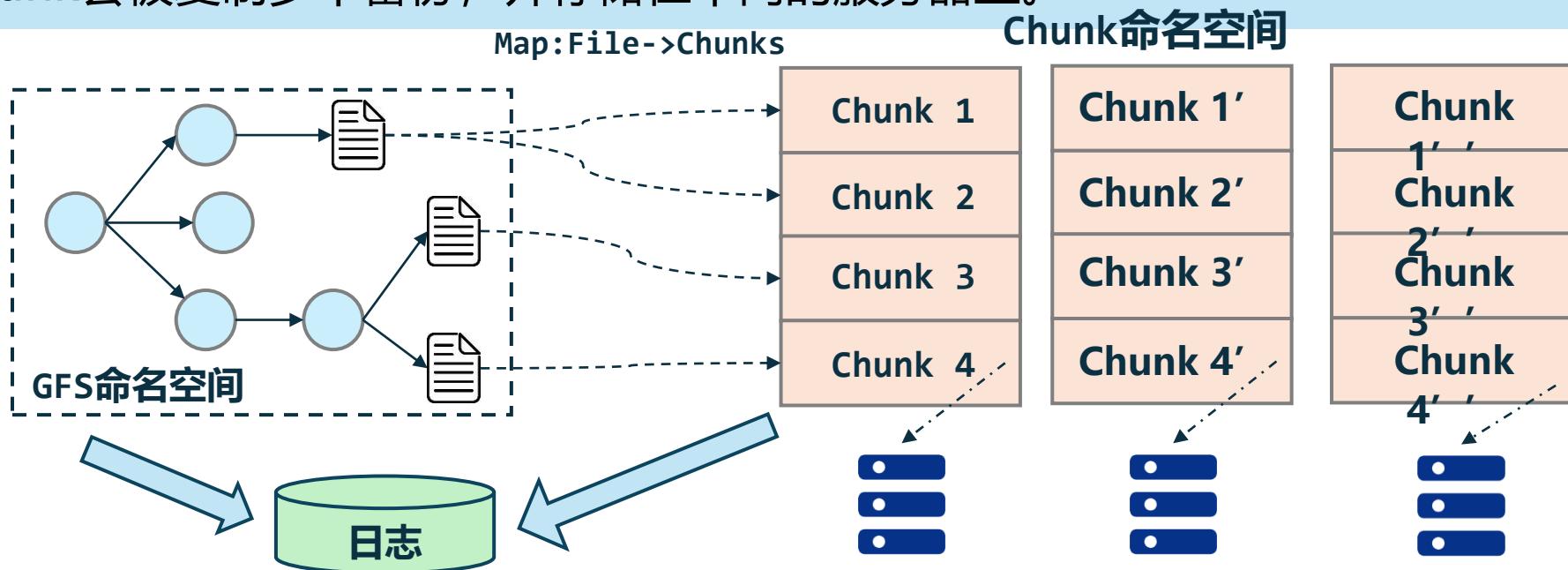
- 主控服务器主要从事系统元数据存储管理以及整个分布式系统的管理。
- **好处是：**整个系统存在一个全局的主控节点，管理起来相对简单。
- **缺点是：**很多服务请求都需要经过“主控服务器”，易成为系统的瓶颈。



GFS整体架构

主控服务器存储的管理数据

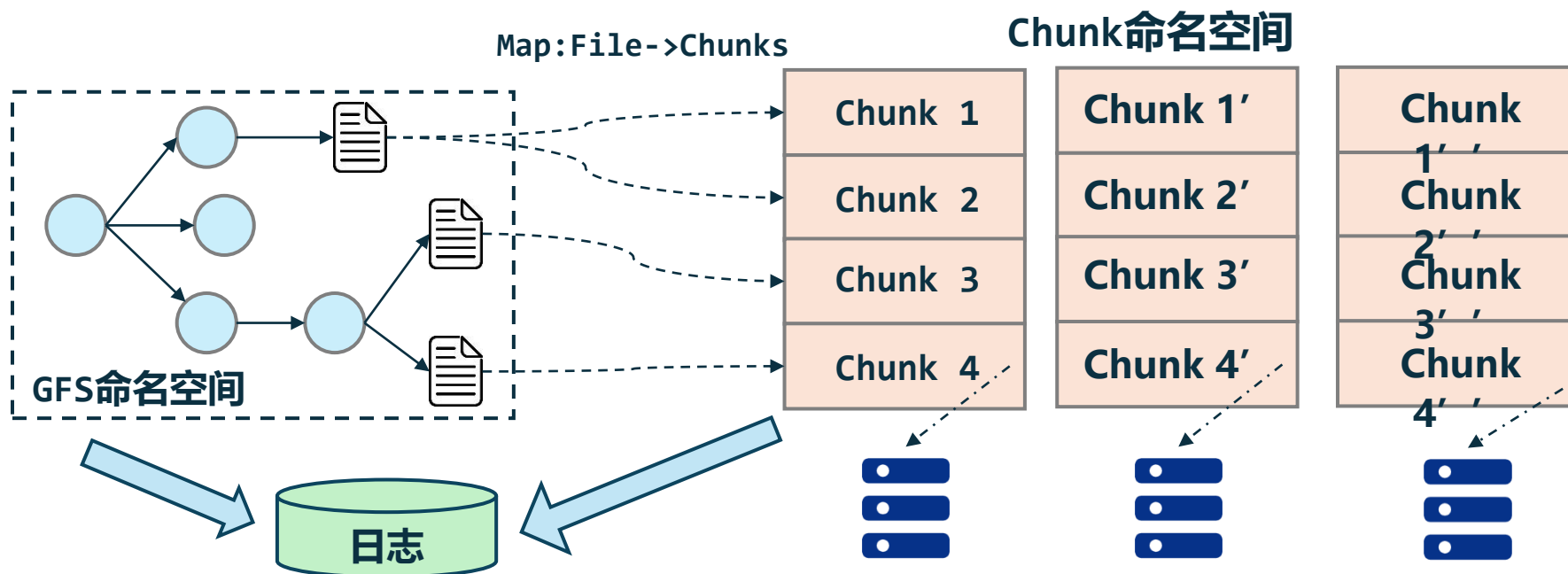
- **GFS和Chunk命名空间**：目录文件和Chunk增删改等信息的记录。
- **从文件到所属Chunk的映射**：因为文件会被切割成众多Chunk，所以系统需要维护这种映射关系。
- **每个Chunk在哪台“Chunk服务器”存储的信息**：每个文件会被切割成若干Chunk，每个Chunk会被复制多个备份，并存储在不同的服务器上。



GFS整体架构

由于管理数据非常重要，所以其安全性必须得到保障，如果管理数据丢失，那么这个GFS系统也就不可用。

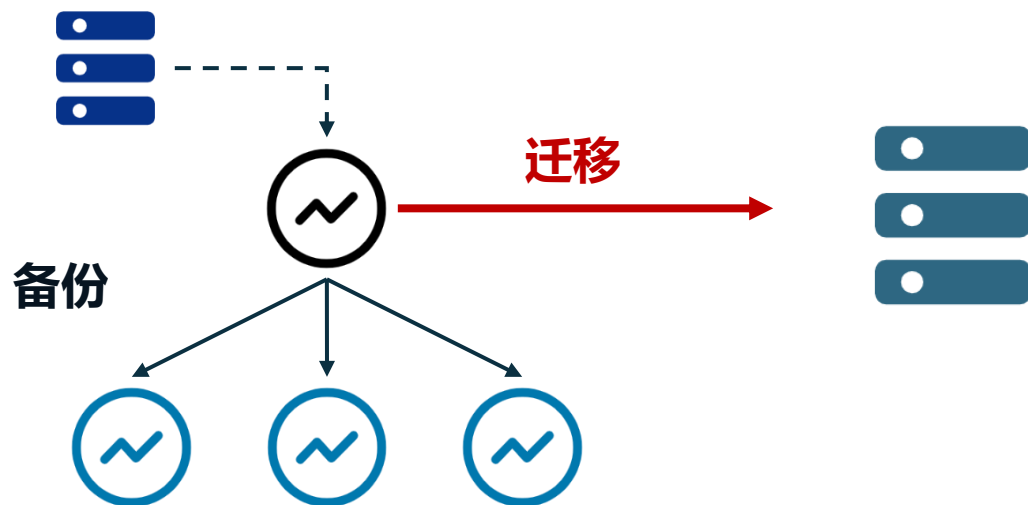
- GFS将命名空间及文件到Chunk映射表记录在系统日志文件内，并且将这个系统日志分别存储在三台机器上，这样就避免了信息丢失的问题。
- 对于Chunk存储在哪个服务器的信息，“主控服务器”在启动时询问每个“Chunk服务器”，之后靠定期询问来保持最新的信息。



GFS整体架构

在对数据进行备份和迁移的时候，GFS重点要考虑两个因素。

- 一是Chunk数据的可用性。也就是说，如果发现Chunk数据不可用，要及时重新备份，以免某个Chunk的所有备份都不可用导致数据丢失。
- 二是要尽可能减少网络传输压力。在不同机器之间传递数据时，因数据量巨大，所以尽可能减少网络传输压力对于系统整体性能表现很重要。



GFS整体架构

为了避免单一“主控服务器”可能存在的单点失效问题，GFS采用了增加另外一台“影子服务器”（Shadow）的方式。

- 当“主控服务器”出现故障无法提供服务时，可由影子服务器接替“主控服务器”行使对应的管理功能。
- 影子服务器不是完整的热备服务器，主控服务器发生故障后，影子服务器可接替主控服务器使客户端可以读取元数据，但作用仅限于此。



GFS执行写操作的整体流程

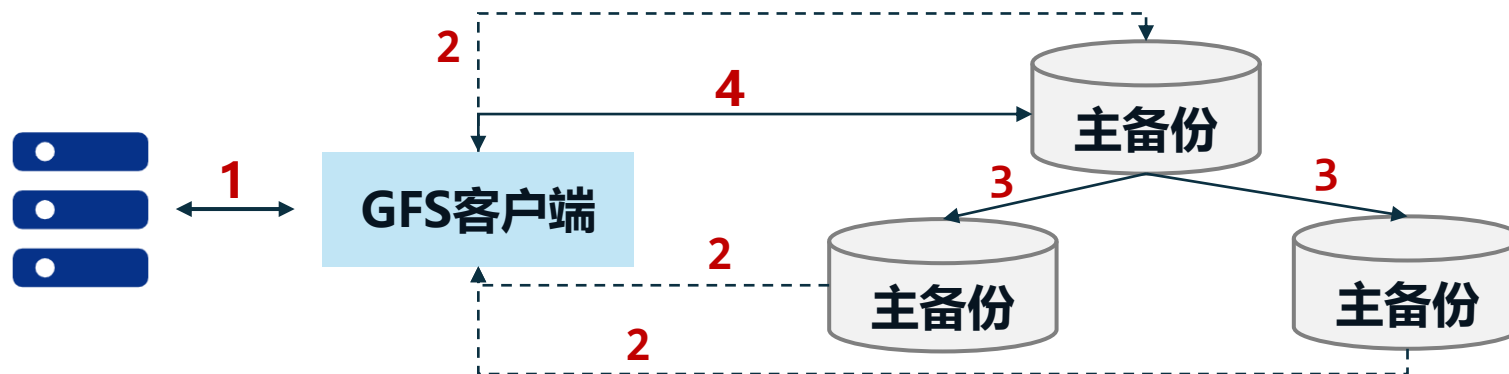
- **GFS客户端和“主控服务器”通信**，获知哪些“Chunk服务器”存储了要写入的Chunk，包括“主备份”和两个“次级备份”的地址数据。
- 客户端将要写入的**数据推送给备份Chunk**，备份Chunk将数据放在缓存，通知是否接收成功，如果都接收成功，通知“主备份”可执行写入操作。
- 主备份将缓存数据写入Chunk，**通知次级备份按照顺序写入数据**，次级备份写完答复主备份写入成功，主备份通知客户端这次写操作成功完成。
- 如果待写入的数据跨Chunk或者需要多个Chunk才能容纳，则客户端会自动将其分解成多个写操作，其执行流程与上述流程一致。



系统交互行为

GFS还提供了原子文件追加操作，可以支持多客户端并发在某个文件尾部追加记录内容，保证每个客户端至少成功追加一次。

- 原子追加操作运行逻辑**唯一区别**在于“主备份”：在接收到客户端的写入通知时，需要判断当前Chunk剩余空间是否足够容纳得下要写入的记录。
- **如果不够**，那么将当前Chunk进行自动填充满并通知所有“次级备份”也如此操作，然后通知客户端让其尝试写入文件的下一个Chunk中。
- **如果空间足够**，那么“主备份”将记录写入Chunk并通知所有“次级备份”也在Chunk的同一偏移处写入记录，之后通知客户端写入成功



HDFS是Hadoop中的大规模分布式文件系统。

- HDFS最初是模仿GFS开发的开源系统，所以在架构上和GFS大致相同。
- 与GFS一样，HDFS适合存储大文件并提供高吞吐量的顺序读 / 写访问，不太适合大量随机读的应用场景，也不适合存储大量小文件等应用场景。

✗ GFS架构中的单一“主控服务器”最主要的两个问题是：

单点失效和水平扩展性不佳。

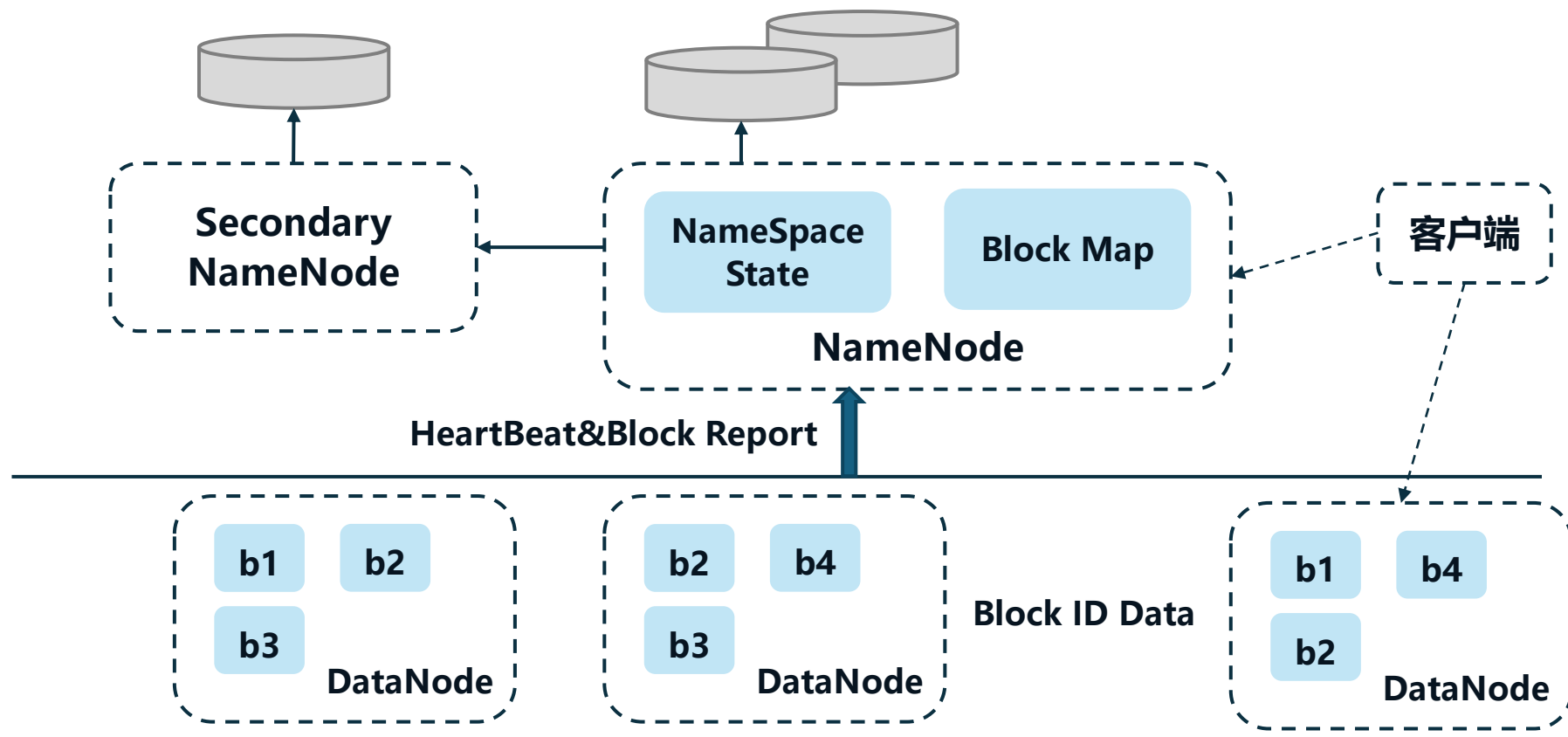
- ✓ **针对这两个问题，从Hadoop 2.0开始提出统一的解决方案。**
- 高可用方案（HighAvailability，HA）用于解决单点失效问题，而NameNode联盟则是为了解决整个系统的水平扩展问题。



HDFS整体架构

HDFS的整体架构如图所示，其由NameNode、DataNode、Secondary NameNode以及客户端构成。

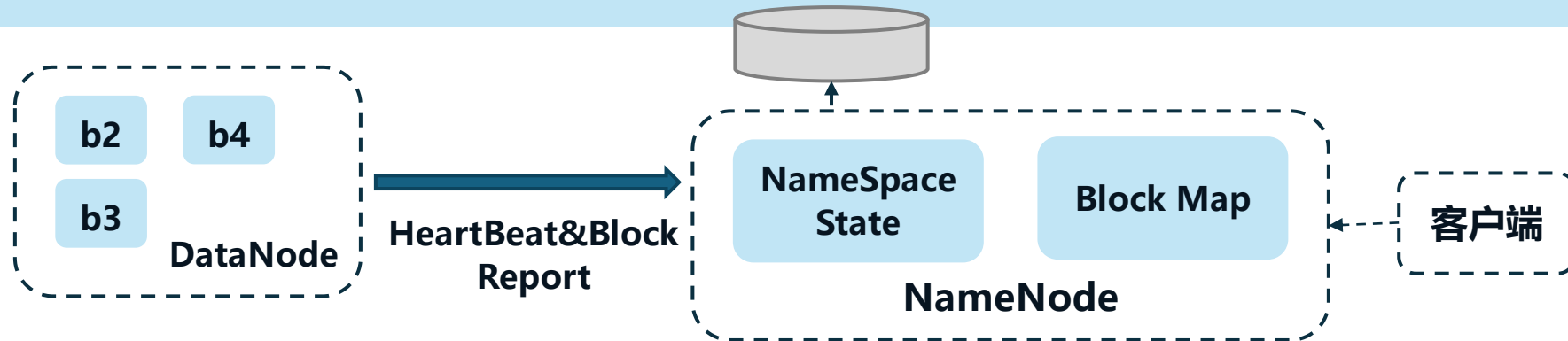
- NameNode类似于GFS的“主控服务器”。
- DataNode类似于GFS的“Chunk服务器”。



HDFS整体架构

NameNode

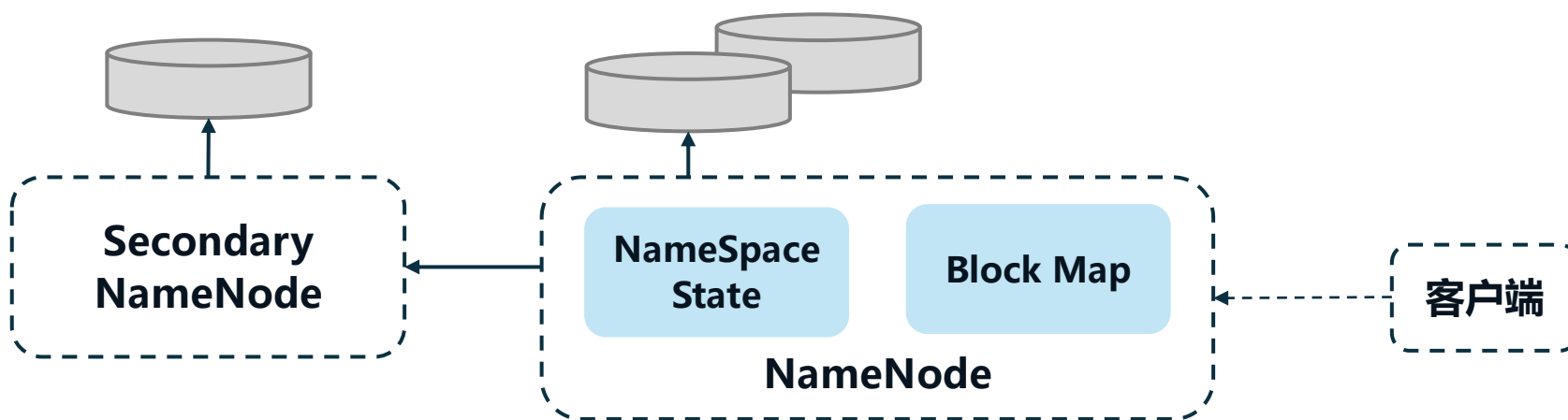
- 负责管理整个分布式文件系统的元数据，包括文件目录树结构、文件到数据块Block的映射关系、Block副本及其存储位置等各种管理数据。
- 磁盘保存两个元数据管理文件。fsimage是内存命名空间元数据在外存的镜像文件，editlog是各种元数据操作的write-ahead-log文件。
- NameNode还负责DataNode的状态监控，两者通过短时间间隔的心跳来传递管理信息和数据信息。
- 如果发现某个DataNode节点发生故障，NameNode会将其负责的Block在其他DataNode机器增加相应备份以维护数据可用性。



HDFS整体架构

Secondary NameNode

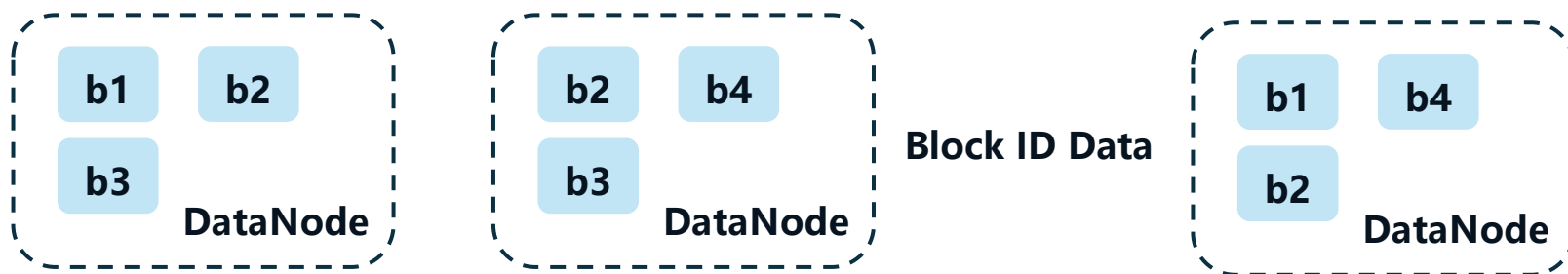
- 职责是定期从NameNode拉取fsimage和editlog文件并对这两个文件进行合并，形成新的fsimage文件并传回给NameNode。
- 目的是减轻NameNode工作压力，NameNode本身不做这种合并操作。
- 本质上Secondary NameNode是个提供检查点功能服务的服务器。



HDFS整体架构

DataNode

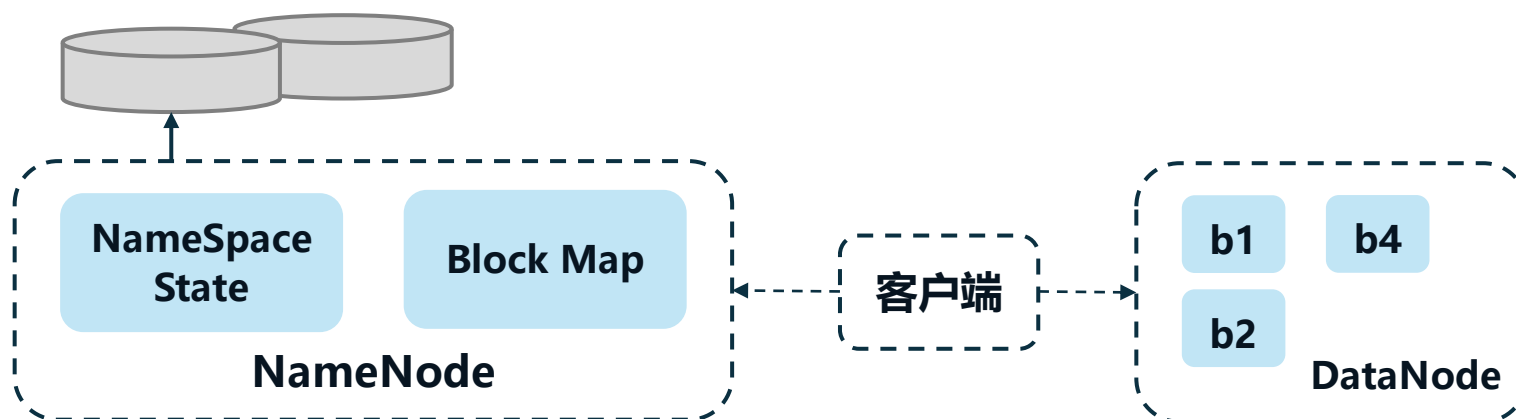
- 类似于GFS的“Chunk服务器”，负责数据块的实际存储和读 / 写工作。
- HDFS语境下将数据块称为Block而非Chunk，Block默认大小为64MB。
- 当客户端上传一个大文件时，HDFS会自动将其切割成固定大小的Block。
- 为保证数据可用性，每个Block以多备份的形式存储，默认备份个数为3。



HDFS整体架构

客户端

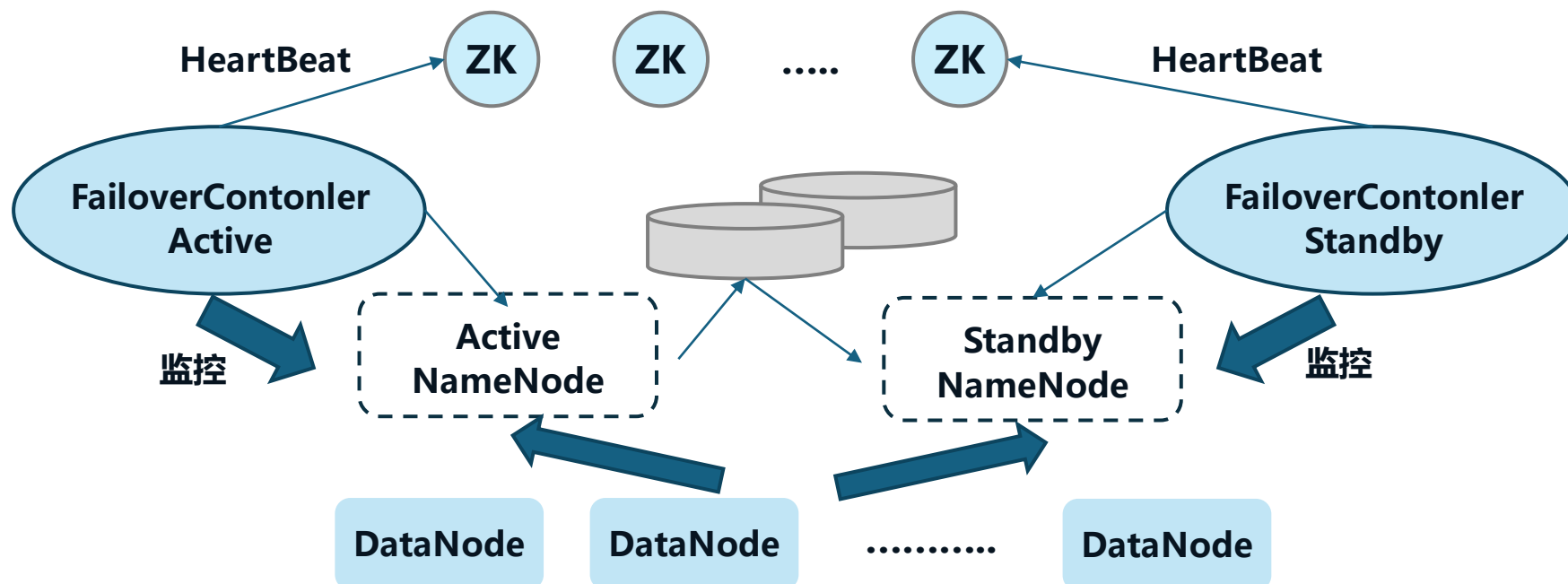
- HDFS客户端和NameNode联系获取所需读 / 写文件的元数据，实际的数据读 / 写都是和DataNode直接通信完成的。
- 其读 / 写流程和GFS的读 / 写流程类似，不同点在于HDFS不支持客户端对同一文件的并发写操作，同一时刻只能有一个客户端在文件末尾进行追加写操作。



HA方案

HA方案

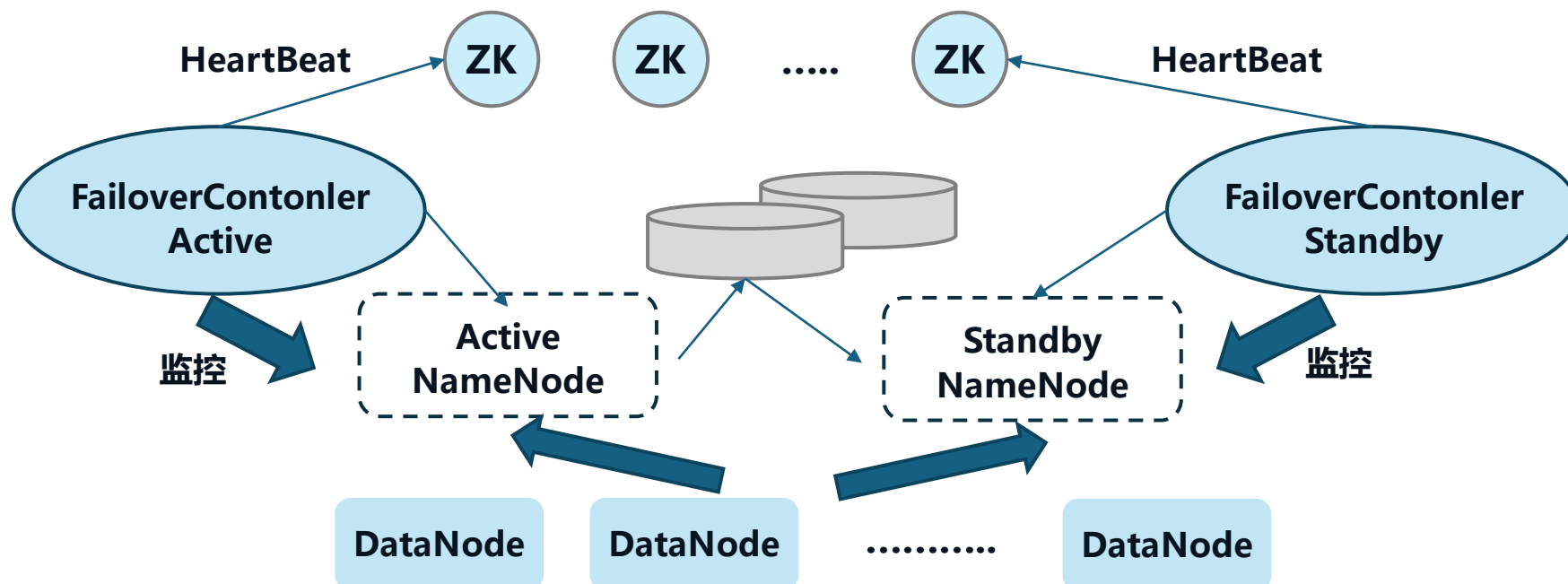
- “主控服务器”由ActiveName Node（简称ANN）和Standby NameNode（简称SNN）一主一从两台服务器构成。
- ANN是当前响应客户端请求的服务器，SNN作为冷备份或者热备份机，在ANN发生故障时接管客户端请求并由SNN转换为ANN。



HA方案

SNN的元数据和ANN保持一致

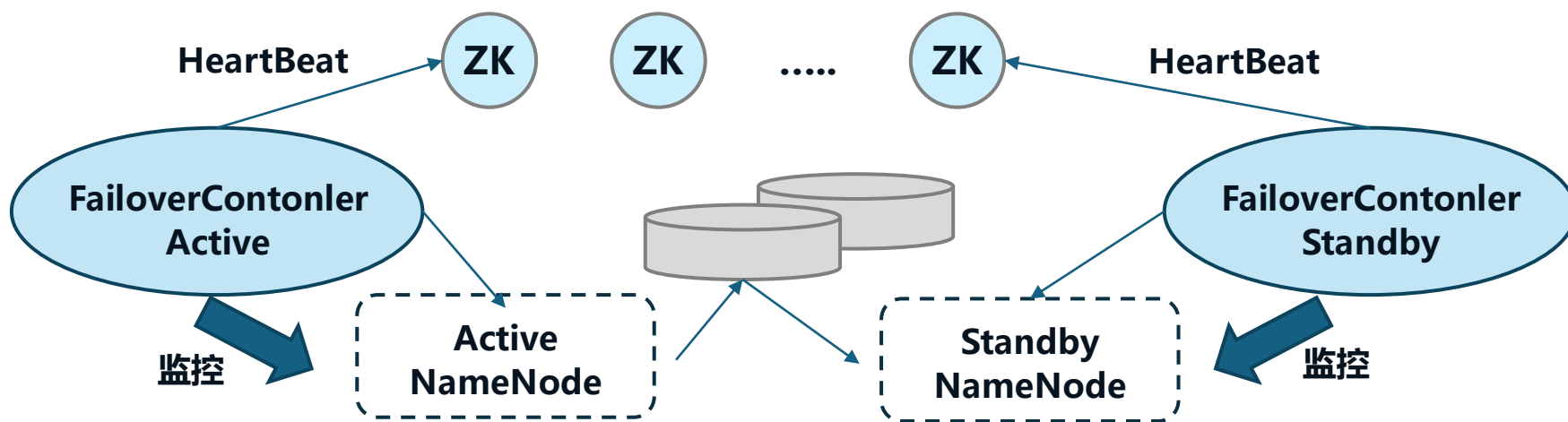
- 使用第三方共享存储（NAS+NFS）来保存目录文件等命名空间元数据，ANN将元数据的更改信息写入第三方存储，SNN从第三方存储不断获取更新的元数据并体现在内存元数据中。
- 所有DataNode同时将心跳信息发送给ANN和SNN。



故障自动切换

HA解决方案采用了独立于NN之外的故障切换控制器（FC）。

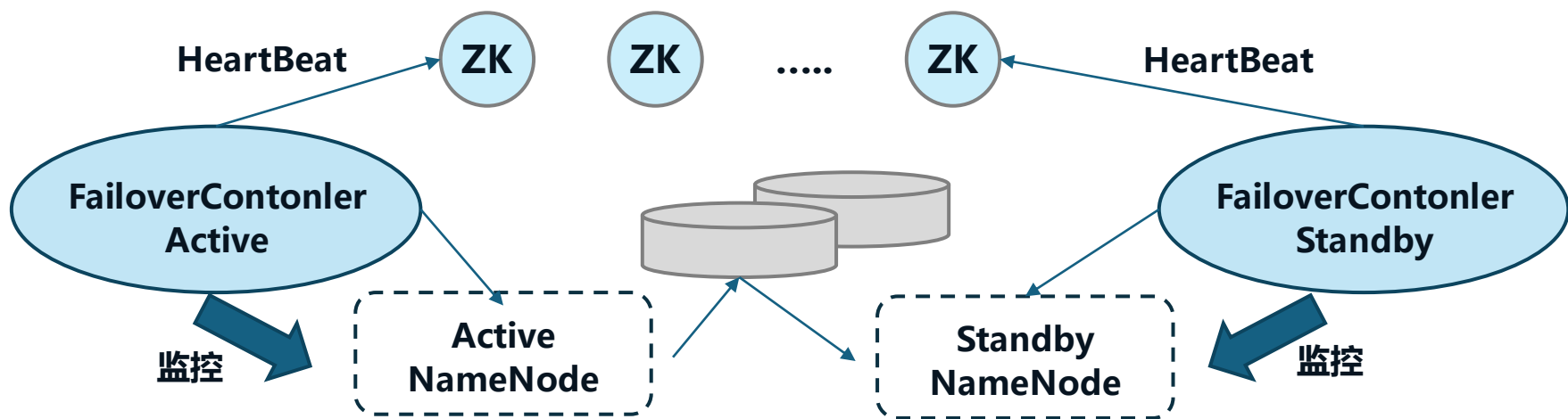
- FC用于监控NN服务器的硬件、操作系统及NN本身等各种健康状况信息。
- 不断地向ZooKeeper写入心跳信息，ZooKeeper用作“领导者选举”。
- 当ANN发生故障时，ZooKeeper重新选举SNN作为“主控服务器”，FC通知SNN从备份机转换为主控机。



为什么要采取独立于NN的FC

- 一方面是因为NN在做垃圾回收（GC）的时候很可能在较长时间（10秒左右）内整个系统无响应，所以无法向ZooKeeper正常写入心跳信息。
- 另外一方面的考虑是，在设计原则上应该将监控程序和被监控程序进行分离而非绑定在一起。

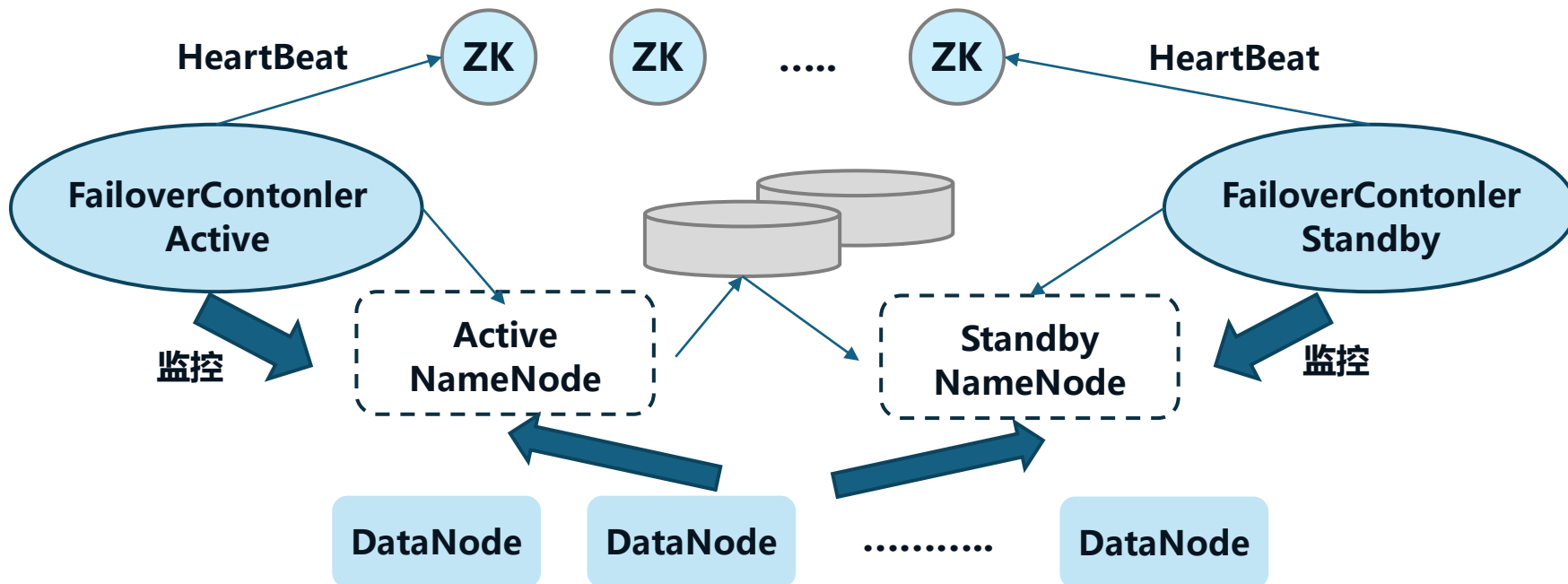
- 一方面是因为NN在做垃圾回收（GC）的时候很可能在较长时间（10秒左右）内整个系统无响应，所以无法向ZooKeeper正常写入心跳信息。
- 另外一方面的考虑是，在设计原则上应该将监控程序和被监控程序进行分离而非绑定在一起。



HA方案

为了防止在故障切换过程中出现脑裂现象，上述HA方案需要在以下3处采取隔离措施（Fencing）。

- **第三方共享存储**：需要保证在任一时刻，只有一个NN能够写入。
- **DataNode**：保证只有一个NN发出与管理数据副本有关的删除命令。
- **客户端**：保证同一时刻只能有一个NN能够对客户端请求发出正确响应。



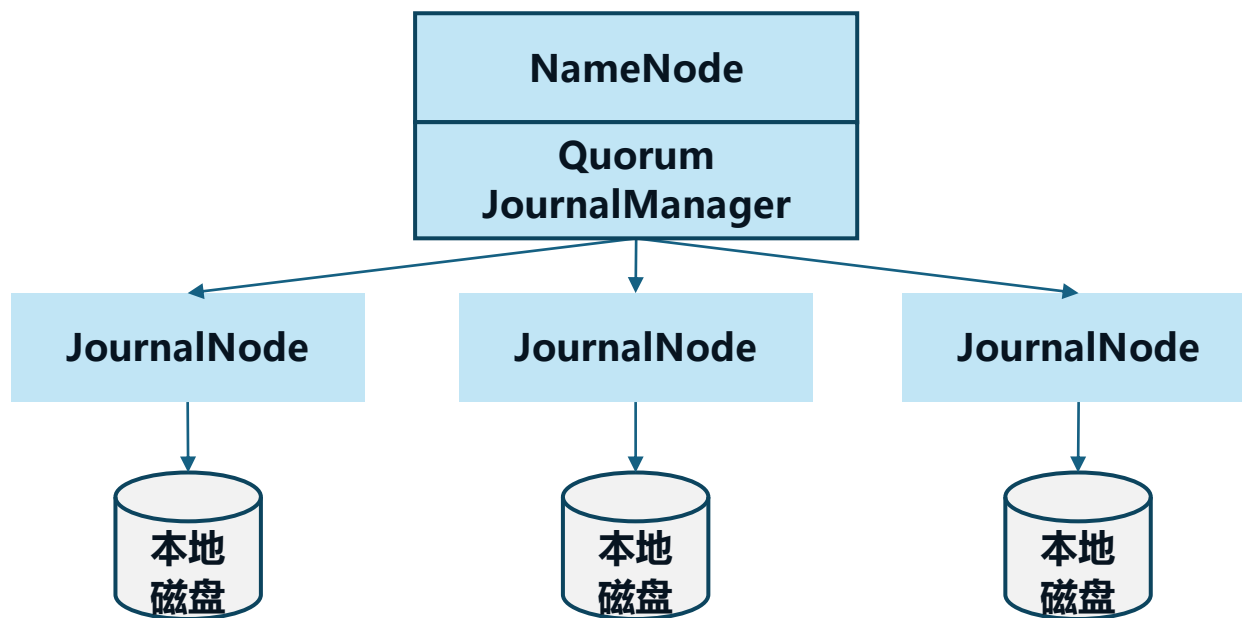
HA方案



- 第三方存储仍然存在单点失效可能
- 需要在多处进行隔离措施以防止脑裂现象出现

✓ **Cloudera提供基于QJM (Quorum Journal Manager) 的HA方案。**

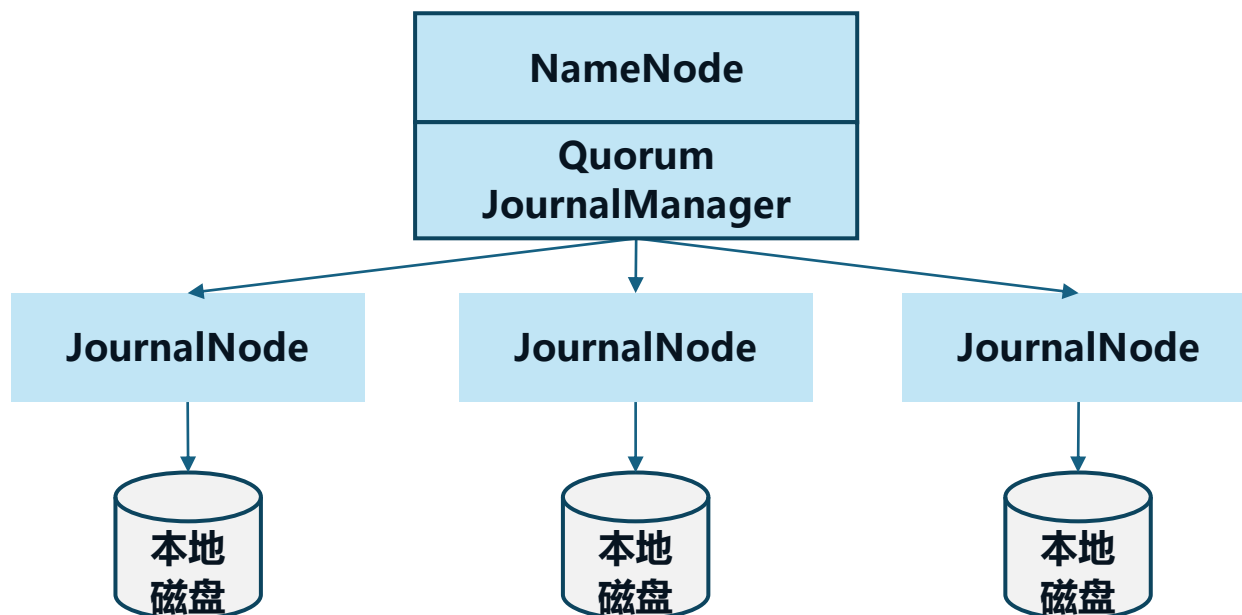
- QJM在 $2F+1$ 个JournalNode中存储NN的editlog，每次写入操作如果有F台服务器返回成功认为成功写入，通过Paxos保证数据的一致性。
- QJM最多可以容忍F个JournalNode同时故障而系统仍然可以正常运行。



HA方案

QJM方案与Hadoop 2.0社区的HA方案比，提供了以下好处。

- 解决**单点失效问题**，可容忍最大故障JournalNode数可通过配置管理。
- 无须配置**额外的第三方存储设备**，这对于目前流行的采用普通商用服务器构建大规模集群的模式来说，减少了整个系统的复杂度和维护成本。
- 无须防脑裂而**单独在多处采用隔离措施**，因为QJM本身内置了该功能。



Hadoop 1.x

Hadoop 1.x中的HDFS由于采取单一NN的架构，会导致系统具有如下缺陷。

- **命名空间可扩展性差：**在单一NN情形下，因为所有命名空间数据都需要加载到内存，所以机器物理内存的大小限制了整个HDFS能够容纳文件的最大个数。
- 尽管DataNode作为实际存储数据块的场所是支持水平扩展的，但是由于NN物理内存限制造成命名空间可扩展性差。



NameNode联盟

Hadoop 1.x

Hadoop 1.x中的HDFS由于采取单一NN的架构，会导致系统具有如下缺陷。

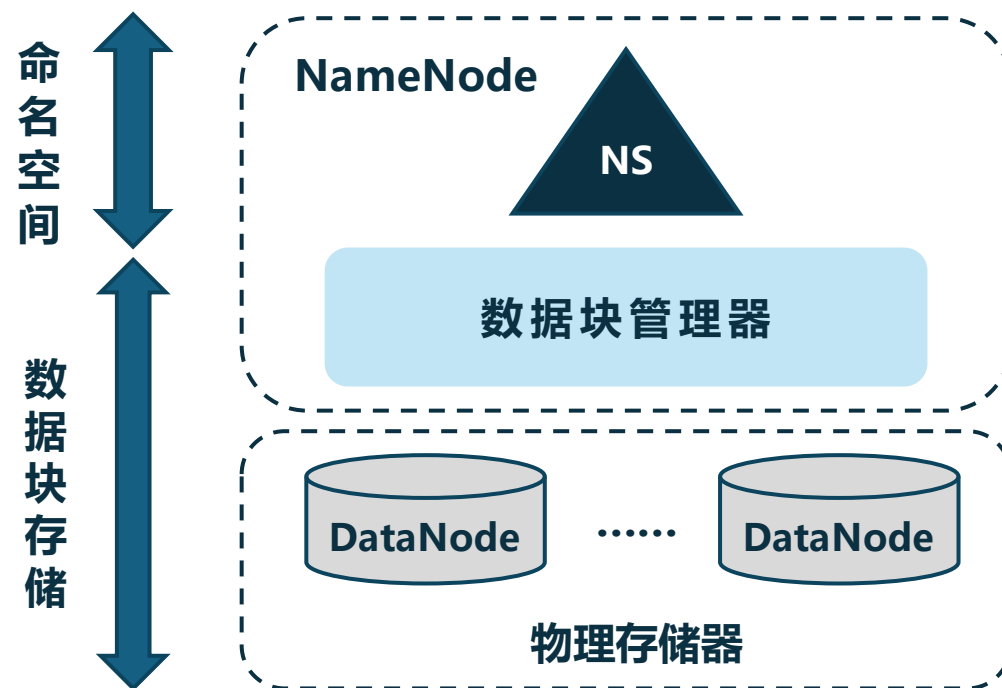
- **性能可扩展性差：**由于所有元数据请求都需要经过NN，单一NN导致所有请求都由一台服务器响应，容易达到机器吞吐极限，造成系统整体性能的提升无法做到水平扩展。
- **隔离性差：**在多租户环境下（Multi-Tenant），单一NN的架构无法在租户之间进行隔离，会造成不可避免的相互影响。



NameNode联盟

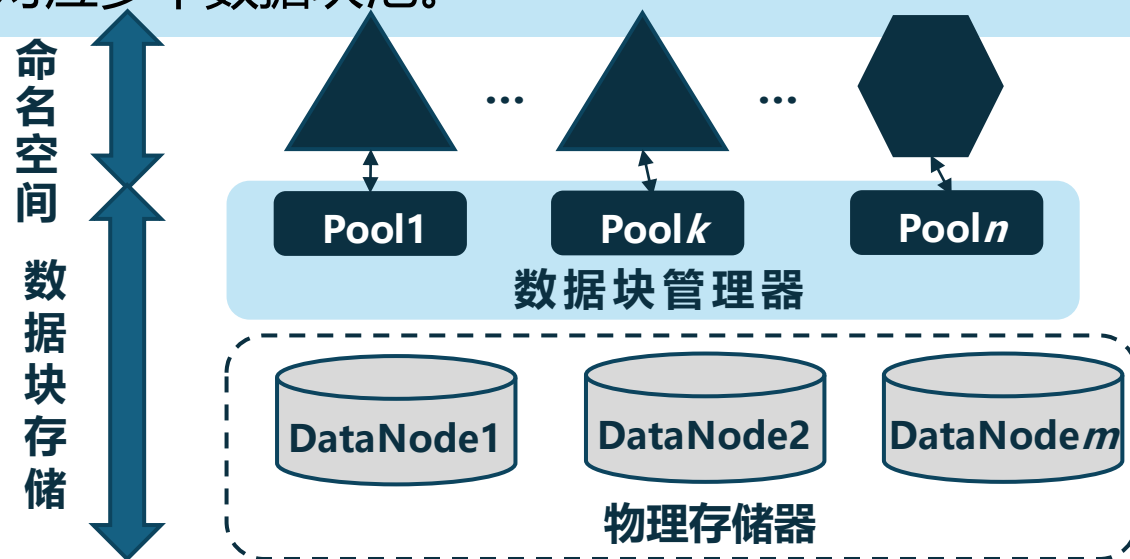
从Hadoop 2.0开始，通过NameNode联盟来解决上述问题。

- 如果将HDFS的功能高度抽象，可以将其划分为3层。
- 低层是物理存储层，其上是数据块管理层（Block Management），最高层是命名空间管理层。



核心思想

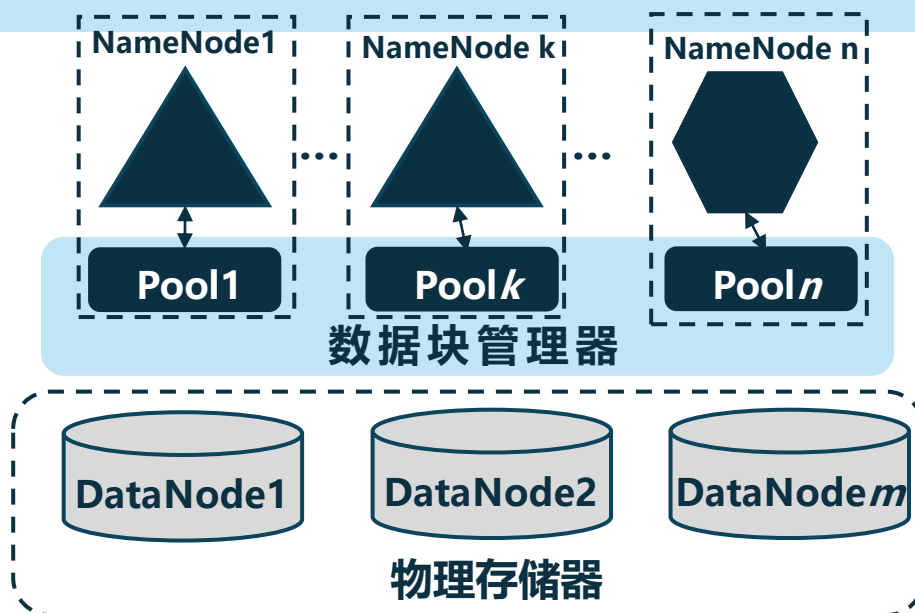
- 将一个大的命名空间**切割成若干子命名空间**，每个子命名空间由单独的NN来负责管理，NN之间独立，相互之间无须做任何协调工作。
- 所有的**DataNode被多个NN共享**，仍然充当实际数据块的存储场所。
- 而**子命名空间和DataNode**之间则由数据块管理层作为中介建立映射关系，数据块管理层由若干数据块池（Pool）构成，每个数据块唯一属于某个固定的数据块池，而一个子命名空间可以对应多个数据块池。



NameNode联盟

具体实现

- 可以将子命名空间及其对应的数据块池管理功能都由相应的NN来承担。
- 也可以将数据块管理层从NN中独立出来，单独形成一个映射层，并由另外一个服务器集群来承担此功能。
- HDFS还是采用了前一种做法，即每个NN同时负责子命名空间及其对应的数据块池的管理，并未将数据块管理层单独抽离出来。



目录

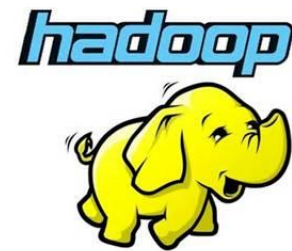
Contents

- 1 MapReduce与计算框架
- 2 数据的分片与路由
- 3 分布式文件系统
- 4 分布式计算框架例析：Hadoop**
- 5 分布式计算框架例析：Spark
- 6 分布式计算框架例析：Storm

- 用于数据密集型分布式应用程序的开源框架
- 灵感来自 Google 的 MapReduce 和 GFS
- 用 Java 实现
- 以造物主之子玩具象命名



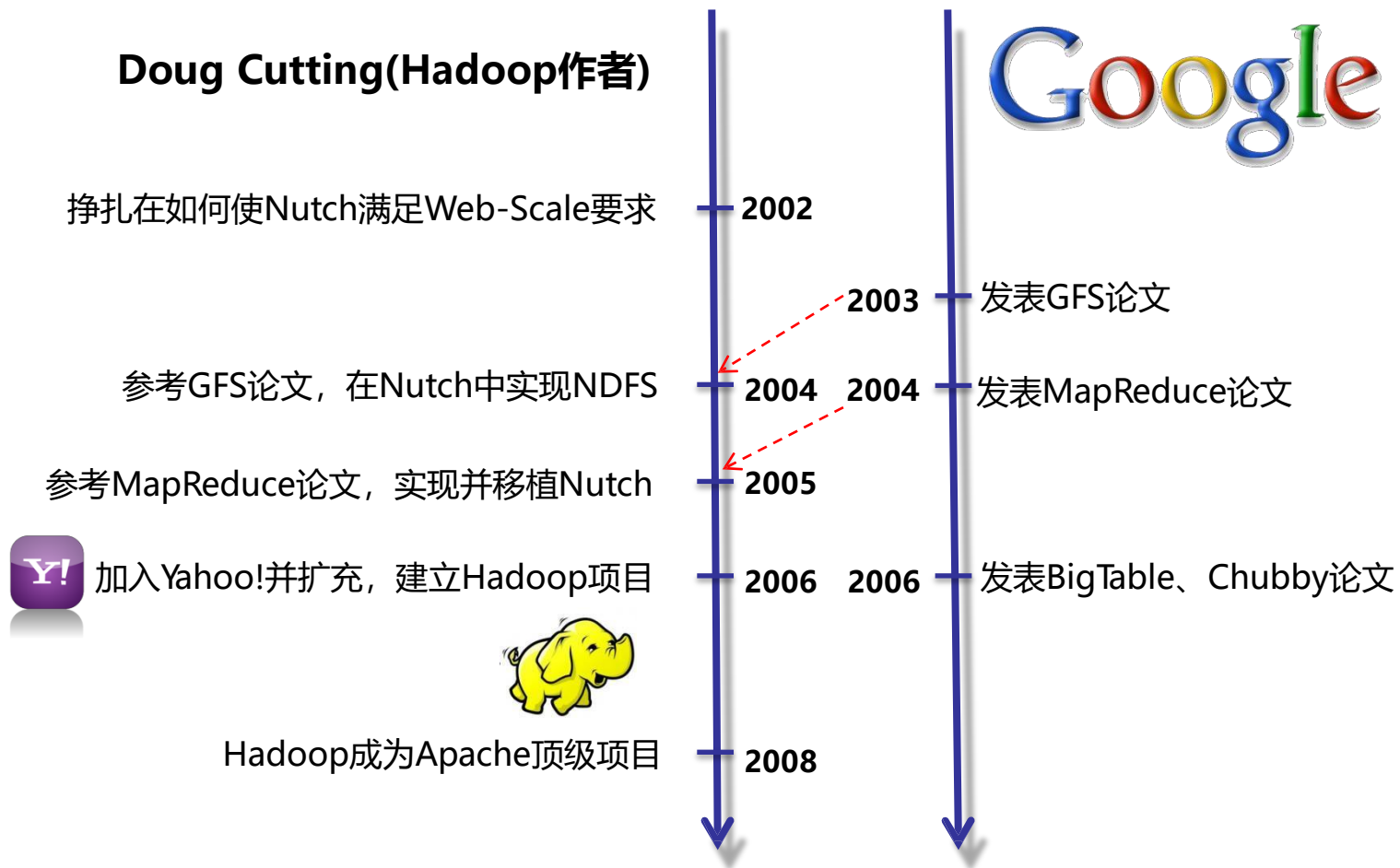
- Hadoop主要包括：
 - MapReduce – 离线大数据分析计算引擎
 - HDFS – 分布式文件系统
 - YARN– 任务执行调度资源管理框架
 - Hbase – NoSQL数据库
 - Hive – 分布式数据仓库



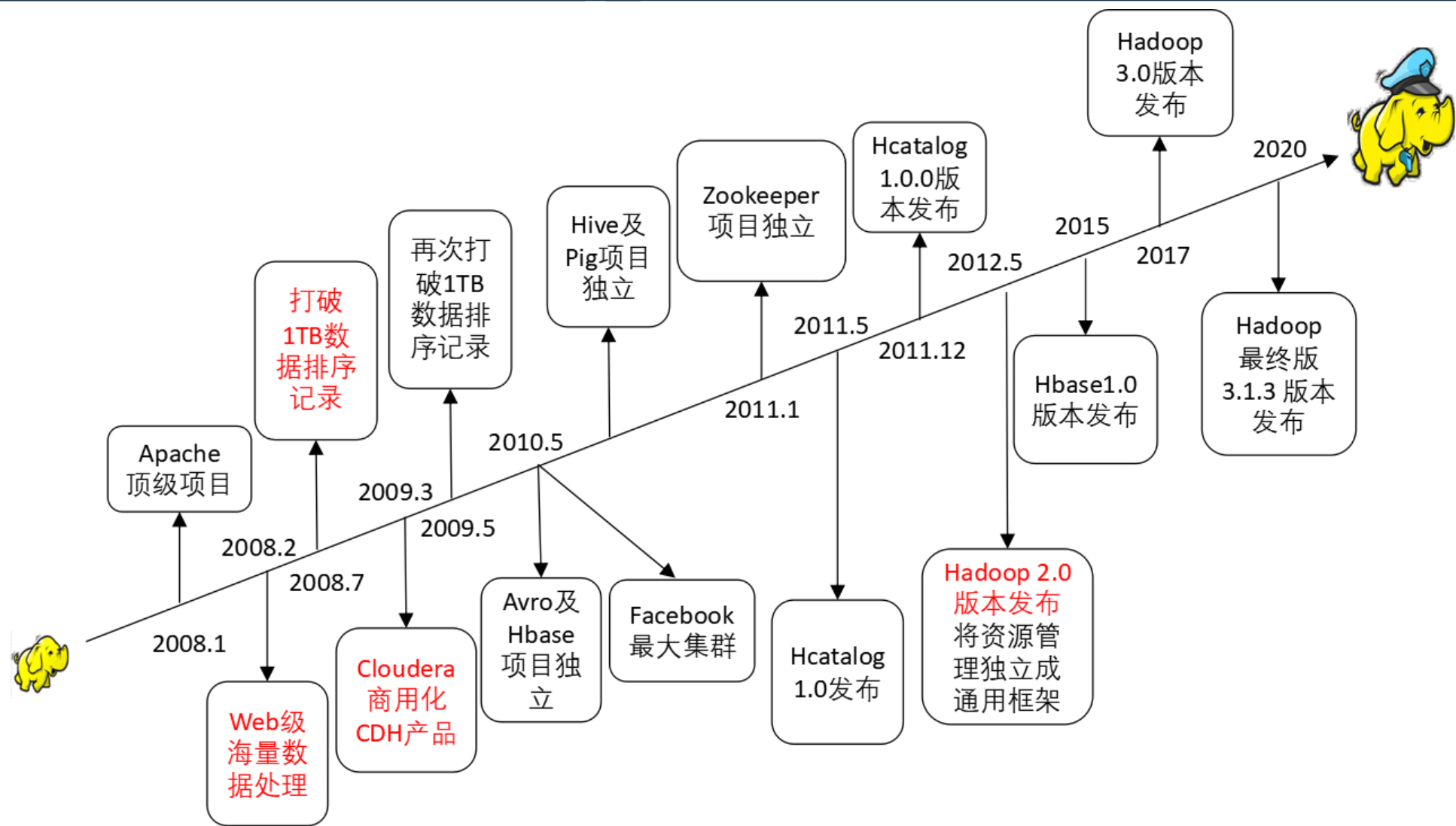
Doug Cutting



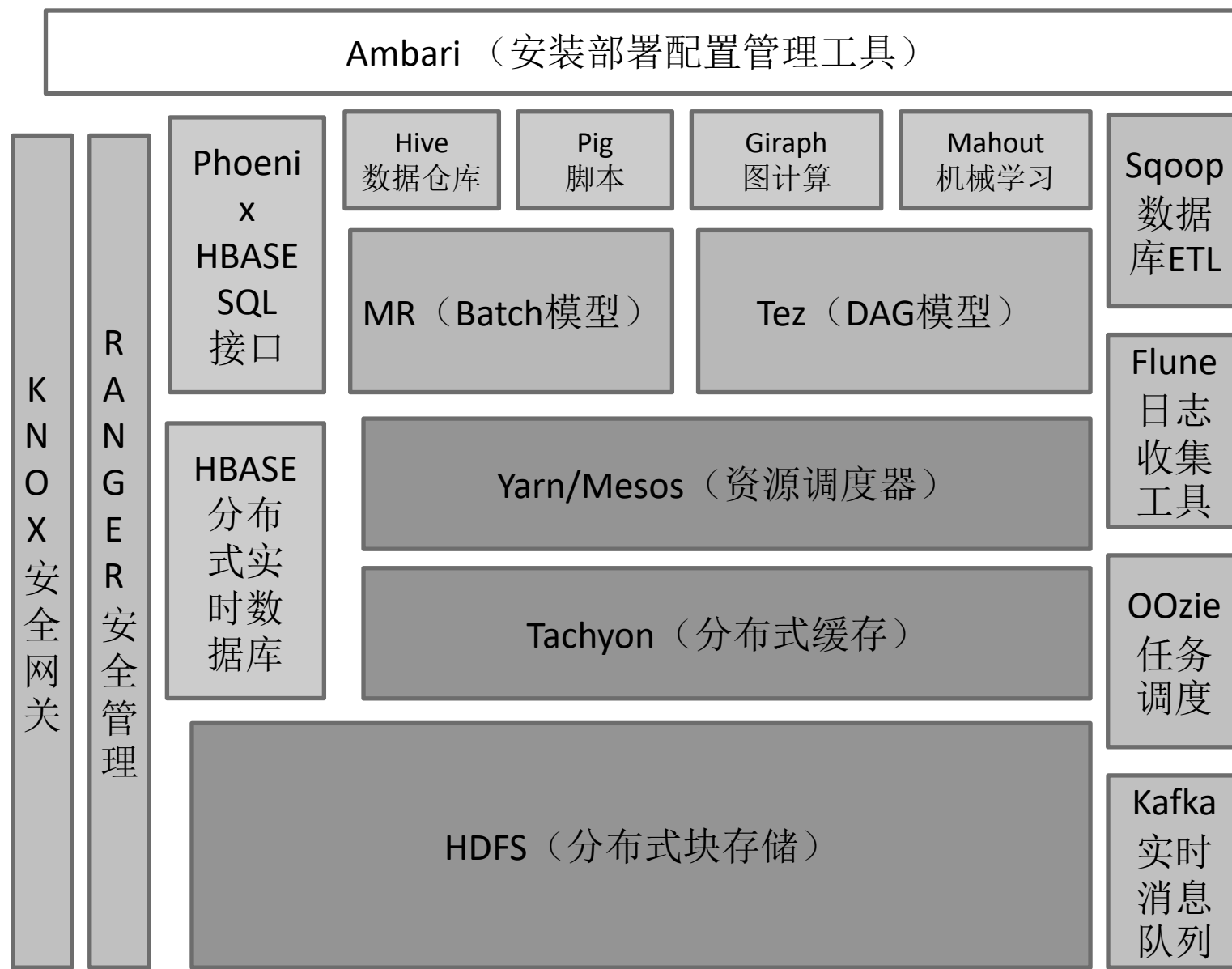
Hadoop发展历程



Hadoop发展历程



Hadoop生态圈



大数据计算框架的实例：Hadoop

- 计算场景: 批处理
- 抽象: Map-Reduce
- API: 重载MapReduce函数

Class Mapper

method map(String input_key, String input_value):

// input_key: text document name

// input_value: document contents

for each word w **in** input_value:

EmitIntermediate(w, "1");

Class Reducer

method reduce(String output_key,
Iterator intermediate_values):

// output_key: a word

// output_values: a list of counts

int result = 0;

for each v **in** intermediate_values:

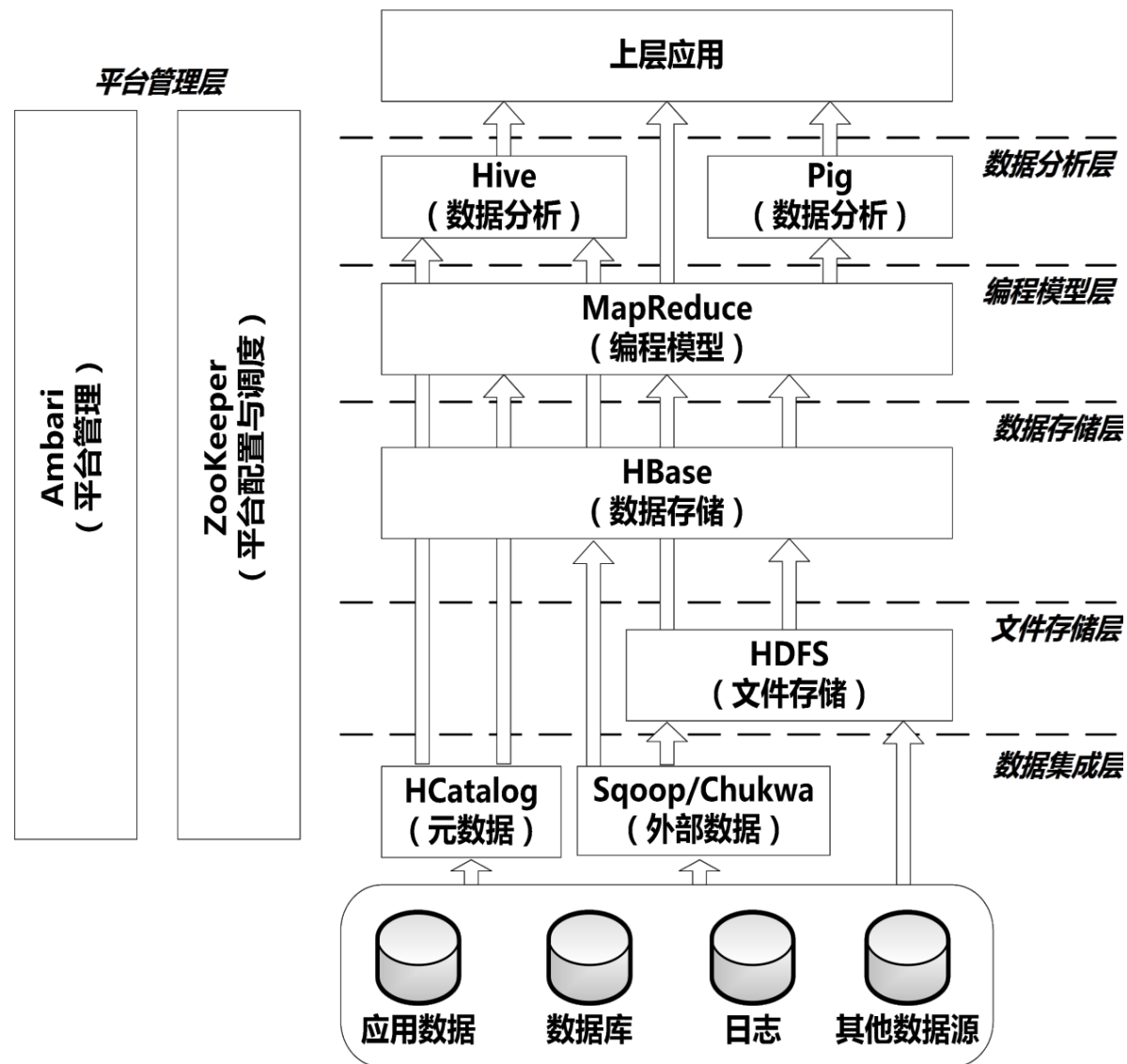
result += ParseInt(v);

Emit(AsString(result));

大数据计算框架的实例：Hadoop

- 系统架构：Hadoop生态
- 数据存储机制：HDFS
- 基本数据操作：数据重分布
- 流程优化：无 (额外的DAG模型生成工具)
- 流程调度：基础任务调度、Map与Reduce函数的执行
- 事务处理：失效结点处理

Hadoop系统结构



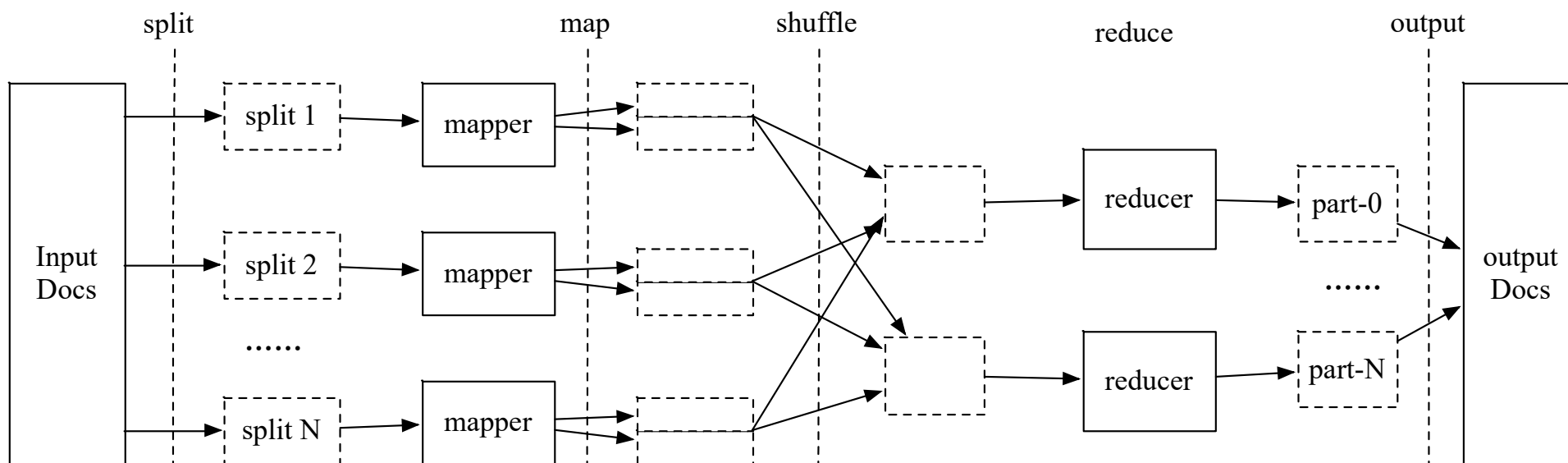
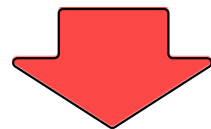
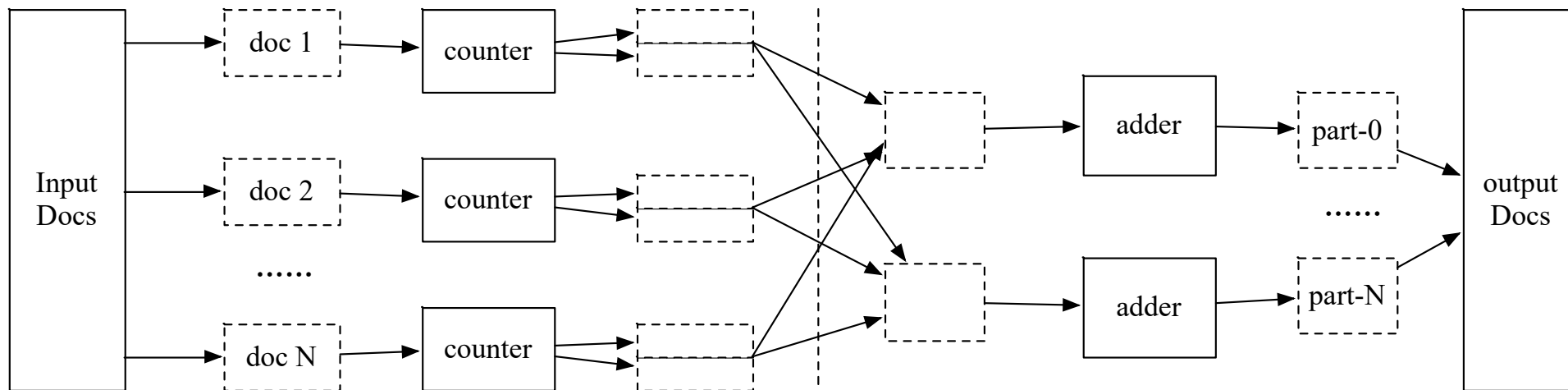
MapReduce版的WordCount

```
public static class TokenizerMapper extends Mapper
<Object, Text, Text, IntWritable>{
    private final static IntWritable one = new IntWritable(1);
    private Text word = new Text();
    public void map(Object key, Text value, Context
        context) throws IOException, InterruptedException{
        StringTokenizer itr = new StringTokenizer(value.toString());
        while (itr.hasMoreTokens()) {
            word.set(itr.nextToken());
            context.write(word, one);
        }
    }

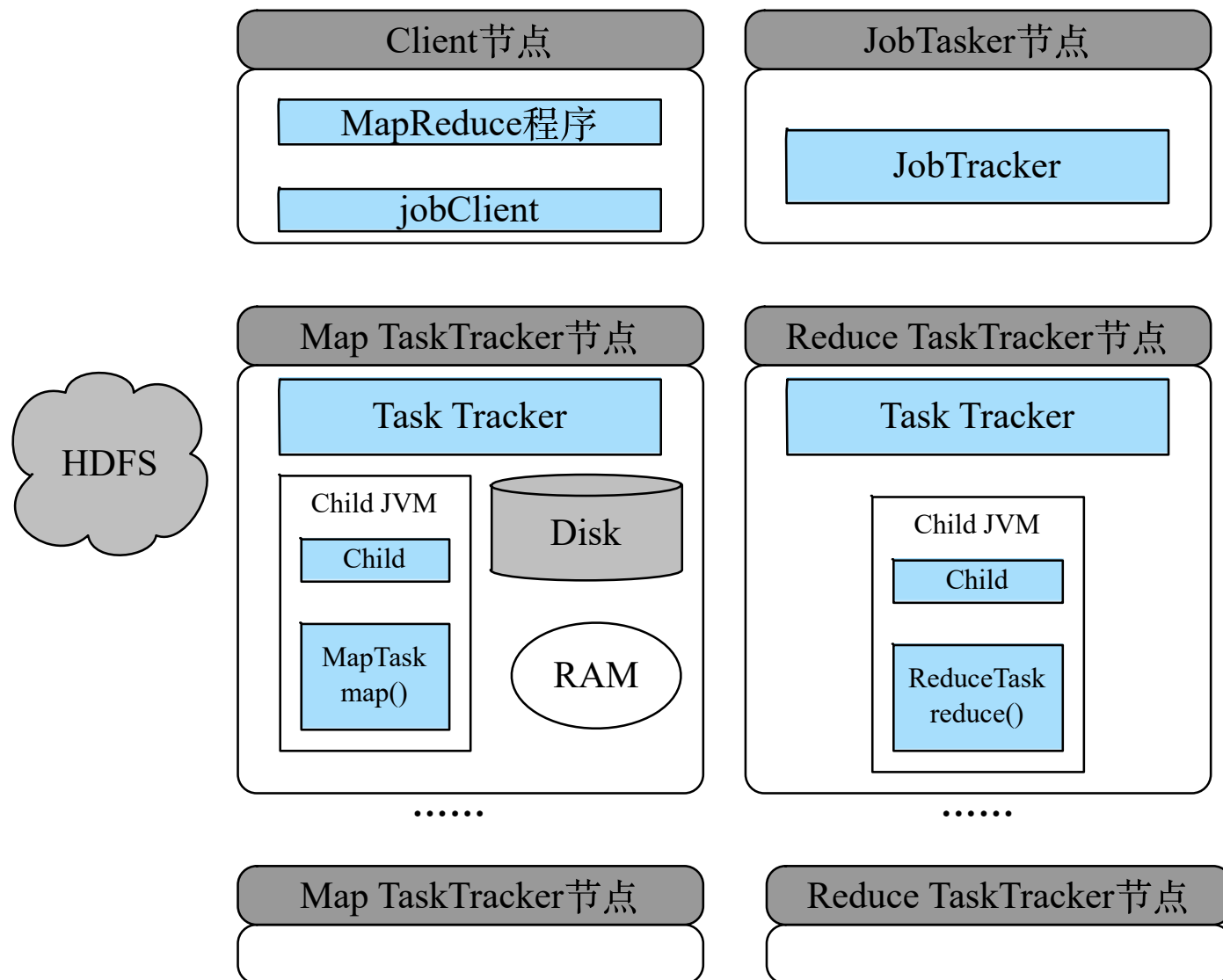
    public static class IntSumReducer extends Reducer
    <Text,IntWritable,Text,IntWritable>{
        private IntWritable result = new IntWritable();
        public void reduce(Text key, Iterable<IntWritable>
            values, Context context) throws IOException{
            int sum = 0;
            for (IntWritable val : values) {
                sum += val.get();
            }
            result.set(sum);
            context.write(key, result);
        }
    }
}
```

```
public static void main(String[] args) throws Exception{
    Configuration conf = new Configuration();
    String[] otherArgs = new GenericOptionsParser(
        conf, args).getRemainingArgs();
    if (otherArgs.length != 2) {
        System.err.println("Usage: wordcount <in> <out>");
        System.exit(2);
    }
    Job job = new Job(conf, "word count");
    job.setJarByClass(WordCount.class);
    job.setMapperClass(TokenizerMapper.class);
    job.setCombinerClass(IntSumReducer.class);
    job.setReducerClass(IntSumReducer.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);
    FileInputFormat.addInputPath(job,
        new Path(otherArgs[0]));
    FileOutputFormat.setOutputPath(job,
        new Path(otherArgs[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
}
```

程序的执行



MapReduce计算框架



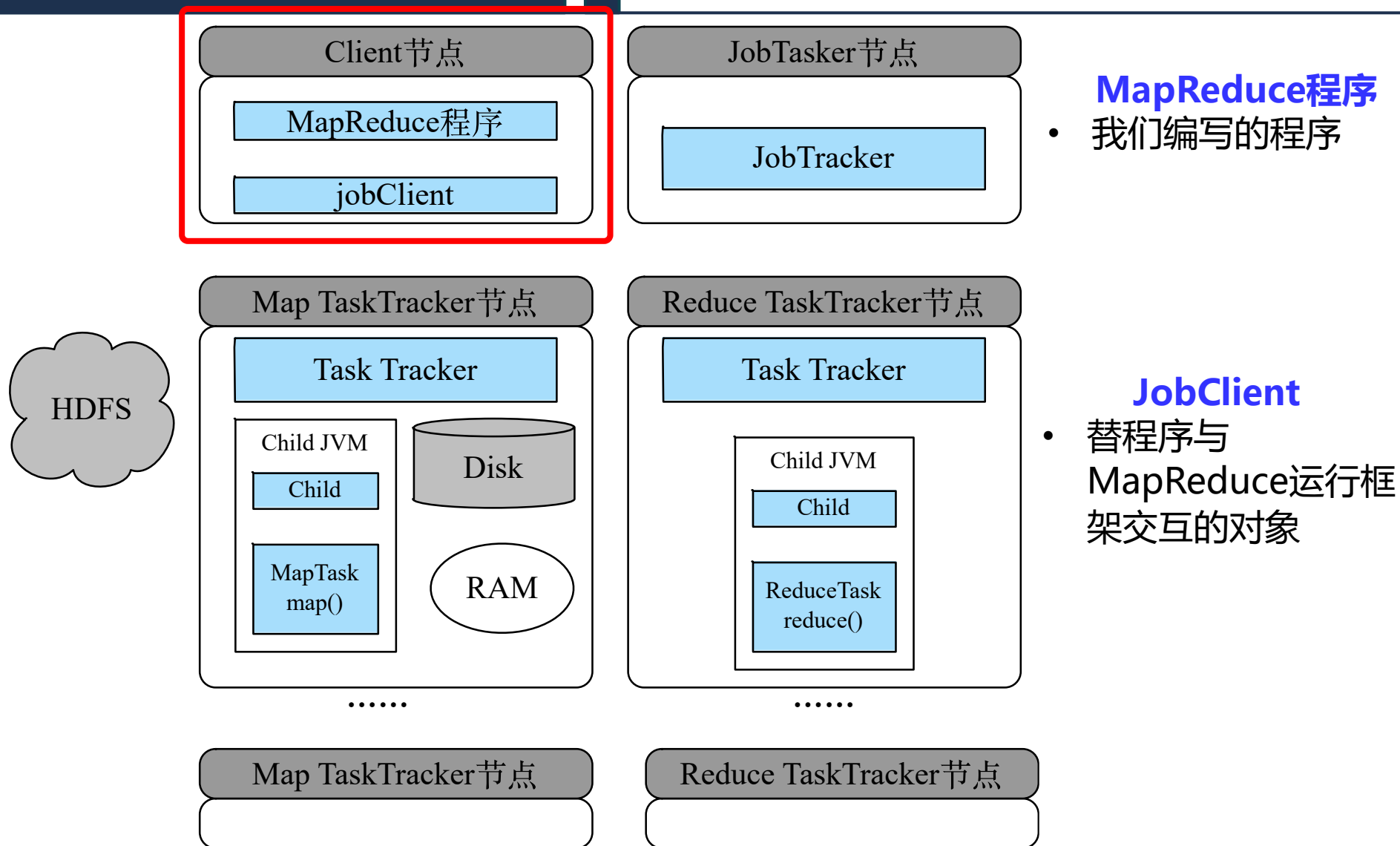
作业 (Job)

- MapReduce程序指定的一个完整计算过程
- 一个作业在执行过程中可以被拆分为若干Map和Reduce任务完成

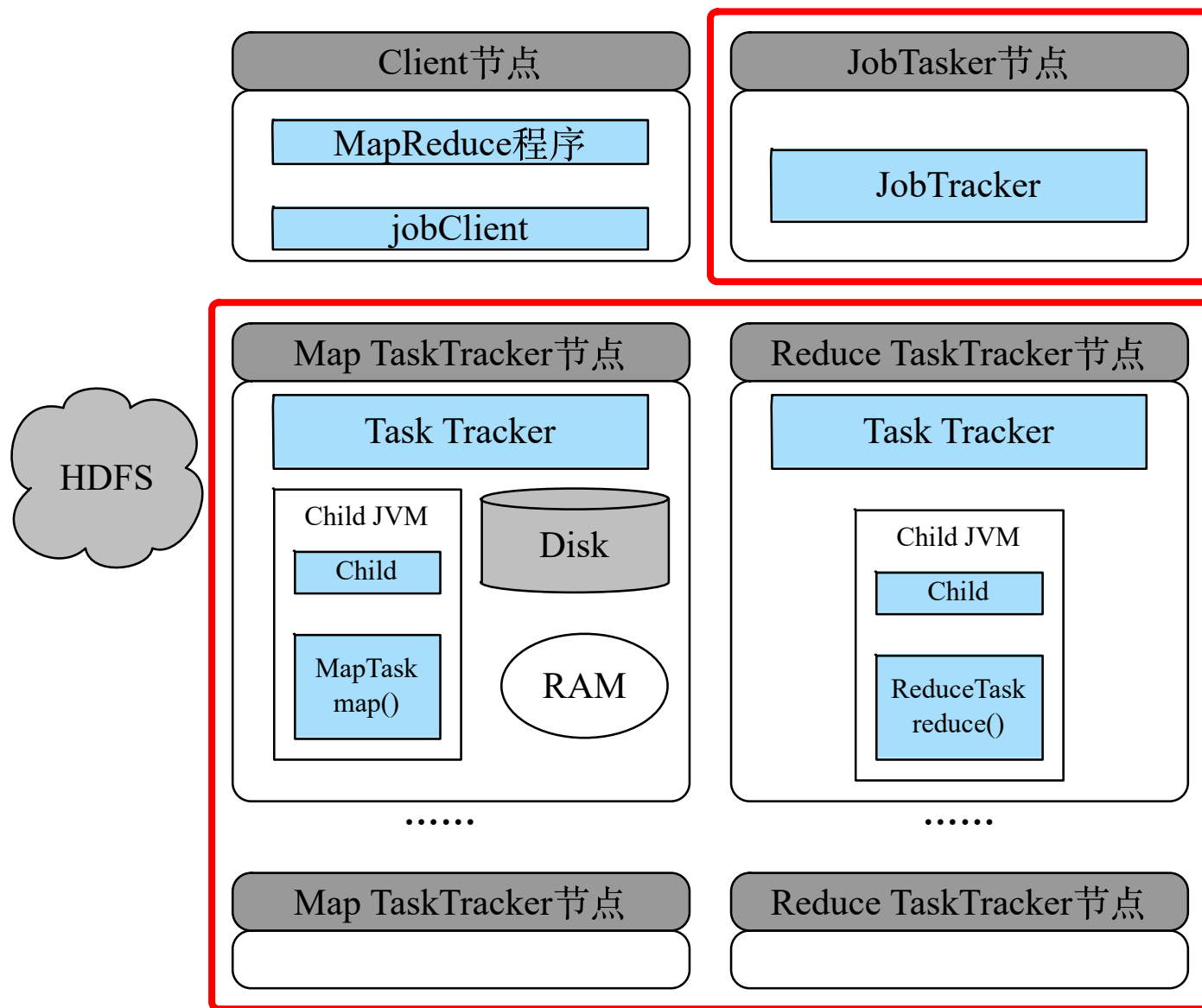
任务 (Task)

- MapReduce框架中进行并行计算的基本事务单元
- 分为Map 和Reduce任务，一个作业通常包含多个任务

MapReduce框架中的角色



MapReduce框架中的角色



JobTracker

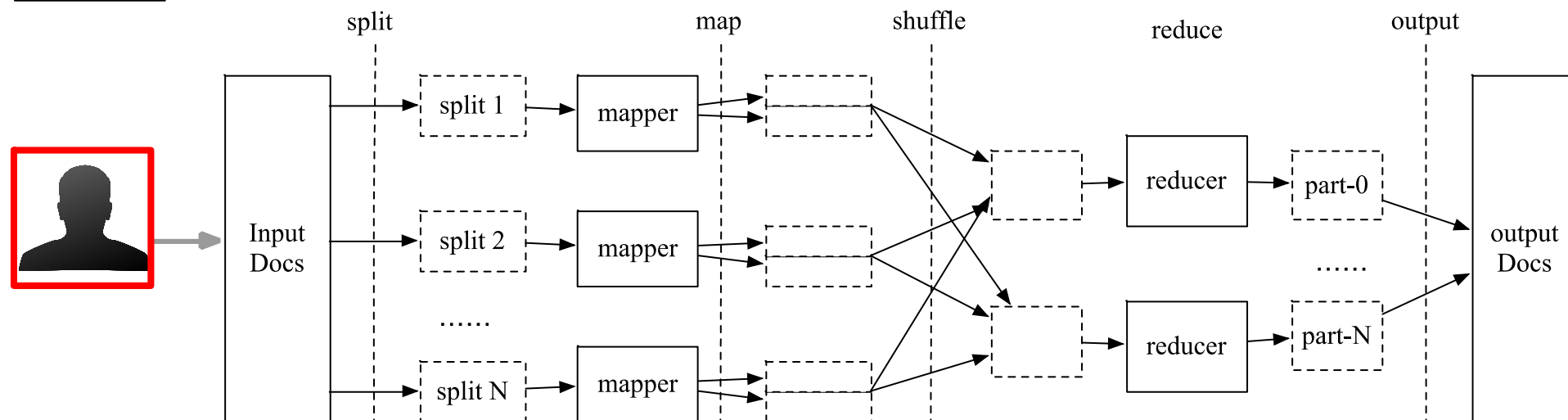
- MapReduce框架的管理者
- 协调MapReduce作业
- 分配任务
- 监控任务

TaskTracker

- 执行JobTracker分配的任务
- 分为Map和Reduce两类

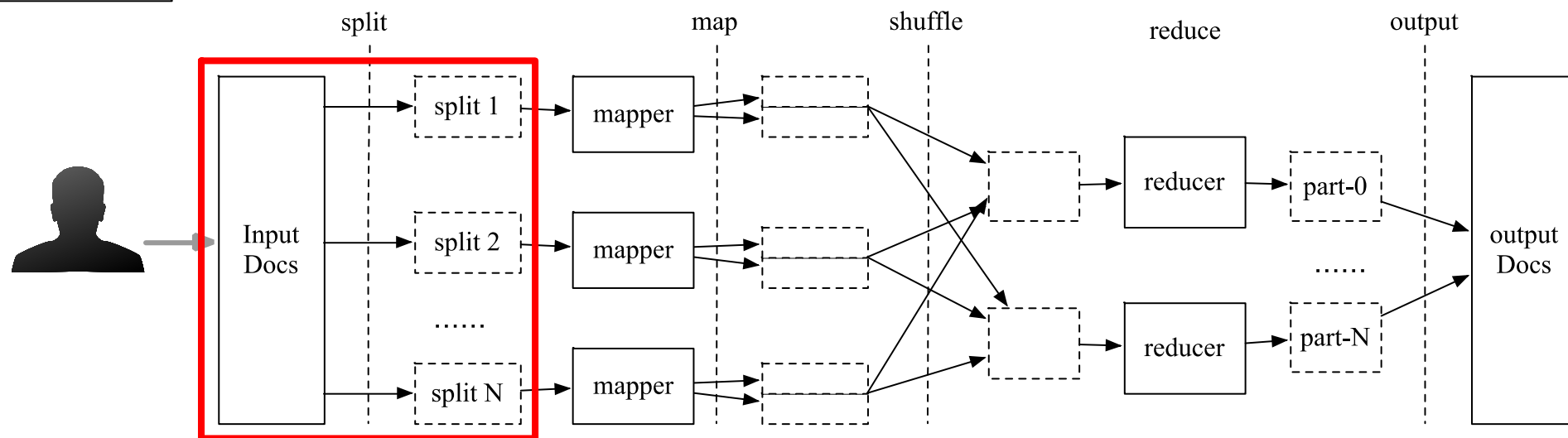
MapReduce运行流程-初始化

1. 作业启动

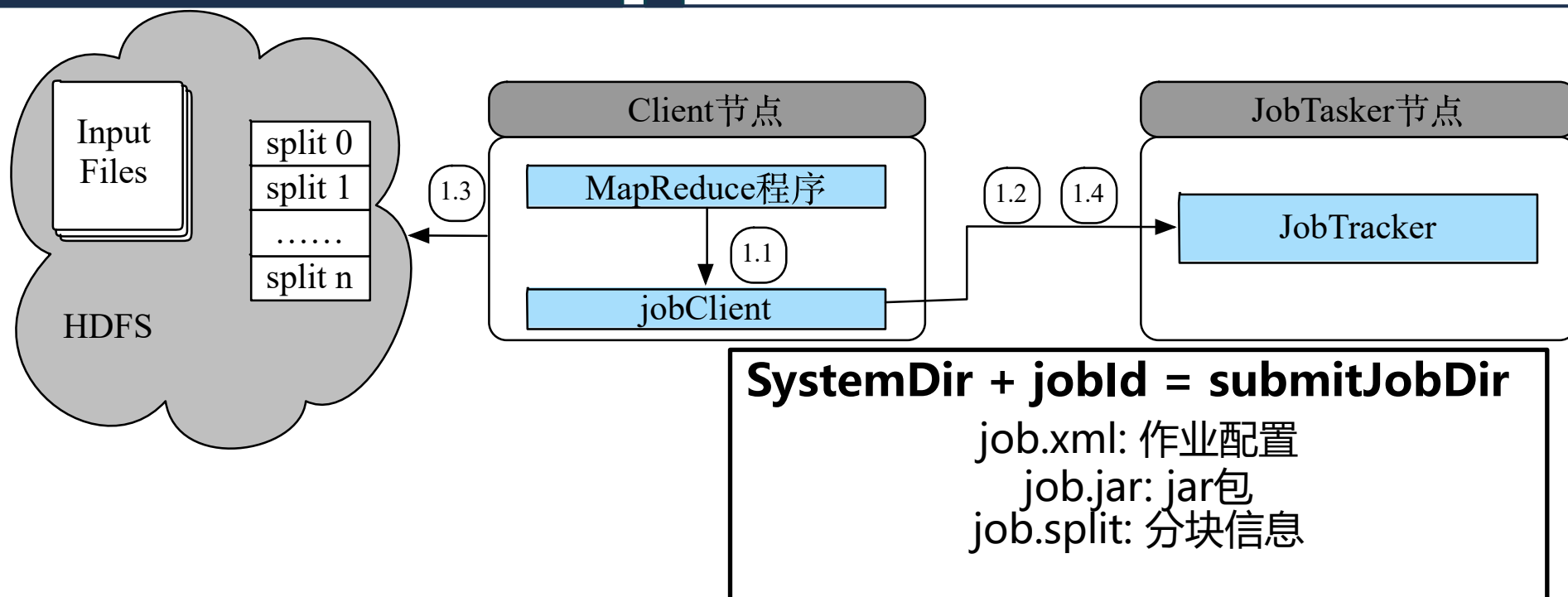


MapReduce运行流程-作业初始化

2. 作业初始化

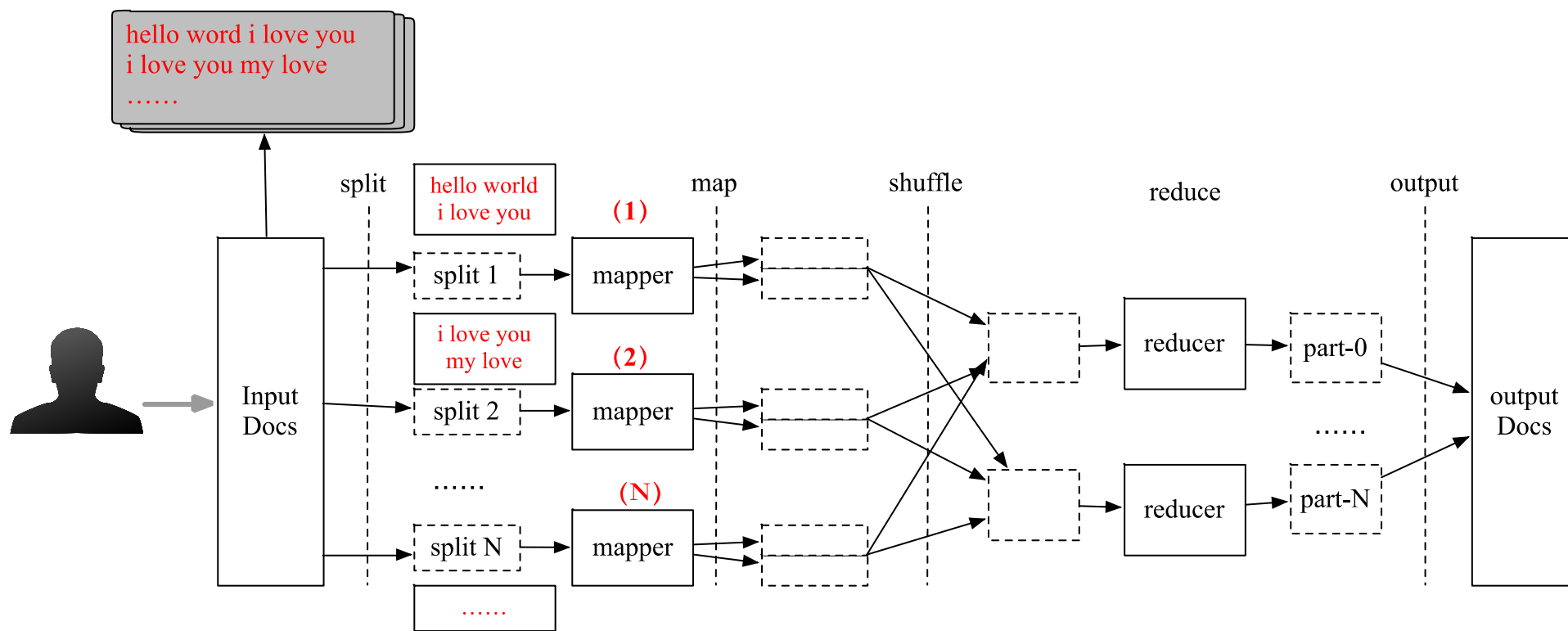


MapReduce运行流程-作业初始化



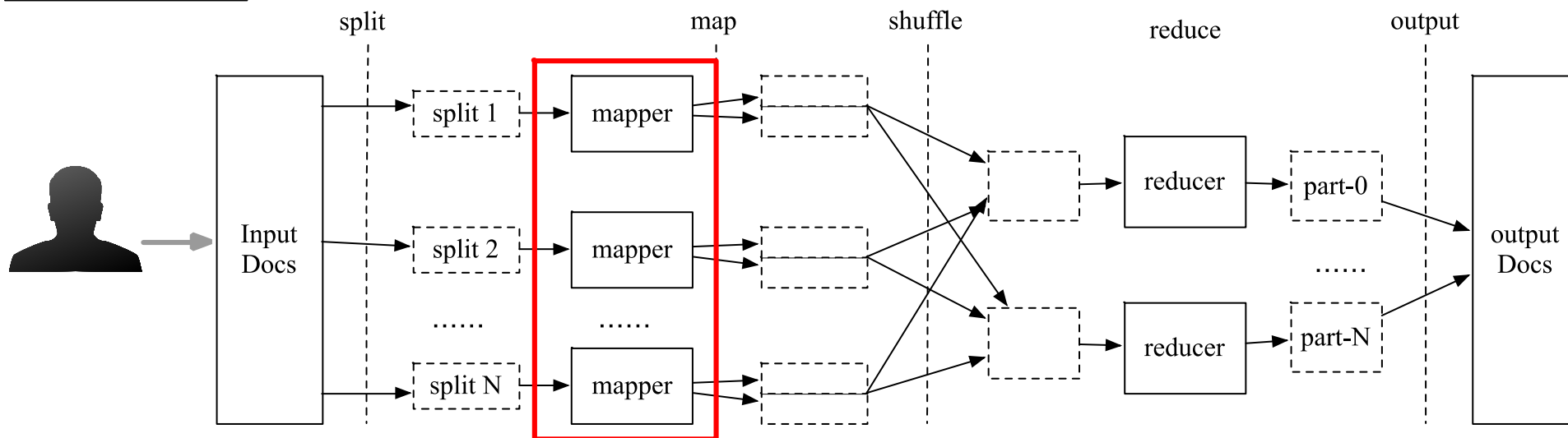
- 1.1** MapReduce程序创建新的JobClient实例
- 1.2** JobClient向JobTracker请求获得一个新的JobId标识本次作业
- 1.3** JobClient将运行作业需要的相关资源放入作业对应的HDFS目录、计算分片数量和map任务数量
- 1.4** 向JobTracker提交作业，并获得作业的状态对象句柄

作业初始化完成后

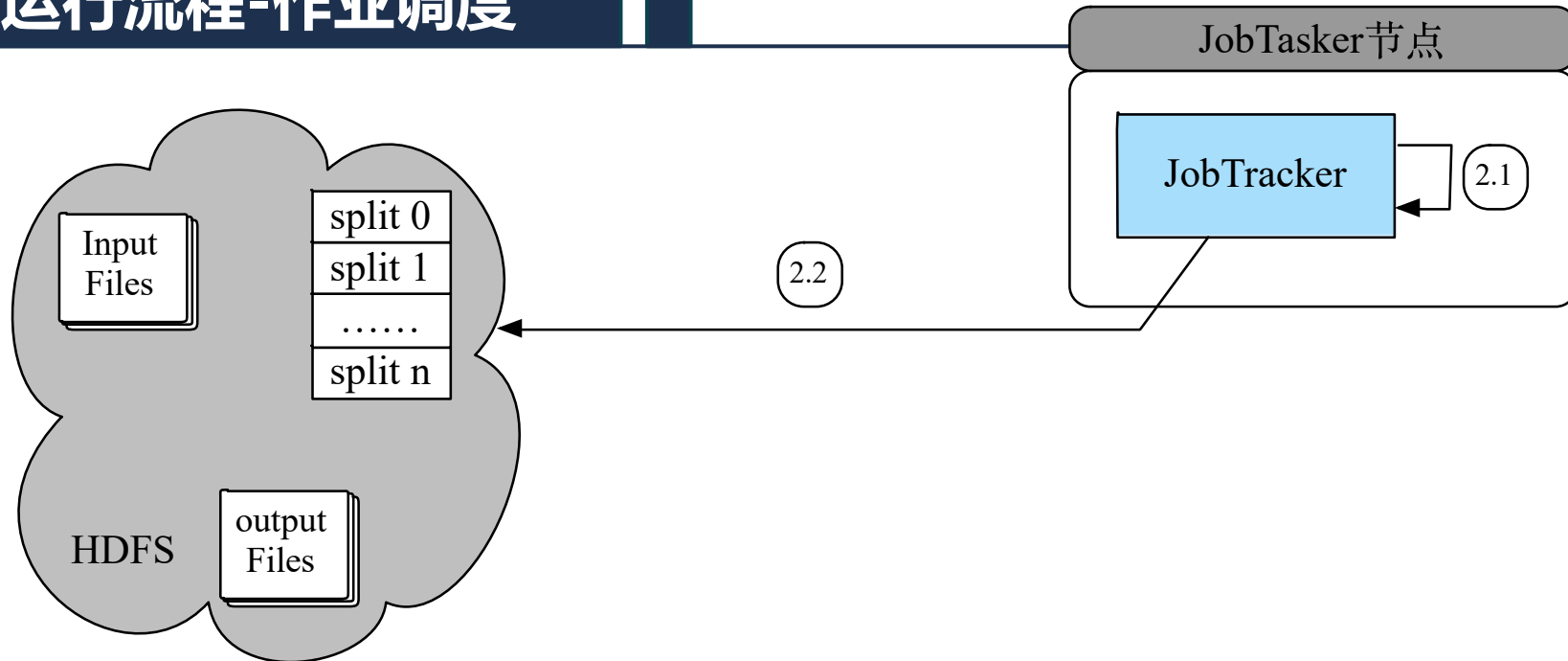


MapReduce运行流程-作业调度

3.作业/任务调度



MapReduce运行流程-作业调度

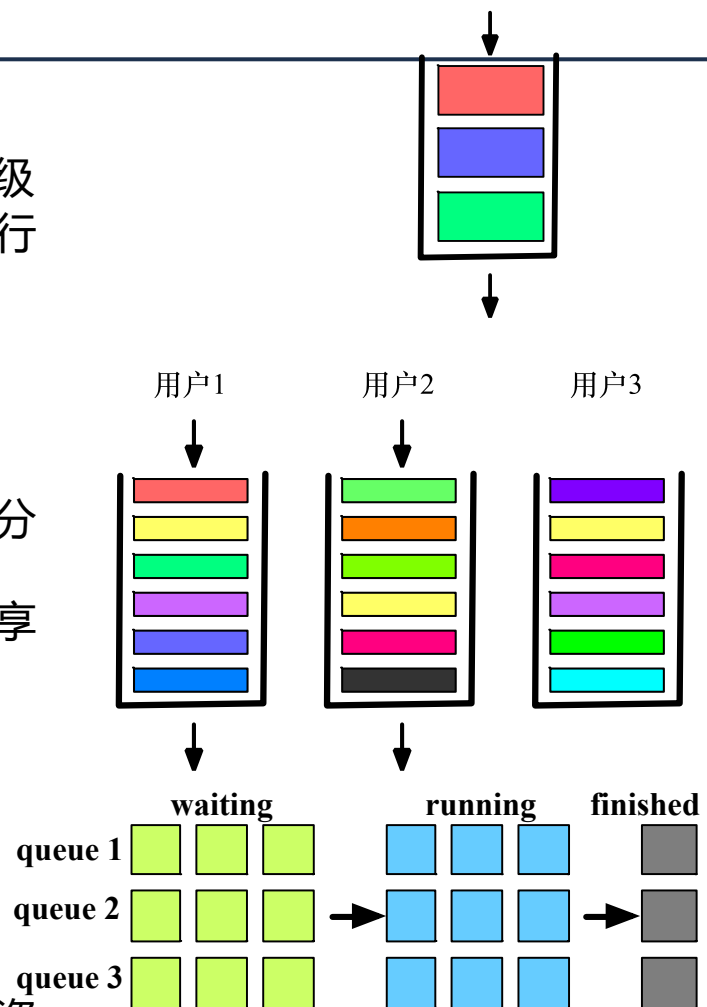


2.1 作业提交请求放入队列等待调度

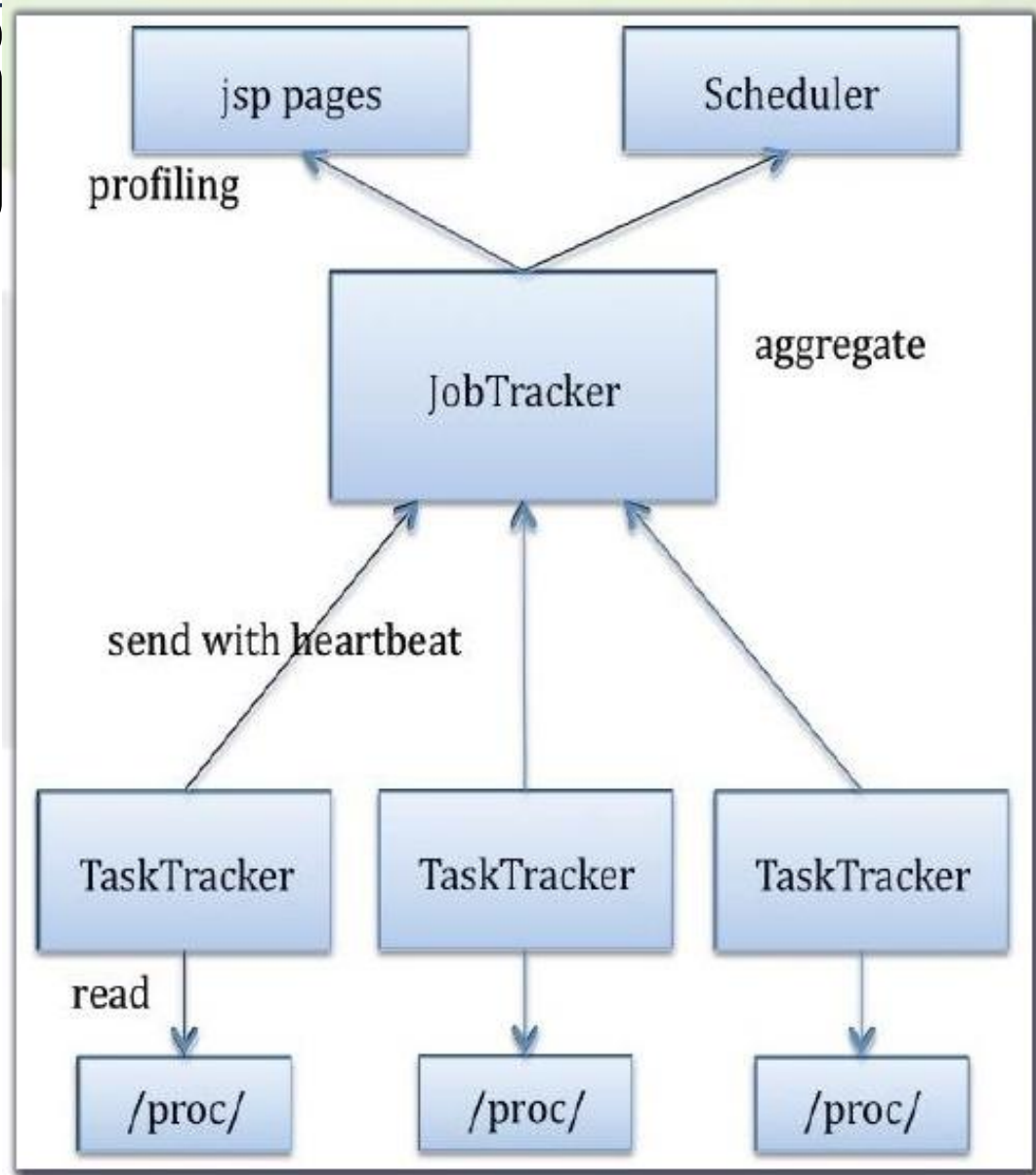
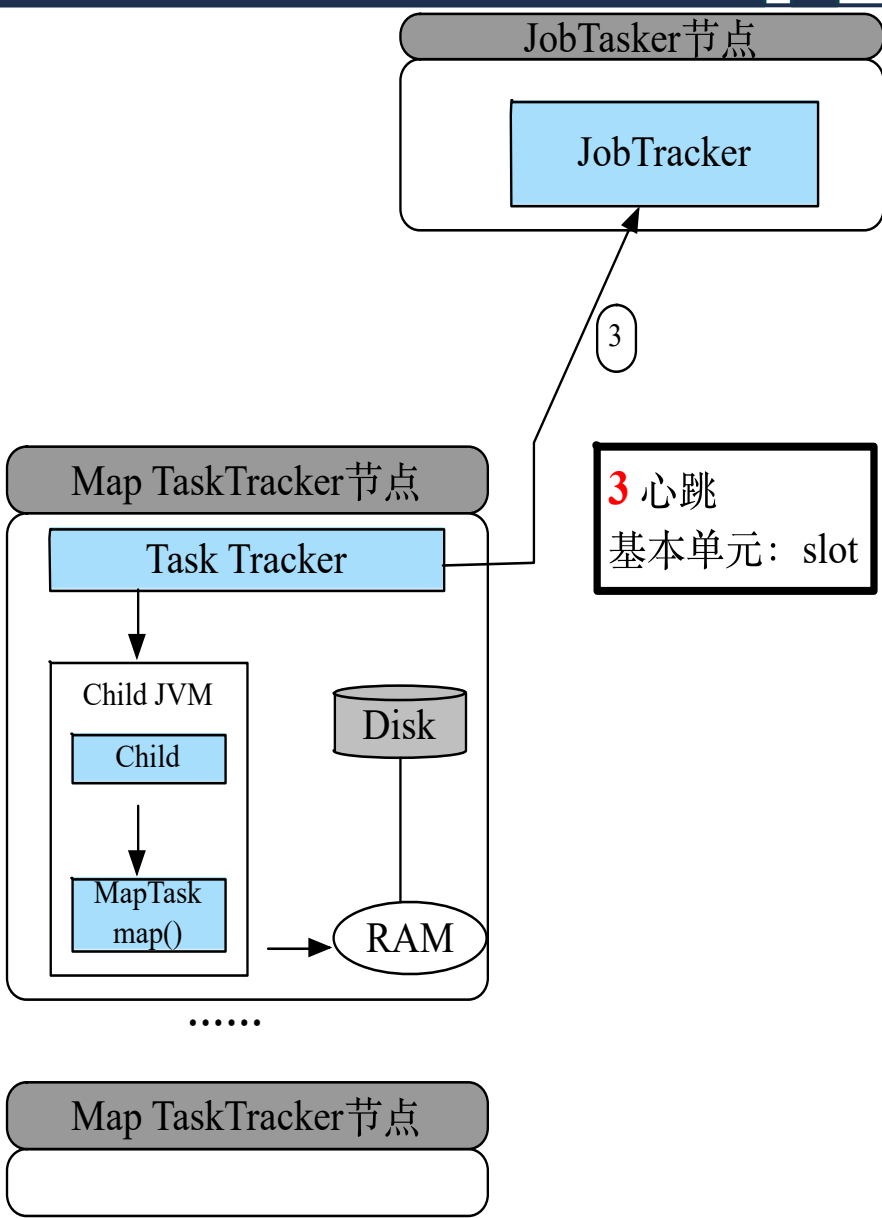
2.2 从HDFS中取出作业分片信息，创建对应数量的TaskInProgress调度和监控Map任务

MapReduce运行流程-作业调度

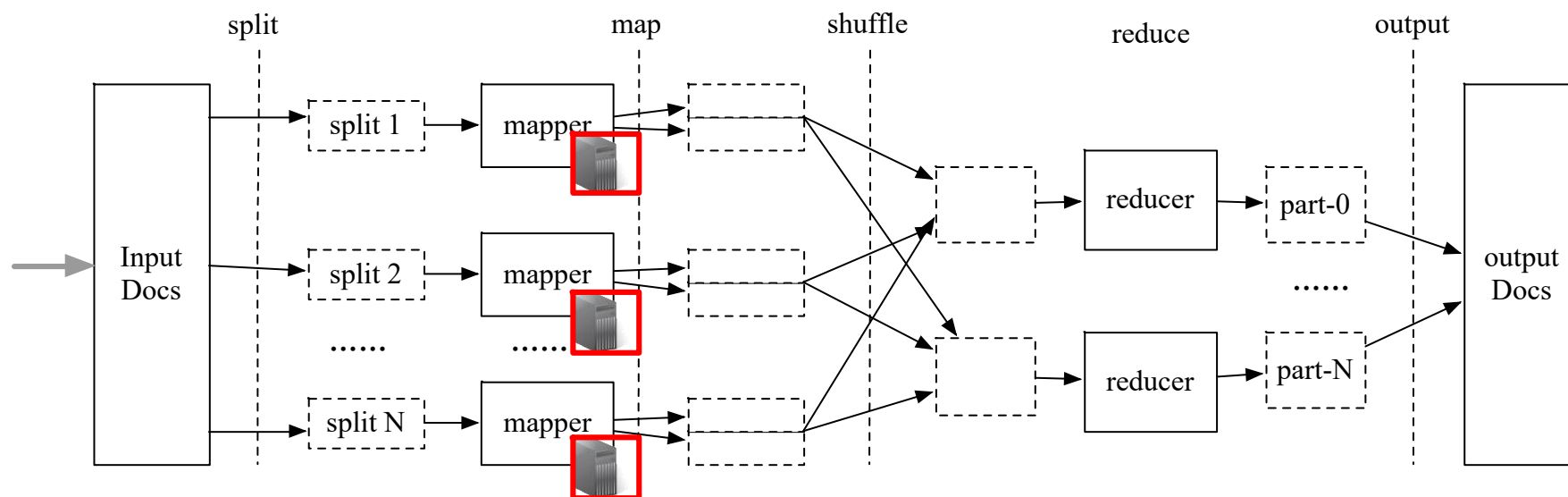
- 先进先出 (FIFO) 调度器 (默认)
 - ✧ VERY_HIGH, HIGH, NORMAL, LOW, VERY_LOW 五个等级
 - ✧ 先按优先级高低、再按作业到达时间、选择下一个被执行的作业
 - ✧ 不支持抢占
- 公平 (Fair) 调度器
 - ✧ 用户公平享用计算能力，不被独占
 - ✧ 每个用户一个资源池，计算资源按照用户设定被公平的分配到这些资源池中
 - ✧ 每个用户可以提交多个作业，会按照公平共享的方式分享该用户占有的资源池
 - ✧ 无作业的资源也可以被其他用户共享
 - ✧ 支持资源抢占
- 能力 (Capacity) 调度器
 - ✧ 采用分层次多队列的形式组织计算资源
 - ✧ 队列内采用优先级的FIFO调度方式
 - ✧ 计算每个队列中正在运行的任务数与其应该分得的计算资源之间的比值，计算一个比值最小的队列，然后按FIFO从队列中选择一个作业执行
 - ✧ 对同一用户提交的作业所占资源总量进行限制



任务分配



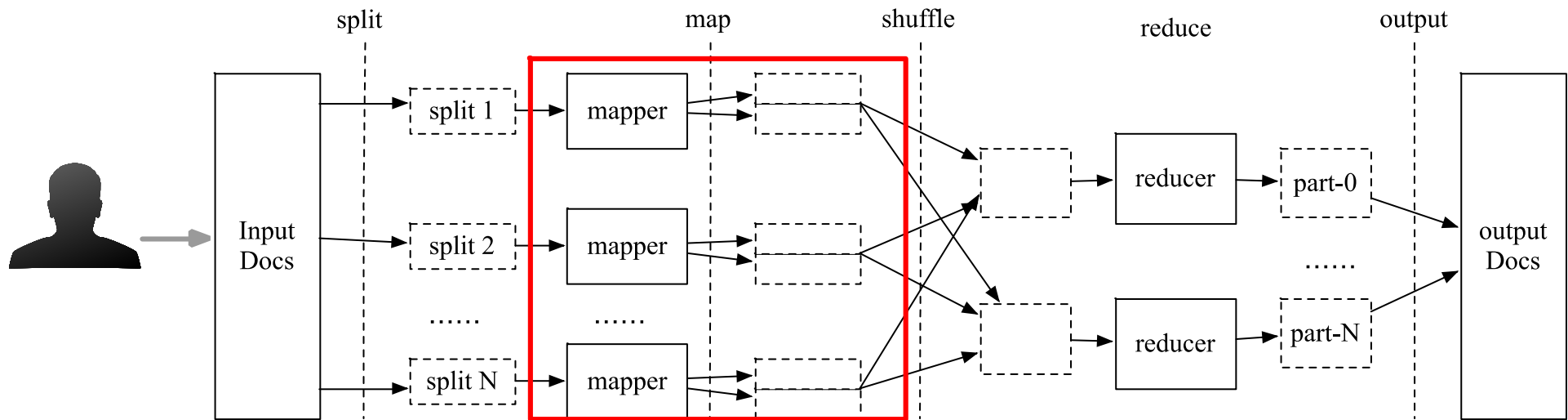
任务分配完成后



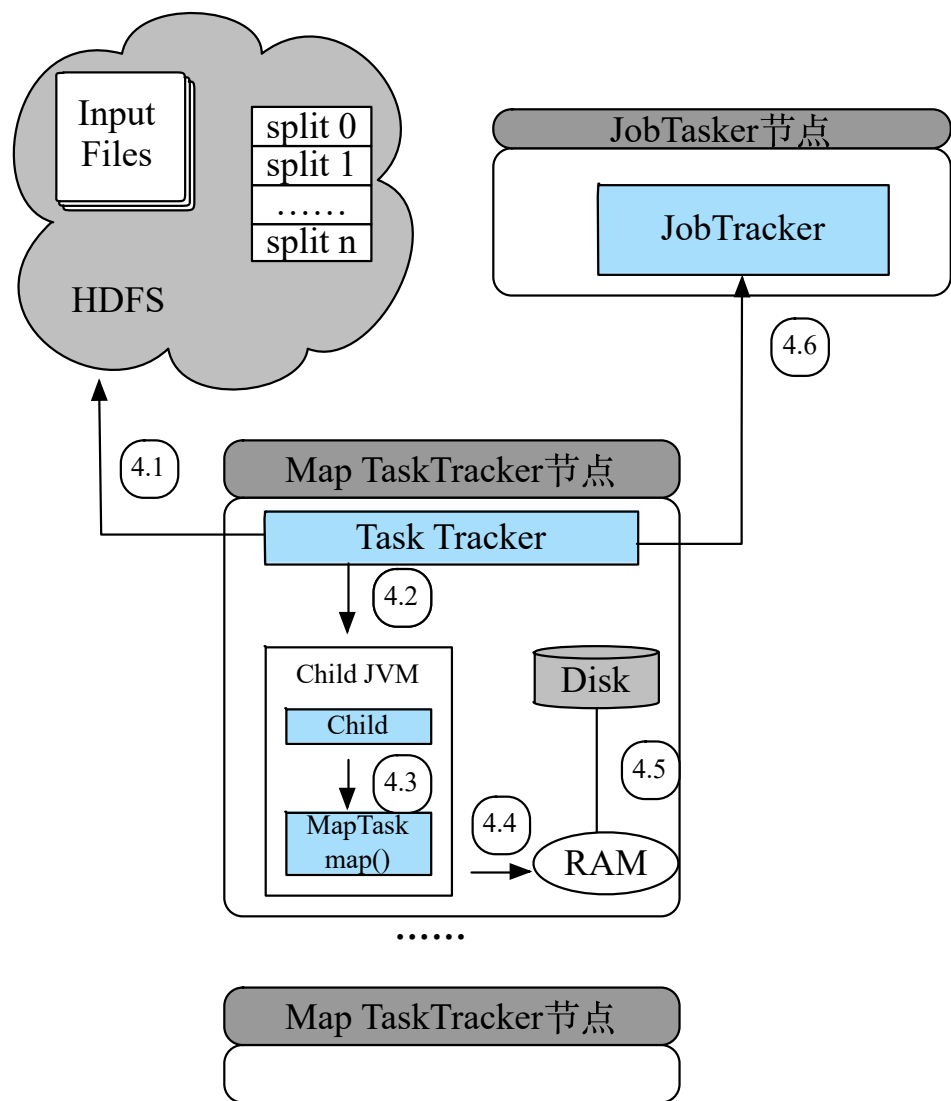
MapReduce运行流程-Map任务执行

```
public static class TokenizerMapper extends Mapper
<Object, Text, Text, IntWritable>{
    private final static IntWritable one = new IntWritable(1);
    private Text word = new Text();
    public void map(Object key, Text value, Context
context) throws IOException, InterruptedException{
        StringTokenizer itr = new StringTokenizer(value.toString());
        while (itr.hasMoreTokens()) {
            word.set(itr.nextToken());
            context.write(word, one);
        }
    }
}
```

4.Map任务执行



MapReduce运行流程-Map任务执行



4.1 从HDFS提取相关资源 (Jar包、数据)

4.2 创建TaskRunner运行Map任务

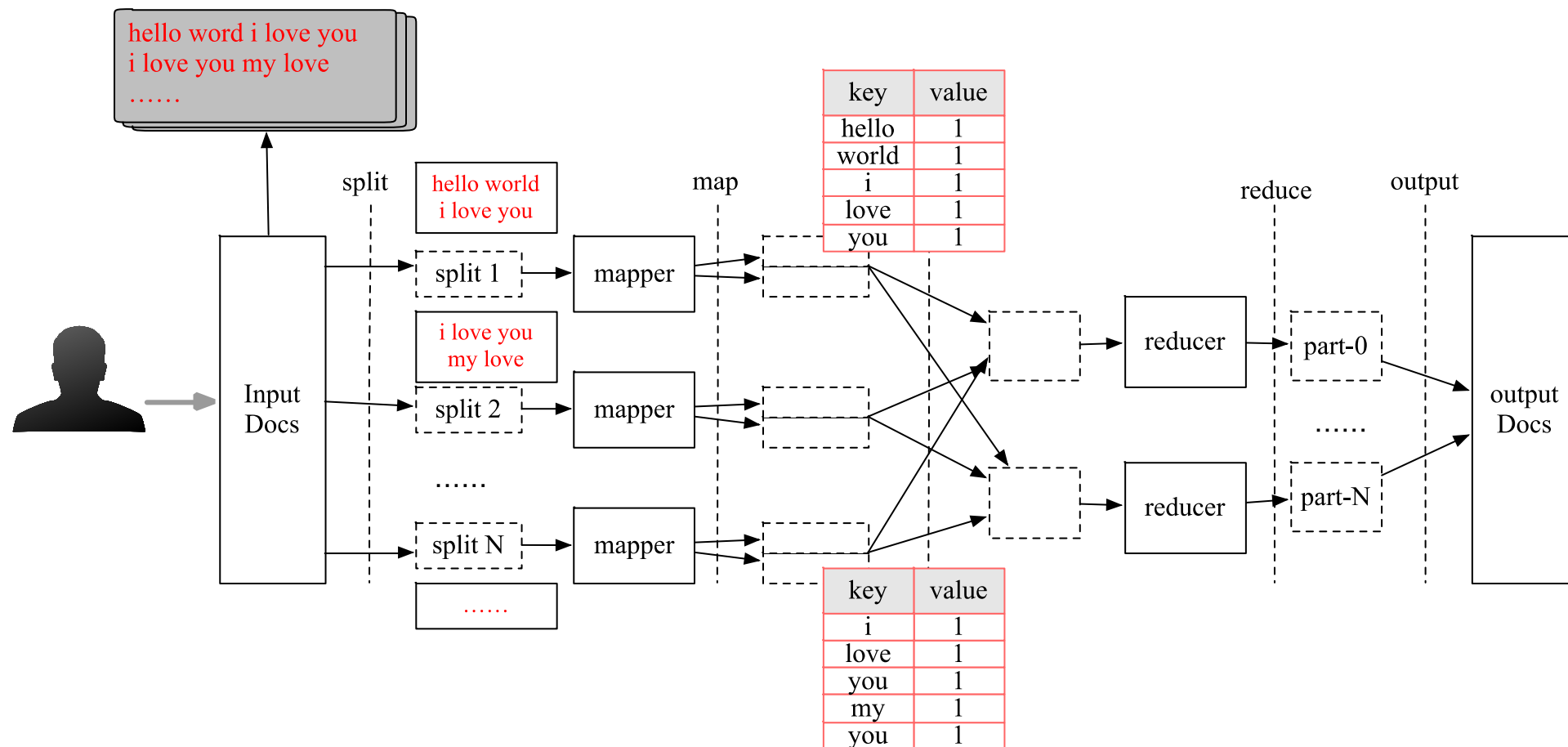
4.3 在单独的JVM中启用MapTask执行map函数

4.4 中间结果数据定期存入缓存

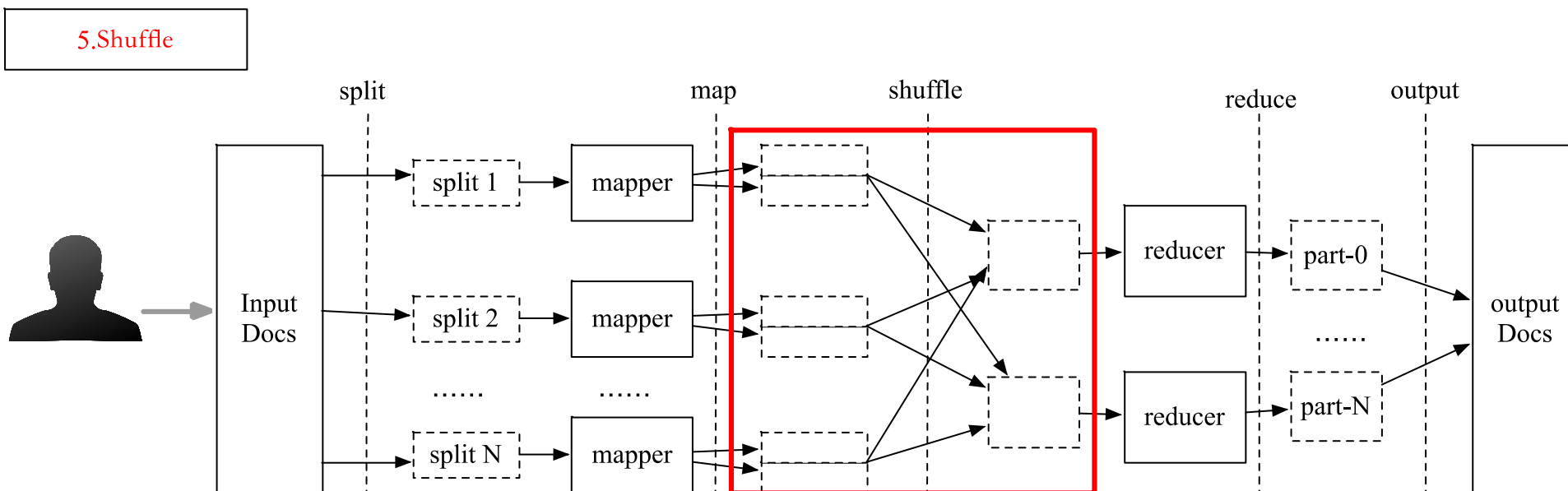
4.5 缓存写入磁盘

4.6 定期报告进度

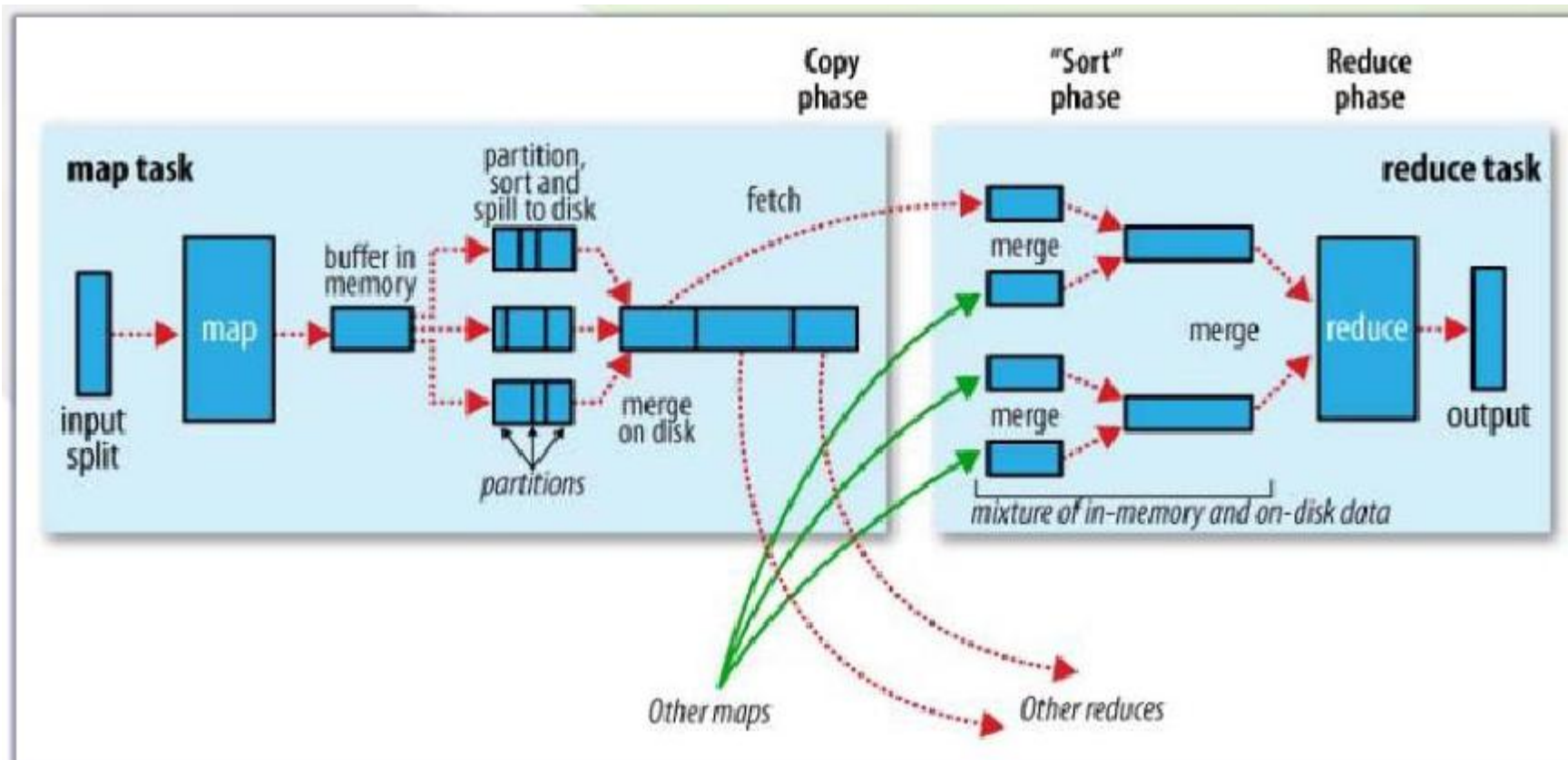
Map任务执行完毕后



MapReduce运行流程-Shuffle

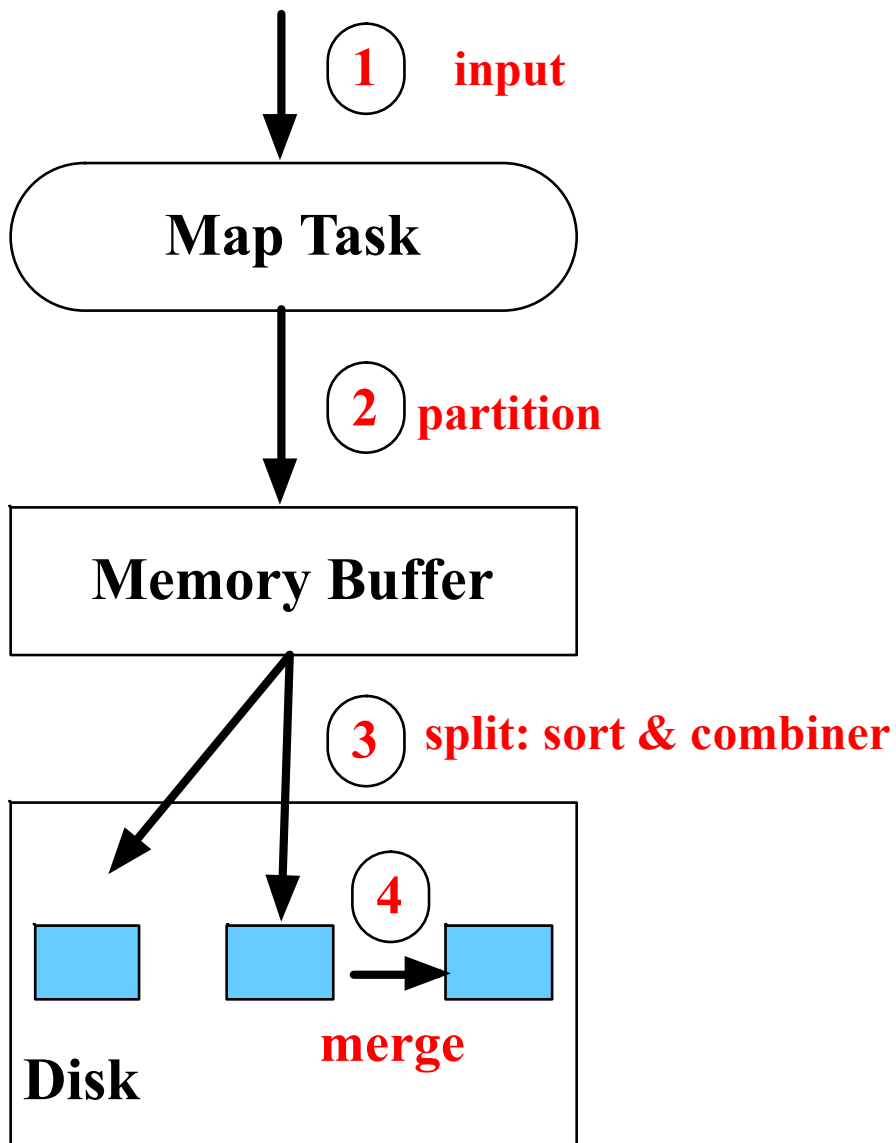


MapReduce运行流程-Shuffle



Map端保存、Partition、Sort、Combine、Reduce端拉取

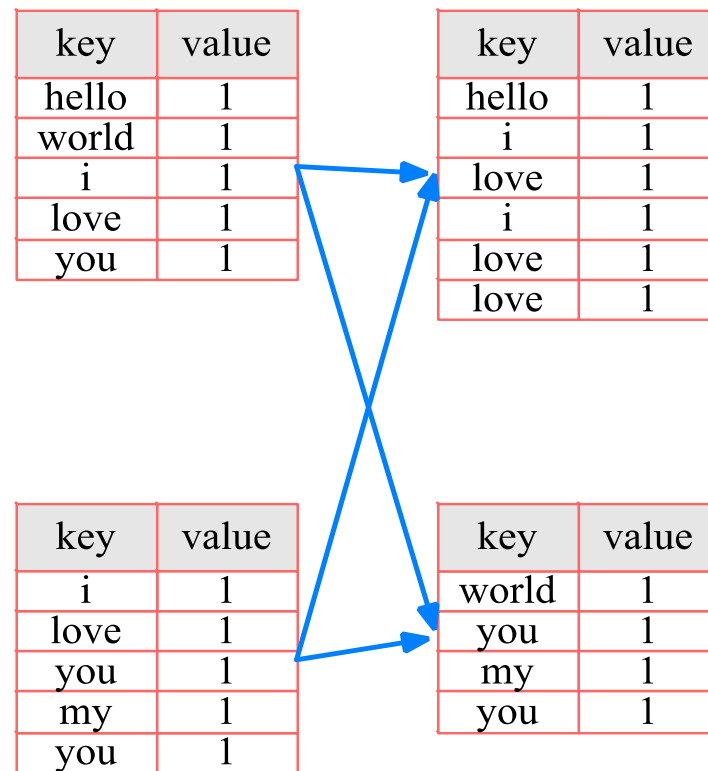
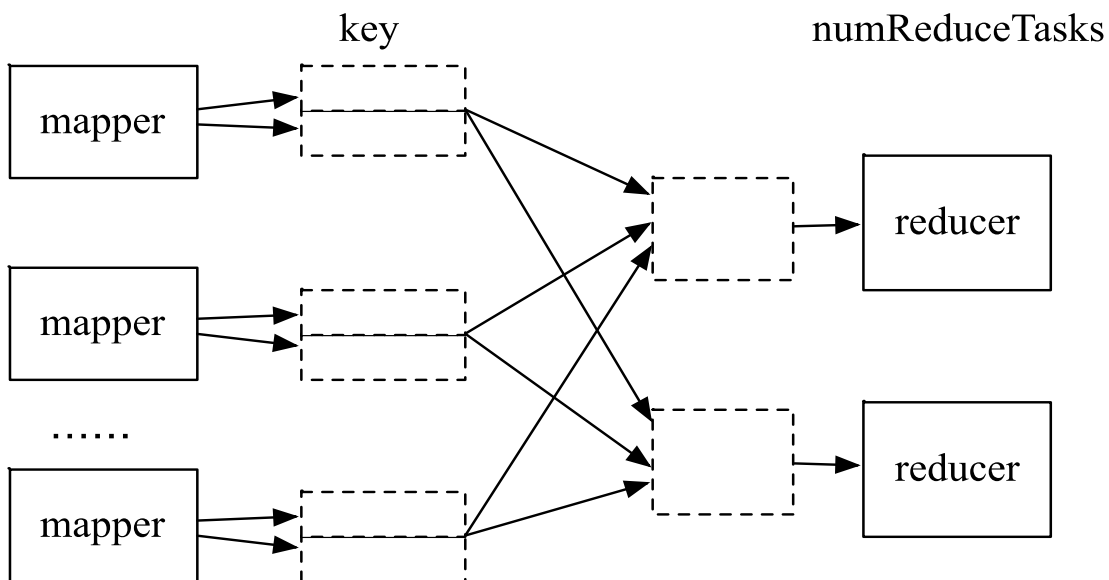
Map端结果保存



- ① 处理输入数据
- ② 寻找对应Reduce (Partition)
- ③ 内存数据溢出到磁盘 (Spill)
 - Sort
 - Combiner
- ④ 合并中间结果文件 (Merge)

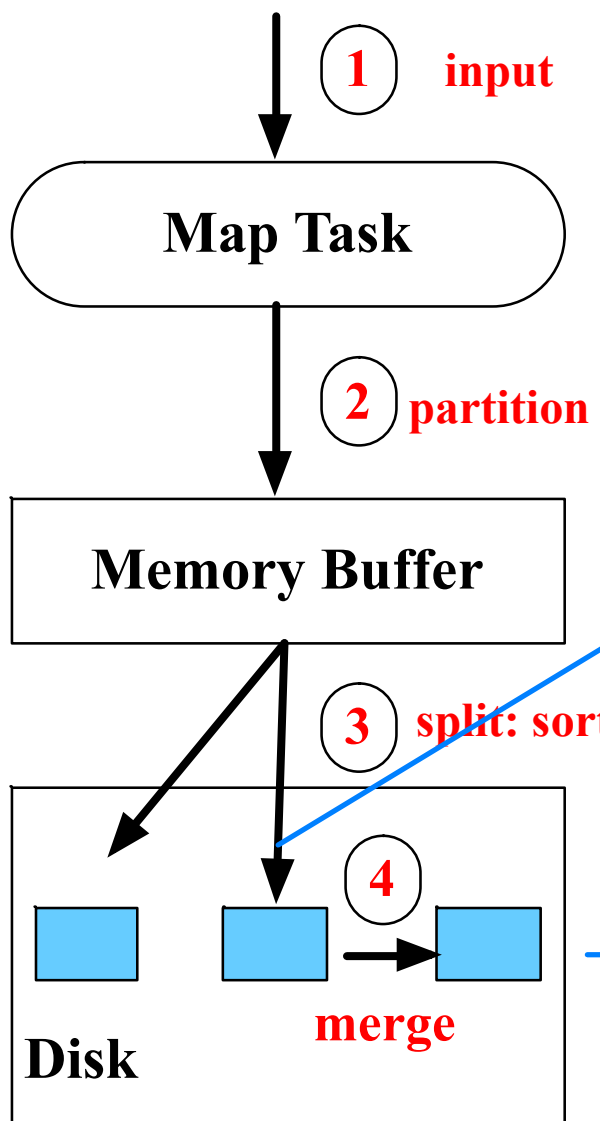
Partition

- 作用：将map的结果发送到reduce
- 要求：负载均衡、效率
- 默认HashPartitioner(key.hashCode() & Integer.MAX_VALUE)%numReduceTasks



- 自定义： `job.setPartitionerClass(MyPartitioner.class);`

Sort



Map后的第1次排序：文件内部快速排序（Sort）

- map函数处理完输入数据之后，会将中间数据存在本机的一个或者几个文件当中，并且针对这些文件内部的记录进行一次快速排序

key	value
i	1
love	1
you	1
my	1
love	1

key	value
i	1
my	1
love	1
you	1
love	1

Map后的第2次排序：多个文件归并排序（Merge）

- Map任务执行完成后会对这些排好序的文件做一次归并排序，并将排好序的结果输出到一个大的文件中

...

key	value
i	1
love	1
you	1
my	1
love	1

key	value
i	1
my	1
love	1
you	1
love	1

Combiner

- 作用：合并Map输出的中间数据，减少数据传输、提高处理效率

key	value
i	1
love	1
you	1
my	1
love	1

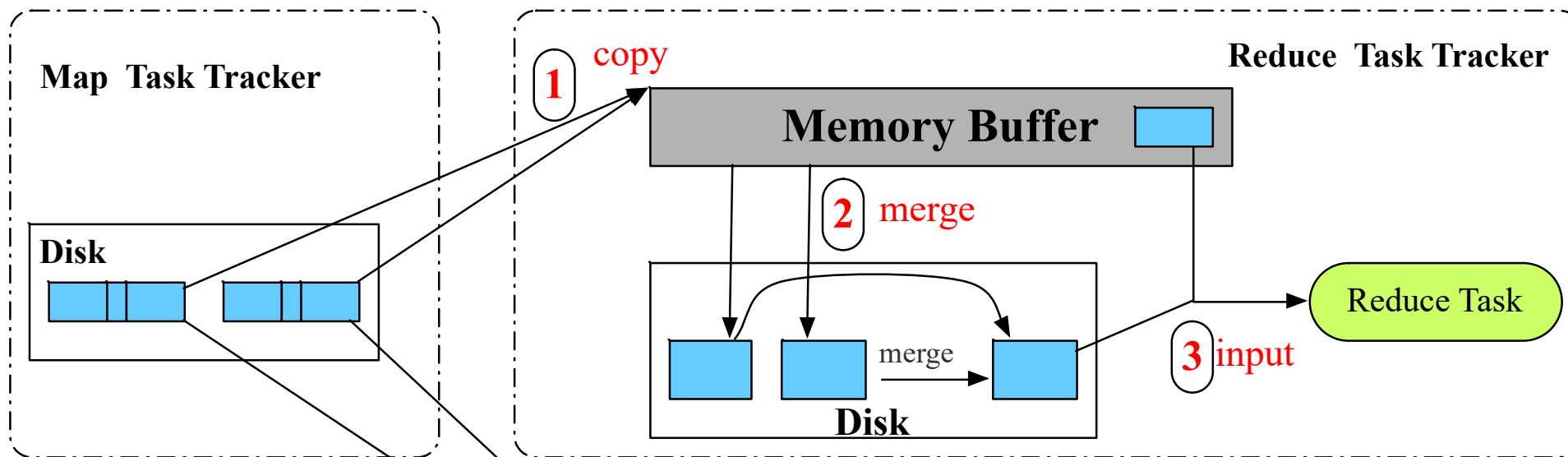


key	value
i	1
my	1
love	2
you	1

```
public static void main(String[] args) throws  
Exception{  
    ...  
    job.setCombinerClass(IntSumReducer.class);  
    ...  
}
```

```
public static class IntSumReducer extends Reducer  
<Text,IntWritable,Text,IntWritable>{  
    private IntWritable result = new IntWritable();  
    public void reduce(Text key, Iterable<IntWritable>  
        values, Context context) throws IOException{  
        int sum = 0;  
        for (IntWritable val : values) {  
            sum += val.get();  
        }  
        result.set(sum);  
        context.write(key, result);  
    }  
}
```

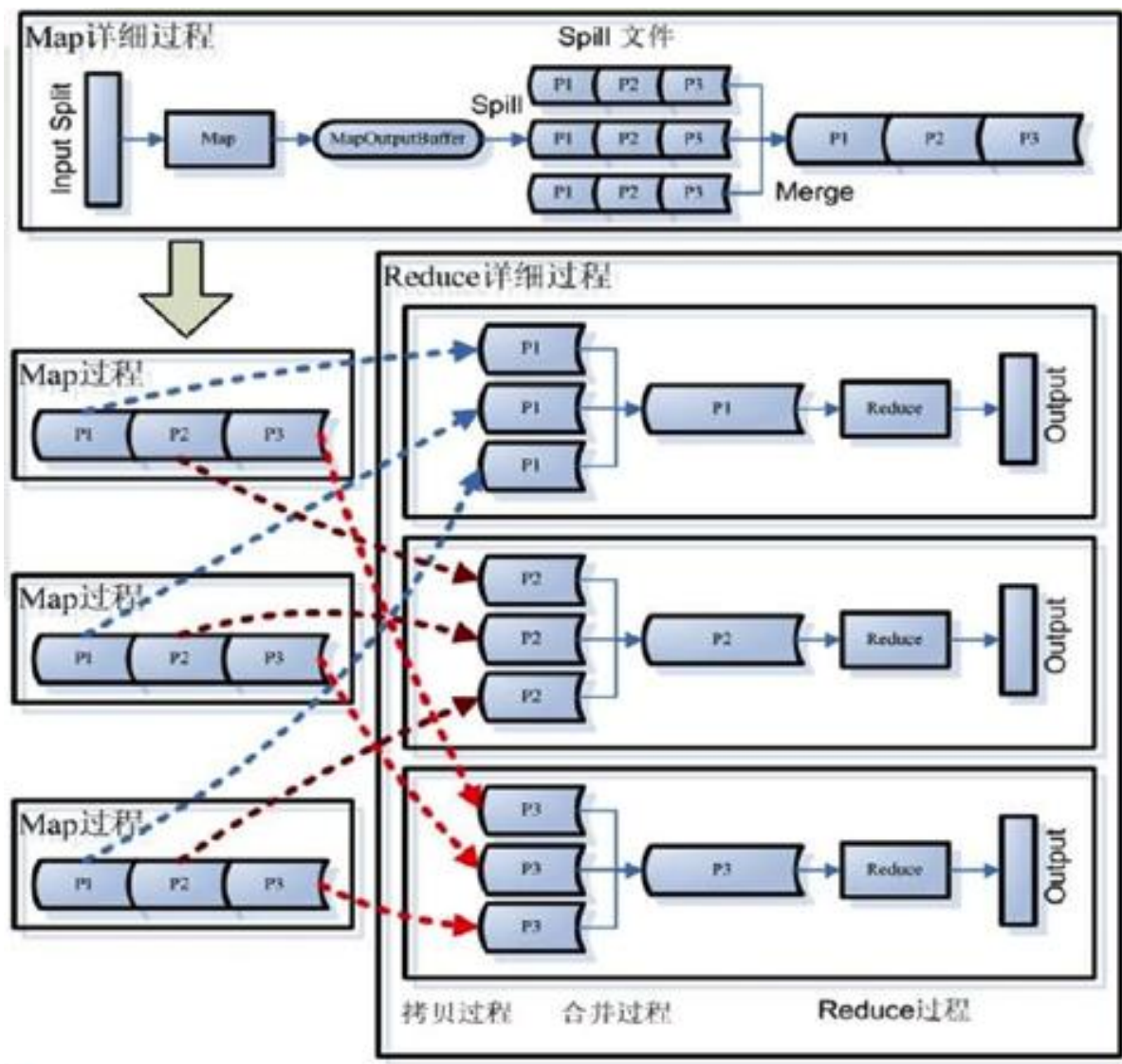

Reduce端拉取数据



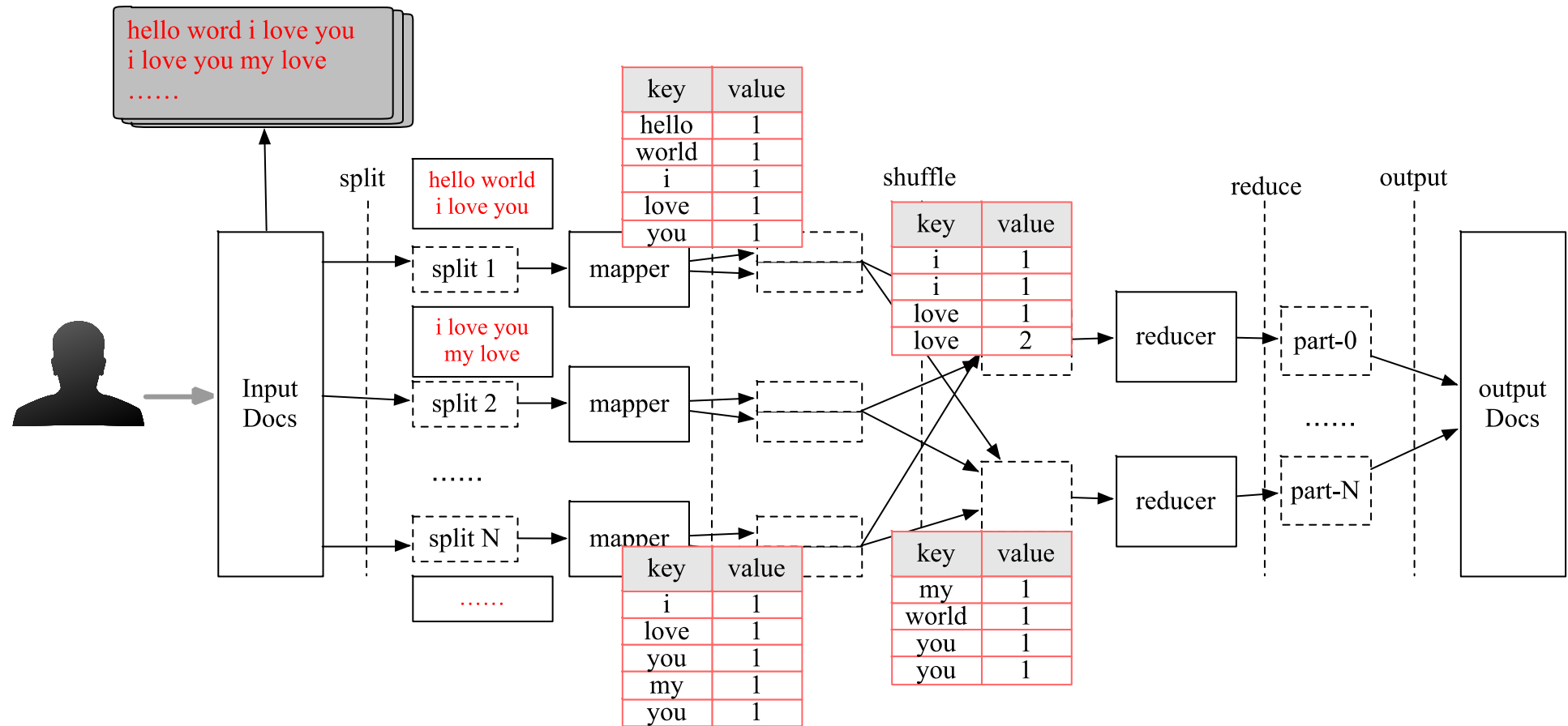
- ① 拉取数据 (Copy)
- ② 合并中间结果文件 (Merge)
- ③ 处理数据

other reduces

Shuffle总结



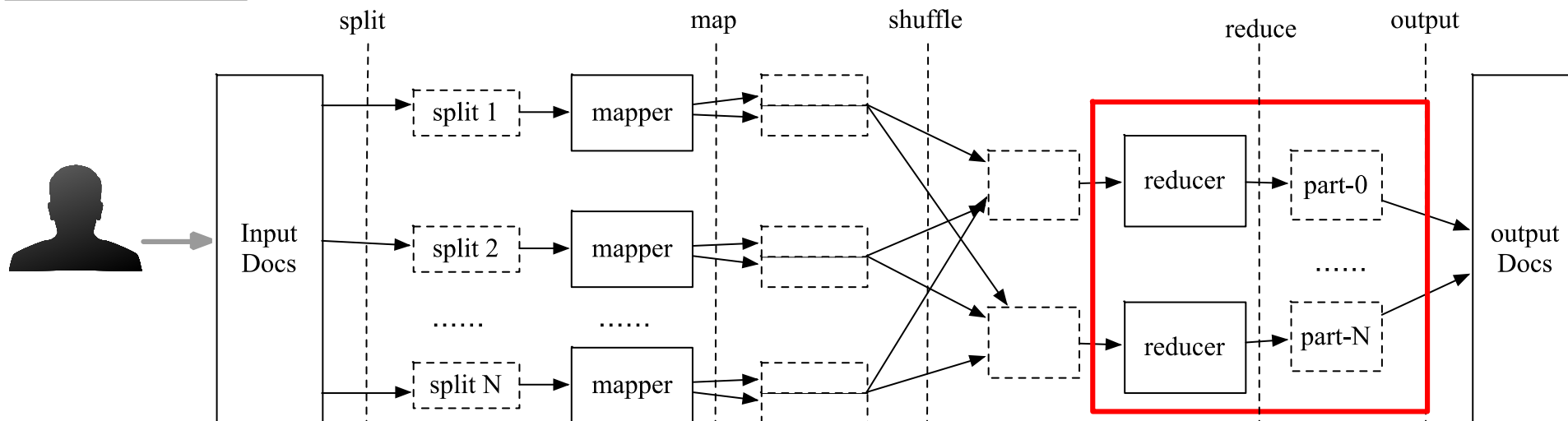
Shuffle完成后



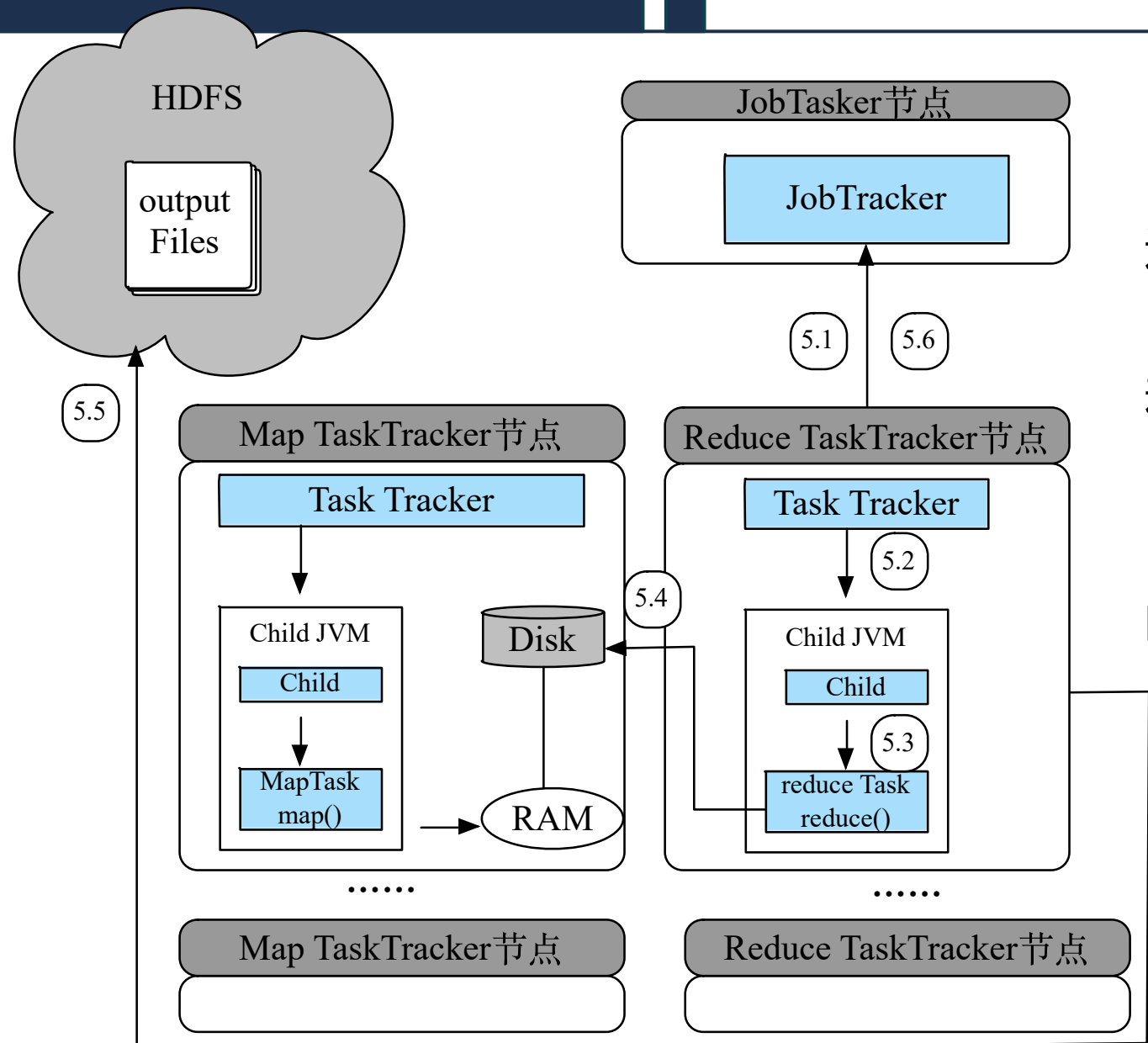
MapReduce运行流程-Reduce任务执行

```
public static class IntSumReducer extends Reducer
<Text,IntWritable,Text,IntWritable>{
    private IntWritable result = new IntWritable();
    public void reduce(Text key, Iterable<IntWritable>
    values, Context context) throws IOException{
        int sum = 0;
        for (IntWritable val : values) {
            sum += val.get();
        }
        result.set(sum);
        context.write(key, result);
    }
}
```

6.Reduce任务执行



MapReduce运行流程-Reduce任务执行



5.1 分配Reduce任务

5.2 创建TaskRunner

运行Reduce任务

5.3 在单独的JVM中启动ReduceTask执行

reduce函数

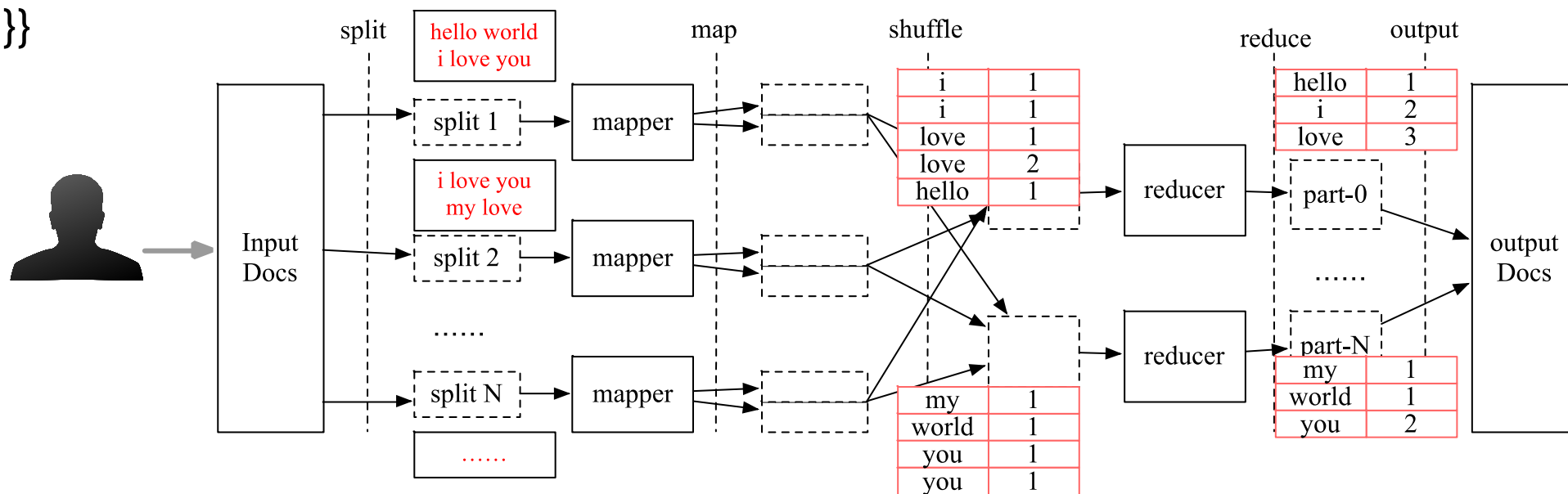
5.4 从Map节点下载中间结果数据

5.5 输出结果临时文件

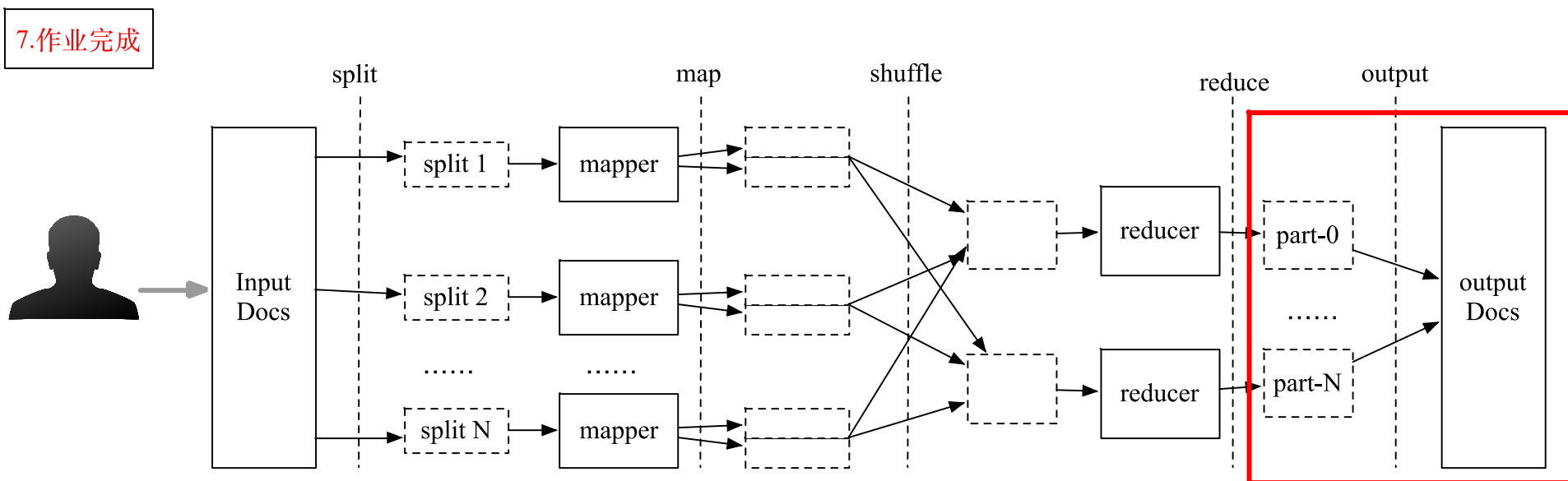
5.6 定期报告进度

Reduce任务执行完成后

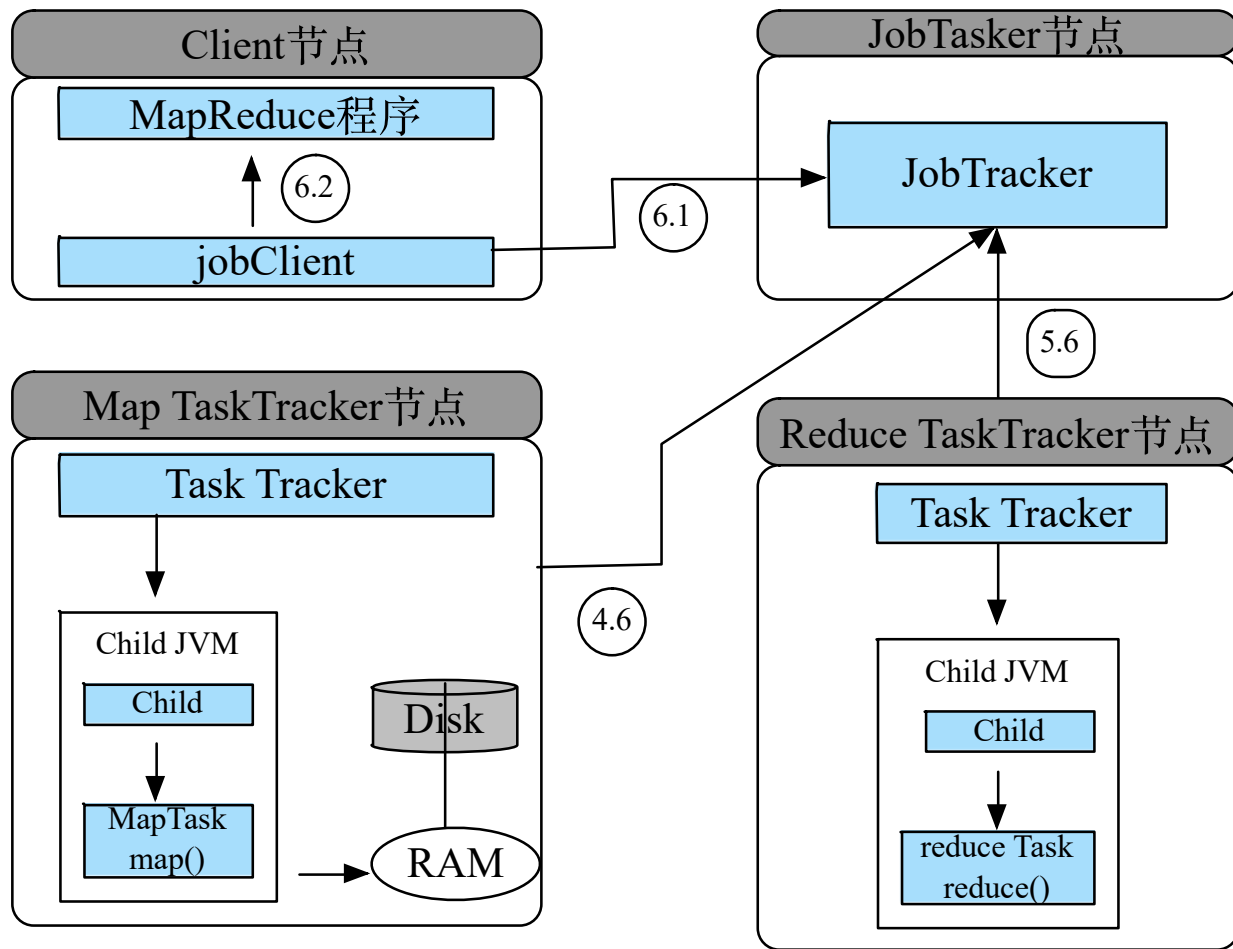
```
public void reduce(Text key, Iterable<IntWritable>
    values, Context context) throws IOException{
    int sum = 0;
    for (IntWritable val : values) {
        sum += val.get();
    }
    result.set(sum);
    context.write(key, result);
}
```



MapReduce运行流程-作业完成

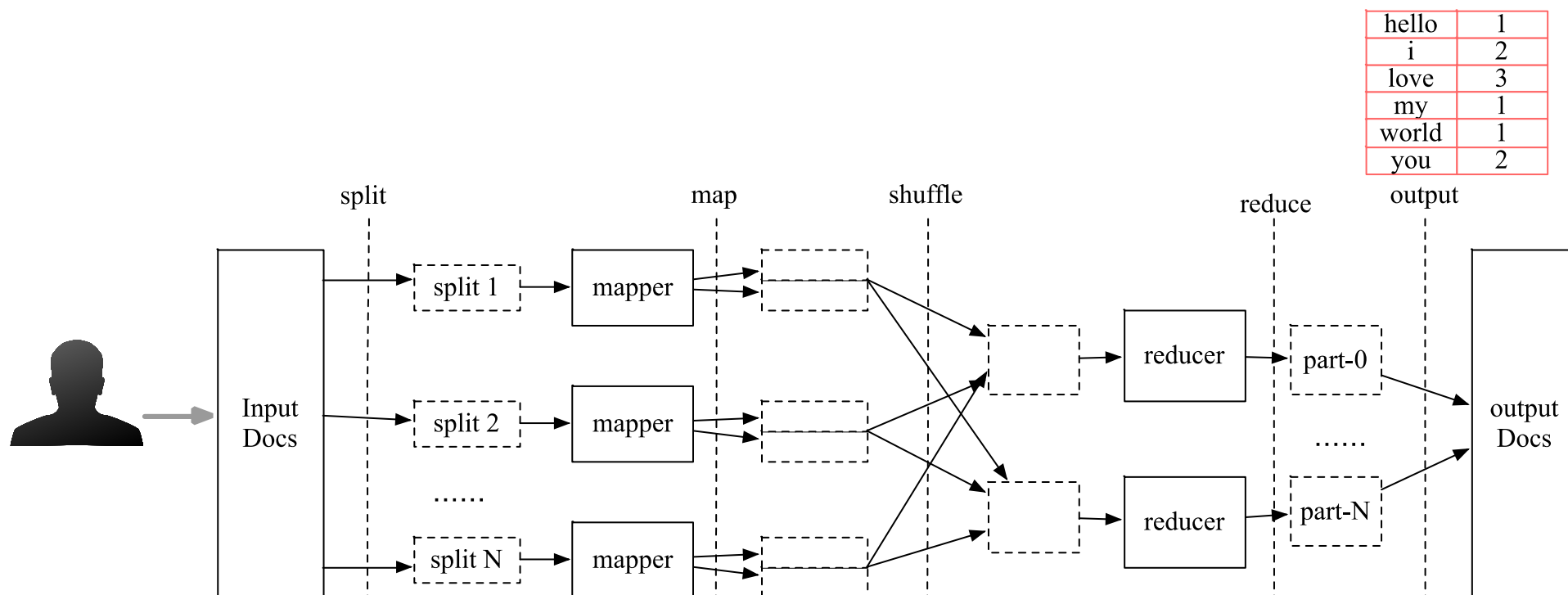


MapReduce运行流程-作业完成



4.6/5.6 进度更新
6.1 JobClient轮询
获知任务完成
6.2 通知用户

作业完成后



计算容错: TaskTracker

● 心跳监测

- `mapred.tasktracker.expiry.interval`, 默认10分钟
- 已完成的任务会正常返回, 未完成的任务则重新分配TaskTracker节点执行

● 黑名单机制

- Job黑名单: 每个Job会维护一个TaskTracker黑名单
- 集群黑名单: 整个集群的TaskTracker黑名单
 - ✓ 当一个job成功结束时, 对该Job黑名单中的tasktracker做如下三个判断
 - 该TaskTracker被4个Job加入了黑名单 (`mapred.max.tracker.blacklists`)
 - 该TaskTracker被加入Job黑名单的次数, 超过了集群中所有tasktracker被加入Job黑名单平均次数的50% (`mapred.cluster.average.blacklist.threshold`)
 - 已经加入黑名单的TaskTracker个数不超过集群总TaskTracker的50%
 - ✓ 以上三条如果均满足, 则将Job黑名单中的该TaskTracker加入集群黑名单, 以后将不在该TaskTracker上调度任何task
- 恢复: 重启TaskTracker

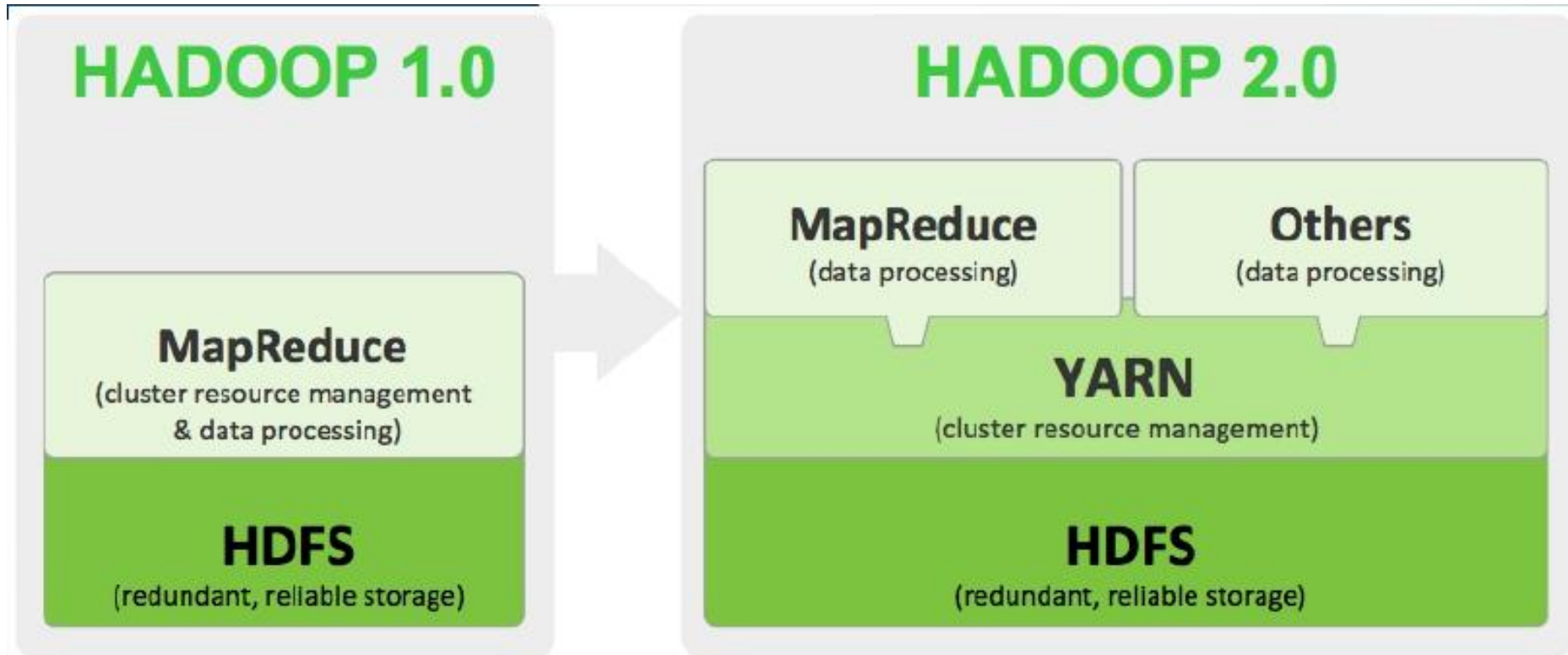
- 主节点失效

主节点中会周期性地设置检查点(checkpoint)，检查整个计算作业的执行情况，一旦某个任务失效，可以从最近有效的检查点开始重新执行，避免从头开始计算的时间浪费。

- 工作节点失效

工作节点失效是很普遍发生的，主节点会周期性地给工作节点发送检测命令，如果工作节点没有回应，则认为该工作节点失效，主节点将终止该工作节点的任务并把失效的任务重新调度到其它工作节点上重新执行。

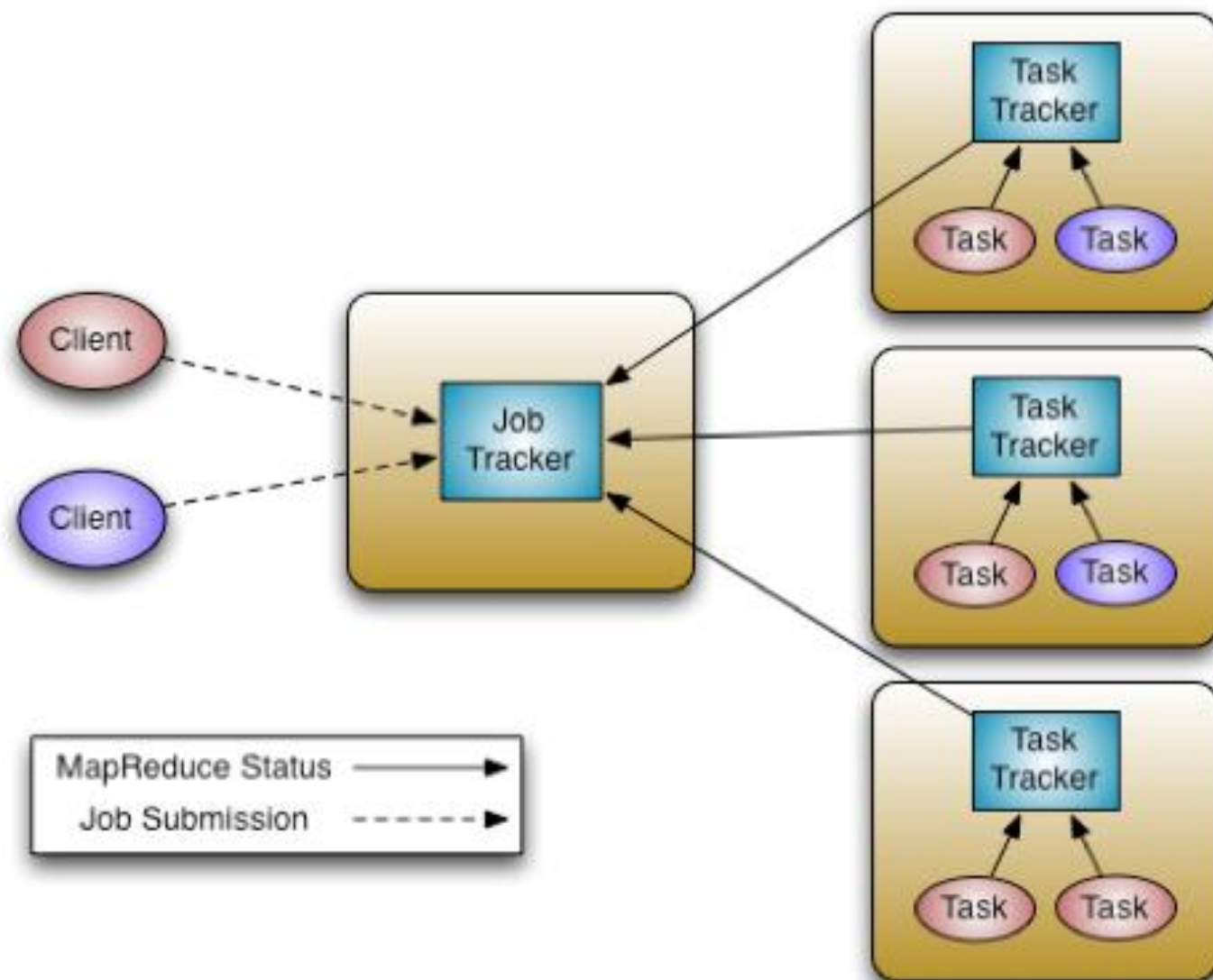
Hadoop1.0 VS Hadoop2.0



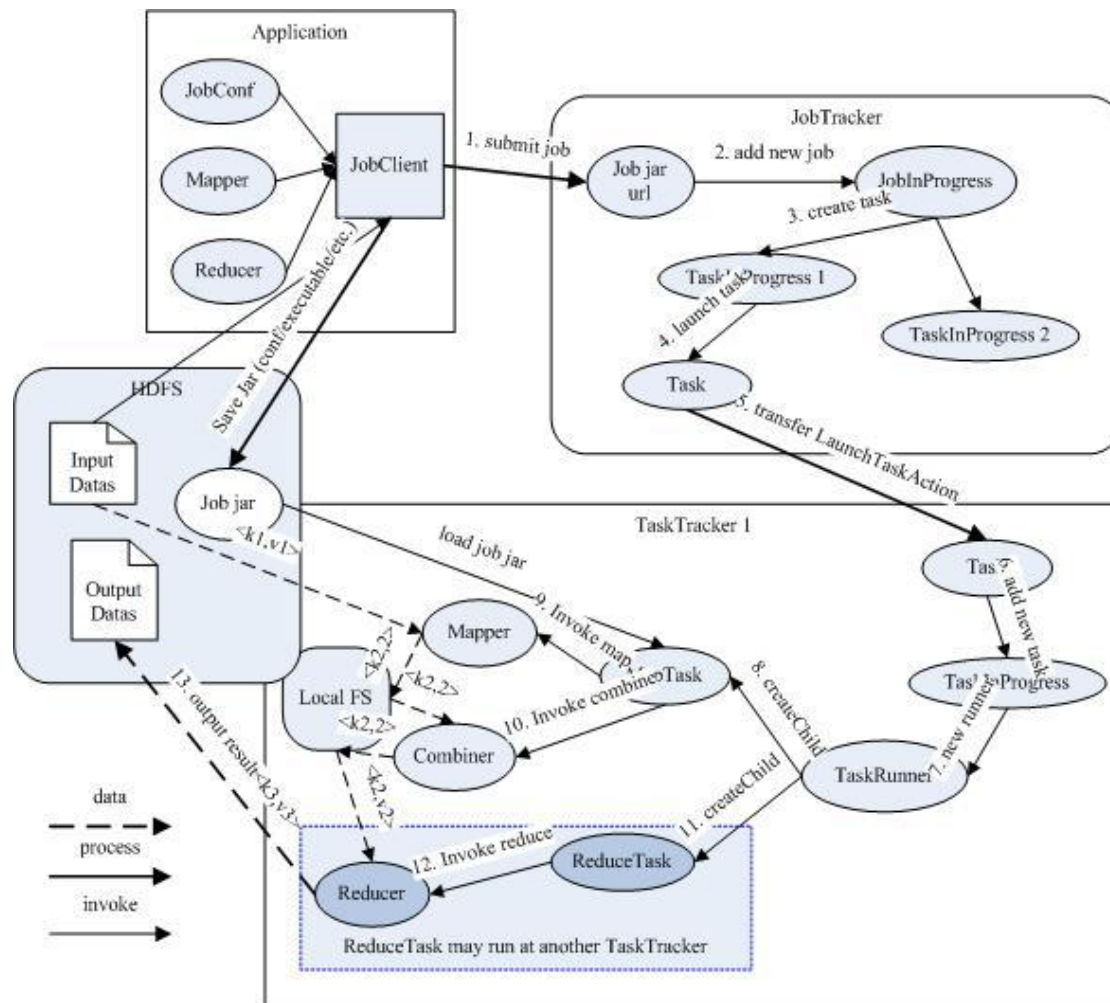
产生背景

- 直接源于**MRv1**在几个方面的缺陷
 - 扩展性受限
 - 单点故障
 - 难以支持**MR**之外的计算
- 多计算框架各自为战，数据共享困难
 - MR**：离线计算框架
 - Storm**：实时计算框架
 - Spark**：内存计算框架

Hadoop1.0 MR调度



Hadoop1.0 MR调度

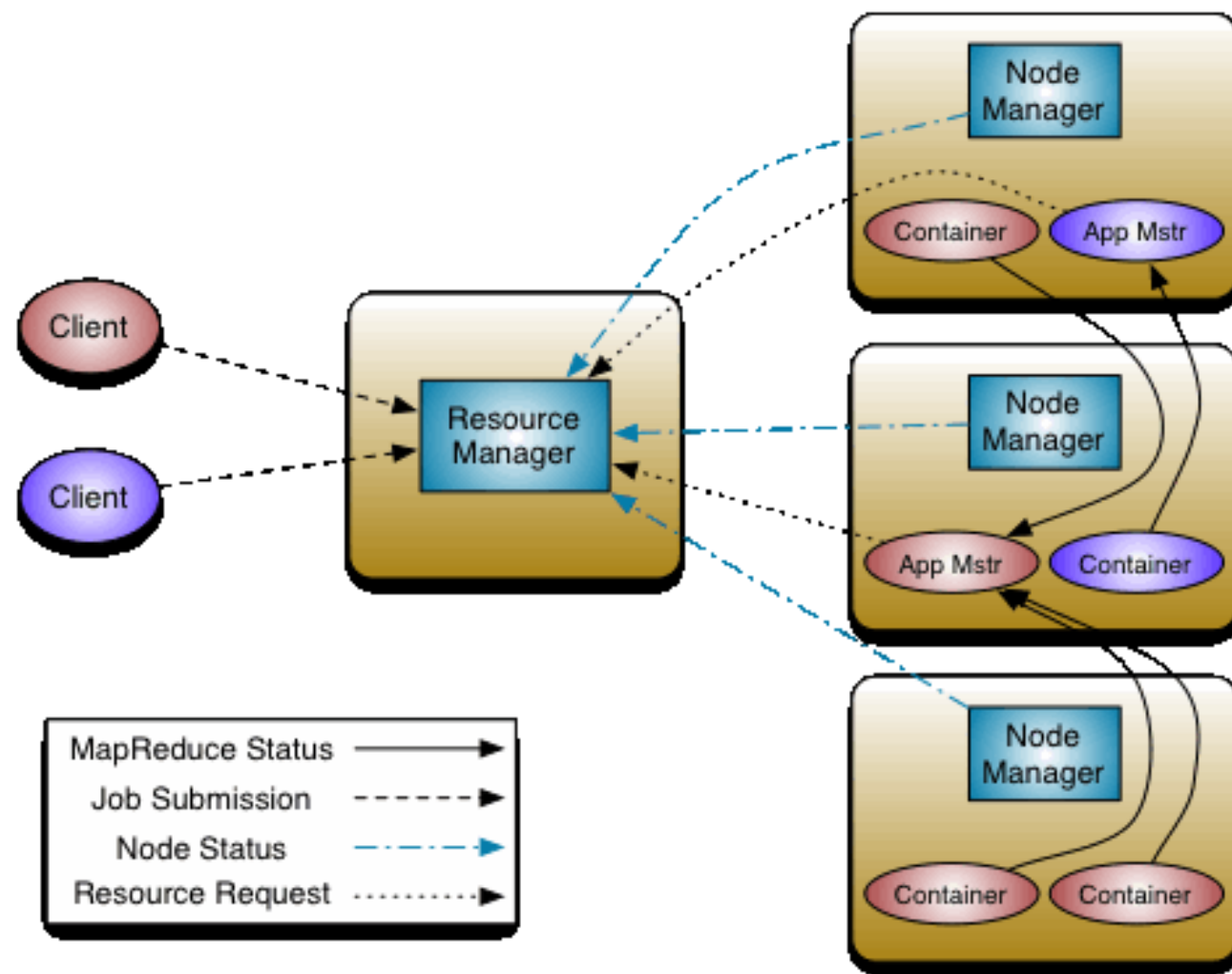


Hadoop MapReduce Framework Architecture

MapReduce 1.0缺点

- JobTracker 是 Map-reduce 的集中处理点，存在单点故障。
- JobTracker 完成了太多的任务，造成了过多的资源消耗，当 map-reduce job 非常多的时候，会造成很大的内存开销。
- 在 TaskTracker 端，以 map/reduce task 的数目作为资源的表示过于简单，没有考虑到 cpu/ 内存的占用情况。
- **MapReduce** 框架在有任何重要的或者不重要的变化（例如 bug 修复，性能提升和特性化）时，都会强制进行系统级别的升级更新。强制让分布式集群系统的每一个用户端同时更新。

Hadoop YARN MR调度

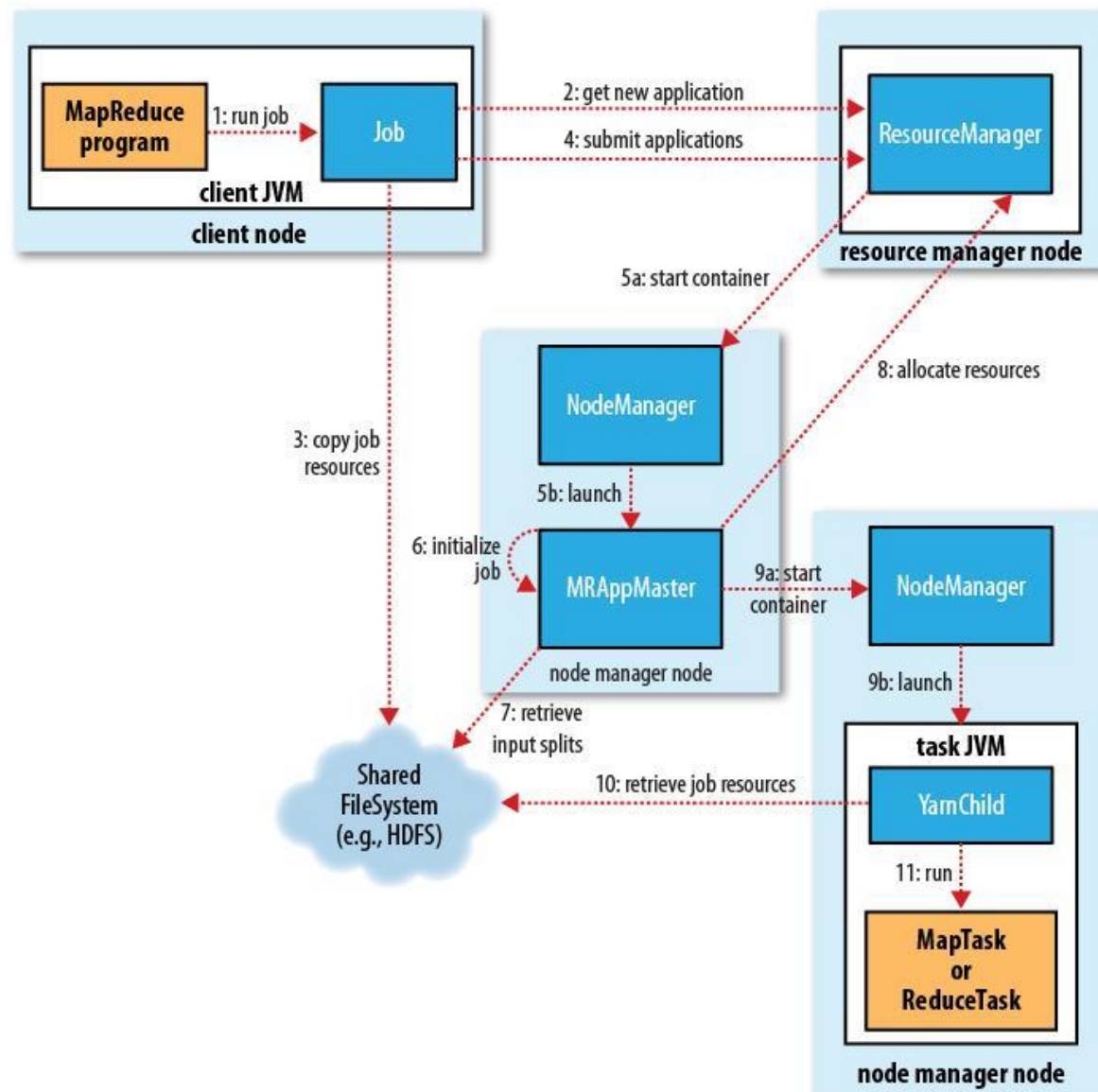


- MR1.0
 - Job Tracker
 - 资源管理
 - 任务调度、监控
- Yarn
 - ResourceManager
 - 调度、启动每一个 Job 所属的 ApplicationMaster、另外监控 ApplicationMaster
 - ApplicationMaster
 - 负责一个 Job 生命周期内的所有工作，类似老的框架中 JobTracker

HadoopYARN 模块组成

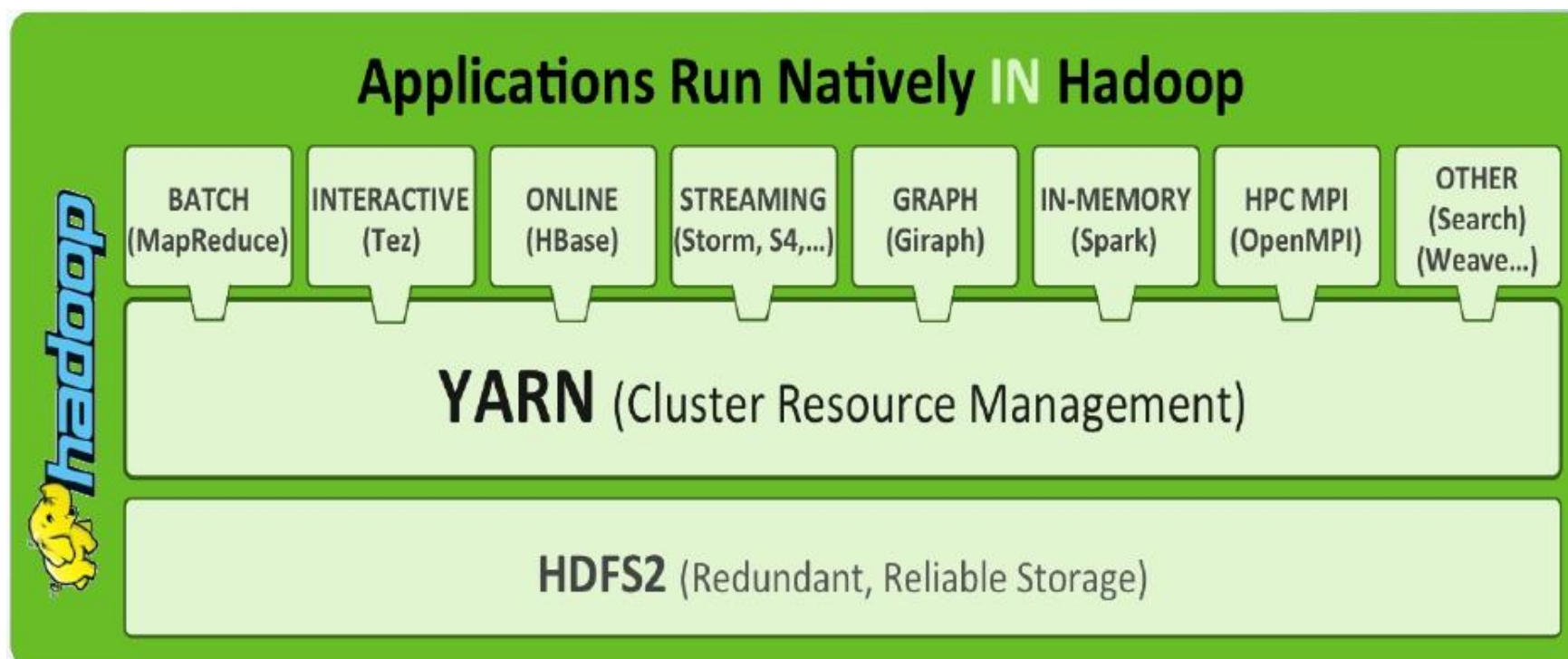
- ResourceManager
 - 处理客户端请求
 - 启动/监控ApplicationMaster
 - 监控NodeManager
 - 资源分配与调度
- NodeManager
 - 单个节点上的资源管理
 - 处理来自ResourceManager的命令
 - 处理来自ApplicationMaster的命令
- ApplicationMaster
 - 数据切分
 - 为应用程序申请资源，并分配给内部任务
 - 任务监控与容错

HadoopYARN 运行流程



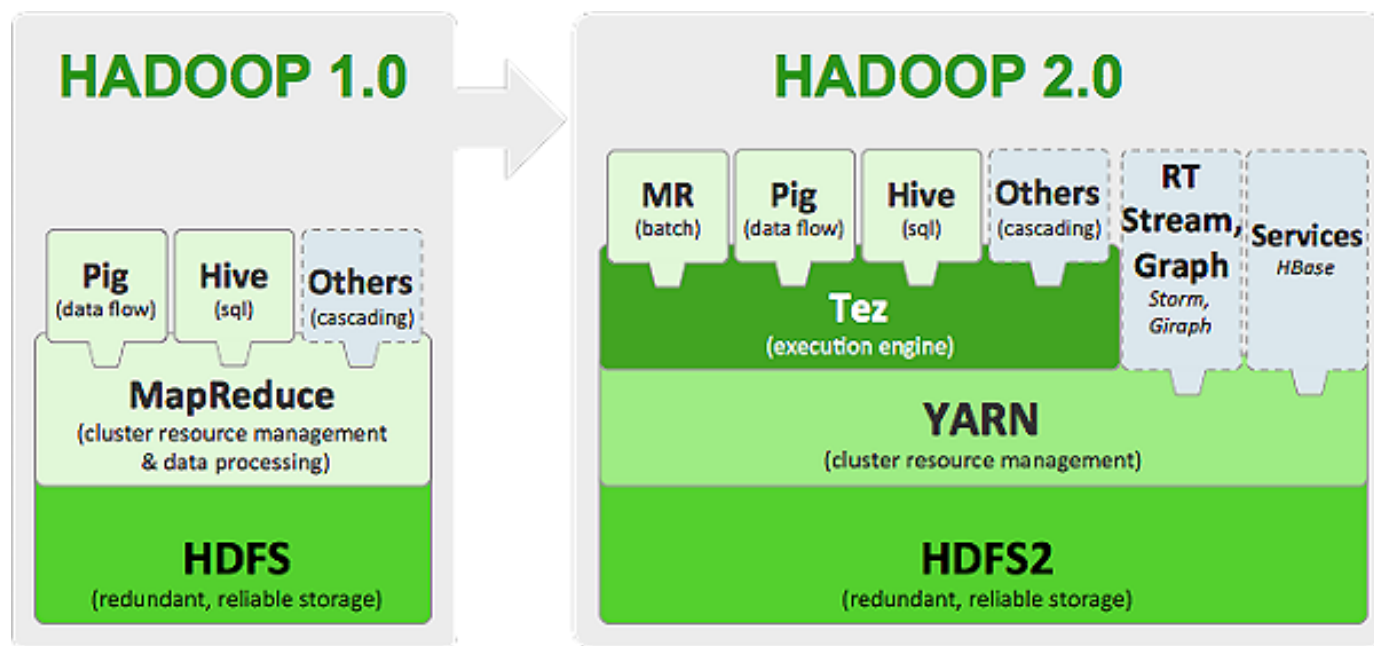
- ResourceManager
 - 存在单点故障;
 - 基于ZooKeeper实现。
- NodeManager
 - 失败后, RM将失败任务告诉对应的AM;
 - AM决定如何处理失败的任务。
- ApplicationMaster
 - 失败后, 由RM负责重启;
 - AM需处理内部任务的容错问题;
 - RMAppMaster会保存已经运行完成的Task, 重启后无需重新运行。

YARN 生态系统



运行在YARN上的计算框架

- 离线计算框架：MapReduce
- DAG计算框架：Tez
 - DAG作业的开源计算框架，支持高度定制化
- 流式计算框架：Storm
- 内存计算框架：Spark



目录

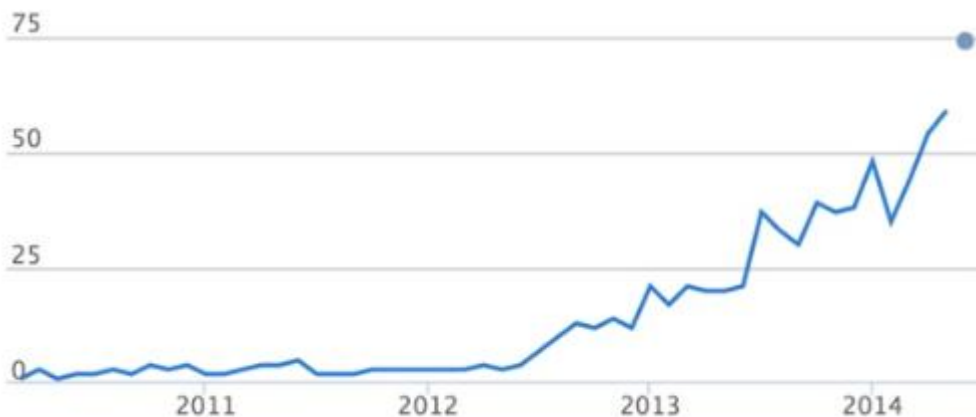
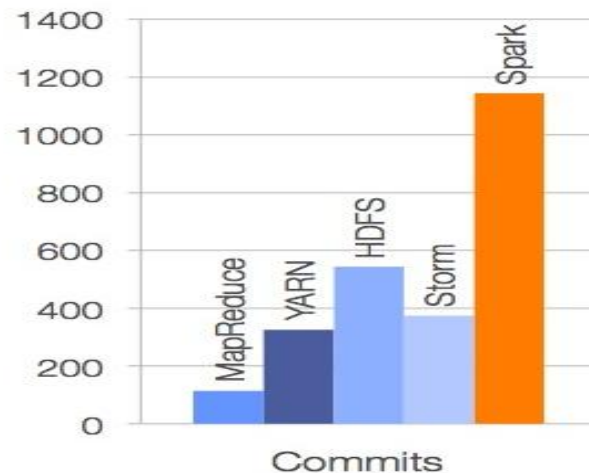
Contents

- 1 MapReduce与计算框架
- 2 数据的分片与路由
- 3 分布式文件系统
- 4 分布式计算框架例析：Hadoop
- 5 分布式计算框架例析：Spark**
- 6 分布式计算框架例析：Storm

Spark的发展历程

- 2009: Spark诞生于伯克利大学 AMPLab
- 2010: 开源
- 2013.6: Apache孵化器项目
- 2014.2: Apache顶级项目

Spark在最近10年内发展迅速，相较于其他大数据平台或框架而言，Spark的代码库最为活跃。



Spark代码贡献者每个月的增长曲线

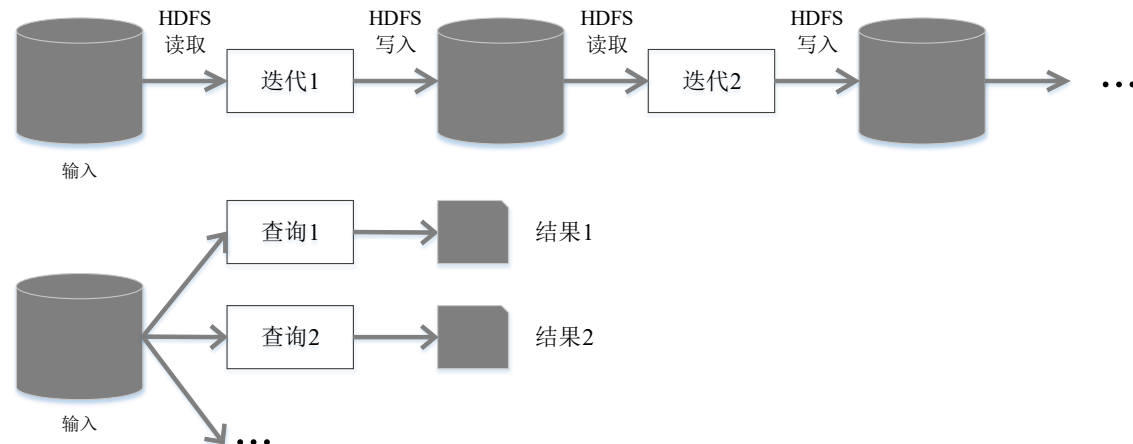
- Spark的Contributor比2014年涨了3倍，达到730人；
- 总代码行数也比2014年涨了2倍多，达到40万行
- Spark应用也越来越广泛，最大的集群来自腾讯——8000个节点，单个Job最大分别是阿里巴巴和Databricks——1PB

Spark的目标

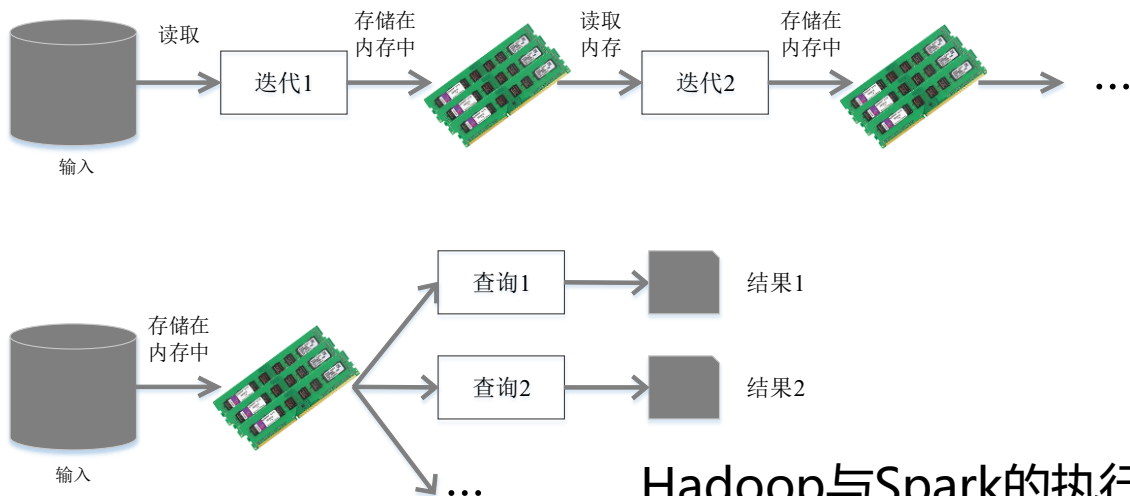
- 为集群运算提供分布式内存抽象，以支持具有工作集的应用程序
- 保留了MapReduce的优秀特性：
 - 容错机制（为崩溃程序和落后程序）
 - 数据局部性
 - 可扩展性

解决方案： 使用 “弹性分布式数据集”
(RDDs) 的增强数据流模型

Spark与Hadoop的对比



(a) Hadoop MapReduce执行流程



(b) Spark执行流程

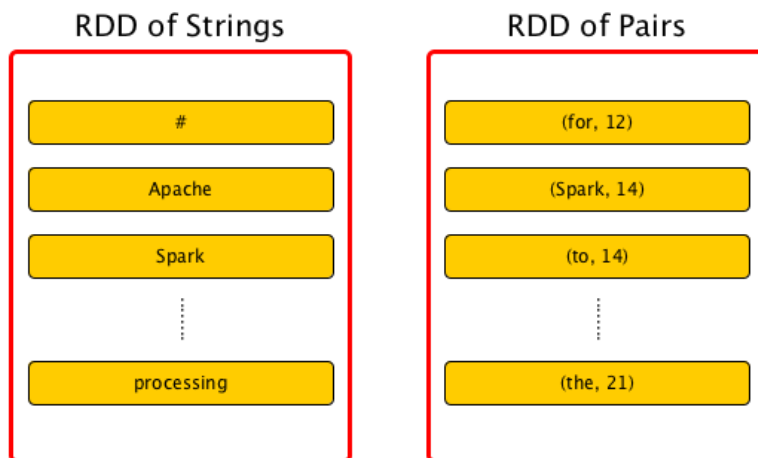
Hadoop与Spark的执行流程对比图

RDD设计背景

- 许多迭代式（比如机器学习、图算法等）和交互式数据挖掘工具的共同之处是，不同计算阶段之间会重用中间结果
- 目前的MapReduce框架都是把中间结果写入到稳定存储（比如磁盘）中，带来了大量的数据复制、磁盘IO和序列化开销
- RDD提供了一个抽象的数据架构，不必担心底层数据的分布式特性，只需将具体的应用逻辑表达为一系列转换处理
- 不同RDD之间的转换操作形成依赖关系，可以实现数据流水处理，避免中间数据存储

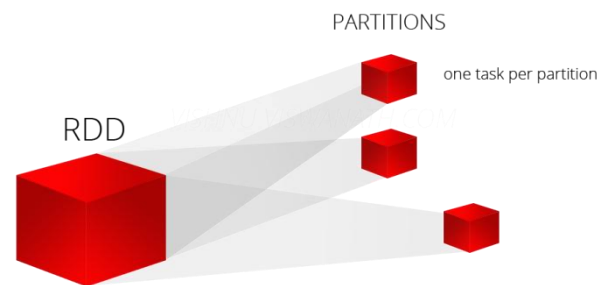
概念区分

- 流水 (pipeline)
- 阻塞 (blocked)



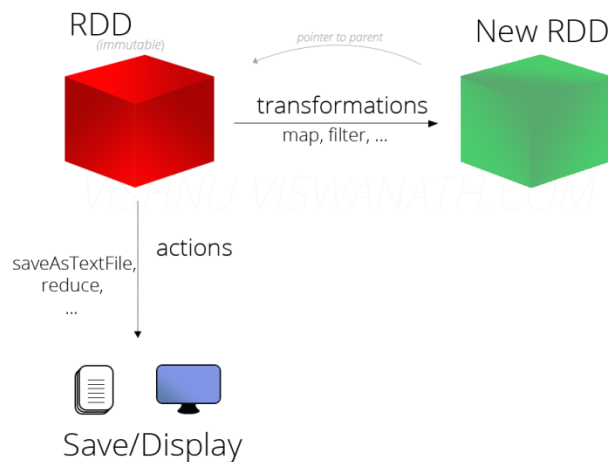
RDD概念

- 一个RDD是一个分布式对象集合，本质上是一个只读的分区记录集合
- 每个RDD可分成多个分区，每个分区就是一个数据集片段
- 一个RDD的不同分区可以被保存到集群中不同的节点上，从而可以在集群中的不同节点上进行并行计算
- RDD提供了一种高度受限的共享内存模型
 - ✓ RDD是只读的记录分区的集合，不能直接修改
 - ✓ 只能基于稳定的物理存储中的数据集创建RDD，或者通过在其他RDD上执行确定的转换操作（如map、join和group by）而创建得到新的RDD



RDD的操作

- RDD提供了一组丰富的操作以支持常见的数据运算，分为“**动作**” (Action) 和“**转换**” (Transformation) 两种类型
- RDD提供的转换接口都非常简单，都是类似map、filter、groupBy、join等粗粒度的数据转换操作，而不是针对某个数据项的细粒度修改（不适合网页爬虫）
- RDD已经被实践证明可以高效地表达许多框架的编程模型（如MapReduce、SQL、Pregel）
- Spark用Scala语言实现了RDD的API，程序员可以通过调用API实现对RDD的各种操作

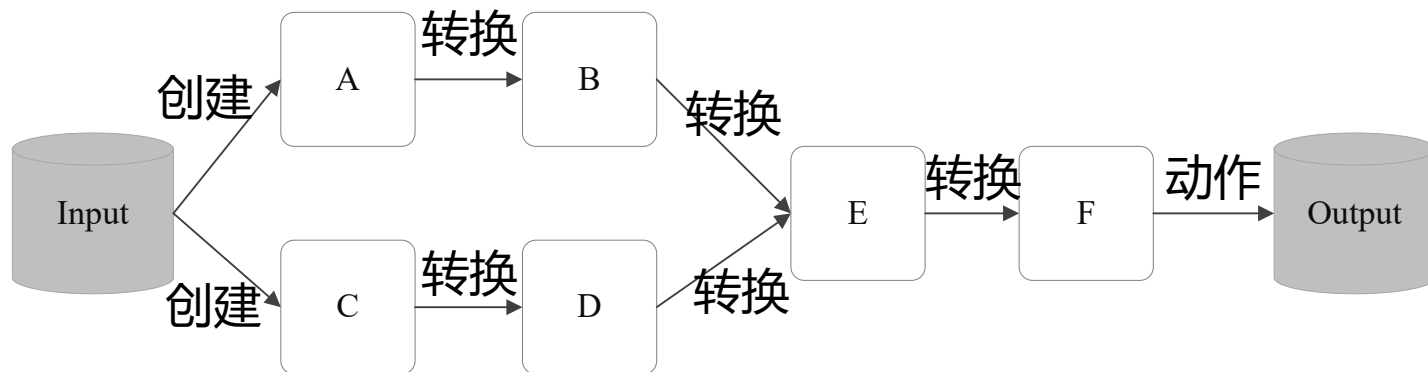


RDD的执行过程

- RDD读入外部数据源进行创建
- RDD经过一系列的转换（Transformation）操作，每一次都会产生不同的RDD，供给下一个转换操作使用
- 最后一个RDD经过“动作”操作进行转换，并输出到外部数据源

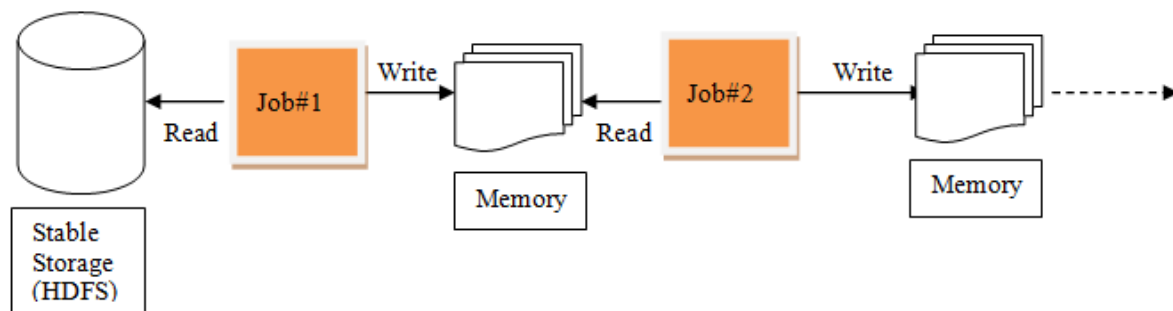
这一系列处理称为一个Lineage（血缘关系），即DAG拓扑排序的结果

优点：惰性调用、管道化、避免同步等待、不需要保存中间结果、每次操作变得简单

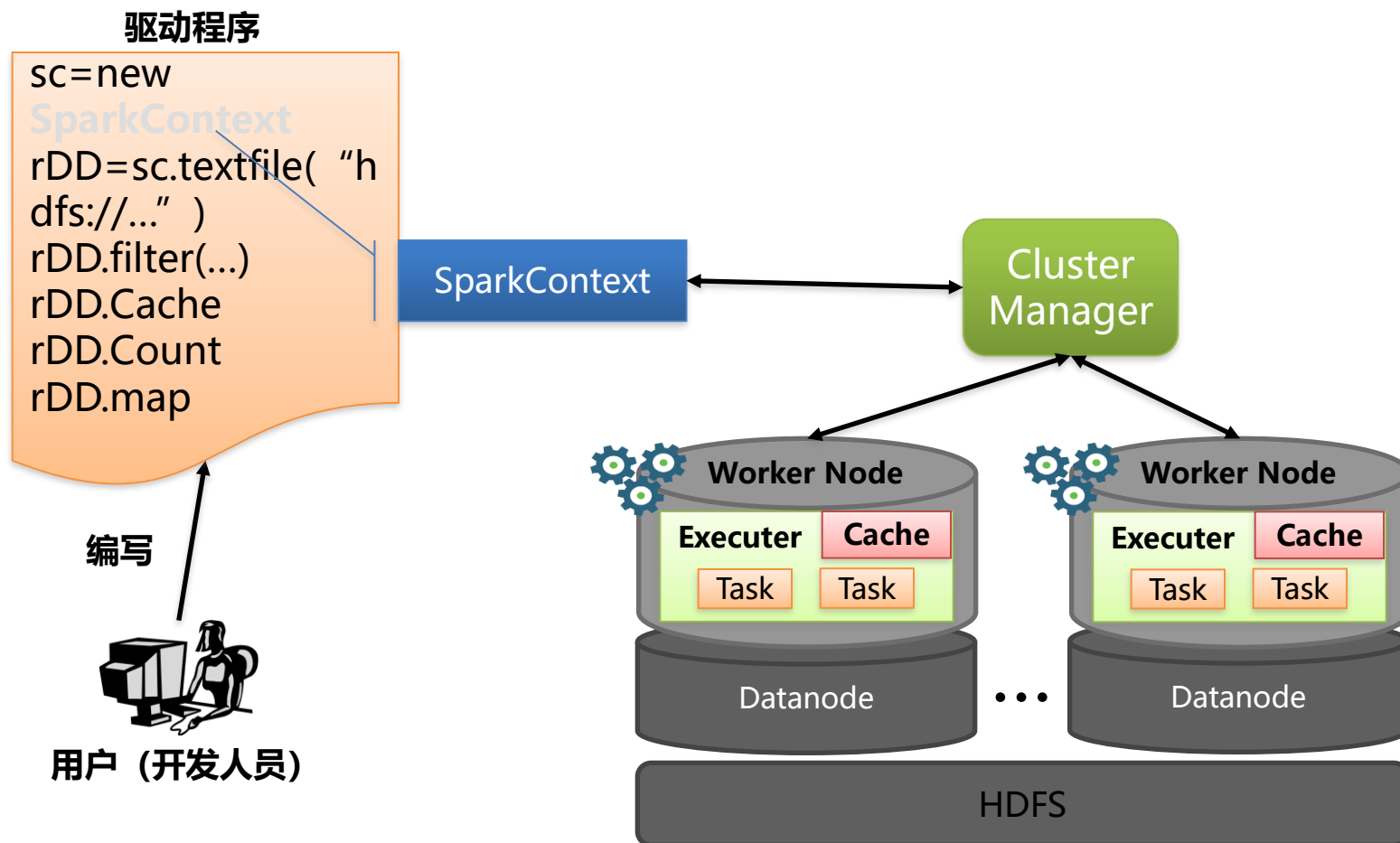


RDD高效的原因

- 高效的容错性
 - ✓ 现有容错机制：数据复制或者记录日志
 - ✓ RDD：血缘关系、重新计算丢失分区、无需回滚系统、重算过程在不同节点之间并行、只记录粗粒度的操作
- 中间结果持久化到内存，数据在内存中的多个RDD操作之间进行传递，避免了不必要的读写磁盘开销
- 存放的数据可以是Java对象，避免了不必要的对象序列化和反序列化



Spark编程模型



Spark编程模型

驱动程序

```
sc=new  
SparkContext  
rDD=sc.textfile( "  
hdfs://..." )  
rDD.filter(...)  
rDD.Cache  
rDD.Count  
rDD.map
```

RDD
(Resilient
Distributed
Dataset)

- 不可变的数据结构
- 存储在内存中（显式）
- 容错
- 并行数据结构
- 可控的分区以优化数据存储
- 可以使用多种的运算符操作

编写



用户（开发人员）

操作RDD的API

- 编程接口：程序员可以执行3种类型的操作

Transformations

- 从现有的数据集中创建新的数据集.
- 惰性计算。它们仅在action执行时才执行。
- 例子：
 - Map(func)
 - Filter(func)
 - Distinct()

Actions

- 返回一个数据到驱动程序或者计算之后向一个存储系统输出数据
- 例子：
 - Count()
 - Reduce(func)
 - Collect
 - Take()

Persistence

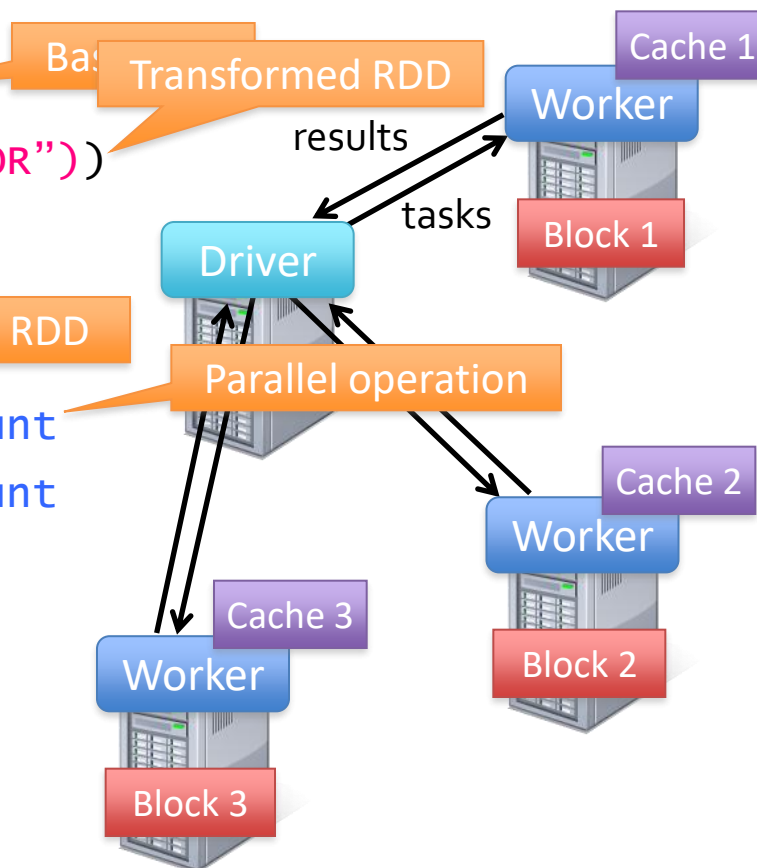
- 为后续操作在内存中缓存数据集
- 可选存在磁盘上或者RAM上, 或混合存储 (存储级别)
- 例子：
 - Persist()
 - Cache()

例1：日志挖掘

- 从日志中加载错误信息到内存，接着交互式地搜索不同的模式

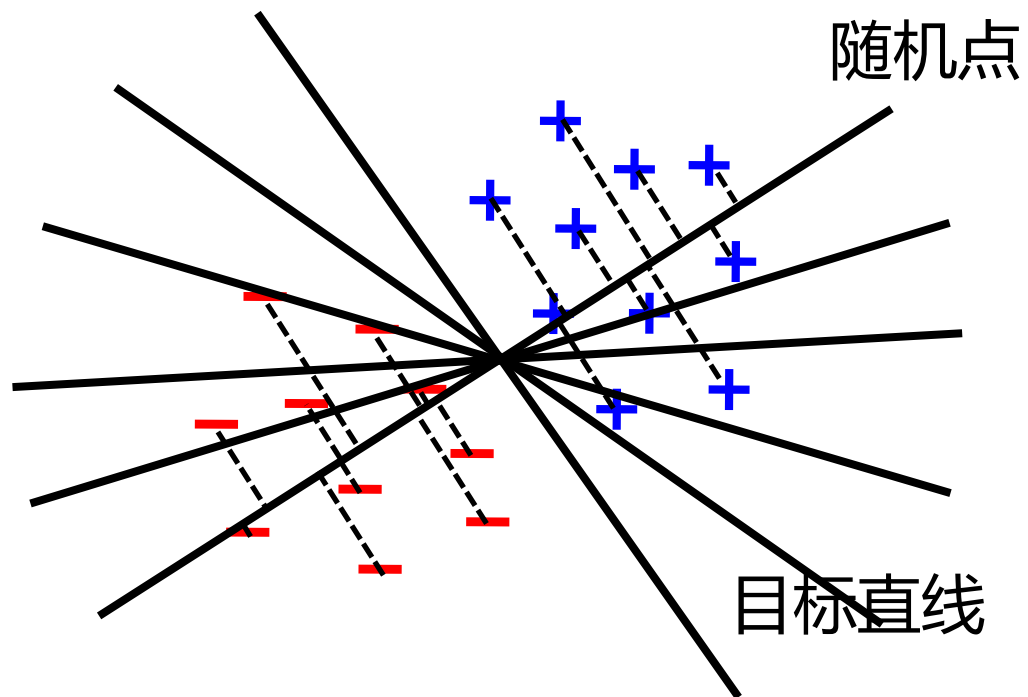
```
lines = spark.textFile("hdfs://...")
errors = lines.filter(_.startsWith("ERROR"))
messages = errors.map(_.split('\t')(2))
cachedMsgs = messages.cache()

cachedMsgs.filter(_.contains("foo")).count
cachedMsgs.filter(_.contains("bar")).count
. . .
```



例2：逻辑回归

- 目的：找到最好的分割两组点的直线



逻辑回归代码

- `val data = spark.textFile(...).map(readPoint).cache()`
- `var w = Vector.random(D)`
- `for (i <- 1 to ITERATIONS) {`
- `val gradient = data.map(p =>`
 - `(1 / (1 + exp(-p.y*(w dot p.x))) - 1) * p.y * p.x`
 - `).reduce(_ + _)`
 - `w -= gradient`
- `}`
- `println("Final w: " + w)`

例3: MapReduce

MapReduce可以用RDD转换来表示

```
res = data.flatMap(rec => myMapFunc(rec))  
            .groupByKey()  
            .map((key, vals) => myReduceFunc(key, vals))
```

或者使用Combiner

```
res = data.flatMap(rec => myMapFunc(rec))  
            .reduceByKey(myCombiner)  
            .map((key, val) => myReduceFunc(key, val))
```

例4：字数统计

Simple Spark Apps: *WordCount*

```
1 public class WordCount {
2     public static class TokenizerMapper
3         extends Mapper<Object, Text, Text, IntWritable>{
4
5         private final static IntWritable one = new IntWritable(1);
6         private Text word = new Text();
7
8         public void map(Object key, Text value, Context context
9             ) throws IOException, InterruptedException {
10             StringTokenizer itr = new StringTokenizer(value.toString());
11             while (itr.hasMoreTokens()) {
12                 word.set(itr.nextToken());
13                 context.write(word, one);
14             }
15         }
16     }
17
18     public static class IntSumReducer
19         extends Reducer<Text, IntWritable, Text, IntWritable> {
20         private IntWritable result = new IntWritable();
21
22         public void reduce(Text key, Iterable<IntWritable> values,
23             Context context
24             ) throws IOException, InterruptedException {
25             int sum = 0;
26             for (IntWritable val : values) {
27                 sum += val.get();
28             }
29             result.set(sum);
30             context.write(key, result);
31         }
32     }
33
34     public static void main(String[] args) throws Exception {
35         Configuration conf = new Configuration();
36         String[] otherArgs = new GenericOptionsParser(conf, args).getRemainingArgs();
37         if (otherArgs.length < 2) {
38             System.err.println("Usage: wordcount <in> <out>");
39             System.exit(2);
40         }
41         Job job = new Job(conf, "word count");
42         job.setJarByClass(WordCount.class);
43         job.setMapperClass(TokenizerMapper.class);
44         job.setCombinerClass(IntSumReducer.class);
45         job.setReducerClass(IntSumReducer.class);
46         job.setOutputKeyClass(Text.class);
47         job.setOutputValueClass(IntWritable.class);
48         for (int i = 0; i < otherArgs.length - 1; ++i) {
49             FileInputFormat.addInputPath(job, new Path(otherArgs[i]));
50         }
51         FileOutputFormat.setOutputPath(job,
52             new Path(otherArgs[otherArgs.length - 1]));
53         System.exit(job.waitForCompletion(true) ? 0 : 1);
54     }
55 }
```

```
1 val f = sc.textFile(inputPath)
2 val w = f.flatMap(l => l.split(" ")).map(word => (word, 1)).cache()
3 w.reduceByKey(_ + _).saveAsText(outputPath)
```

WordCount in 3 lines of Spark

Simple Spark Apps: *WordCount*

Scala:

```
val f = sc.textFile("README.md")
val wc = f.flatMap(l => l.split(" ")).map(word => (word, 1)).reduceByKey(_ + _)
wc.saveAsTextFile("wc_out")
```

Python:

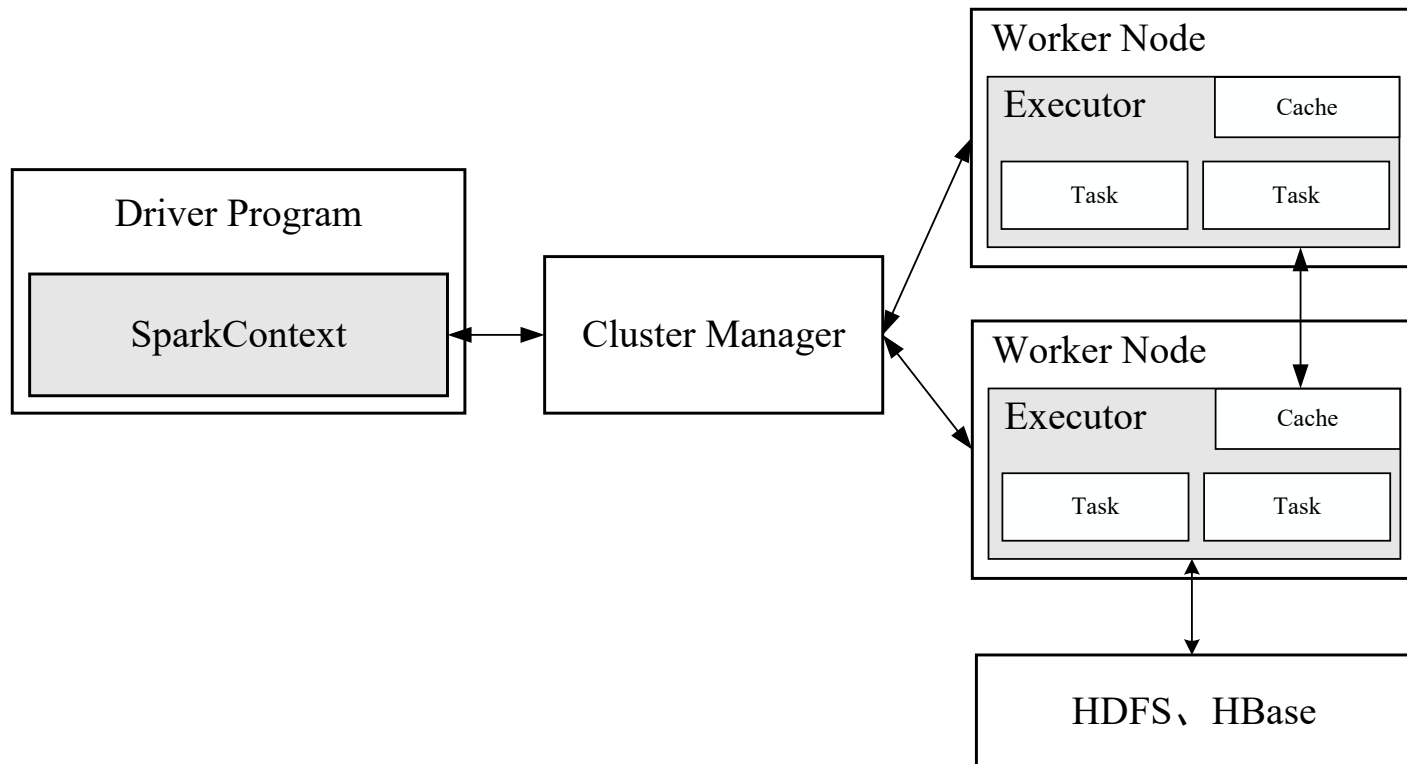
```
from operator import add
f = sc.textFile("README.md")
wc = f.flatMap(lambda x: x.split(' ')).map(lambda x: (x, 1)).reduceByKey(add)
wc.saveAsTextFile("wc_out")
```

基本概念

- DAG：是Directed Acyclic Graph（有向无环图）的简称，反映RDD之间的依赖关系
- Executor：是运行在工作节点（WorkerNode）的一个进程，负责运行Task
- 应用（Application）：用户编写的Spark应用程序
- 任务（Task）：运行在Executor上的工作单元
- 作业（Job）：一个作业包含多个RDD及作用于相应RDD上的各种操作
- 阶段（Stage）：是作业的基本调度单位，一个作业会分为多组任务，每组任务被称为阶段，或者也被称为任务集合，代表了一组关联的、相互之间没有Shuffle依赖关系的任务组成的任务集

架构设计

- Spark运行架构包括集群资源管理器（Cluster Manager）、运行作业任务的工作节点（Worker Node）、每个应用的任务控制节点（Driver）和每个工作节点上负责具体任务的执行进程（Executor）
- 资源管理器可以自带Mesos或YARN



SparkContext

SparkContext处于DriverProgram核心位置，所有与Cluster、Worker Node交互的操作都需要SparkContext来完成

```
val sc = new SparkContext(master="local[*]",  
                           appName="SparkMe App", new SparkConf)  
val lines = sc.textFile(...).cache()  
val c = lines.count()  
println(s"There are $c lines in $fileName")
```

Spark context

RDD graph

DAGScheduler

Task Scheduler

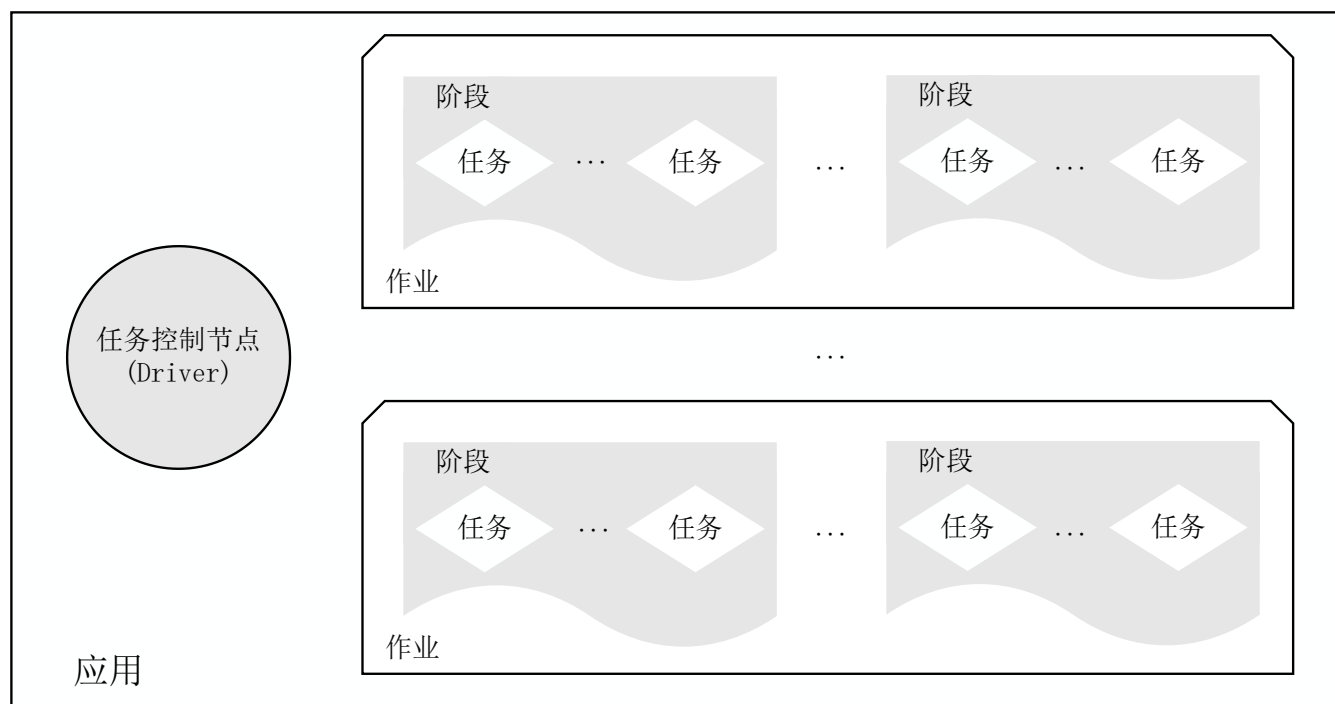
Scheduler Backend

Listener Bus

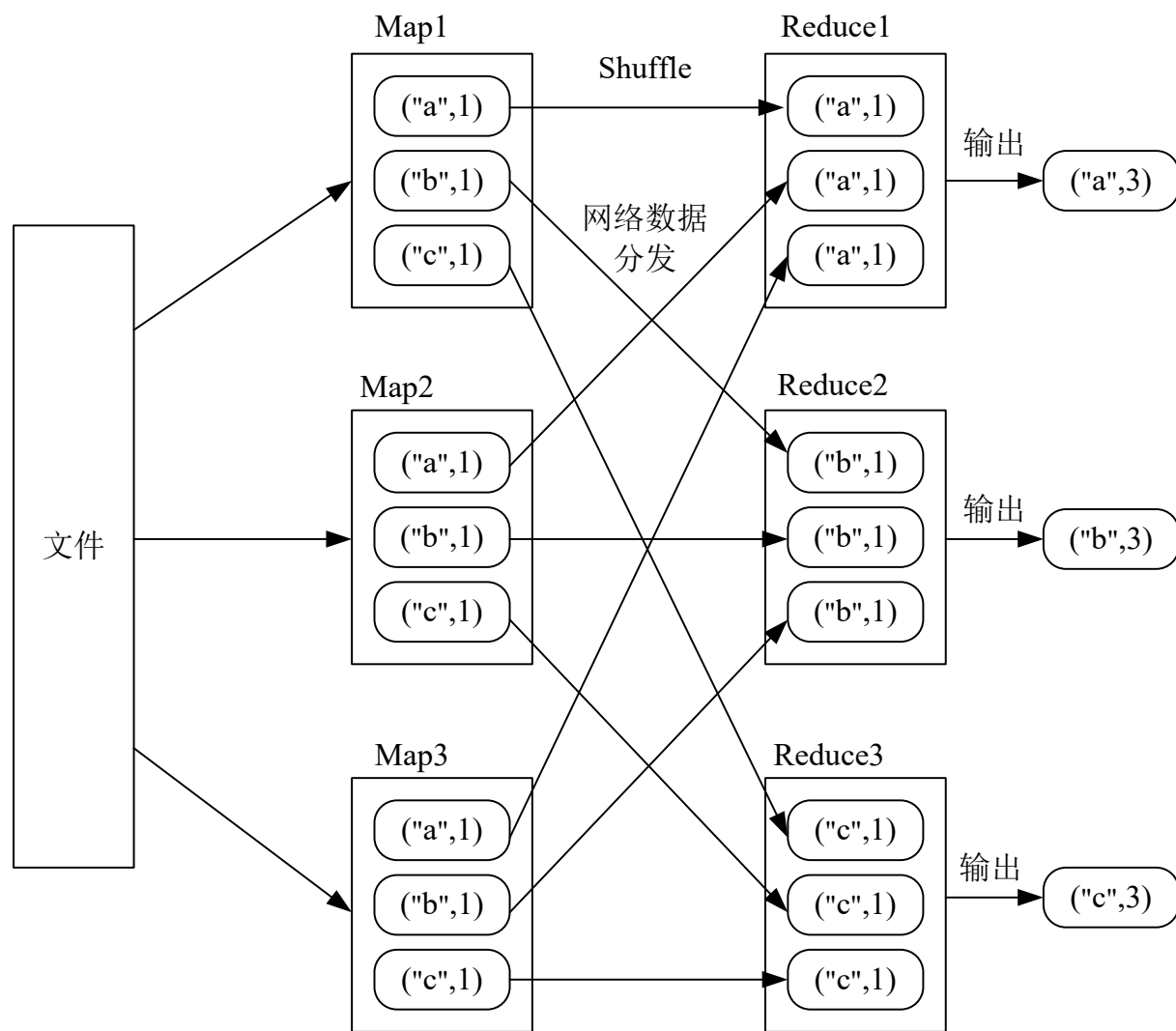
Block Manager

架构设计

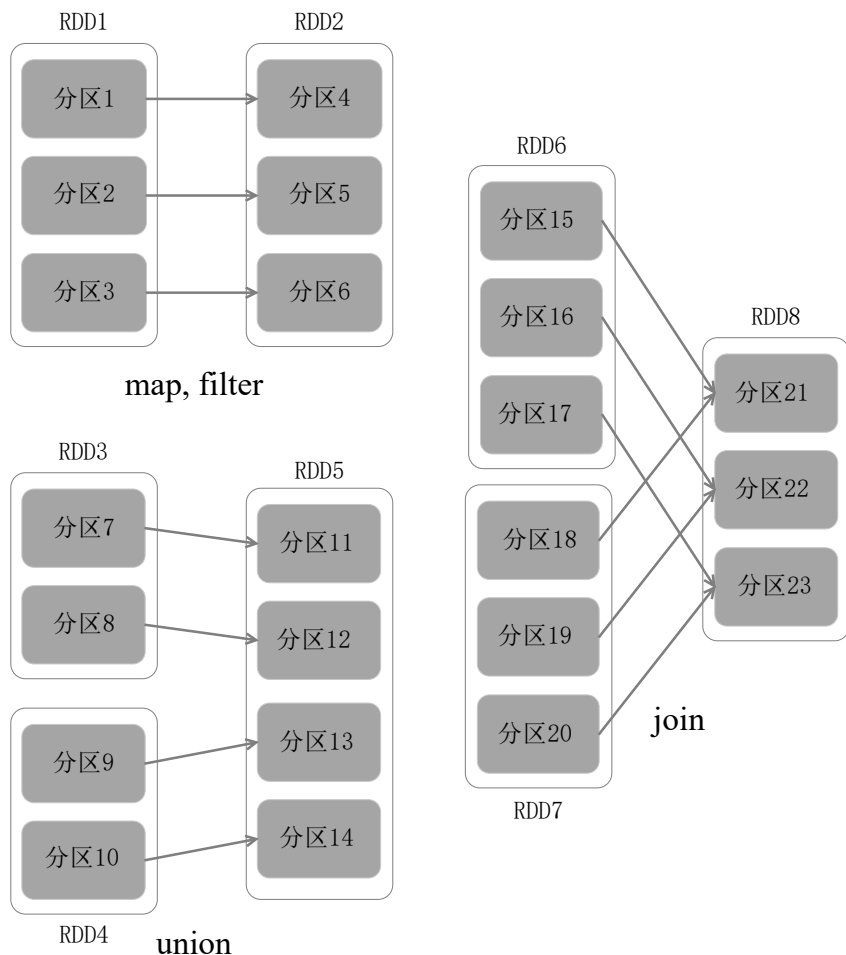
- 一个应用由一个Driver和若干个作业构成，一个作业由多个阶段构成，一个阶段由多个没有Shuffle关系的任务组成
- 当执行一个应用时，Driver会向集群管理器申请资源，启动Executor，并向Executor发送应用程序代码和文件，然后在Executor上执行任务，运行结束后，执行结果会返回给Driver，或者写到HDFS或者其他数据库中



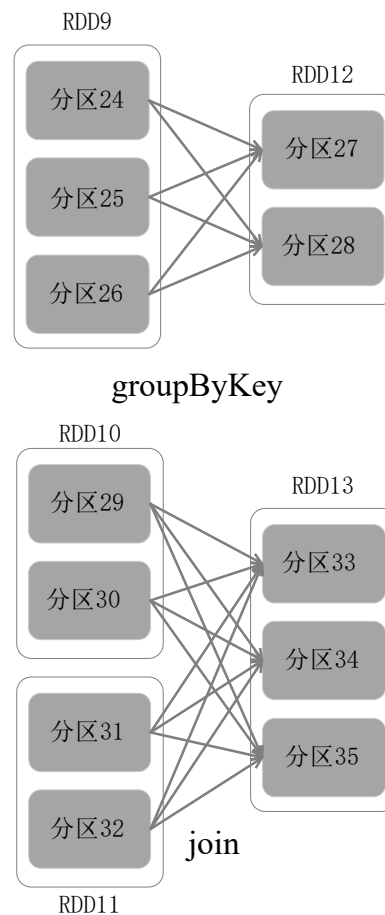
Shuffle操作



窄依赖和宽依赖



(a)窄依赖



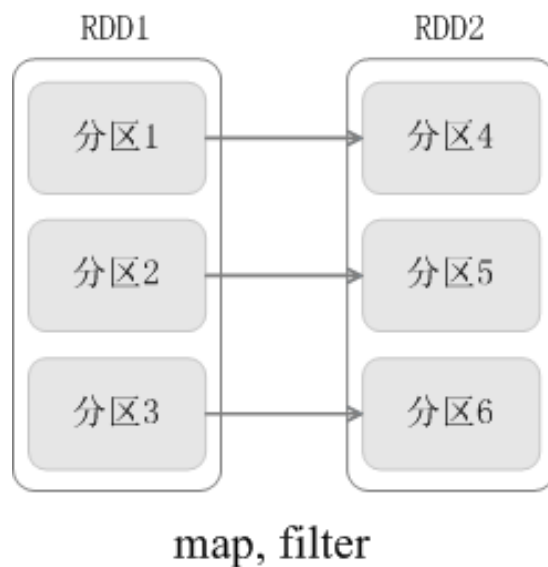
(b)宽依赖

- ✓ **窄依赖:** 一个父RDD的分区对应于一个子RDD的分区或多个父RDD的分区对应于一个子RDD的分区
- ✓ **宽依赖:** 存在一个父RDD的一个分区对应一个子RDD的多个分区

阶段的划分

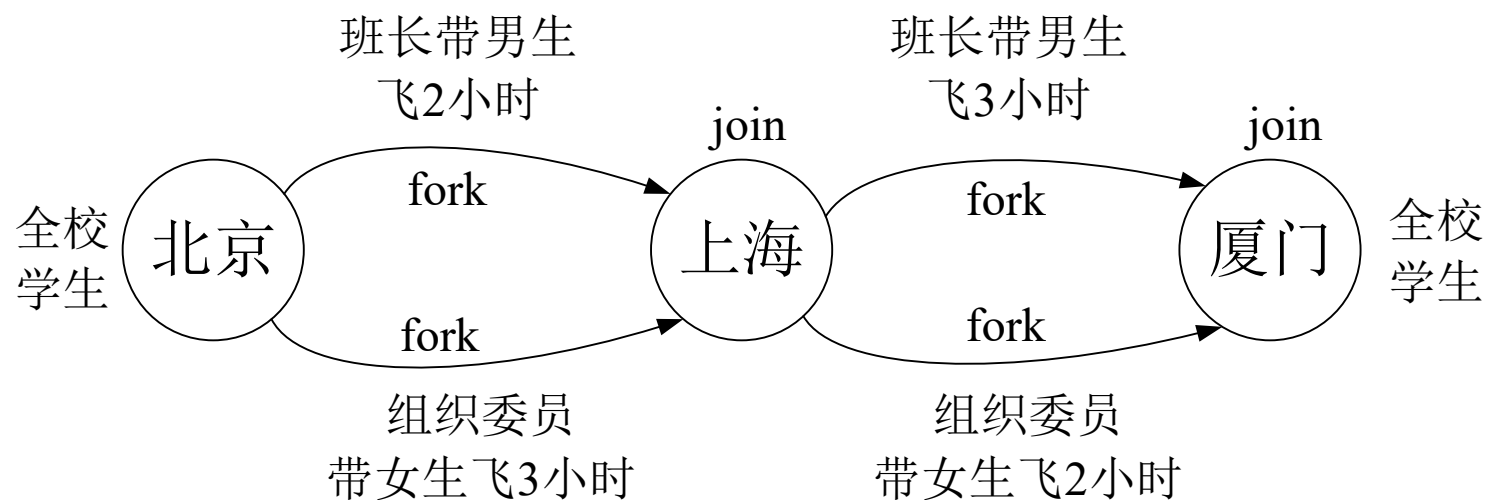
- 根据DAG 图中的RDD 依赖关系，把一个作业分成多个阶段。
- 阶段划分的依据是窄依赖和宽依赖：窄依赖对于作业的优化很有利，宽依赖无法优化
- 逻辑上，每个RDD 操作都是一个fork/join（一种用于并行执行任务的框架），把计算fork 到每个RDD 分区，完成计算后对各个分区得到的结果进行join 操作，然后fork/join下一个RDD 操作

- ✓ **窄依赖**：一个父RDD的分区对应于一个子RDD的分区或多个父RDD的分区对应于一个子RDD的分区
- ✓ **宽依赖**：存在一个父RDD的一个分区对应一个子RDD的多个分区



fork/join的优化原理

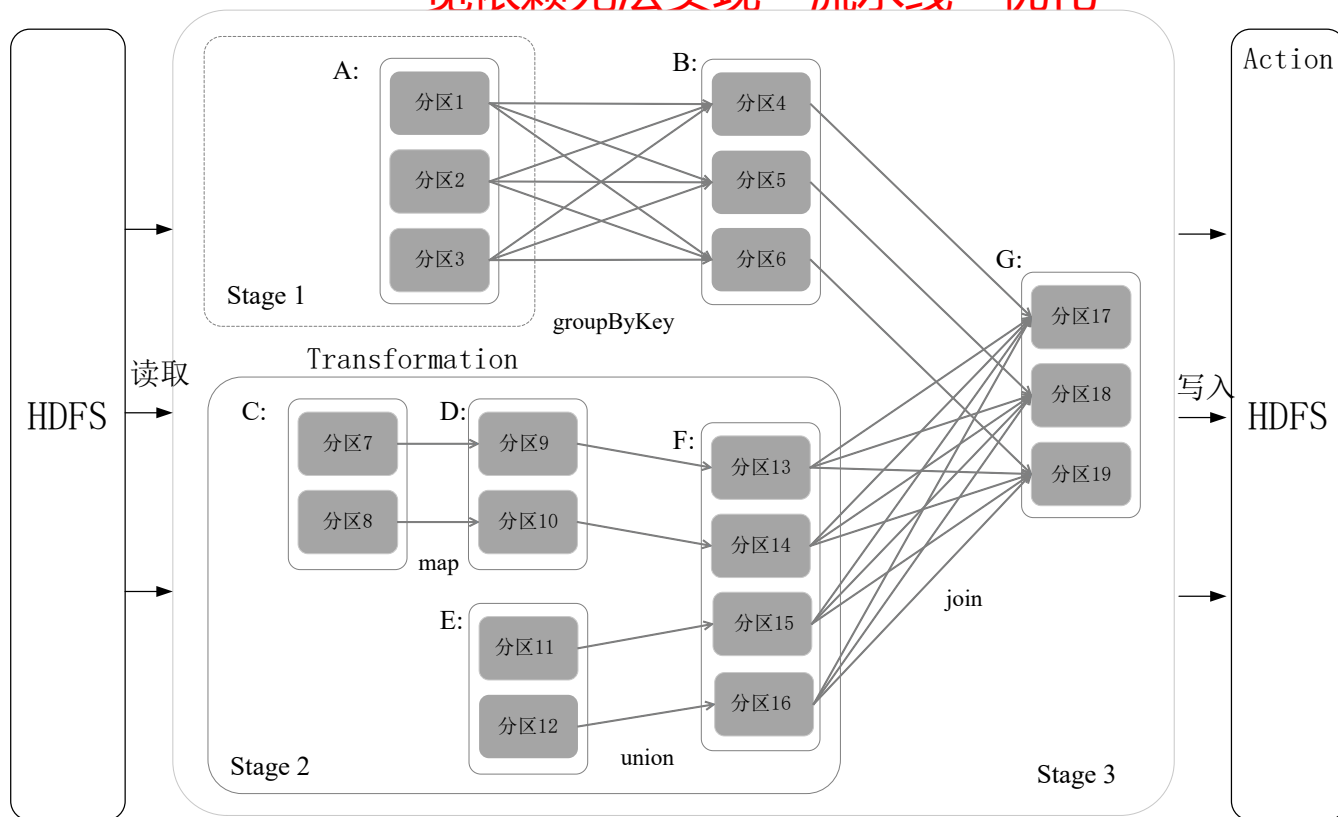
举例：一个学校（含2个班级）完成从北京到厦门的旅行



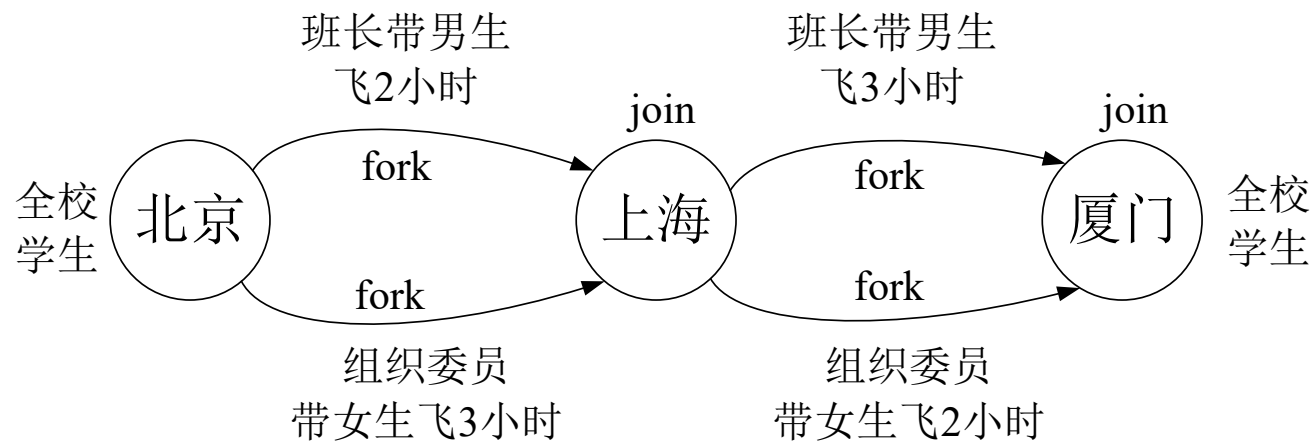
阶段的划分

宽依赖往往对应着shuffle操作(多对一,汇总,多节点), 需要在运行过程中将同一个父RDD的分区传入到不同的子RDD分区中, 中间可能涉及多个节点之间的数据传输; 而窄依赖的每个父RDD的分区只会传入到一个子RDD分区中, 通常可以在一个节点内完成转换

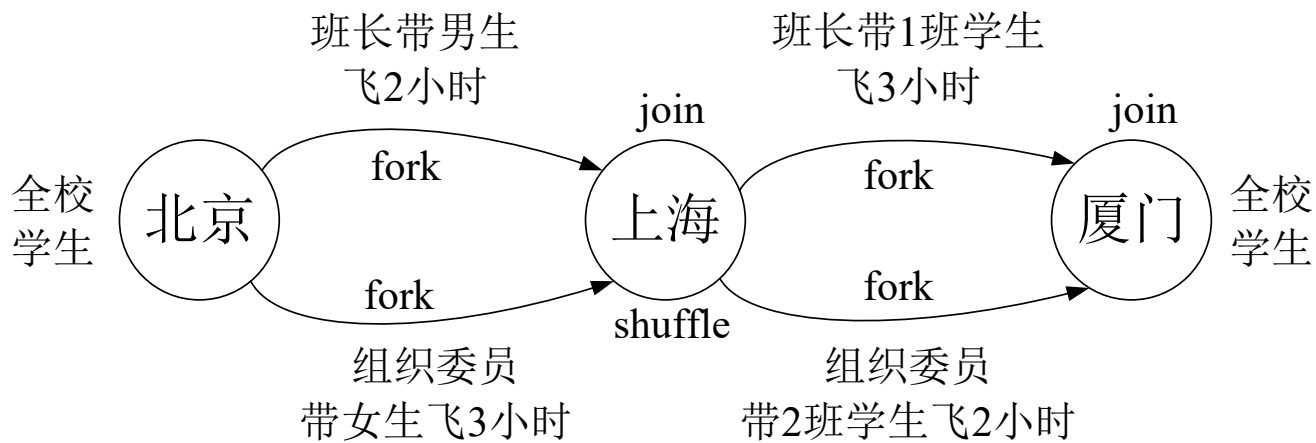
窄依赖可以实现“流水线”优化
宽依赖无法实现“流水线”优化



阶段的划分



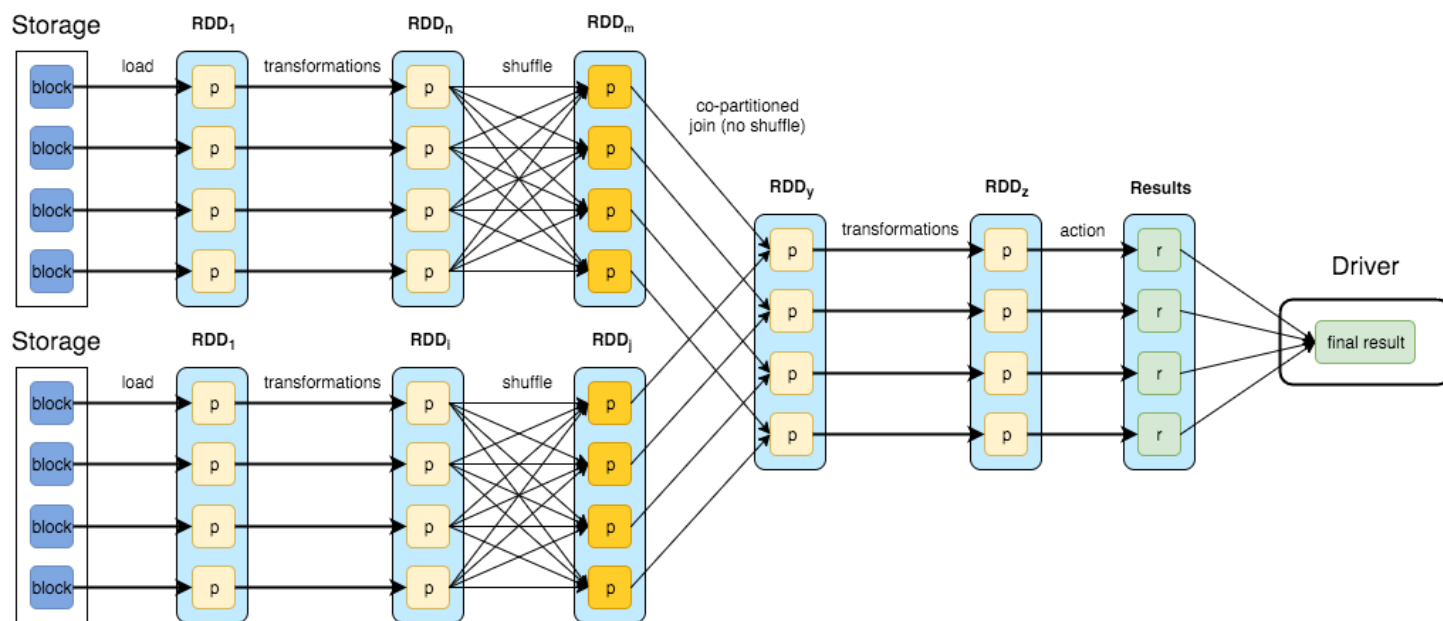
(a) 窄依赖



(a) 宽依赖

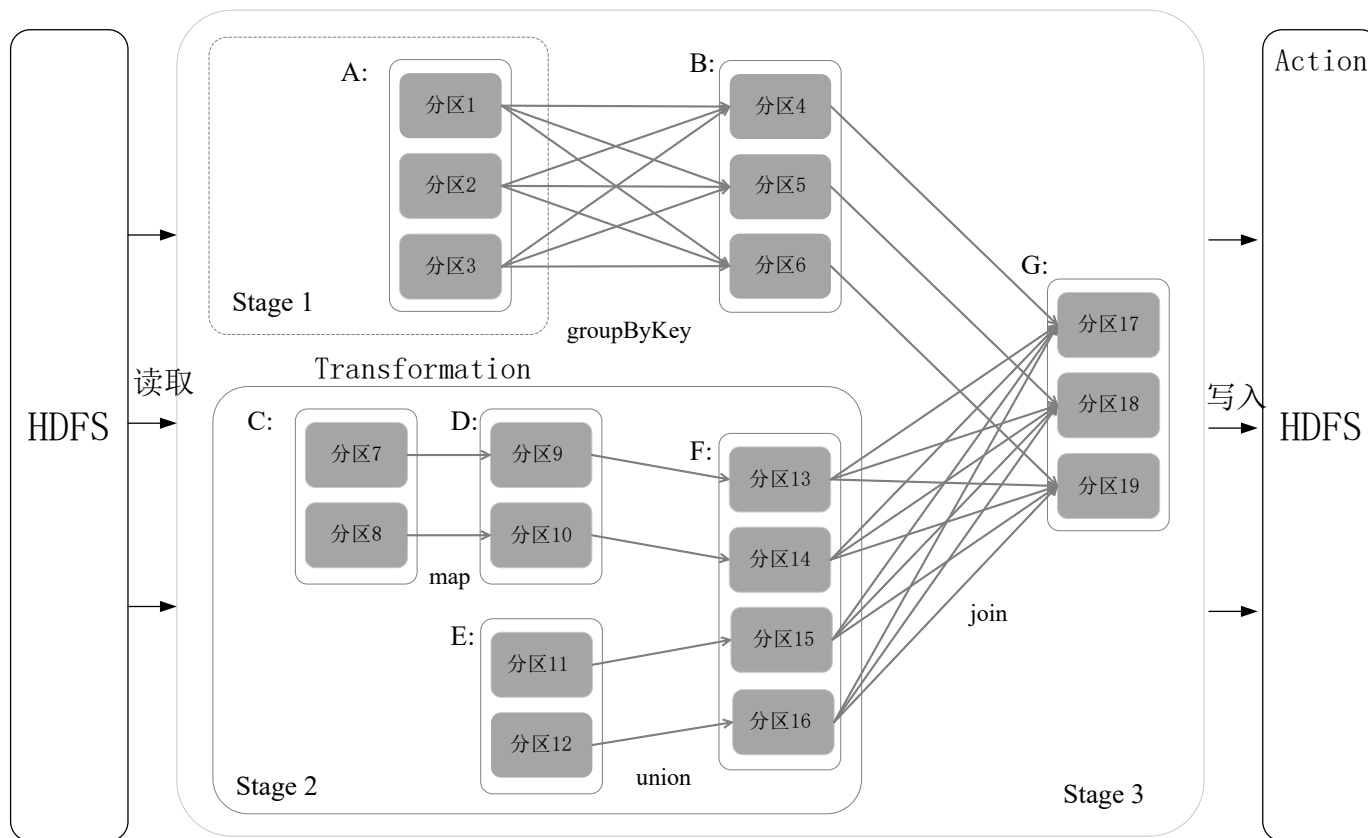
划分的方法

- 通过分析各个RDD的依赖关系生成了DAG，再通过分析各个RDD中的分区之间的依赖关系来决定如何划分Stage
- 具体步骤(贪心法):
 - ✓ 在DAG中进行反向解析，遇到宽依赖就断开
 - ✓ 遇到窄依赖就把当前的RDD加入到Stage中
 - ✓ 将窄依赖尽量划分在同一个Stage中，可以实现流水线计算



划分的实例

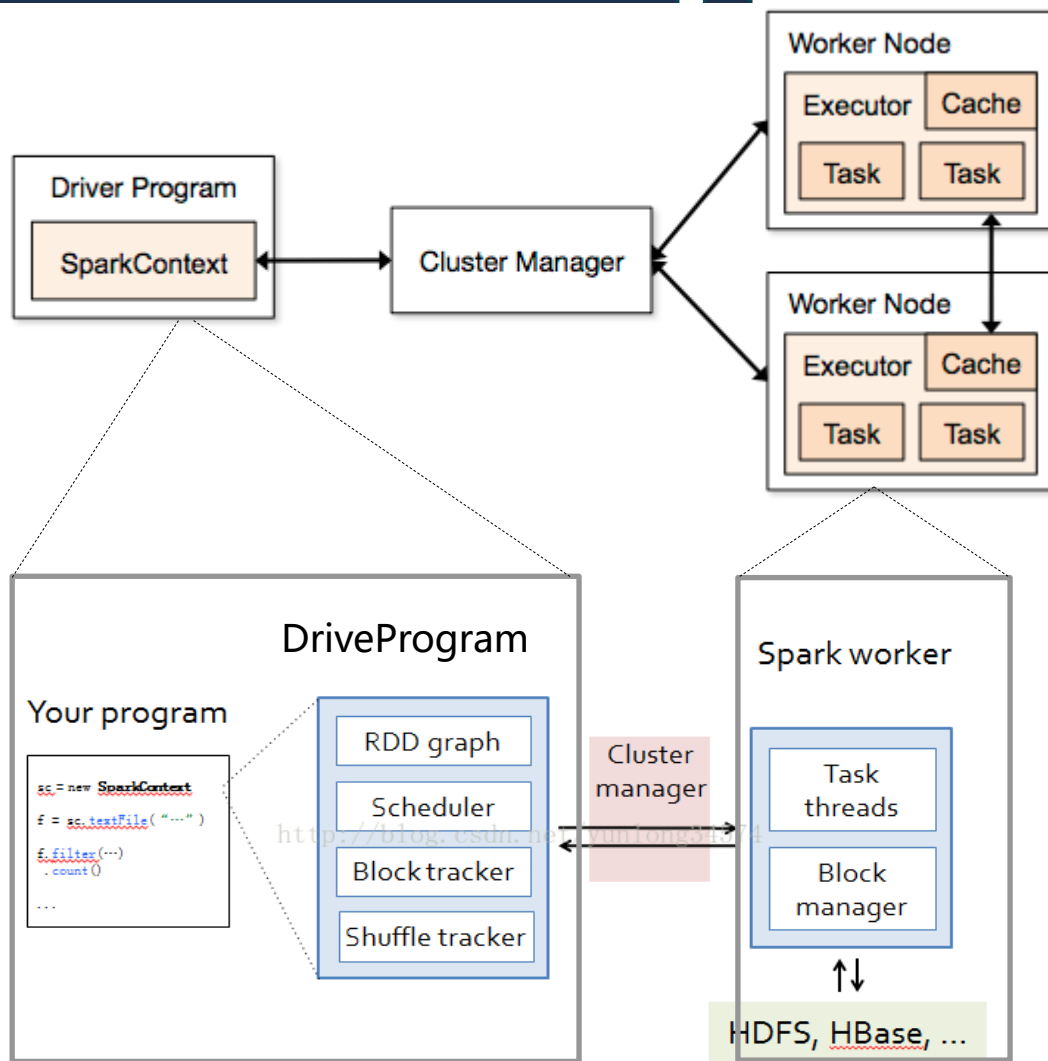
任务被分成三个Stage，在Stage2中，从map到union都是窄依赖，这两步操作可以形成一个流水线操作



流水线操作实例

分区7通过map操作生成的分区9，可以不用等待分区8到分区10这个map操作的计算结束，而是继续进行union操作，得到分区13，这样流水线执行大大提高了计算的效率

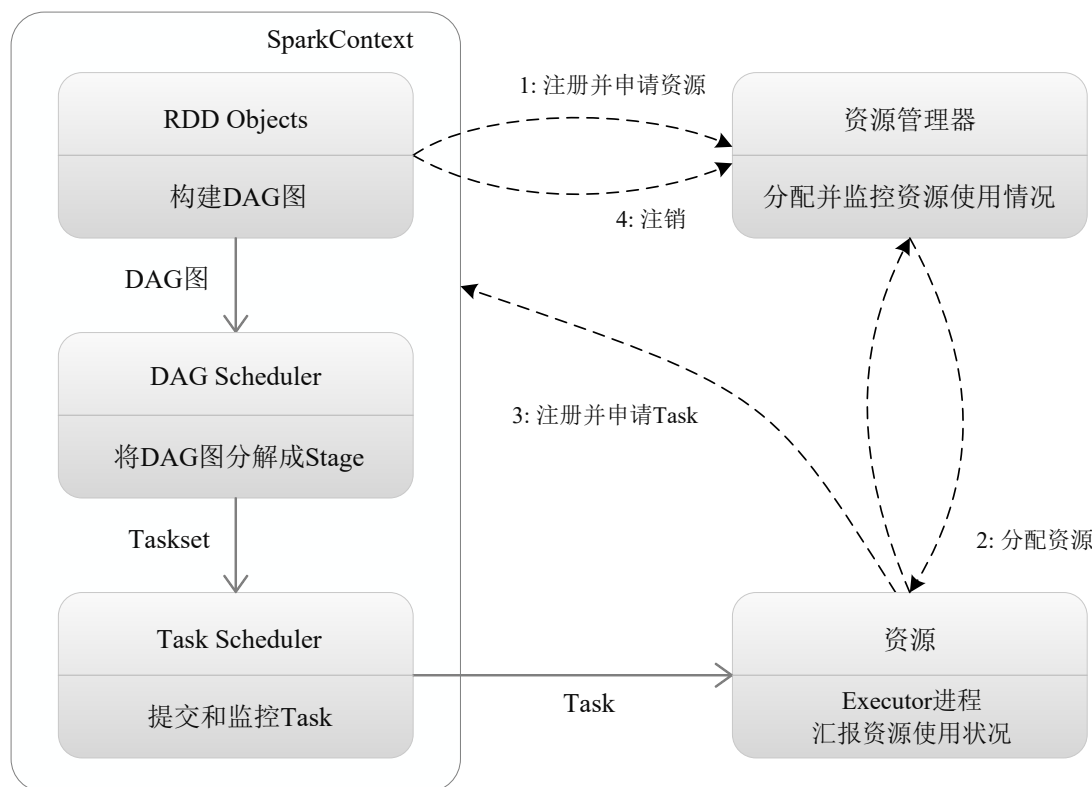
Spark端到端流程



整体流程

- Spark的Driver Program (简称Driver)包含用户的应用程序
- Driver完成task的解析和生成
- Driver向Cluster Manager (集群资源管理器) 申请运行task需要的资源。
- 集群资源管理器为task分配满足要求的节点, 并在节点按照要求创建Executor
- 创建的Executor向Driver注册。
- Driver将spark应用程序的代码和文件传送给分配的executor
- executor运行task, 运行完之后将结果返回给Driver或者写入HDFS或其他介质。

Spark运行基本流程



SparkContext对象代表了和一个集群的连接

(1) 首先为应用构建起基本的运行环境，即由Driver创建一个SparkContext，进行资源的申请、任务的分配和监控

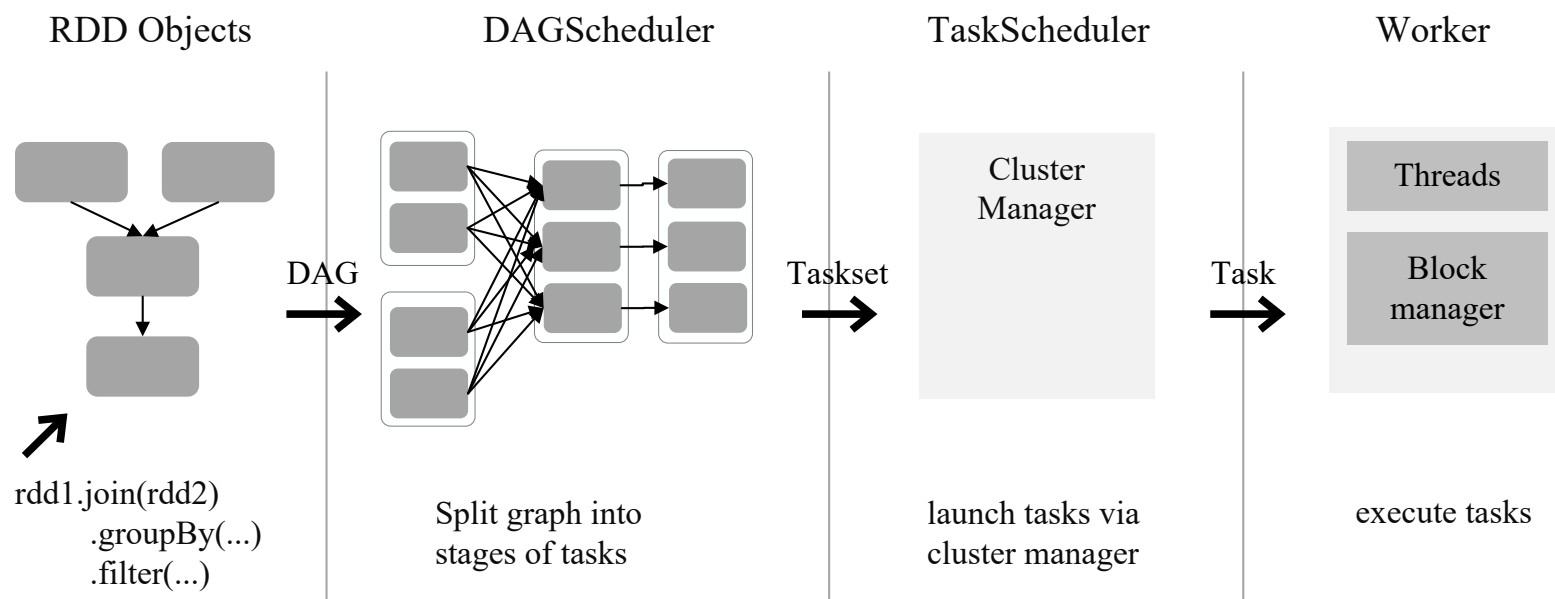
(2) 资源管理器为Executor分配资源，并启动Executor进程

(3) SparkContext根据RDD的依赖关系构建DAG图，DAG图提交给DAGScheduler解析成Stage，然后把一个个TaskSet提交给底层调度器TaskScheduler处理；Executor向SparkContext申请Task，Task Scheduler将Task发放给Executor运行，并提供应用程序代码

(4) Task在Executor上运行，把执行结果反馈给TaskScheduler，然后反馈给DAGScheduler，运行完毕后写入数据并释放所有资源

RDD运行过程

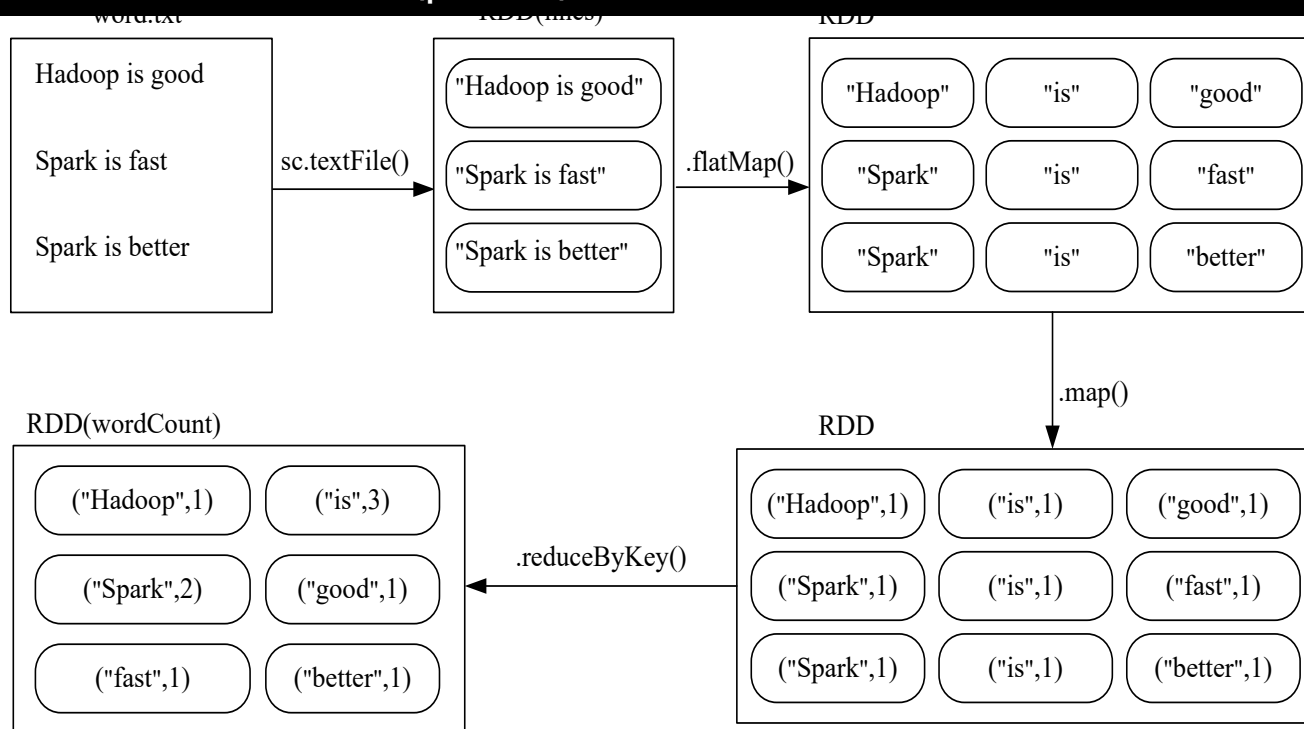
- (1) 创建RDD对象;
- (2) SparkContext负责计算RDD之间的依赖关系, 构建DAG;
- (3) DAGScheduler负责把DAG图分解成多个Stage, 每个Stage中包含了多个Task, 每个Task会被TaskScheduler分发给各个WorkerNode上的Executor去执行。



综合案例

本地文件word.txt包含很多行文本，每行文本由多个单词构成，单词之间用空格分隔。可以使用如下语句进行词频统计（即统计每个单词出现的次数）：

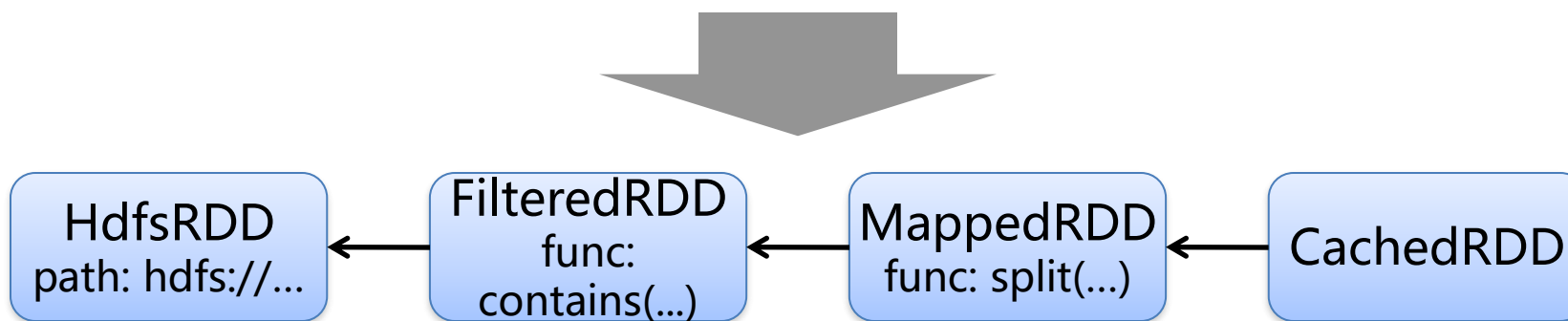
```
scala> val lines = sc. //代码一行放不下，可以在圆点后回车，在下行继续输入
| textFile("file:///usr/local/spark/mycode/wordcount/word.txt")
scala> val wordCount = lines.flatMap(line => line.split(" ")).
| map(word => (word, 1)).reduceByKey((a, b) => a + b)
scala> wordCount.collect()
scala> wordCount.foreach(println)
```



RDD容错机制

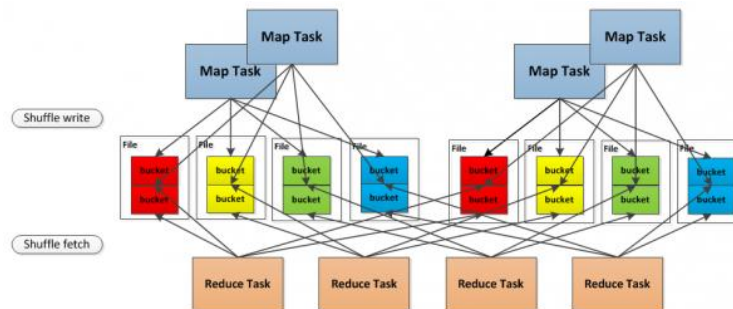
- RDD维护着可以用来重建丢失分区的信息
- 例如:

```
cachedMsgs = textFile(...).filter(_.contains( "error" ))  
                        .map(_.split( '\t' )(2))  
                        .cache()
```



Spark中的存储机制

- RDD缓存：包括基于内存和磁盘的缓存
 - ✓ 分区和数据块是一一对应的
 - ✓ 在内部建立RDD分区和数据块之间的映射，需要读取缓存的RDD时，根据映射关系取得分区对应的数据块
 - ✓ 内存缓存=哈希表+存取策略
 - ✓ 一个数据块对应着文件系统中的文件，文件名和块名称的映射关系是通过哈希算法计算所得的
- Shuffle数据的持久化
 - Shuffle数据块必须是在磁盘上进行缓存的
 - Shuffle数据块的存储有两种方式：
 - 将Shuffle数据块映射成文件
 - 将Shuffle数据块映射成文件中的一段



目录

Contents

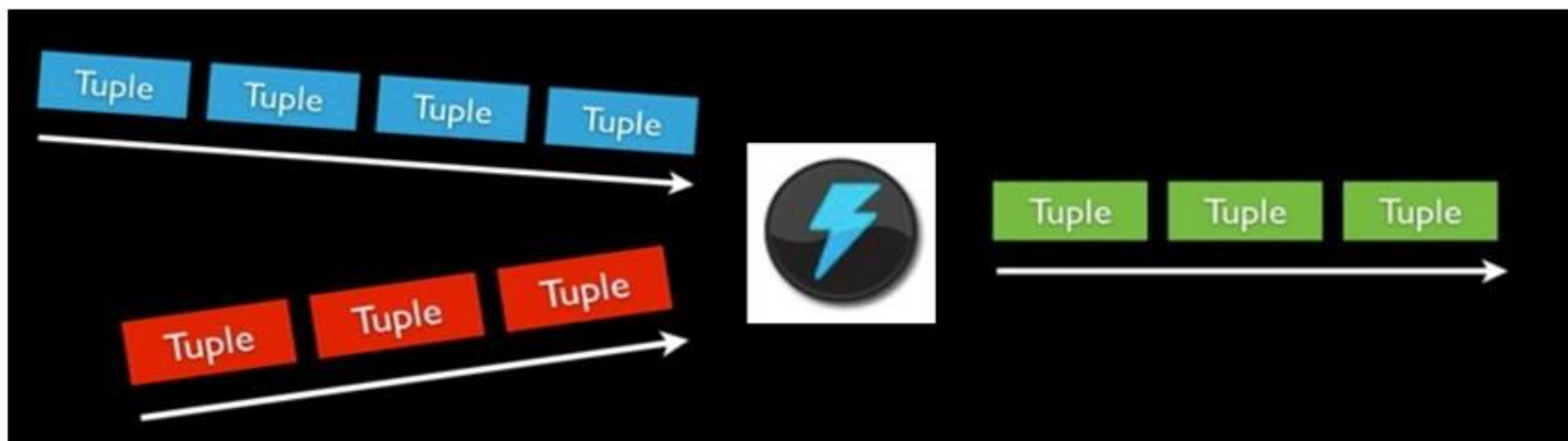
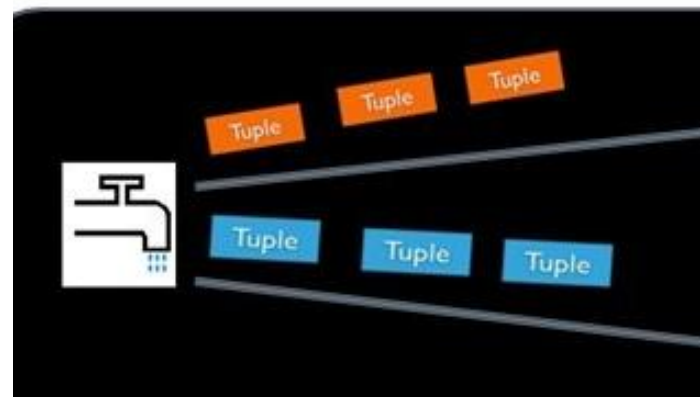
- 1 MapReduce与计算框架
- 2 数据的分片与路由
- 3 分布式文件系统
- 4 分布式计算框架例析：Hadoop
- 5 分布式计算框架例析：Spark
- 6 分布式计算框架例析：Storm**

流计算框架

- 业界诞生了许多专门的流数据实时计算系统来满足各自需求
- 目前有三类常见的流计算框架和平台：商业的流计算平台、开源流计算框架、公司为支持自身业务开发的流计算框架
- 商业平台：IBM InfoSphere Streams和IBM StreamBase
- 较为常见的是开源流计算框架，代表如下：
 - Twitter Storm：免费、开源的分布式实时计算系统，可简单、高效、可靠地处理大量的流数据
 - Yahoo! S4 (Simple Scalable Streaming System)：开源流计算平台，是通用的、分布式的、可扩展的、分区容错的、可插拔的流式系统
- 公司为支持自身业务开发的流计算框架：
 - Facebook Puma
 - Dstream (百度)
 - 银河流数据处理平台 (淘宝)

流计算的例子

- 统计某个服务被访问的客户端地域分布情况
 - 日志中记录了客户端IP
 - 把IP转换成地域
 - 按照地域进行统计



Hadoop处理流计算

- Hadoop貌似就可以轻松搞定
 - 日志存HDFS
 - 运行MapReduce程序
 - map做ip提取，转换成地域
 - reduce以地域为key聚合，计数统计
 - 从HDFS取出结果

Hadoop为什么不适合流计算?

- 如果有时效性要求呢?
 - 小时级：还行，每小时跑一个MapReduce Job
 - 10分钟：还凑合能跑
 - 5分钟：够呛了，等槽位可能要几分钟呢
 - 1分钟：算了吧，启动Job就要几十秒呢
 - 秒级：...
- 分析MapReduce不满足时效性要求的原因
 - 一批数据启动一次，处理完进程停止
 - 启动本身是需要时间的：输入切分、调度、起进程
 - 共享集群Job比较杂，可能需要等待资源
 - 所有数据都需要读写磁盘

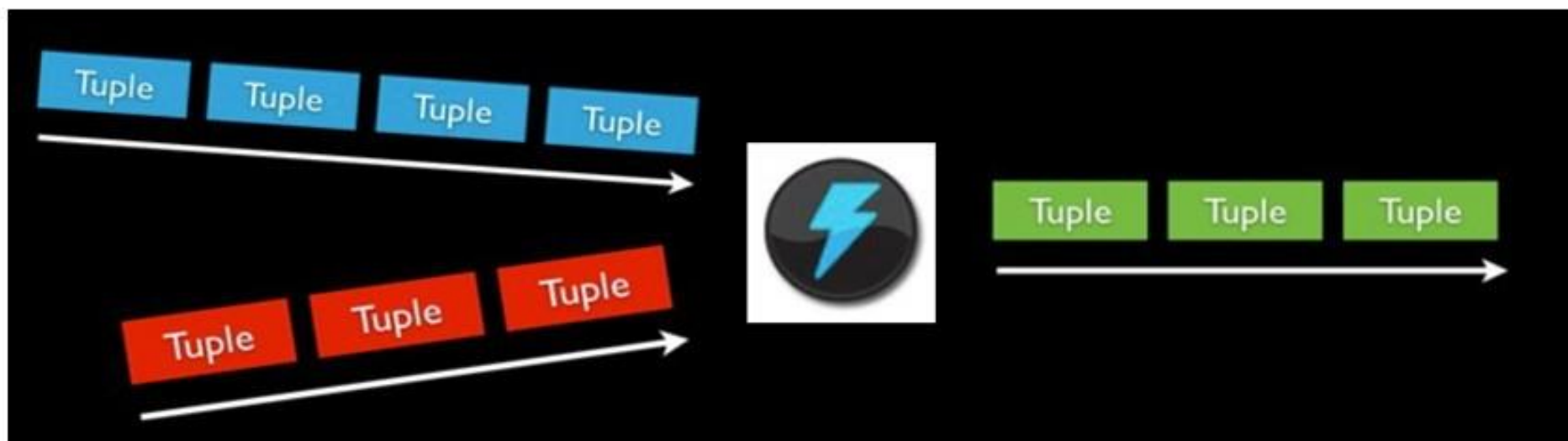


APACHE
STORMTM

Distributed • Resilient • Real-time

Storm的动机

- 解决方案
 - 进程常驻运行
 - 数据在内存中
- Storm正好适合这种需求



Storm简介

- Storm是个实时的、分布式以及具备高容错的计算系统。

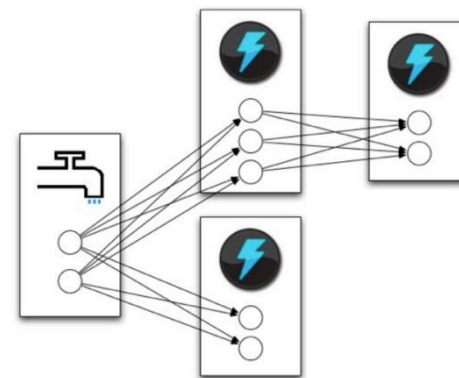
- 分布式

- 水平扩展：通过加机器、提高并发数就提高处理能力
 - 自动容错：自动处理进程、机器、网络异常

- 实时：数据不写磁盘，延迟低（毫秒级）

- 流式：不断有数据流入、处理、流出

- 开源：twitter开源，社区很活跃



- 同Hadoop一样Storm也可以处理大批量的数据，然而Storm在保证高可靠性的前提下还可以让处理进行的更加实时；
- Storm同样还具备容错和分布计算这些特性，这就让Storm可以扩展到不同的机器上进行大批量的数据处理。

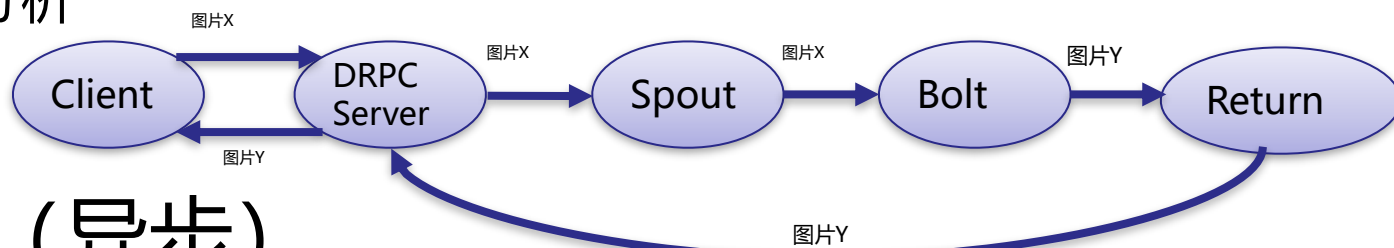
Storm和其他大数据计算平台对比

- Storm vs. MapReduce
 - 常驻运行
 - 流式处理：数据来一点处理一点
 - 实时处理：数据在内存中不写磁盘
 - DAG模型：可以组合多个阶段
- Storm vs. queue+worker系统
 - 维护简单：无需维护queue，queue和worker对应关系
 - 扩展简单：加机器，提高并发，重新提交
 - 自动容错：进程、机器、网络异常，消息可重发

- 请求应答（同步）

- DRPC

- 实时图片处理
 - 实时网页分析



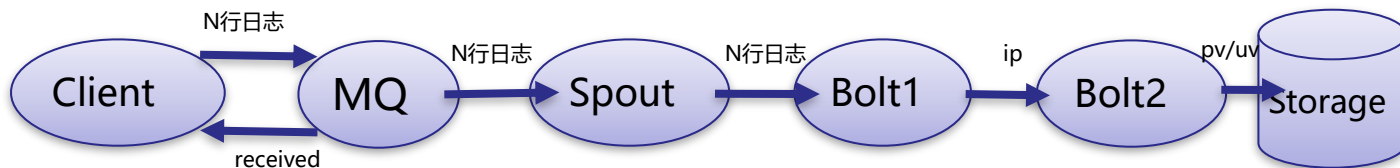
- 流式处理（异步）

- 逐条处理

- 数据之间无关系：如实时日志格式标准化入库

- 分析统计

- 数据之间有关系（聚合等）：如日志pv/uv统计、访问热点统计



应用场景

●数据流处理

可用来实时处理新数据和更新数据库，兼具容错性和可扩展性。

●连续计算

可进行连续查询并把结果即时反馈给客户端。

例如：把Twitter上的热门话题发送到浏览器中。

●分布式远程程序调用

可用来并行处理密集查询。其拓扑结构是一个等待调用信息的分布函数，当它收到一条调用信息后，会对查询进行计算，并返回查询结果。

例如：可以做并行搜索或者处理大集合的数据。

Storm的特点

- 编程模型简单

类似于MapReduce降低了并行批处理复杂性，Storm降低了进行实时处理的复杂性。

- 可以使用各种编程语言

可以在Storm之上使用各种编程语言。默认支持Clojure、Java、Ruby和Python。要增加对其他语言的支持，只需实现一个简单的Storm通信协议即可。

- 容错性

Storm会管理工作进程和节点的故障。

- 水平扩展

计算是在多个线程、进程和服务器之间并行进行的。

- 可靠的消息处理

Storm保证每个消息至少能得到一次完整处理。任务失败时，它会负责从消息源重试消息。

- 快速

系统的设计保证了消息能得到快速的处理

- 本地模式

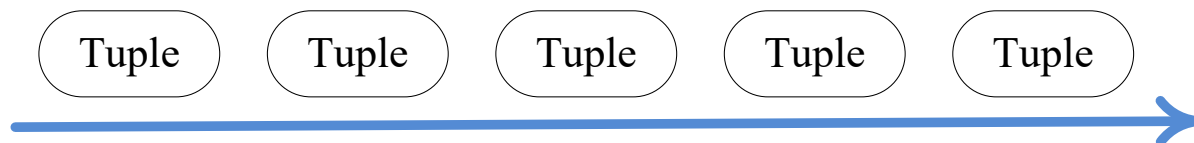
可以在处理过程中完全模拟Storm集群，可以快速进行开发和单元测试。

Storm处理对象的抽象

- **Streams:** Storm将流数据Stream描述成一个无限的Tuple序列，这些Tuple序列会以分布式的方式并行地创建和处理

Streams

无界的Tuple序列



- 每个tuple是一堆值，每个值有一个名字，并且每个值可以是任何类型
- Tuple应该是一个Key-Value的Map，由于各个组件间传递的tuple的字段名称已经事先定义好了，所以Tuple只需要按序填入各个Value，所以就是一个Value List（值列表）

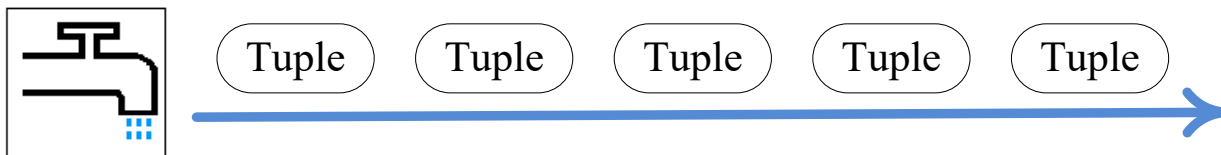
Field1	Field2	Field3	Field4
--------	--------	--------	--------

流处理方式的抽象

- **Spout:** Storm认为每个Stream都有一个源头，并把这个源头抽象为Spout
- 通常Spout会从外部数据源（队列、数据库等）读取数据，然后封装成Tuple形式，发送到Stream中。Spout是一个主动的角色，在接口内部有个nextTuple函数，Storm框架会不停地调用该函数

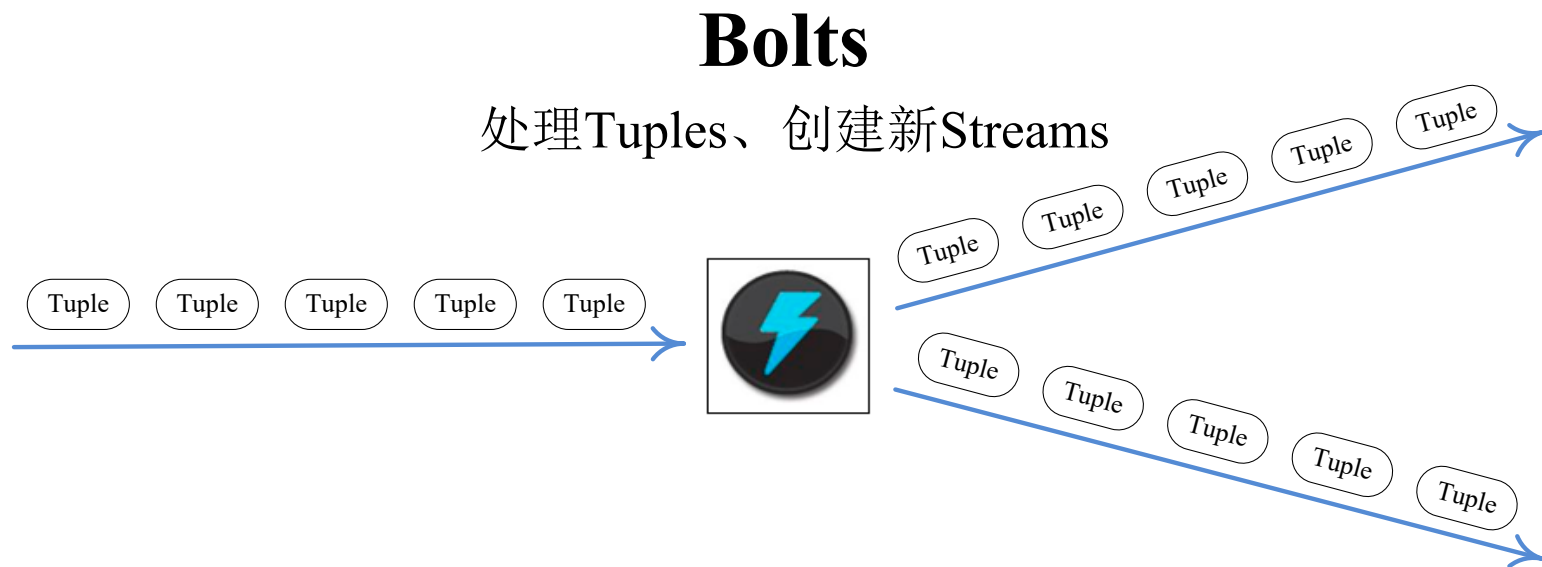
Spouts

Streams的来源



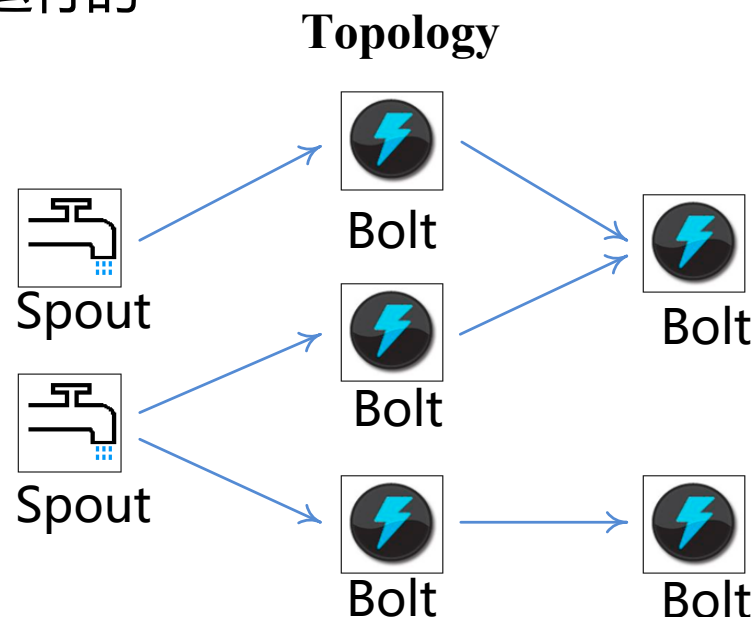
状态转换的抽象

- **Bolt**: Storm将Streams的状态转换过程抽象为Bolt。Bolt即可以处理Tuple，也可以将处理后的Tuple作为新的Streams发送给其他Bolt
- Bolt可以执行过滤、函数操作、Join、操作数据库等任何操作
- Bolt是一个被动的角色，其接口中有一个execute(Tuple input)方法，在接收到消息之后会调用此函数，用户可以在此方法中执行自己的处理逻辑



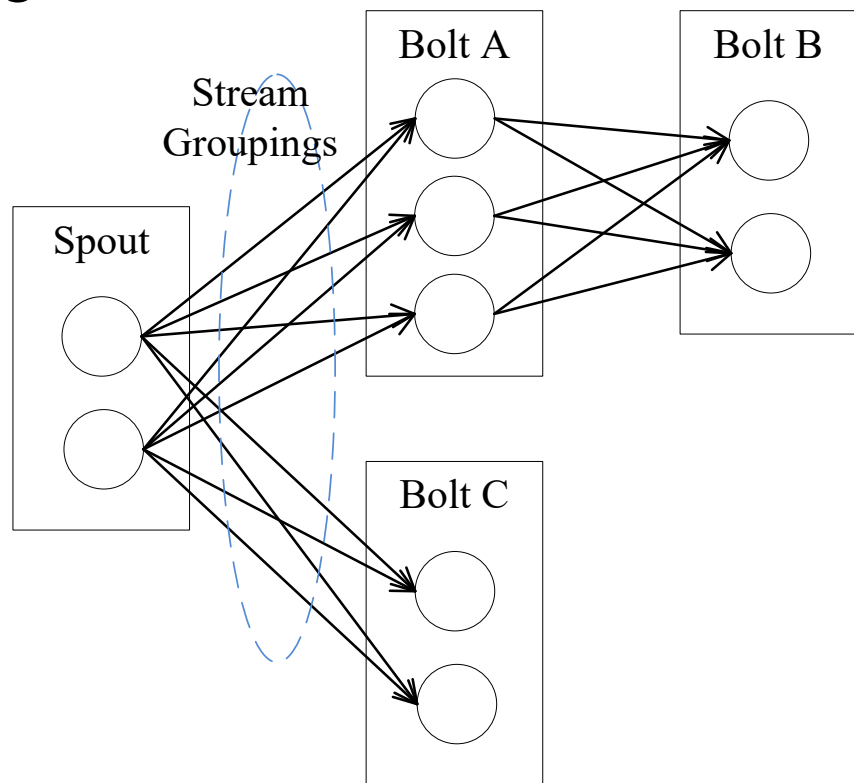
操作关系的抽象

- **Topology**: Storm将Spouts和Bolts组成的网络抽象成Topology, 它可以被提交到Storm集群执行。Topology可视为流转换图, 图中节点是一个Spout或Bolt, 边则表示Bolt订阅了哪个Stream。当Spout或者Bolt发送元组时, 它会把元组发送到每个订阅了该Stream的Bolt上进行处理
- Topology里面的每个处理组件 (Spout或Bolt) 都包含处理逻辑, 而组件之间的连接则表示数据流动的方向
- Topology里面的每一个组件都是并行运行的
- 在Topology里面可以指定每个组件的并行度, Storm会在集群里面分配那么多的线程来同时计算
- 在Topology的具体实现上, Storm中的Topology定义仅仅是一些Thrift结构体 (二进制高性能的通信中间件), 支持各种编程语言进行定义



组件数据关系的抽象

- **Stream Groupings:** Storm中的Stream Groupings用于告知Topology如何在两个组件间（如Spout和Bolt之间，或者不同的Bolt之间）进行Tuple的传送。每一个Spout和Bolt都可以有多个分布式任务，一个任务在什么时候、以什么方式发送Tuple就是由Stream Groupings来决定的



Stream Grouping的方式

- (1) ShuffleGrouping: 随机分组, 随机分发Stream中的Tuple, 保证每个Bolt的Task接收Tuple数量大致一致
- (2) FieldsGrouping: 按照字段分组, 保证相同字段的Tuple分配到同一个Task中
- (3) AllGrouping: 广播发送, 每一个Task都会收到所有的Tuple
- (4) GlobalGrouping: 全局分组, 所有的Tuple都发送到同一个Task中
- (5) NonGrouping: 不分组, 和ShuffleGrouping类似, 当前Task的执行会和它的被订阅者在同一个线程中执行
- (6) DirectGrouping: 直接分组, 直接指定由某个Task来执行Tuple的处理

Storm开发API

- Java API
 - Spout
 - nextTuple(): 回调函数, 循环触发
 - ack(id): 回调函数, 消息成功处理时触发
 - fail(id): 回调函数, 消息超时时触发
 - Bolt
 - execute(Tuple input): 回调函数, 数据触发
 - collector.emit(tuple): 通过collector向下游发送tuple
 - collector.ack(tuple): 通过collector确认已经成处理输入tuple
- 一定要用Java吗?
NO, ShellBolt

Storm例程

```
• public class MD5Topology {
•     public static class MD5Bolt extends BaseBasicBolt {
•         @Override
•         public void execute(Tuple tuple, BasicOutputCollector collector) {
•             String input = tuple.getString(0); // 获取来自DRPCSpout的实际输入数据
•             String output = MD5Util.getMD5Str(input);
•             // 往下游ReturnBolt emit数据
•             // 第一个字段是计算的结果, 这里是md5串
•             // 第二个字段是来自DRPCSpout的return-info, 是一个json串, 包括drpc request id, server host、
port
•             collector.emit(new Values(output, tuple.getString(1)));
•         }
•         @Override
•         public void declareOutputFields(OutputFieldsDeclarer declarer) {
•             declarer.declare(new Fields("result", "return-info")); // 声明输出两个字段, 和emit是对应的
•         }
•     }
•     public static void main(String[] args) throws Exception {
•         TopologyBuilder builder = new TopologyBuilder();

•         builder.setSpout("DRPCSpout", new DRPCSpout(args[0]), 2);
•         builder.setBolt("MD5Bolt", new MD5Bolt(), 4) // 参数依次是spout/bolt id, spout/bolt对象, 并发度
•             .shuffleGrouping("DRPCSpout"); // 指定上游以及grouping方式
•         builder.setBolt("ReturnBolt", new ReturnResults(), 2)
•             .shuffleGrouping("MD5Bolt");

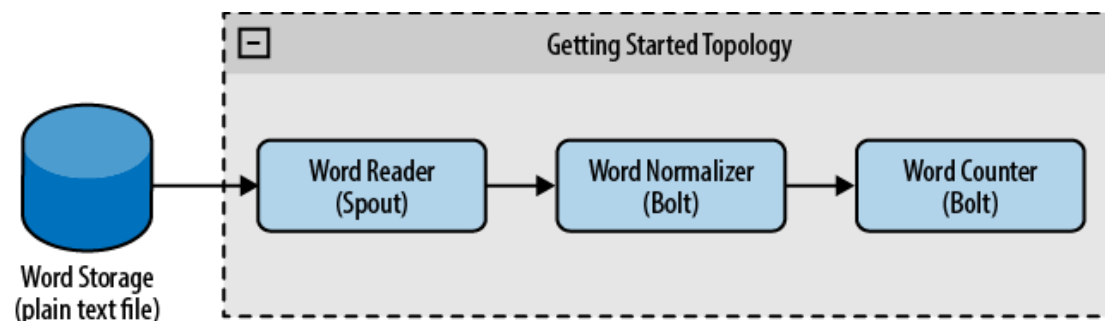
•         Config conf = new Config();
•         conf.setNumWorkers(4); // 设置worker个数
•         StormSubmitter.submitTopology(args[0], conf, builder.createTopology());
•     }
• }
```

Storm实战 - 应用开发

- 编译打包成md5.jar
- 提交topology
 - \$STORM_HOME/bin/storm jar md5.jar MD5Topology md5
- 在webui查看topology状态
- 发送drpc请求
 - incubator-storm/storm-core/src/py/storm/DistributedRPC-remote -h host:port -f execute md5 abcd

Wordcount例子

- 实现一个简单的topology来统计一个文件中的每个单词出现的次数。
- 为了实现这个topology，使用一个spout来负责读取单词，第一个bolt来标准化单词，第二个bolt来为单词计数：



```
public class WordReader implements IRichSpout {  
    .....  
    public void open(Map conf, TopologyContext context,  
        SpoutOutputCollector collector) {  
        try {  
            this.context = context;  
            this.fileReader = new  
                FileReader(conf.get("wordsFile").toString());  
        } catch (FileNotFoundException e) {  
            throw new RuntimeException("Error reading file  
                [" + conf.get("wordFile") + "]);  
        }  
        this.collector = collector;  
    }  
    .....  
}
```

spout

```
public void nextTuple() {
    if(completed){
        try {Thread.sleep(1);} catch (InterruptedException e)
        {
            //Do nothing
        }
        return;
    }
    String str;
    BufferedReader reader = new BufferedReader(fileReader);
    try{
        while((str = reader.readLine()) != null){
            this.collector.emit(new Values(str));}
    }catch(Exception e){
        throw new RuntimeException("Error reading tuple",e);
    }finally{
        completed = true;
    }
}
```

- **WordNormalizer**

负责获取行并且标准化行。它会将行分隔成单词，将单词转化成小写并且trim单词，发送给下一个Bolt.

- **WordCounter**

负责为单词计数。

main

```
public static void main(String[] args) throws InterruptedException {  
    //Topology definition  
    TopologyBuilder builder = new TopologyBuilder();  
    builder.setSpout("word-reader",new WordReader());  
    builder.setBolt("word-normalizer", new WordNormalizer()).shuffleGrouping("word-  
        reader");  
    builder.setBolt("word-counter", new WordCounter(),2).fieldsGrouping("word-  
        normalizer", new Fields("word"));  
    //Configuration  
    Config conf = new Config();  
    conf.put("wordsFile", args[0]);  
    conf.setDebug(false);  
    //Topology run  
    conf.put(Config.TOPOLOGY_MAX_SPOUT_PENDING, 1);  
    Creating Our First Topology | 17  
    LocalCluster cluster = new LocalCluster();  
    cluster.submitTopology("Getting-Started-Topologie", conf,builder.createTopology());  
    Thread.sleep(1000);  
    cluster.shutdown();  
}
```

Spout

- Ispout接口

- BaseRichSpout, ClojureSpout, DRPCSpout, FeederSpout, FixedTupleSpout, MasterBatchCoordinator, NoOpSpout, RichShellSpout, RichSpoutBatchTriggerer, ShellSpout, SpoutTracker, TestPlannerSpout, TestWordSpout, TransactionalSpoutCoordinator

Method Summary

void	ack (java.lang.Object msgId) Storm has determined that the tuple emitted by this spout with the msgId identifier has been fully processed.
void	activate () Called when a spout has been activated out of a deactivated mode.
void	close () Called when an ISpout is going to be shutdown.
void	deactivate () Called when a spout has been deactivated.
void	fail (java.lang.Object msgId) The tuple emitted by this spout with the msgId identifier has failed to be fully processed.
void	nextTuple () When this method is called, Storm is requesting that the Spout emit tuples to the output collector.
void	open (java.util.Map conf, TopologyContext context, SpoutOutputCollector collector) Called when a task for this component is initialized within a worker on the cluster.

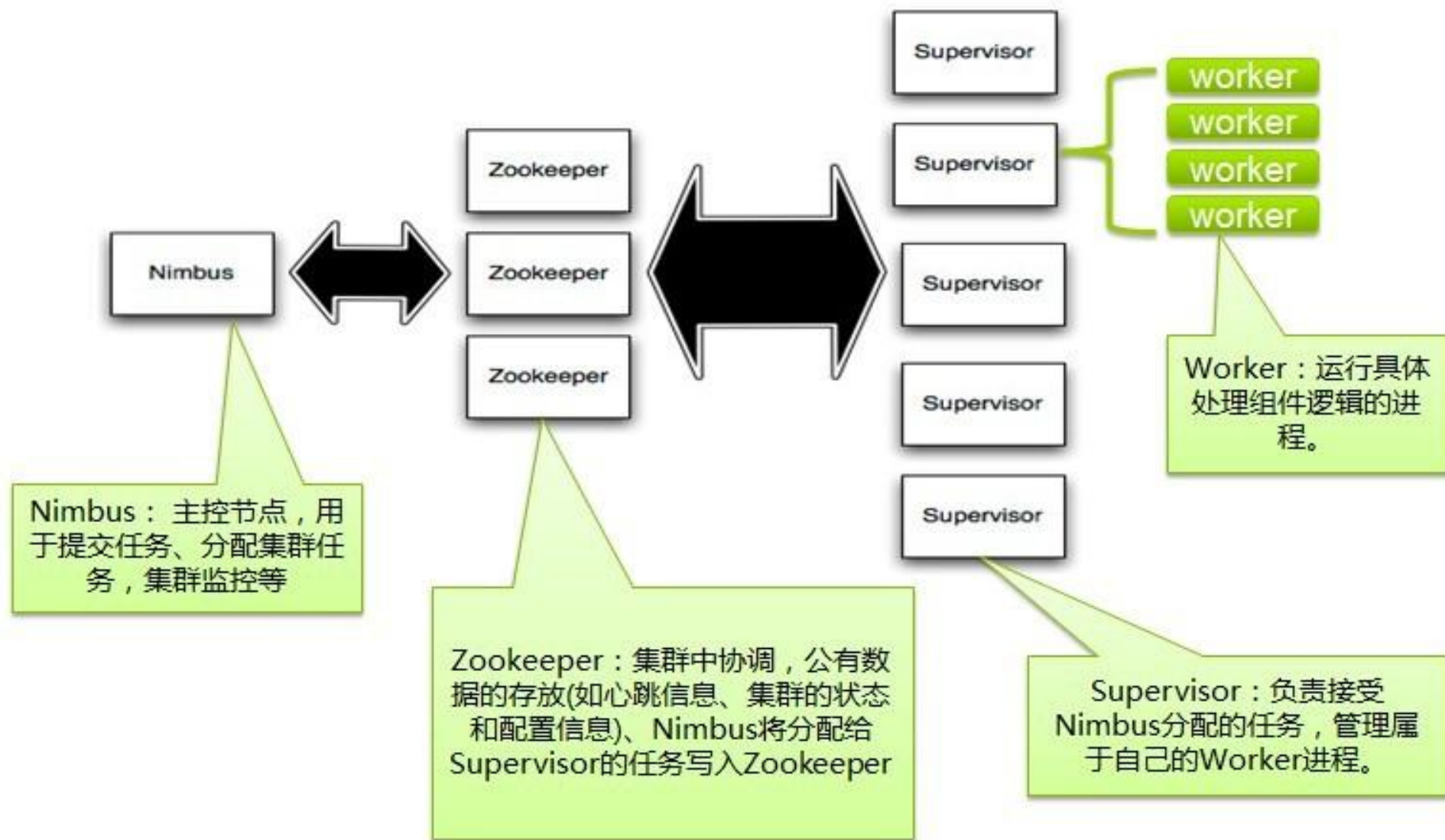
Bolts

- Ibolt接口

- BaseRichBolt, BasicBoltExecutor, BatchBoltExecutor, BoltTracker, ClojureBolt, CoordinatedBolt, JoinResult, KeyedFairBolt, NonRichBoltTracker, ReturnResults, BaseShellBolt, ShellBolt, TestAggregatesCounter, TestGlobalCount, TestPlannerBolt, TransactionalSpoutBatchExecutor, TridentBoltExecutor, TupleCaptureBolt

Method Summary	
void	<u>cleanup()</u> Called when an IBolt is going to be shutdown.
void	<u>execute(Tuple input)</u> Process a single tuple of input.
void	<u>prepare(java.util.Map stormConf, TopologyContext context, OutputCollector collector)</u> Called when a task for this component is initialized within a worker on the cluster.

技术架构



技术架构

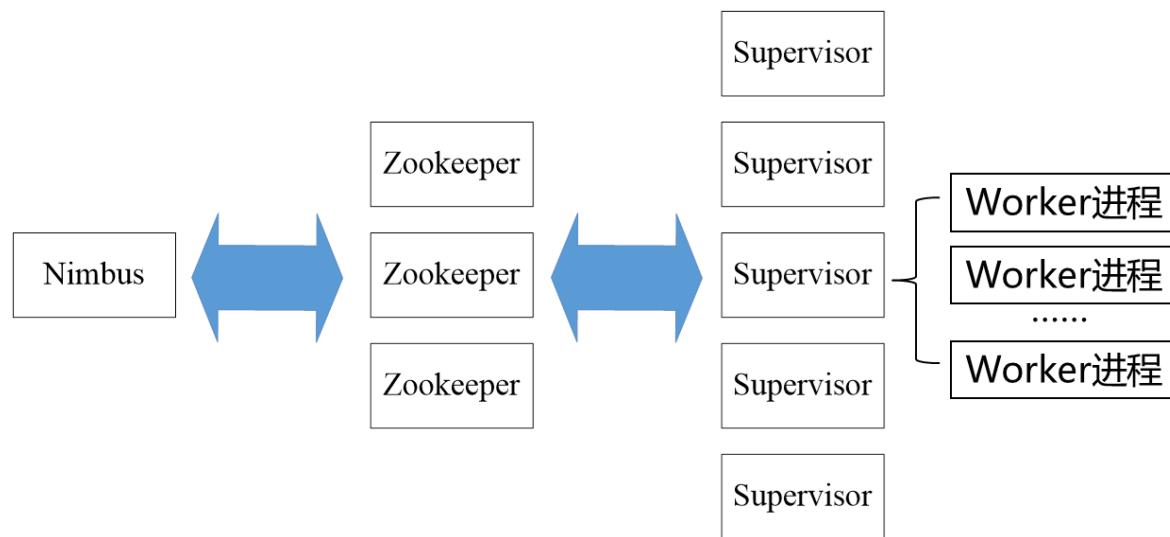
	Hadoop	Storm
系统角色	JobTracker	Nimbus
	TaskTracker	Supervisor
	Child	Worker
应用名称	Job	Topology
组件接口	Mapper/Reducer	Spout/Bolt

关键组件

- Storm集群采用 “**Master—Worker**” 的节点方式
- **Nimbus**（主节点）：主节点通常运行一个后台程序——Nimbus，用于响应分布在集群中的节点，分配任务和监测故障。这个很类似于Hadoop中的Job Tracker。
- **Supervisor**（工作节点）：工作节点同样会运行一个后台程序 Supervisor，用于收听工作指派并基于要求运行工作进程。每个工作节点都是topology中一个子集的实现。而Nimbus和Supervisor之间的协调则通过Zookeeper系统或者集群。

关键组件

- **Zookeeper:** Zookeeper是完成Supervisor和Nimbus之间协调的服务。而应用程序实现实时的逻辑则被封装进Storm中的“topology”。 topology则是一组由Spouts（数据源）和Bolts（数据操作）通过Stream Groupings进行连接的图
- Storm使用Zookeeper来作为分布式协调组件，负责Nimbus和多个Supervisor之间的所有协调工作。借助于Zookeeper，若Nimbus进程或Supervisor进程意外终止，重启时也能读取、恢复之前的状态并继续工作，使得Storm极其稳定



Storm框架设计

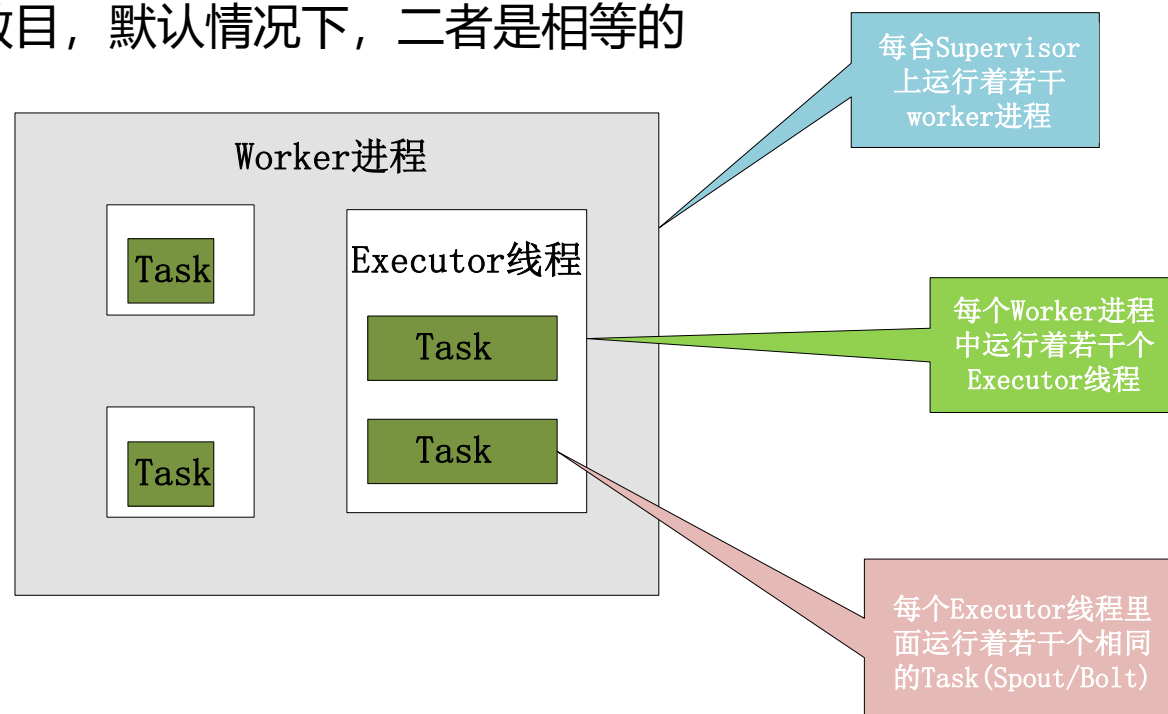
- Storm运行任务的方式与Hadoop类似：Hadoop运行的是MapReduce作业，而Storm运行的是“Topology”
- 但两者的任务大不相同，主要的不同是：MapReduce作业最终会完成计算并结束运行，而Topology将持续处理消息（直到人为终止）

Storm和Hadoop架构组件功能对应关系

	Hadoop	Storm
应用名称	Job	Topology
系统角色	JobTracker	Nimbus
	TaskTracker	Supervisor
组件接口	Map/Reduce	Spout/Bolt

Worker、Executor和Task的关系

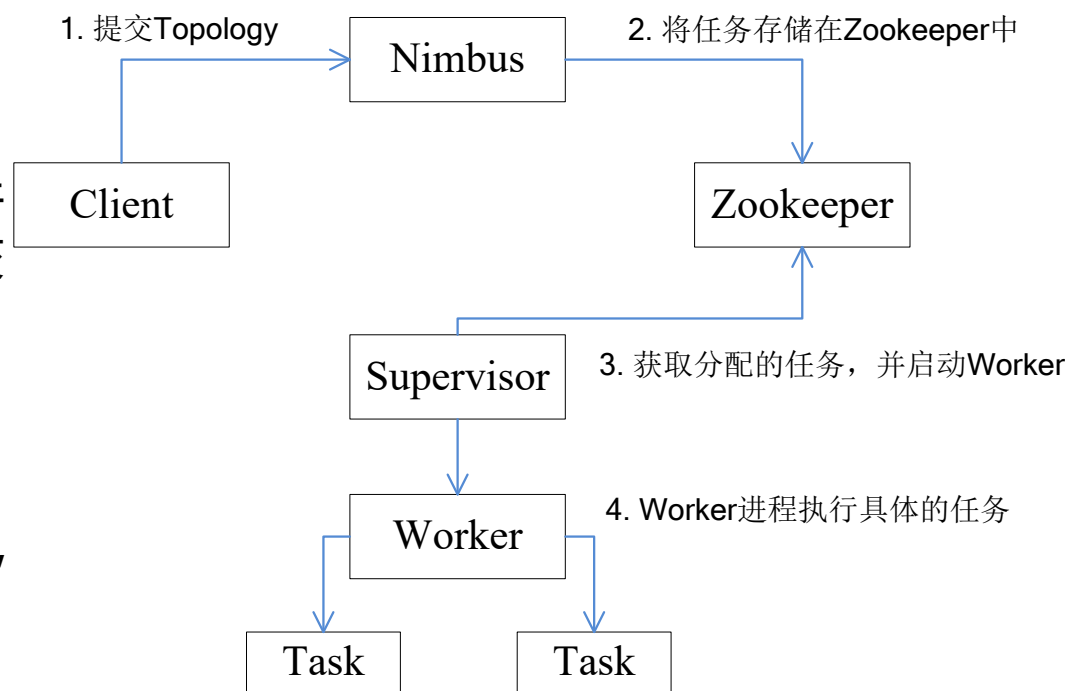
- (1) worker: 每个worker进程都属于一个特定的Topology，每个Supervisor节点的worker可以有多个，每个worker对Topology中的每个组件（Spout或Bolt）运行一个或者多个executor线程来提供task的运行服务
- (2) executor: executor是产生于worker进程内部的线程，会执行同一个组件的一个或者多个task。
- (3) task: 实际的数据处理由task完成，在Topology的生命周期中，每个组件的task数目是不会发生变化的，而executor的数目却不一定。executor数目小于等于task的数目，默认情况下，二者是相等的



Storm框架设计

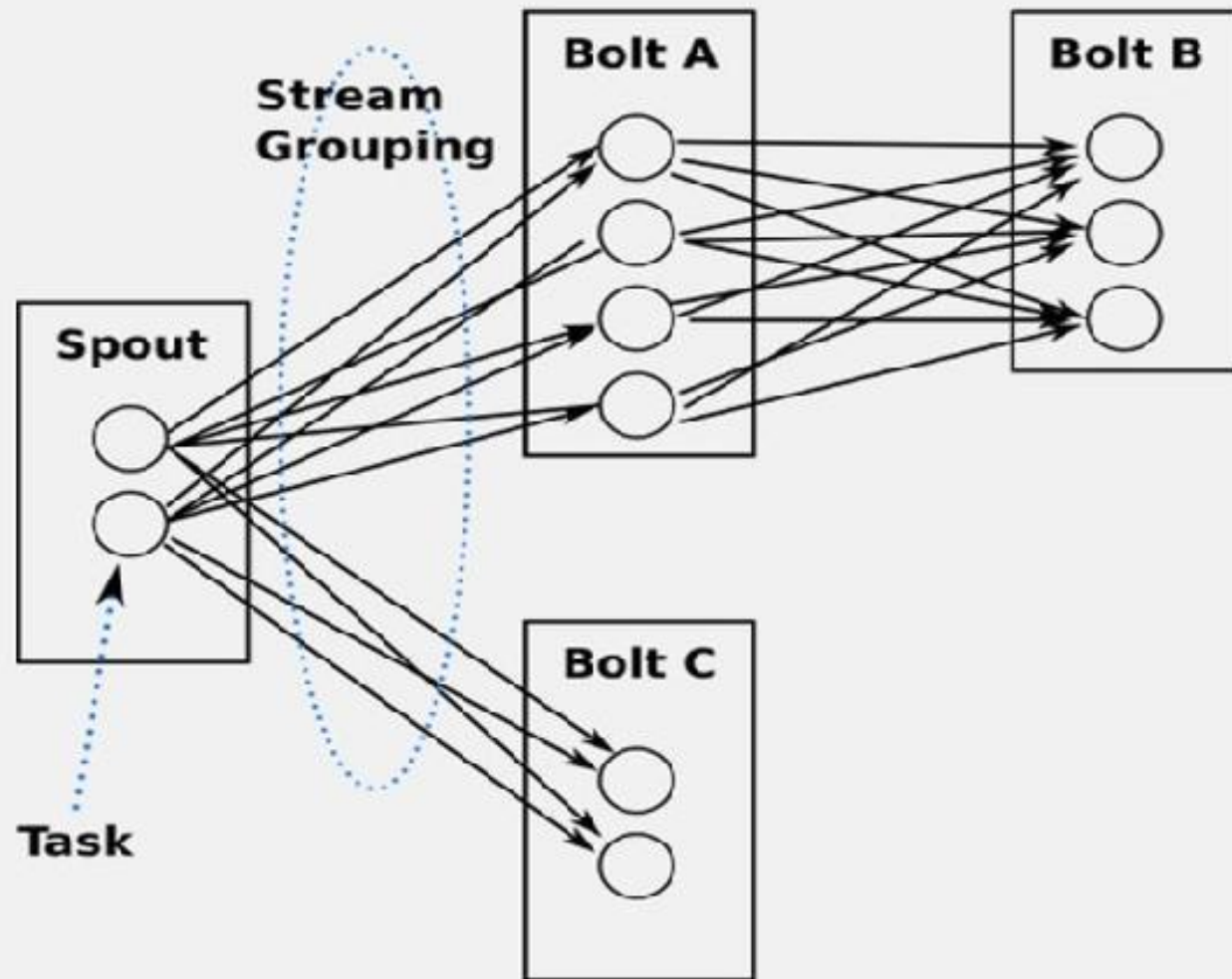
• 基于这样的架构设计，Storm的工作流程如下图所示：

- 所有Topology任务的提交必须在Storm客户端节点上进行，提交后，由Nimbus节点分配给其他Supervisor节点进行处理
- Nimbus节点首先将提交的Topology进行分片，分成一个个Task，分配给相应的Supervisor，并将Task和Supervisor相关的信息提交到Zookeeper集群上
- Supervisor会去Zookeeper集群上认领自己的Task，通知自己的Worker进程进行Task的处理
- 说明：在提交了一个Topology之后，Storm就会创建Spout/Bolt实例并进行序列化。之后，将序列化的组件发送给所有的任务所在的机器(即Supervisor节点)，在每一个任务上反序列化组件



Storm工作流程示意图

Topology



Stream Groupings: Stream Grouping定义了一个流在Bolt任务间该如何被切分。这里有Storm提供的6个Stream Grouping类型：

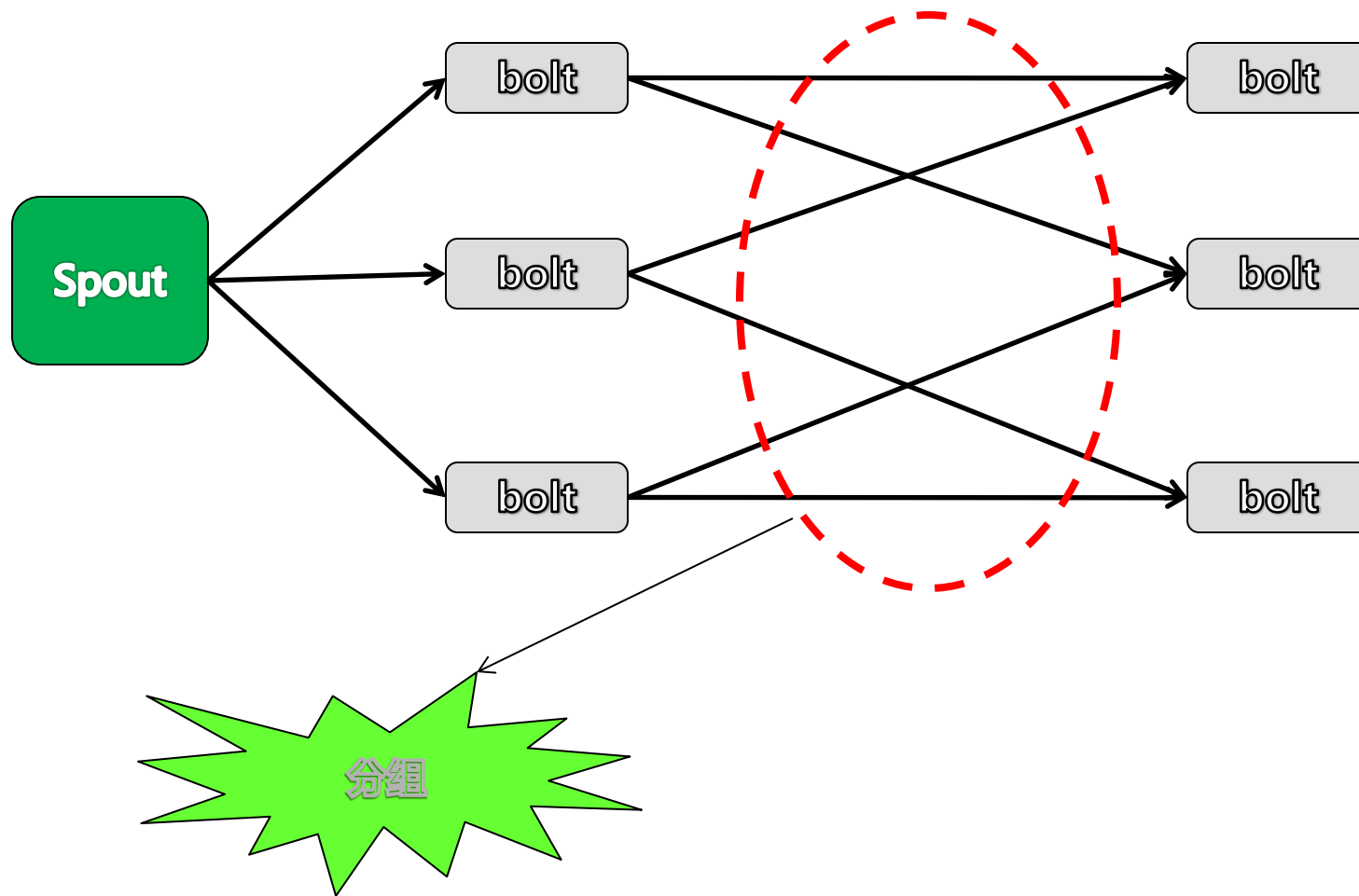
- 1. **随机分组** (Shuffle grouping)：随机分发tuple到Bolt的任务，保证每个任务获得相等数量的tuple。
- 2. **字段分组** (Fields grouping)：根据指定字段分割数据流，并分组。例如，根据“user-id”字段，相同“user-id”的元组总是分发到同一个任务，不同“user-id”的元组可能分发到不同的任务。

关键组件

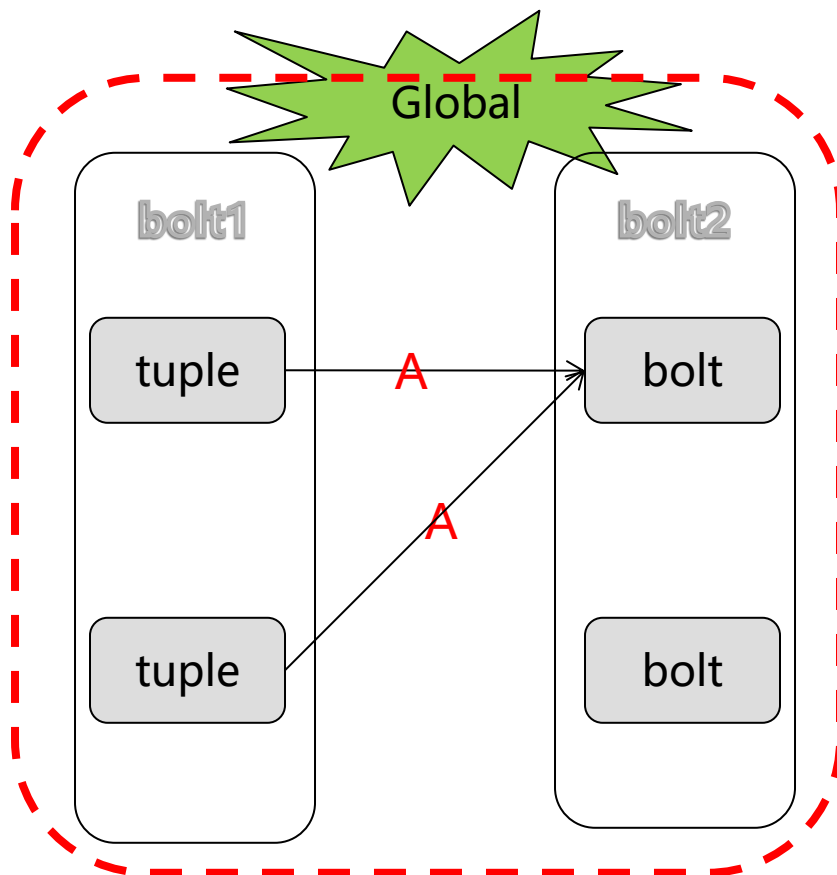
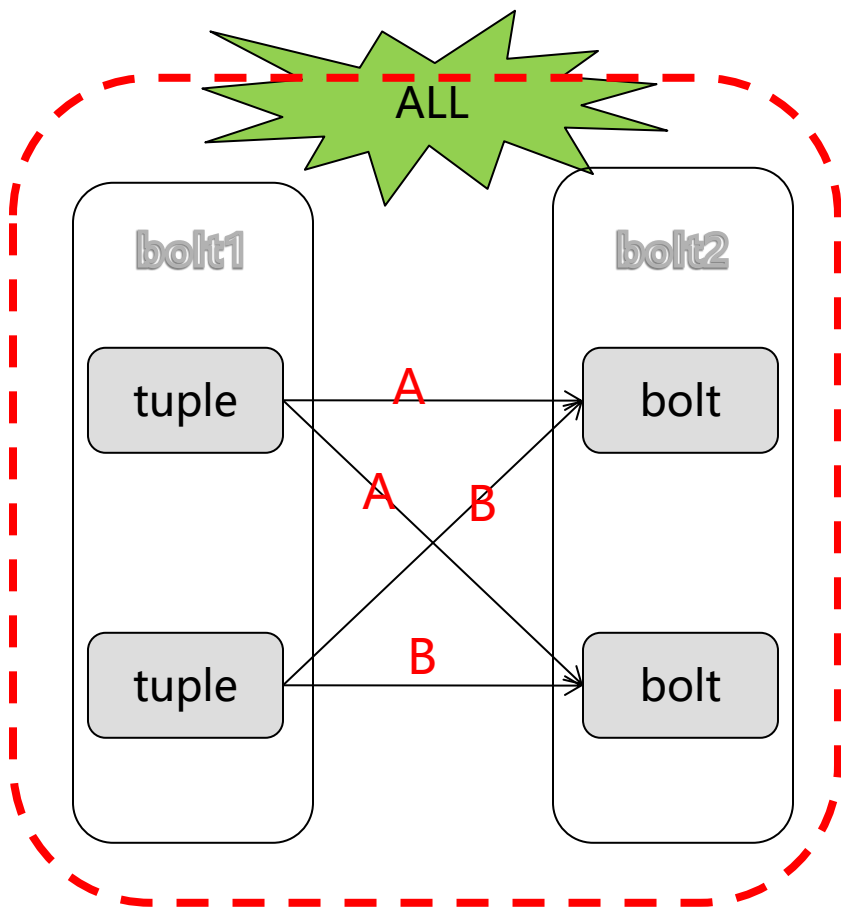
- 3. **全部分组** (All grouping) : tuple被复制到bolt的所有任务。这种类型需要谨慎使用。
- 4. **全局分组** (Global grouping) : 全部流都分配到bolt的同一个任务。明确地说, 是分配给ID最小的那个task。
- 5. **无分组** (None grouping) : 你不需要关心流是如何分组。目前, 无分组等效于随机分组。但最终, Storm 将把无分组的Bolts放到Bolts或Spouts订阅它们的同一线程去执行 (如果可能) 。
- 6. **直接分组** (Direct grouping) : 这是一个特别的分组类型。元组生产者决定tuple由哪个元组处理者任务接收。

分组

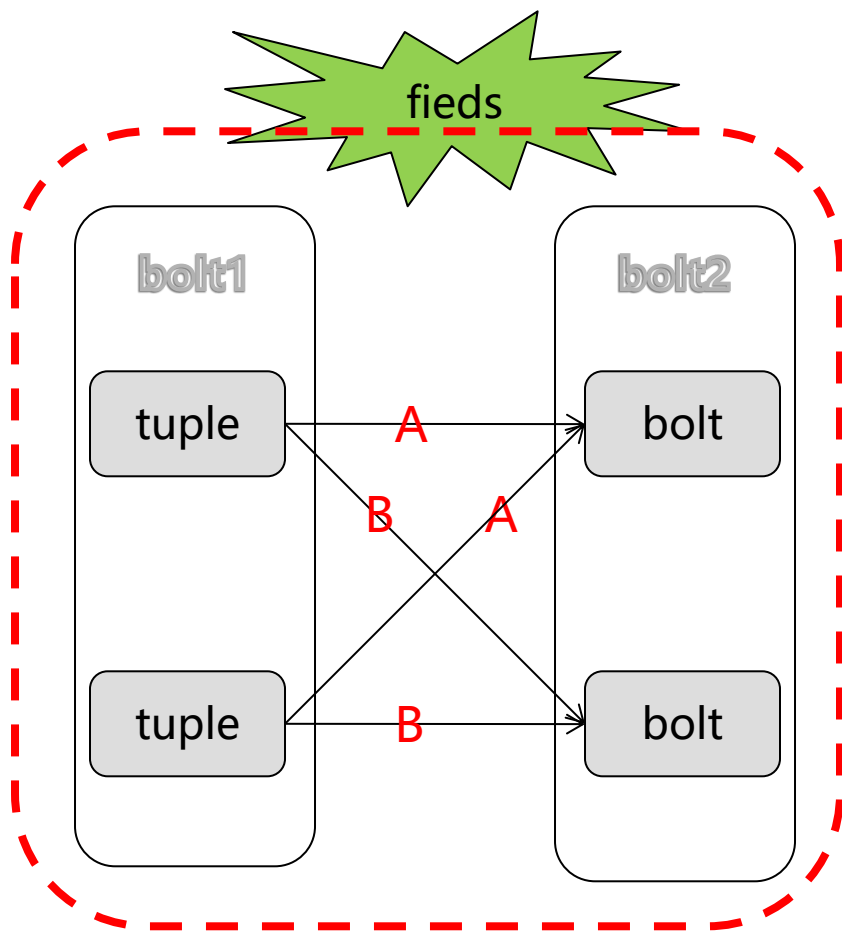
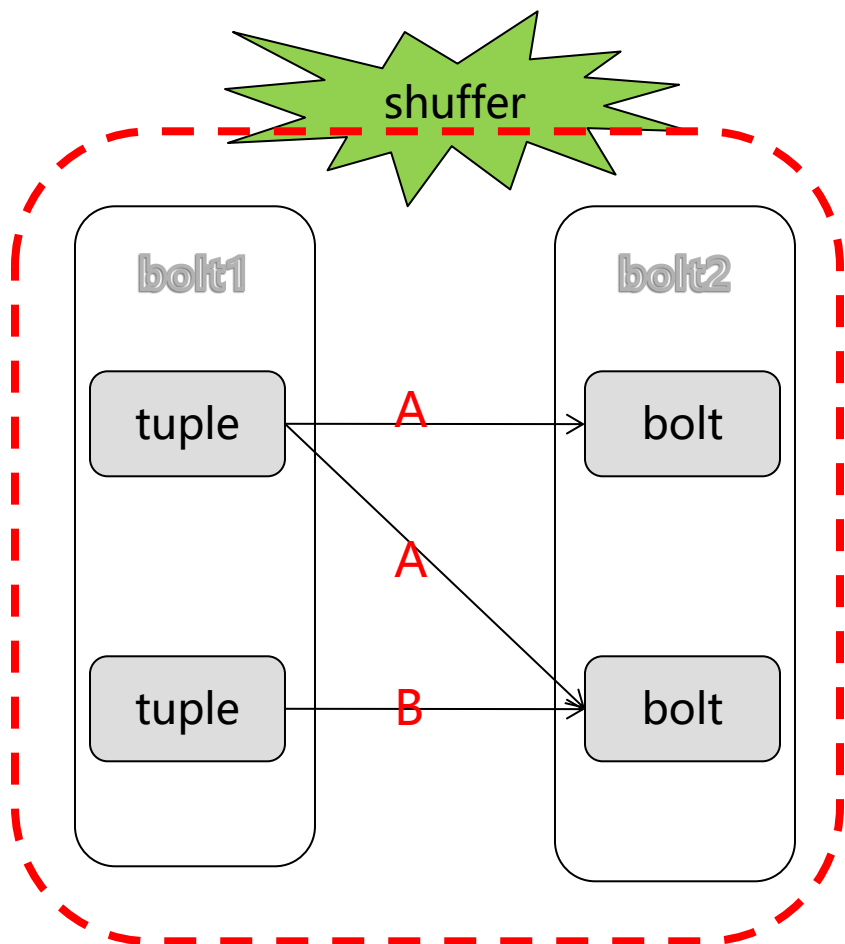
- 分组



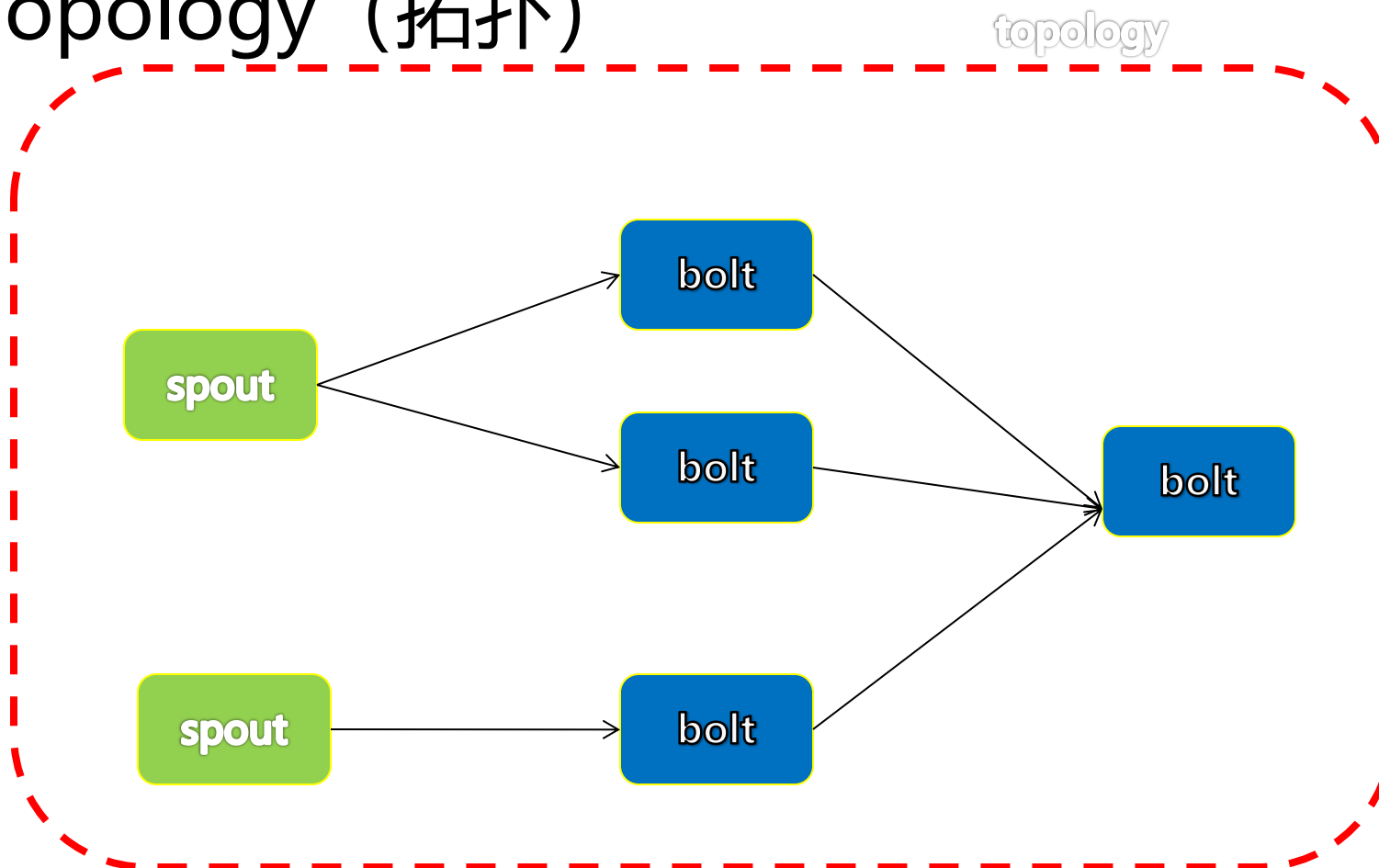
几种重要的分组模式



几种重要的分组模式

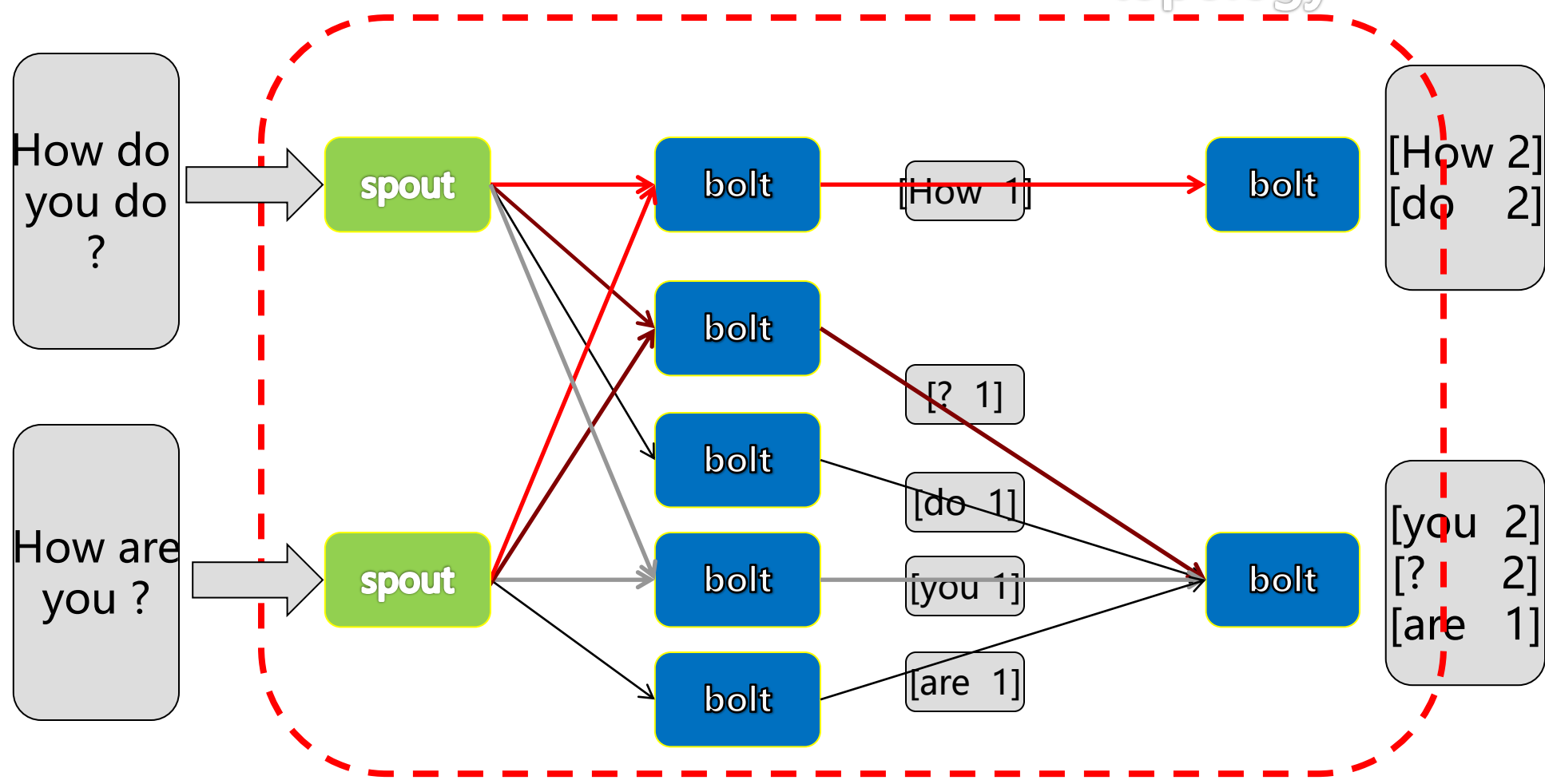


Topology (拓扑)

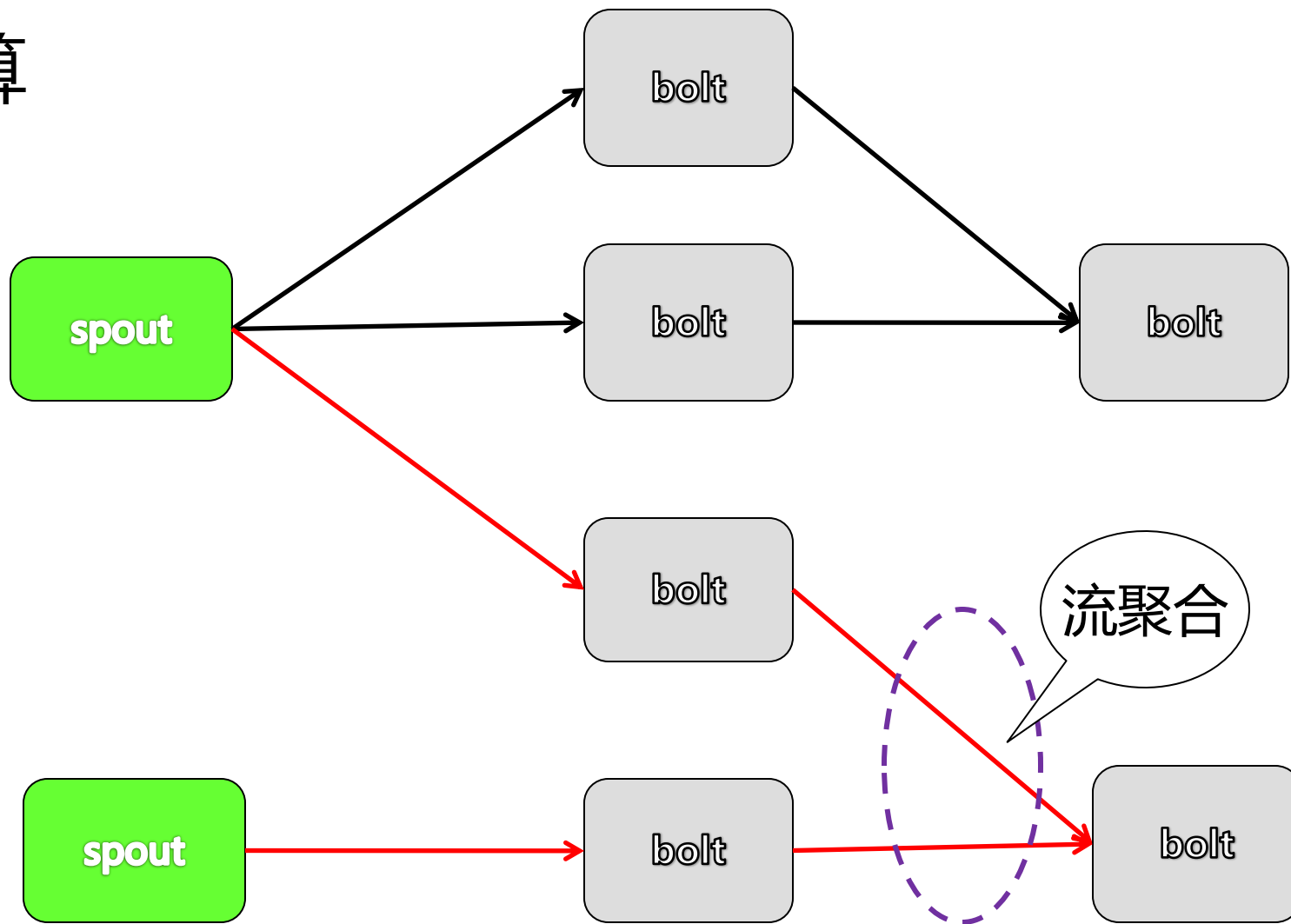


拓扑示例

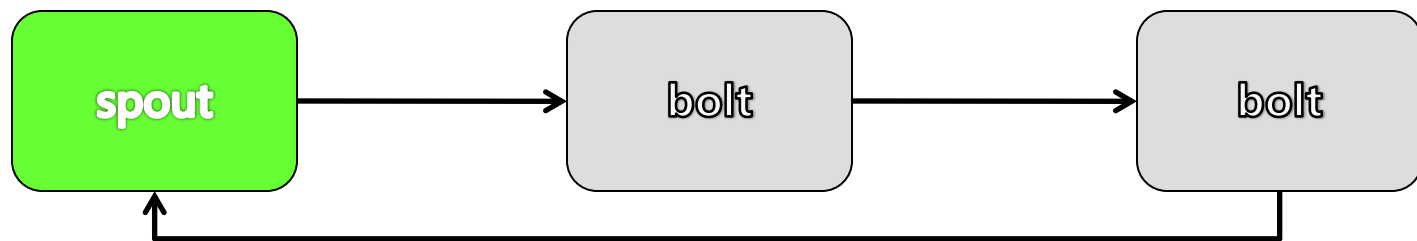
Topology (拓扑) ---统计单词数量



流式计算

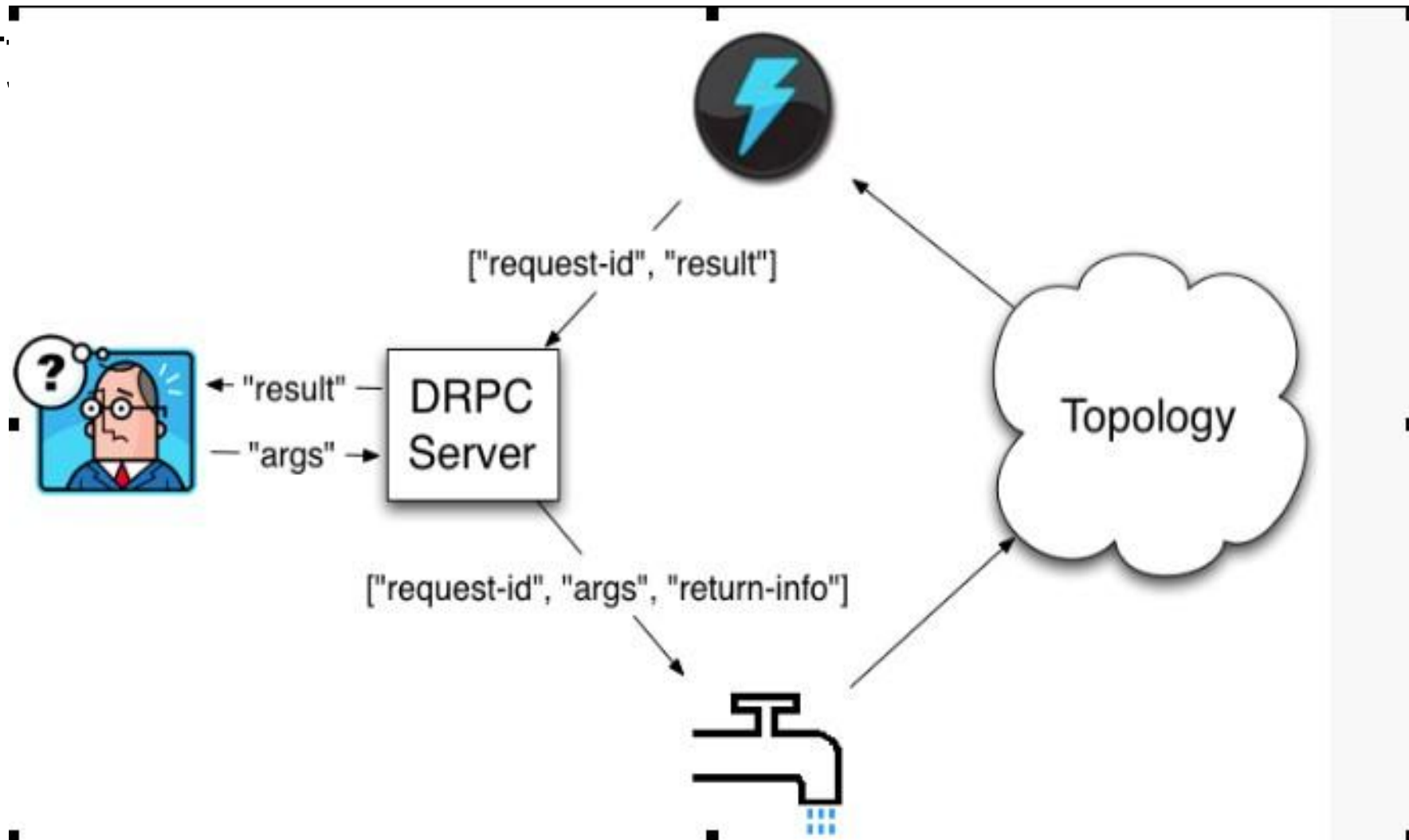


- 持续计算



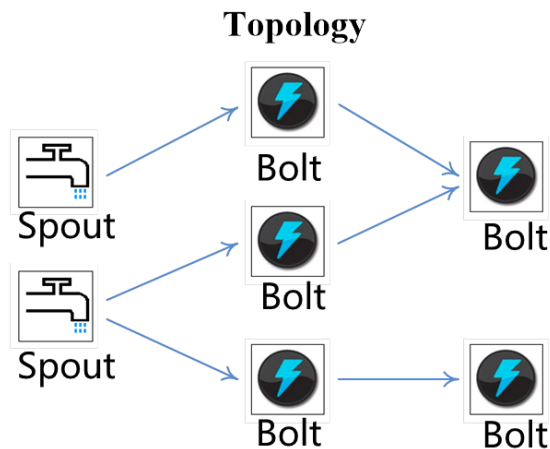
计算模式

- 分布



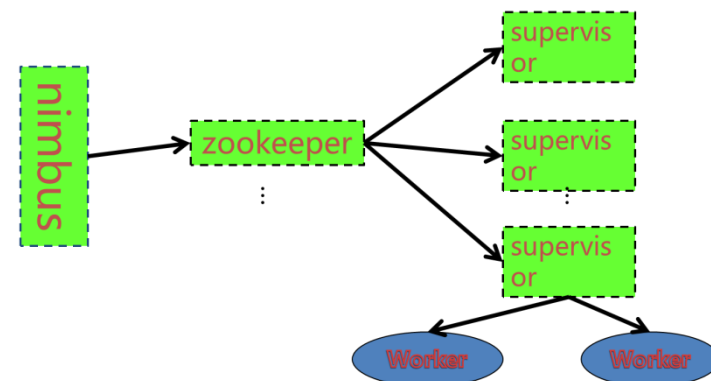
Storm容错—任务级失败

- **Bolt任务crash引起的消息未被应答**。此时，acker中所有与此Bolt任务关联的消息都会因为超时而失败，对应的Spout的fail方法将被调用。
- **acker任务失败**。如果acker任务本身失败了，它在失败之前持有的所有消息都将超时而失败。Spout的fail方法将被调用。
- **Spout任务失败**。在这种情况下，与Spout任务对接的外部设备(如MQ)负责消息的完整性。例如，当客户端异常时，kestrel队列会将处于pending状态的所有消息重新放回队列中。



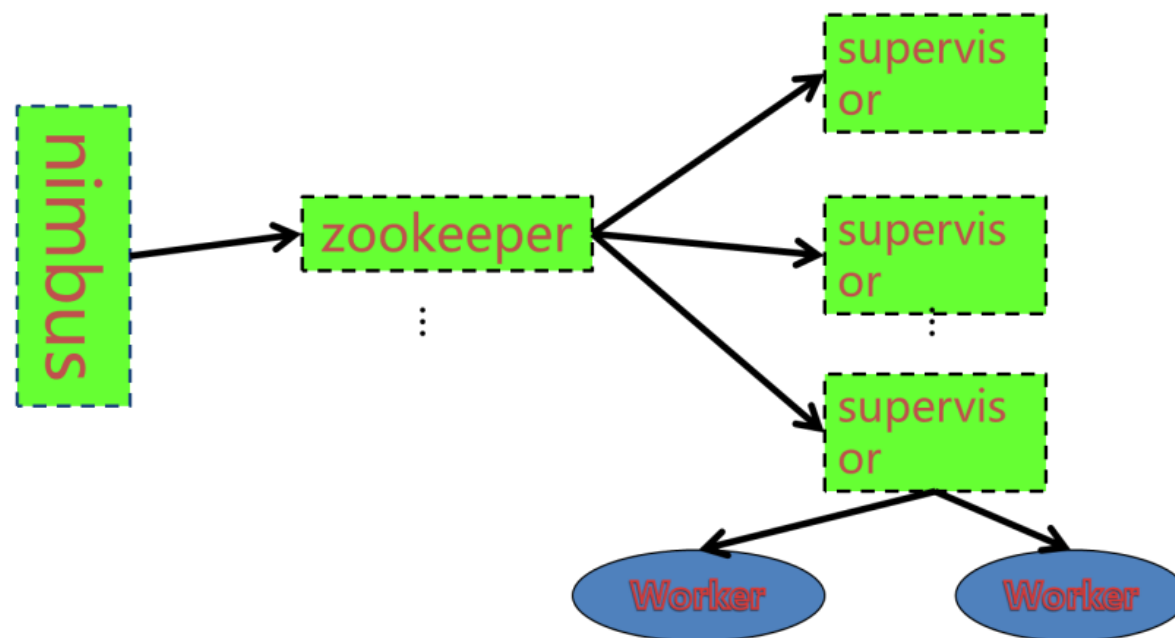
Storm容错—任务槽(slot)故障务级失败

- **Worker失败**。每个Worker中包含数个Bolt(或Spout)任务。Supervisor负责监控这些任务，当worker失败后会尝试在本机重启它，如果它在启动时连续失败了一定的次数，无法发送心跳信息到Nimbus，Nimbus将在另一台主机上重新分配worker。
- **Supervisor失败**。Supervisor是无状态(所有的状态都保存在Zookeeper或者磁盘上)和快速失败(每当遇到任何意外的情况，进程自动毁灭)的，因此Supervisor的失败不会影响当前正在运行的任务，只要及时将他们重新启动即可。
- **Nimbus失败**。Nimbus也是无状态和快速失败的，因此Nimbus的失败不会影响当前正在运行的任务，但是当Nimbus失败时，无法提交新的任务，只要及时将它重新启动即可。



Storm容错—集群节点(机器)故障

- **Storm集群中的节点故障**。此时Nimbus会将此机器上所有正在运行的任务转移到其他可用的机器上运行。
- **Zookeeper集群中的节点故障**。Zookeeper保证少于半数的机器宕机系统仍可正常运行，及时修复故障机器即可。



Storm容错—Nimbus节点故障

- 如果失去了Nimbus节点，Worker也会继续执行。另外，如果worker死亡，Supervisor也会继续重启他们。但是，没有Nimbus，Worker不会在必要时(例如，失去一个Worker的主机)被安排到其他主机，客户端也无法提交任务。所以Nimbus“在某种程度上”是单点故障。在实践中，这不是一个大问题，因为Nimbus守护进程死亡，不会发生灾难性问题。

